NEURAL NETWORKS / DAY 1

Neural network foundations

From tensors and single neurons to a network you can train and check by hand

Full day · 09:00–17:00 | 7 teaching parts | 2 laboratories | NumPy · scikit-learn · PyTorch

Today's plan

09:00	Welcome, five-day route, neural networks in context	Lecture
09:45	Tensors, shapes, and tools	Lecture
10:20	Break	Break
10:35	Lab 1: Sonar baseline pipeline, then debrief	Lab
11:40	From neuron to perceptron and XOR	Lecture
12:30	Lunch	Lunch
13:30	Multilayer networks; losses and gradient descent	Lecture
14:45	Break	Break
15:00	Backpropagation, training loop, first networks in code	Lecture
15:55	Lab 2: NumPy network on Sonar, debrief, exit check	Lab

Five days from a single neuron to attention

DAY 1 · TODAY

Neural network foundations

- Tensors and shapes
- Neuron, perceptron
- MLPs, activations
- Losses, gradients
- Backpropagation

Check a forward and backward pass

→

DAY 2

Training neural networks reliably

- Learning curves
- Initialization
- Normalization
- Optimizers
- Regularization

Reproduce a training experiment

→

DAY 3

Energy modelling capstone

- Data audits
- ESP gas lock
- PE log curve
- Sensor signals
- Error analysis

Defend a model comparison

→

DAY 4

Convolutional neural networks

- Convolution, pooling
- Architectures
- Transfer learning
- Segmentation
- Seismic, borehole

Check an image or signal model

→

DAY 5

Sequence models and attention

- Windows, baselines
- RNN, LSTM, TCN
- Attention
- Transformers
- Forecast intervals

Evaluate a future forecast

Schematic course map. Day 3 applies dense networks to the course energy data before CNNs and sequence models.

LEARNING OUTCOMES

What you will be able to do by 17:00

1. Place neural networks within AI and machine learning, and judge when one is worth trying.
2. Track batch, feature, and output shapes through a dense network.
3. Explain why XOR needs a hidden layer and what hidden units compute.
4. Match output layers, targets, and losses for regression and classification.
5. Compute one forward and backward pass by hand and verify it numerically.
6. Train a first network in NumPy, scikit-learn, or PyTorch against a baseline.

PRACTICE

4 min · Pairs

Warm-up: where would a network earn its place?

1. Name one prediction task from your work with a fixed set of numeric inputs.
2. What simple model would you compare a network against?
3. What evidence would convince your team that the network is better?

PART 1 OF 7

01

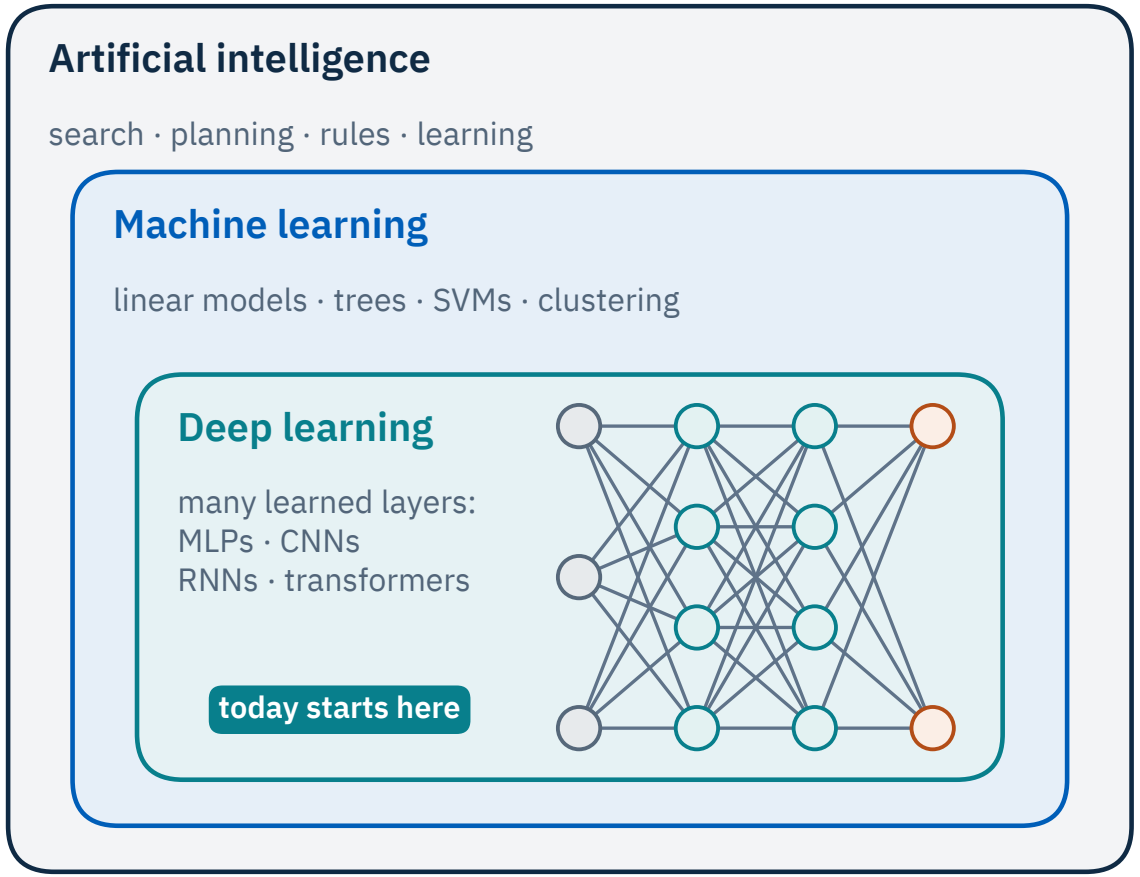
Neural networks in context

One model family among many; it must earn its place.

- AI, machine learning, and deep learning
- Eight milestones since 1943
- Where networks meet energy data
- When a network is worth trying

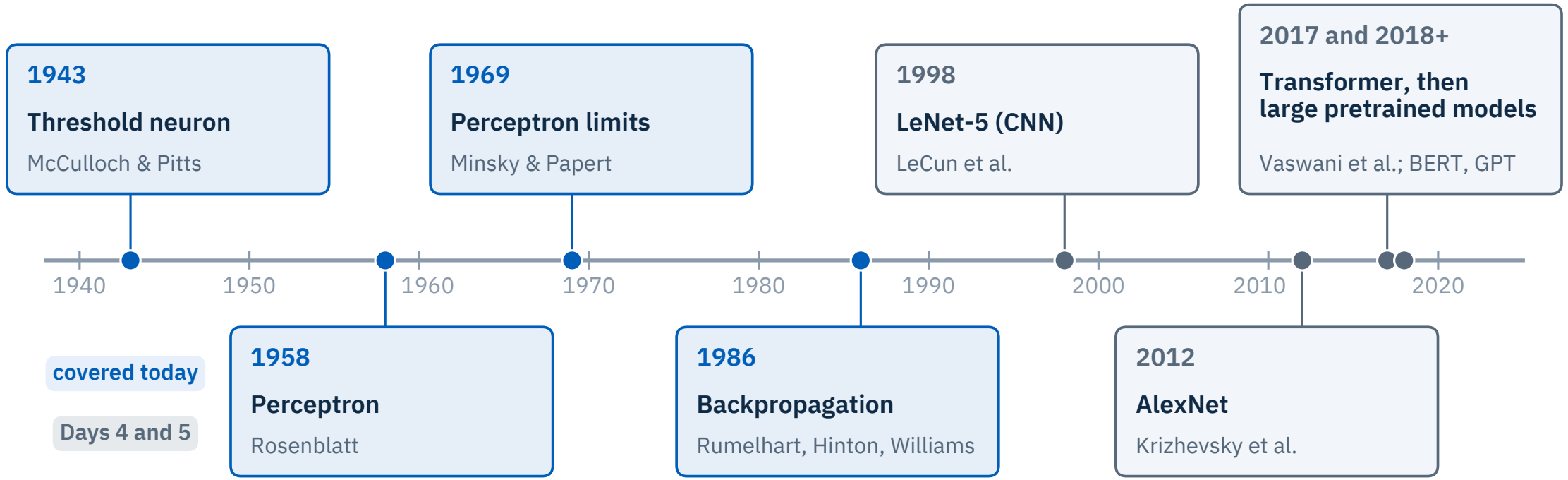
Deep learning is one branch of machine learning

- AI is the broad field; not all of it learns from data.
- Machine learning fits a model's parameters to data.
- **Deep learning** composes many learned layers.
- The same networks serve classification, regression, and representation learning.



Schematic. The examples in each field are illustrative, not exhaustive.

Eight milestones from threshold units to transformers



Schematic timeline; dates are publication years of the cited works. Blue: covered today; grey: covered on Days 4 and 5.

Data, compute, and software made depth practical

Data

- Large labelled collections, such as ImageNet
- Sensors and logs recorded at scale
- Energy data: often few independent units

Compute

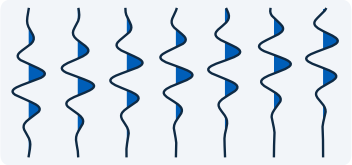
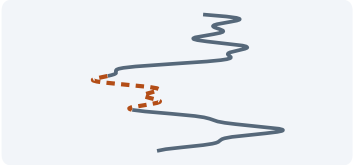
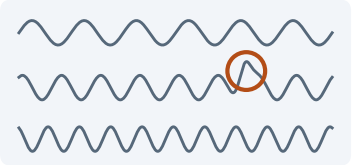
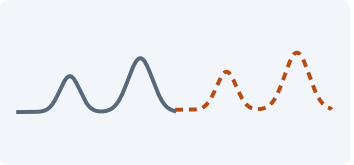
- Matrix products map onto GPU hardware
- Many training runs become affordable
- Larger and deeper models fit in memory

Methods and software

- ReLU units, better initialization, dropout
- Automatic differentiation frameworks
- Open-source code and pretrained models

◆ None of these removes the need for a baseline and an honest split.

Where neural networks meet energy data

Seismic interpretation	Log prediction	Production forecasting	Equipment health	Power and load forecasting
				
image patches (B, 1, H, W)	log curves (B, D)	rate history (B, T, F)	sensor channels (B, T, F)	weather, history (B, T, F)
CNN	MLP, 1-D CNN	LSTM, TCN	MLP, autoencoder	MLP, LSTM
fault or facies map	missing curve (PE)	future rates	anomaly score	next-day profile
Day 4	Day 3	Day 5	Days 2 to 5	Day 5

Schematic. Common data–architecture pairings, not recommendations; every case still needs a baseline. Chips show where the course covers each.

A neural network needs the same evidence as other models

1. **Frame** — the target, the inputs known at prediction time, the decision it supports.
2. **Baseline** — the simplest defensible non-neural model on the same rows.
3. **Split** — by well, asset, or time, matching how the model will be used.
4. **Select** — architecture, learning rate, and threshold on validation data only.
5. **Test** — once, on held-out data; then inspect errors by group.

◆ A flexible model exploits leakage as readily as any other model; it does not remove it.

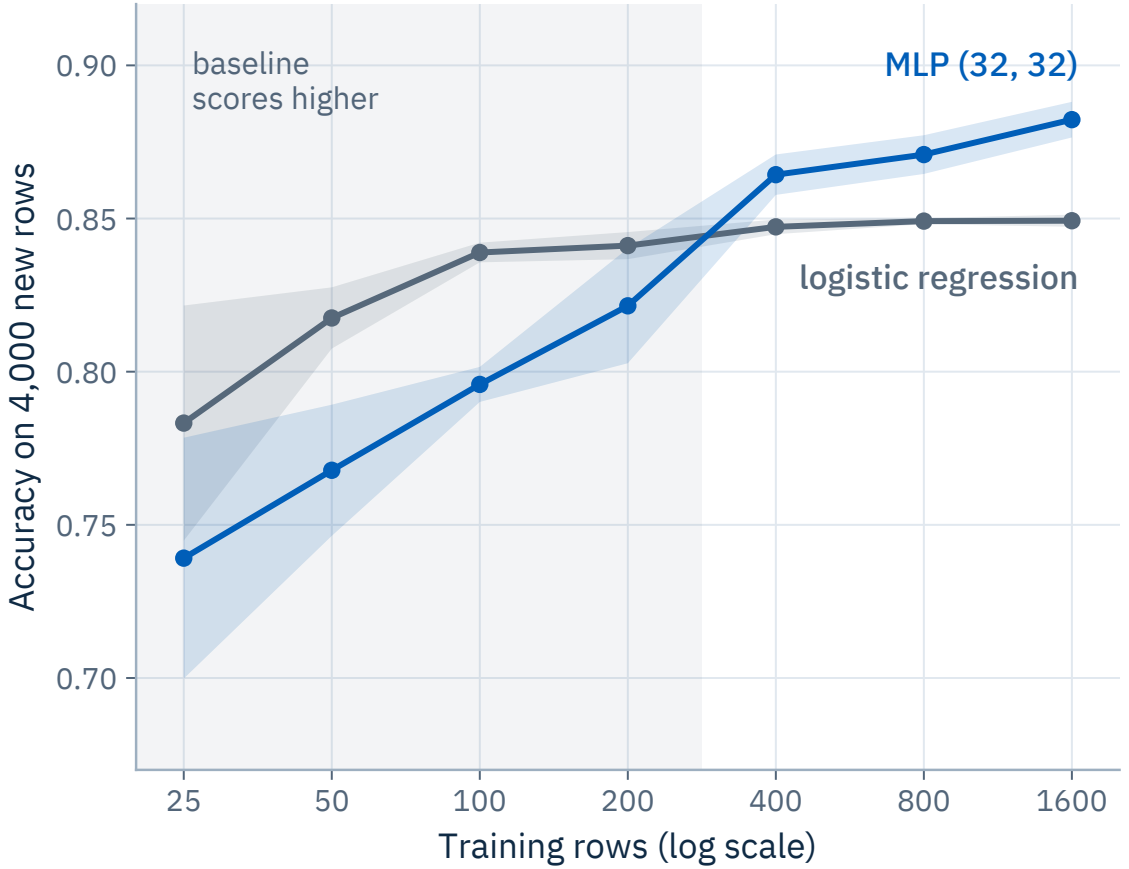
The data type decides whether a network is worth trying

Data and task	Useful initial comparison	Main concern
Small tabular dataset	Linear model and boosted trees	High variance from limited data
Images	CNN or pretrained visual model	Labels and domain shift
Ordered signals	Linear lag model, 1-D CNN, RNN	Correct sequence construction
Complex representations	Task-specific neural architecture	Compute and validation evidence

◆ Today's Sonar data (208 rows, 60 numeric features) is a small tabular problem: expect high variance.

On small samples, a simple baseline can beat a network

- Same synthetic task; only the number of training rows changes.
- Up to 200 rows, logistic regression scores higher.
- From 400 rows the MLP pulls ahead: 0.88 vs 0.85 at 1,600.
- In energy data, count independent wells or assets, not rows.



Computed: synthetic two moons plus 4 noise features; lines show the mean of 8 resamples (seeds 0–7), bands ±1 sd.

Flexibility is paid for in data, compute, and care

✓ What networks offer

- Learn features from raw signals and images
- One framework for many output types
- Transfer from pretrained models

vs

✗ What they cost

- More data, tuning, and compute
- More ways to fail silently
- Harder to explain and to validate

◆ Much of today builds habits that turn silent failures into loud ones.

PRACTICE

5 min · Pairs

Would you start with a neural network here?

1. 300 wells, 12 log-derived features per depth sample → facies class.
2. Two years of 1-minute ESP sensor data → gas-lock warning an hour ahead.
3. 50,000 labelled seismic image patches → fault or no fault.
4. Daily wind-farm output and weather forecasts → next-day power.

PART 2 OF 7

02

Tensors, shapes, and tools

Most first bugs show in the shapes before the loss.

- The Python stack and what each tool is for
- Tensors, ranks, and the batch axis
- Broadcasting, vectorization, and devices
- The Sonar data and a baseline pipeline

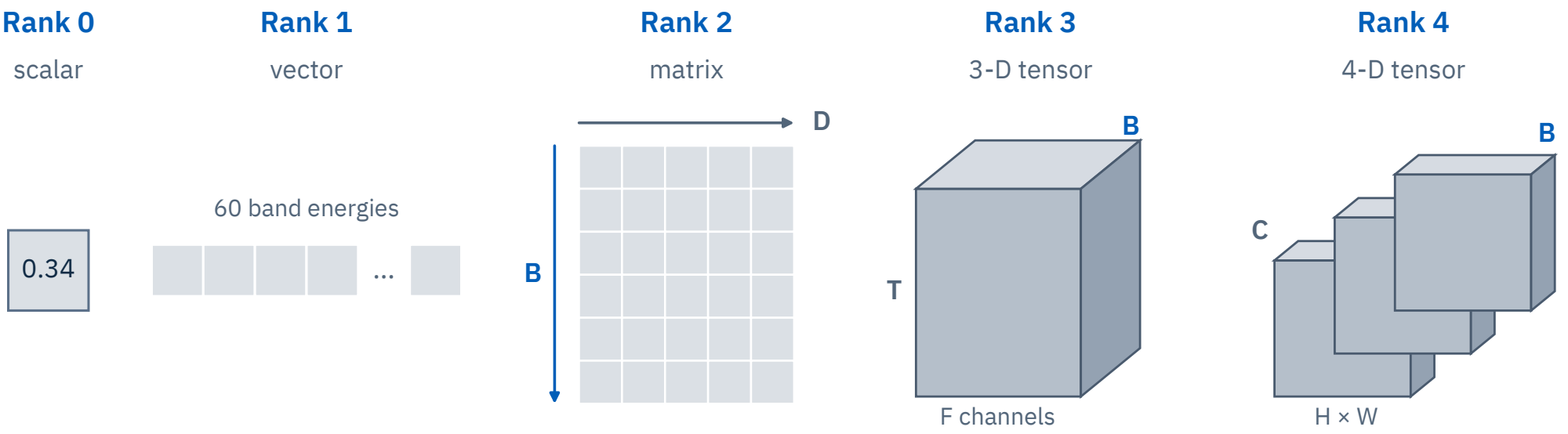
The Python stack has distinct responsibilities

Tool	Responsibility	First check
NumPy	Arrays and vectorized calculations	Shape and dtype
pandas	Tables, labels, joins, and timestamps	Row alignment and schema
Matplotlib	Scientific plots	Axes, units, and sampling
scikit-learn	Baselines and preprocessing	Fitting inside training folds
PyTorch / Keras	Differentiable models and optimization	Output-loss compatibility

◆ Keep notebooks restartable: run top to bottom before trusting any number.

A tensor is an array with a fixed number of axes

The **rank** is the number of axes; the shape lists the length of each axis.



Schematic. B = batch, D = features, T = time steps, F = channels, C = image channels, H × W = image size.

Energy data arrive in a few standard tensor shapes

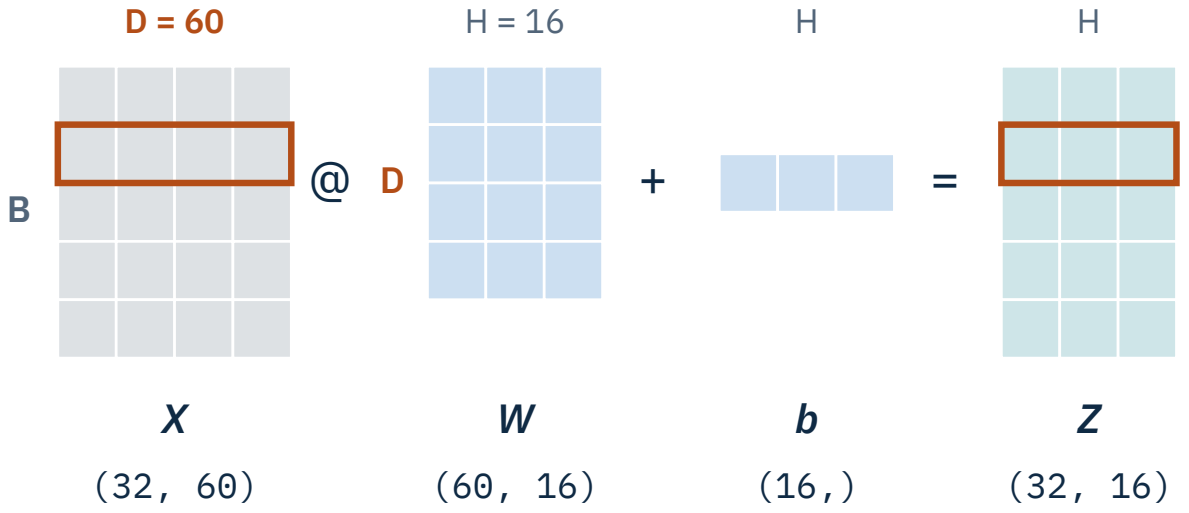
Data	Tensor shape	Axes
Log samples as a table	(B, D)	depth samples × curves
Sonar echoes (today)	(B, 60)	echoes × frequency bands
ESP sensor windows	(B, T, F)	windows × time steps × channels
Seismic patches	(B, 1, H, W)	patches × amplitude channel × height × width
Load or production history	(B, T, 1)	windows × time steps × one value

◆ First question for any dataset: what is one example, and what does B count?

One batch, one shape contract

$$Z = XW + b$$

- Rows are examples: X is (B, D), Z is (B, H).
- Inner sizes must match: (B, D) @ (D, H).
- W never contains B, so the batch size is free.
- PyTorch `nn.Linear(D, H)` stores its weight as (H, D), the transpose of this W.



`(32, 60) @ (60, 16)` inner sizes must match

row i of Z uses only row i of X (outlined)

Schematic with B = 32, D = 60, H = 16. Row i of Z depends only on row i of X.

Broadcasting stretches size-1 axes, sometimes silently

- Shapes align from the last axis; missing or size-1 axes are copied.
- A bias of shape (H,) added to (B, H) is the intended use.
- (B, 1) minus (B,) silently becomes (B, B).
- Reshape targets to match predictions before any loss.

Adding a bias: (3, 2) + (2,) ✓

1	2		10	20		11	22
3	4	+			=	13	24
5	6					15	26

b is copied down the batch

A loss with mismatched shapes: (3, 1) − (3,) ✗

0.9			1	0	1	−0.1	0.9	−0.1
0.2	−				=	−0.8	0.2	−0.8
0.7						−0.3	0.7	−0.3

result: (3, 3)

diagonal = intended

others = nonsense

MSE intended 0.047 · MSE computed 0.313 · no error raised

Exact NumPy arithmetic. The mismatched loss is 0.313 instead of 0.047, and no error is raised.

PRACTICE

4 min · Individual

Predict the shape before you run the cell

X has shape (32, 60), W (60, 16), b (16,), y_pred (32, 1), and y (32,).

1. `X @ W + b`
2. `(X @ W).mean(axis=0)`
3. `y_pred - y`
4. `W @ X`

Vectorized calculation replaces nested loops

```python
import numpy as np

X = np.array([[1., 2.], [3., 4.]])      # 2 examples, 2 features
W = np.array([[0.5, -1.], [1., 0.5]])   # 2 inputs -> 2 units
b = np.array([0., 1.])
Z = X @ W + b                           # (2, 2): one row per example
A = np.maximum(Z, 0.)                   # ReLU, elementwise
assert A.shape == (2, 2)
print(A)
```

OUTPUT

```
[[2.5 1. ]
 [5.5 0. ]]
```

One matrix product computes every unit for every example at once.

PyTorch tensors mirror NumPy arrays

```
import numpy as np
import torch

a = np.arange(6, dtype=np.float32).reshape(2, 3)
t = torch.from_numpy(a)              # shares memory with a
print(t.shape, t.dtype, t.ndim)
print((t @ t.T).shape)               # (2, 3) @ (3, 2)
a[0, 0] = 100.                       # change the NumPy array...
print(t[0, 0])                       # ...and the tensor sees it
```

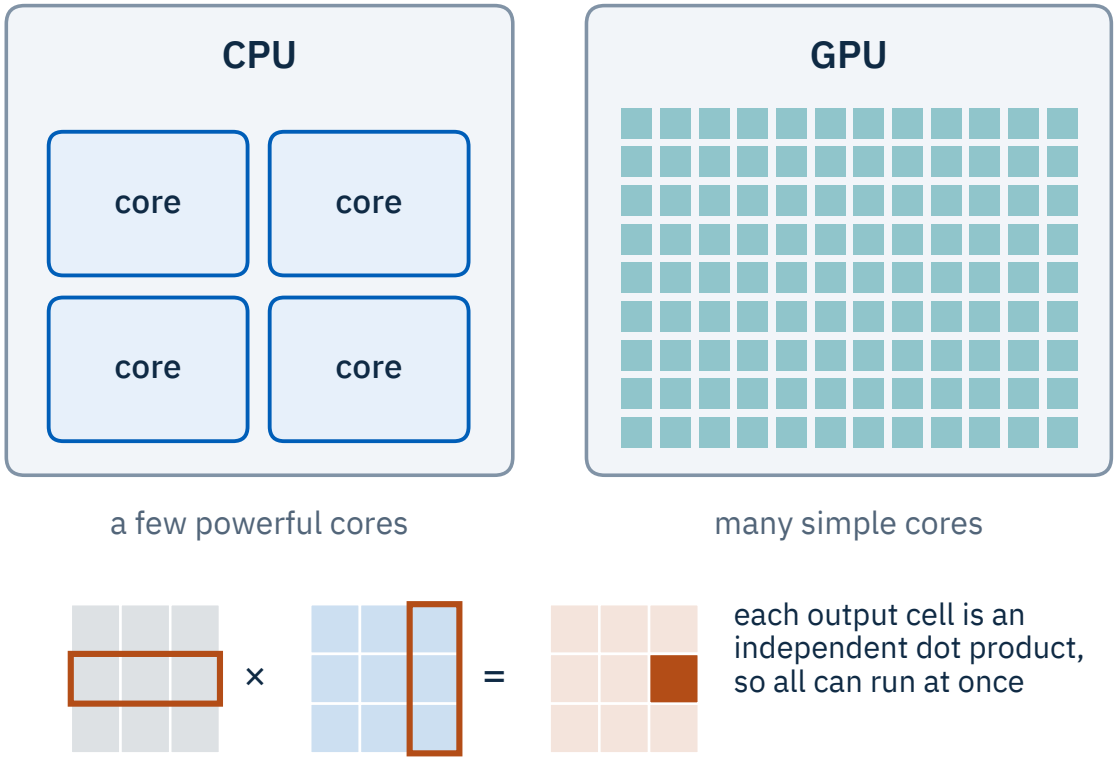
OUTPUT

```
torch.Size([2, 3]) torch.float32 2
torch.Size([2, 2])
tensor(100.)
```

`torch.from_numpy` shares memory; `torch.tensor` copies. NumPy defaults to float64, PyTorch layers to float32.

GPUs run many identical operations at once

- A matrix product is many independent dot products.
- GPUs trade per-core flexibility for massive parallelism.
- Small tabular networks, like today's, train quickly on a CPU.
- Model and data must be on the same device.



Schematic; core counts are illustrative, not a specific device.

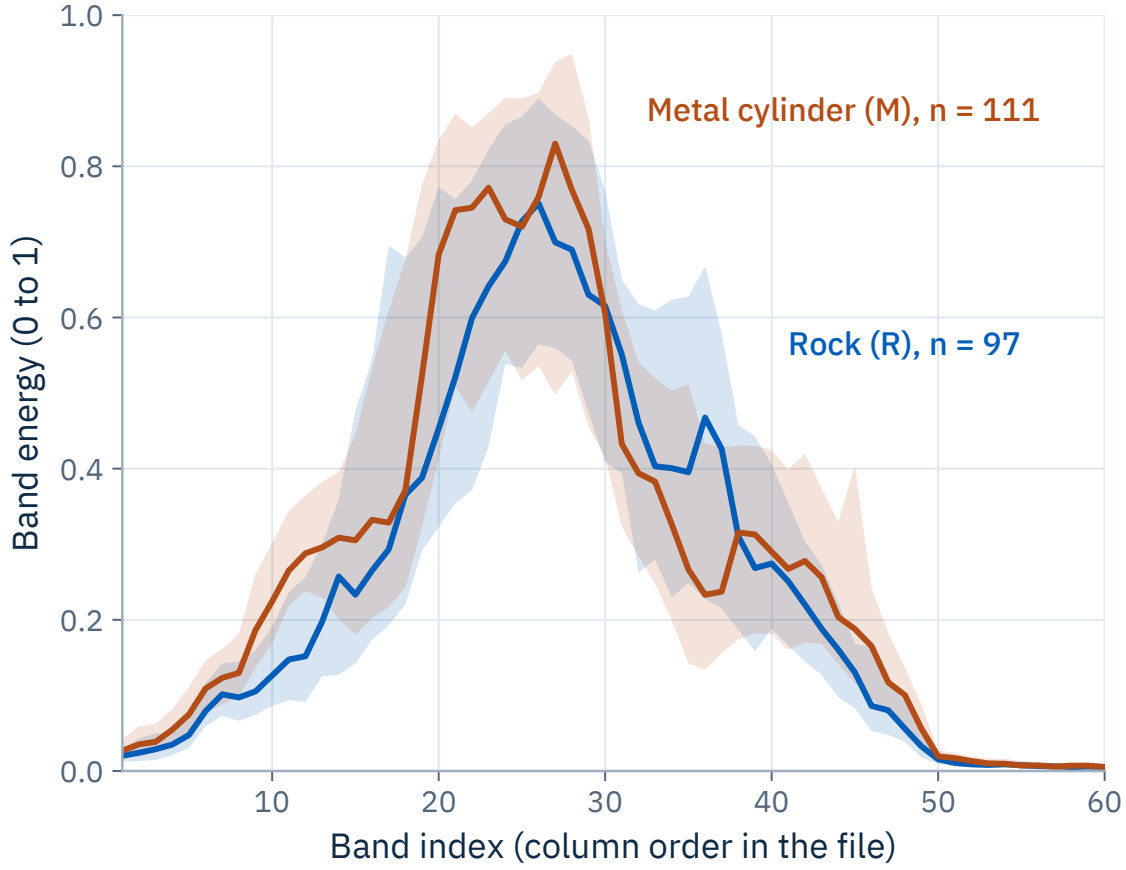
A reliable table-to-tensor path

1. **Join** — attach labels by a verified key, never by row position after a sort.
2. **Select** — list predictor columns explicitly and keep their order fixed.
3. **Split** — by well, asset, or time before fitting anything.
4. **Scale** — fit imputers and scalers on training rows only.
5. **Convert** — cast to float32 and check for NaN and infinite values.

- Numeric row IDs are not features: drop them before training.

Lab data: 208 sonar echoes with 60 band energies each

- Labels: R = rock, M = metal cylinder, not "mineral".
- 97 rock and 111 metal echoes; values already in [0, 1].
- The classes overlap heavily in every single band.
- With 208 rows, one random split gives a noisy score.



Computed from sonar.all-data.csv: median and interquartile range of each band for each class.

Baseline pipeline for Sonar

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y_raw, random_state=42)
baseline = make_pipeline(StandardScaler(),
                         LogisticRegression(max_iter=1000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(baseline, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.753 +/- 0.061
```

The 42 test rows stay untouched until one final evaluation.

LABORATORY

50 min

Lab 1: a Sonar baseline you can defend

- Load, explore, and encode the labels; print the label mapping.
- Build three pipelines: logistic regression, random forest, RBF SVM.
- Compare them with shuffled 5-fold CV on the training rows only.
- Tune the best of them with GridSearchCV on the same folds.
- Test the tuned model once; note what 42 rows can establish.

NOTEBOOK

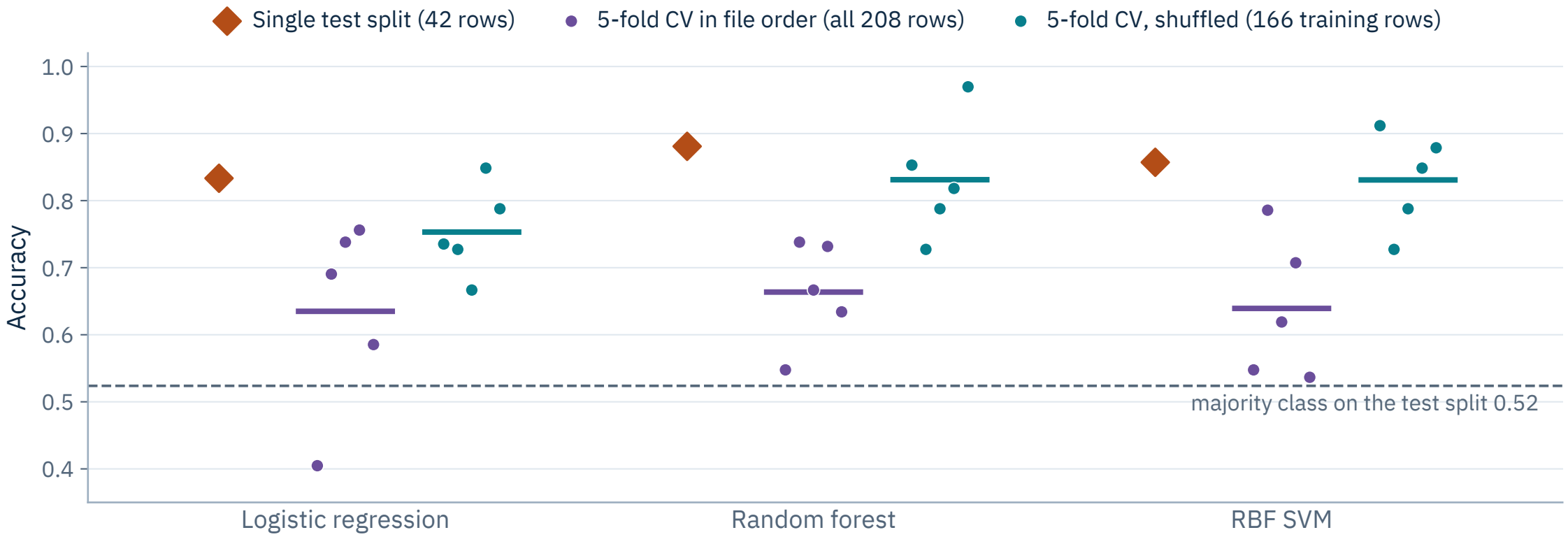
`exercises/01_sklearn_pipeline/sklearn_Pipeline_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table: CV mean ± sd per model, one final test score
- The printed label mapping and class counts
- Two sentences on the uncertainty of your best score

DISCUSSION

Lab 1 debrief: the evaluation protocol moves the score



Computed from sonar.all-data.csv with the Lab 1 pipelines. Dots: single folds; bars: fold means; diamonds: the 42-row test split.

DISCUSSION

Lab 1 debrief: three habits to keep all week

Print the label mapping

- Mine = 1, Rock = 0 in all labs
- LabelEncoder picks M = 0
- Precision and recall depend on it

Keep the test set for last

- Choosing by test accuracy spends the test set
- Select with CV, then test once
- Report the test score with its n

Treat row order as structure

- Unshuffled folds follow the file order
- The two protocols differ by over 10 points
- Group or order splits when metadata allows

◆ The baseline to beat this afternoon: 0.75 CV accuracy (logistic), 0.83 (forest, SVM).

PART 3 OF 7

03

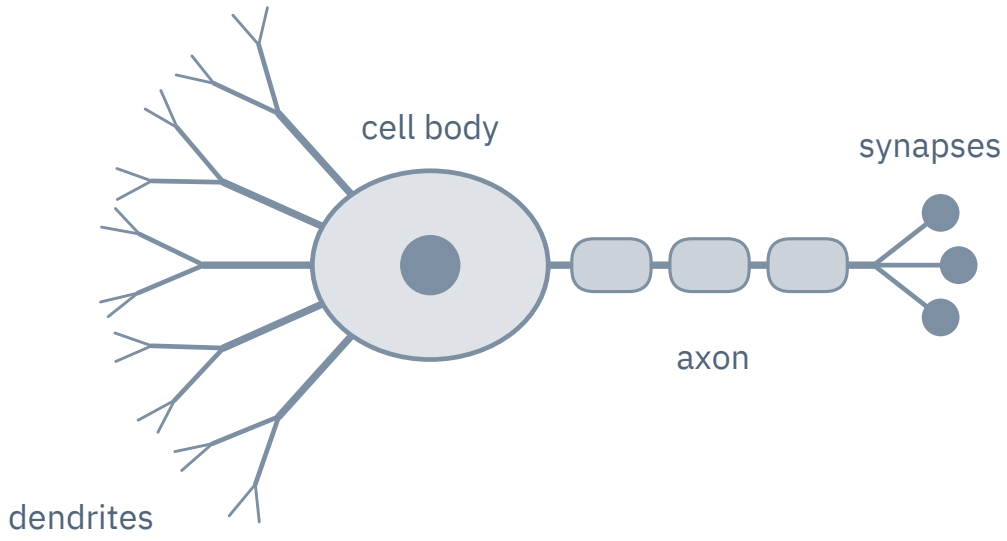
From neuron to perceptron

A weighted sum and a threshold draw one straight line.

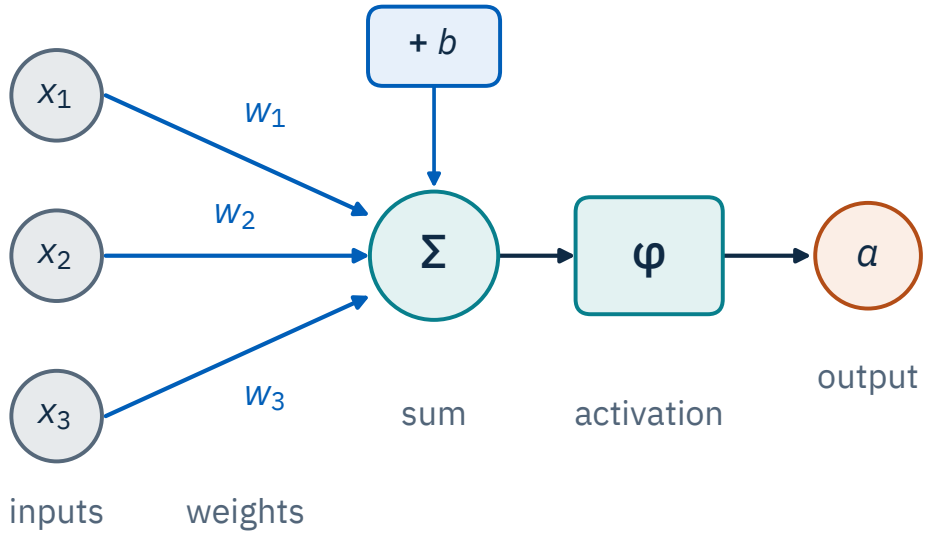
- Biological inspiration and its limits
- The artificial neuron and the perceptron rule
- Linear separability and XOR
- A hidden layer that solves XOR

The artificial neuron borrows a loose biological metaphor

Biological neuron (sketch)



Artificial unit (a formula)



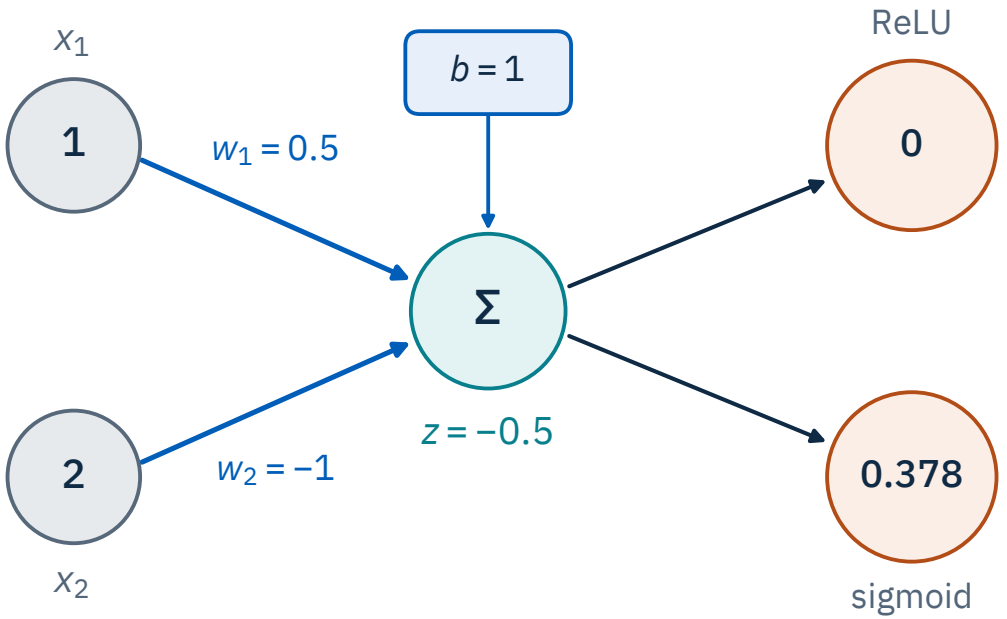
What the analogy leaves out: spike timing · computation inside dendrites · chemical signalling · adaptation over time

Schematic drawing, not anatomically accurate. The analogy motivated the model; the equations define it.

Weighted inputs, bias, and activation

$$z = \sum_{j=1}^{d} w_j x_j + b, \qquad a = \phi(z)$$

- The weights set each input's contribution.
- The bias shifts the operating point.
- The **activation** φ shapes the response.



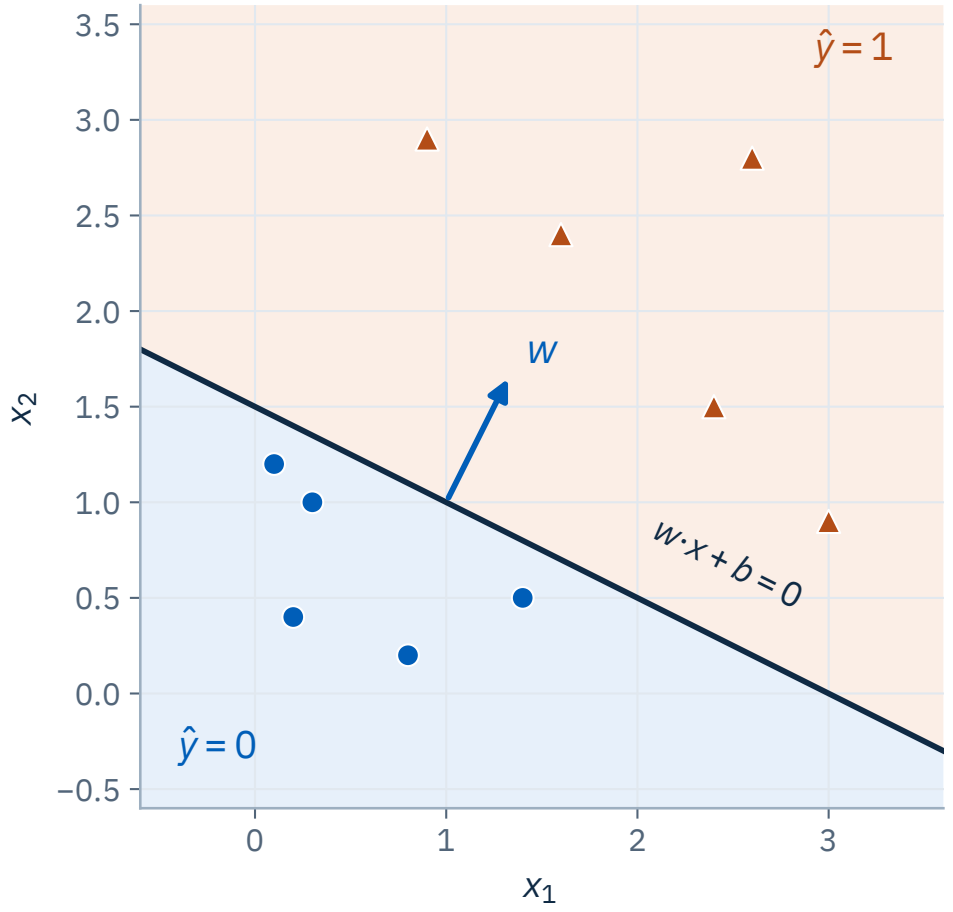
$$z = 0.5 \cdot 1 + (-1) \cdot 2 + 1 = -0.5$$

Exact arithmetic for inputs (1, 2), weights (0.5, −1), and bias 1.

A perceptron thresholds a weighted sum into a class

$$\hat{y} = \text{step}(w \cdot x + b)$$

- step(z) = 1 if z > 0, otherwise 0.
- The boundary $w \cdot x + b = 0$ is a straight line (a plane in more dimensions).
- The weight vector w is perpendicular to the boundary; b shifts it.

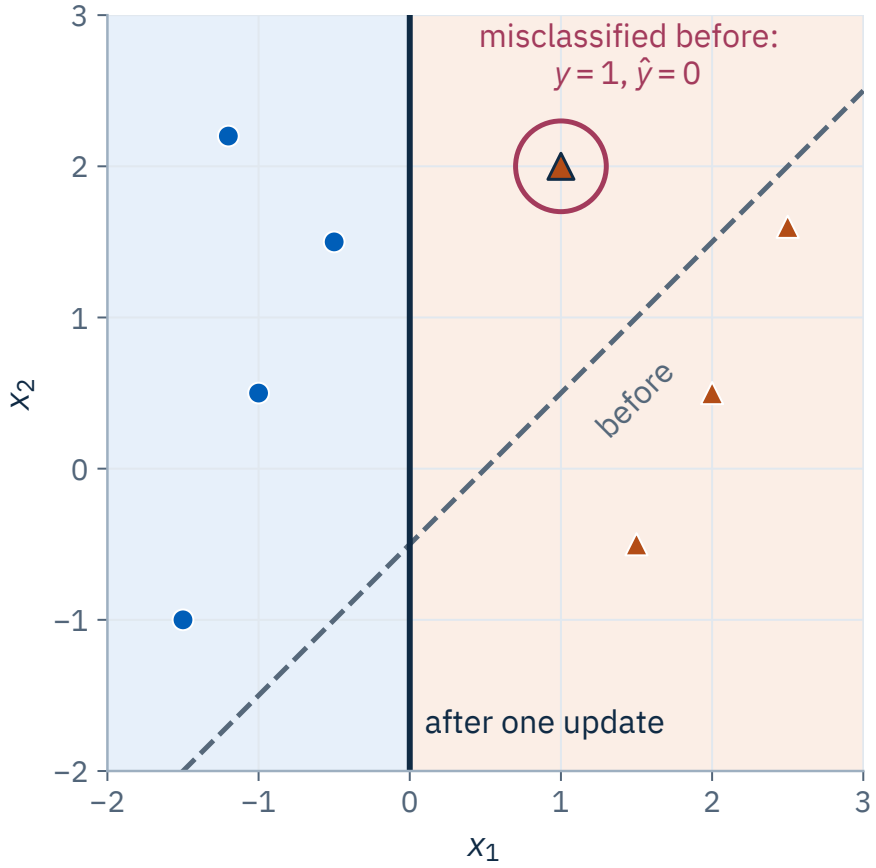


Exact geometry for w = (1, 2) and b = −3; illustrative points.

The perceptron rule moves the boundary toward mistakes

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

- Correct predictions leave the weights unchanged.
- Worked update: $x = (1, 2)$, $y = 1$, $\hat{y} = 0$, $\eta = 0.5$.
- w: $(1, -1) \to (1.5, 0)$; b: $-0.5 \to 0$.
- On separable data the rule stops after finitely many mistakes.



Exact update. Dashed: boundary before; solid: after one update. The ringed point is now classified correctly.

PRACTICE

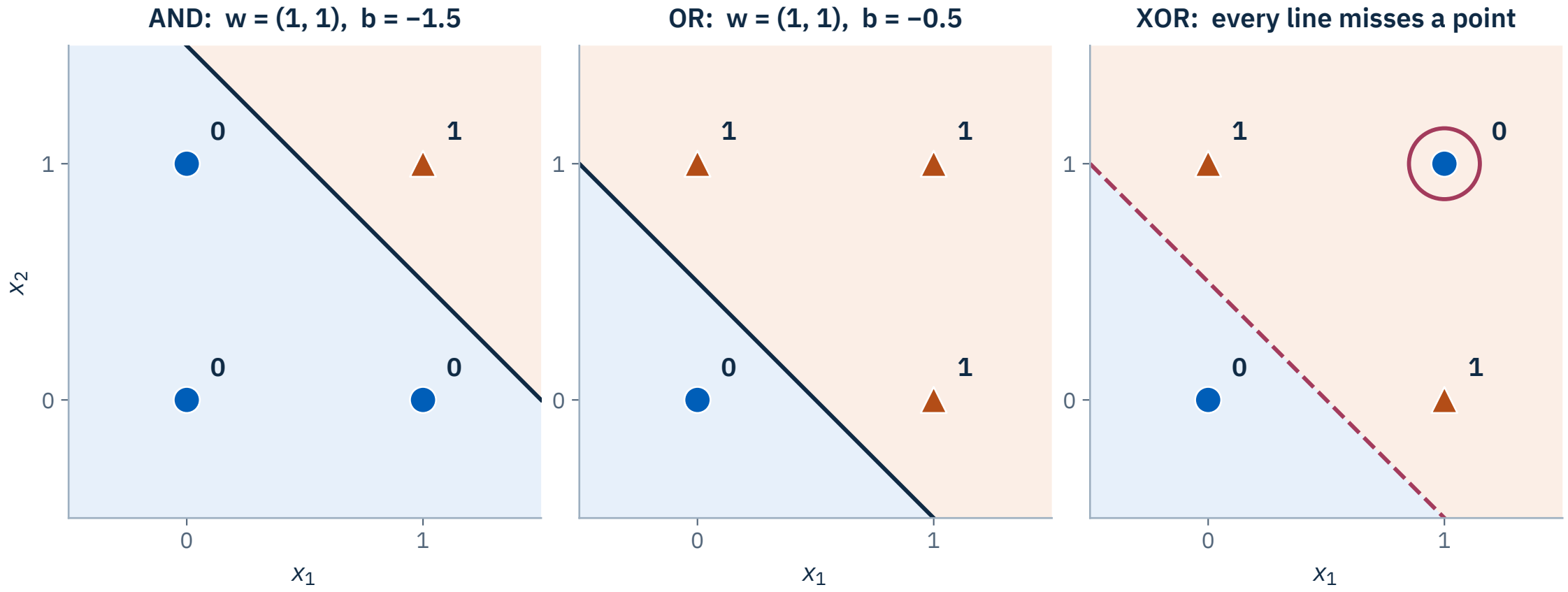
5 min · Individual

Update a perceptron by hand

Start from $w = (0.2, -0.4)$, $b = 0$, and learning rate $\eta = 0.5$.

1. Present $x = (1, 1)$ with $y = 1$: compute z, $\hat{y}$, and the new w and b.
2. Then present $x = (2, -1)$ with $y = 0$ and update again.
3. Are both examples now classified correctly?

AND and OR are linearly separable; XOR is not



Exact threshold units. On XOR even the best line, here the OR line, misclassifies (1, 1); the perceptron rule keeps making mistakes on every pass.

PRACTICE

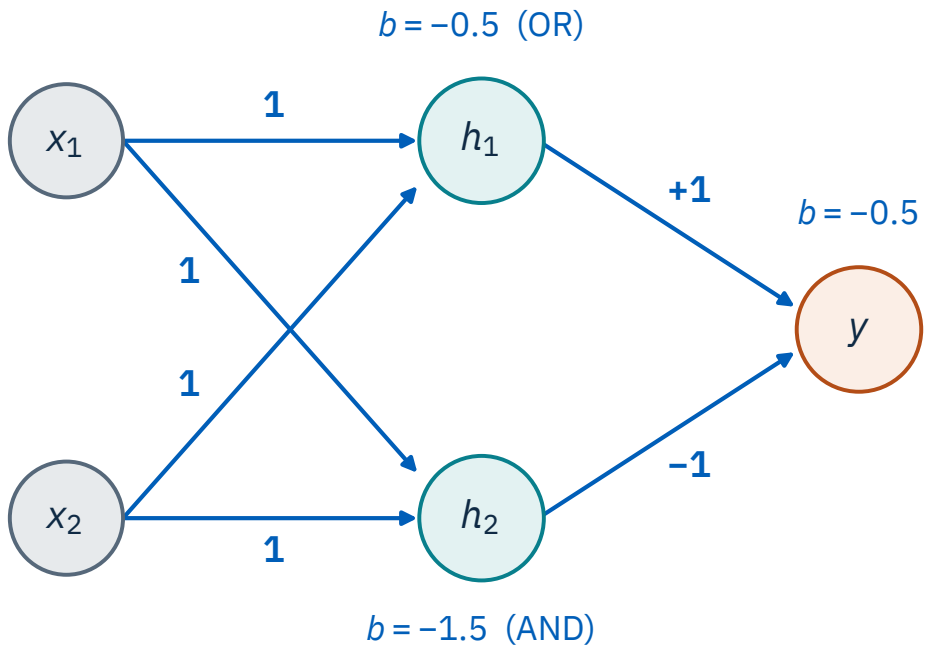
5 min · Pairs

Show that no single threshold unit computes XOR

A unit outputs 1 when $w_1x_1 + w_2x_2 + b > 0$ and 0 otherwise.

1. Write the four inequalities that XOR requires.
2. Add the two inequalities for the inputs (0, 1) and (1, 0).
3. Show that the result contradicts the other two.

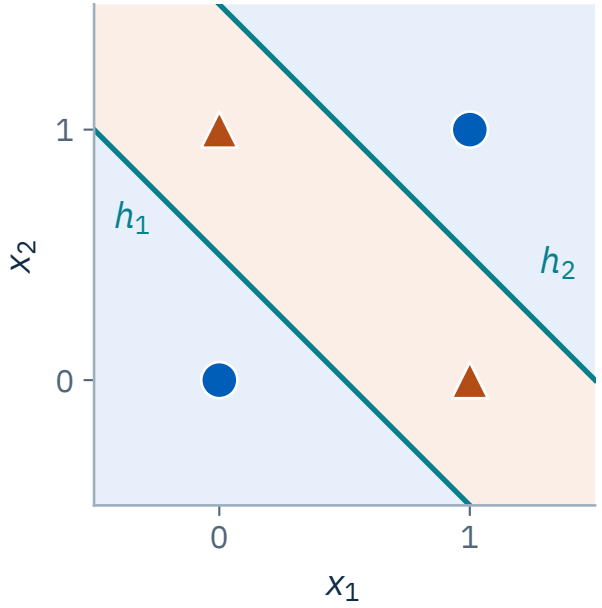
Two hidden units solve XOR: OR and not AND



all units: step(z) = 1 if z > 0, else 0

x_1	x_2	h_1	h_2	y
0	0	0	0	**0**
0	1	1	0	**1**
1	0	1	0	**1**
1	1	1	1	**0**

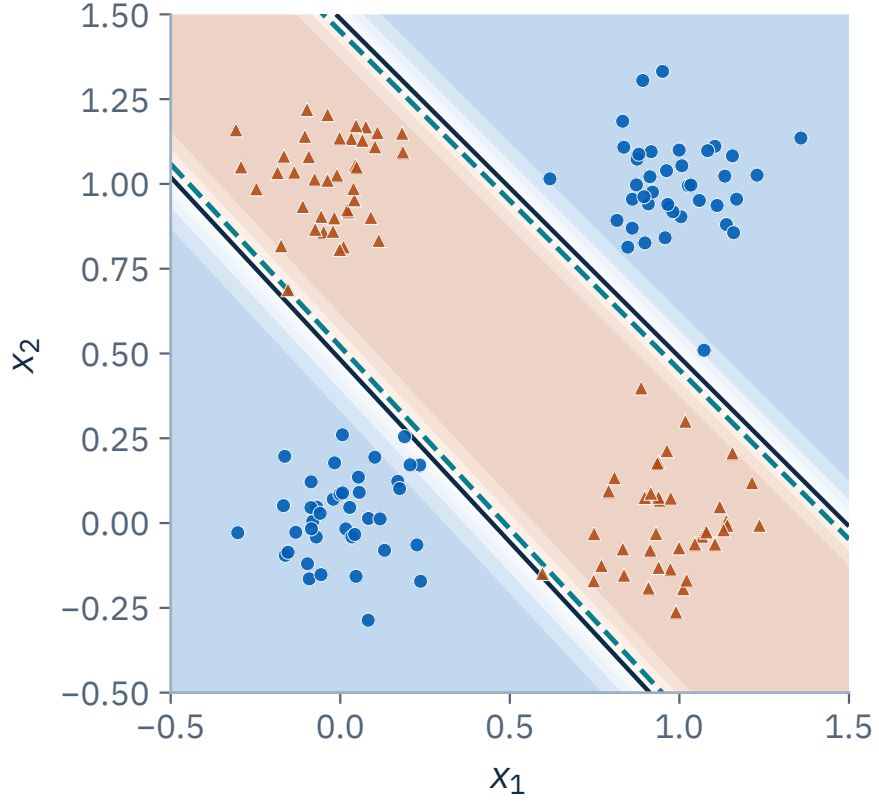
$y = h_1$ AND NOT h_2 = XOR



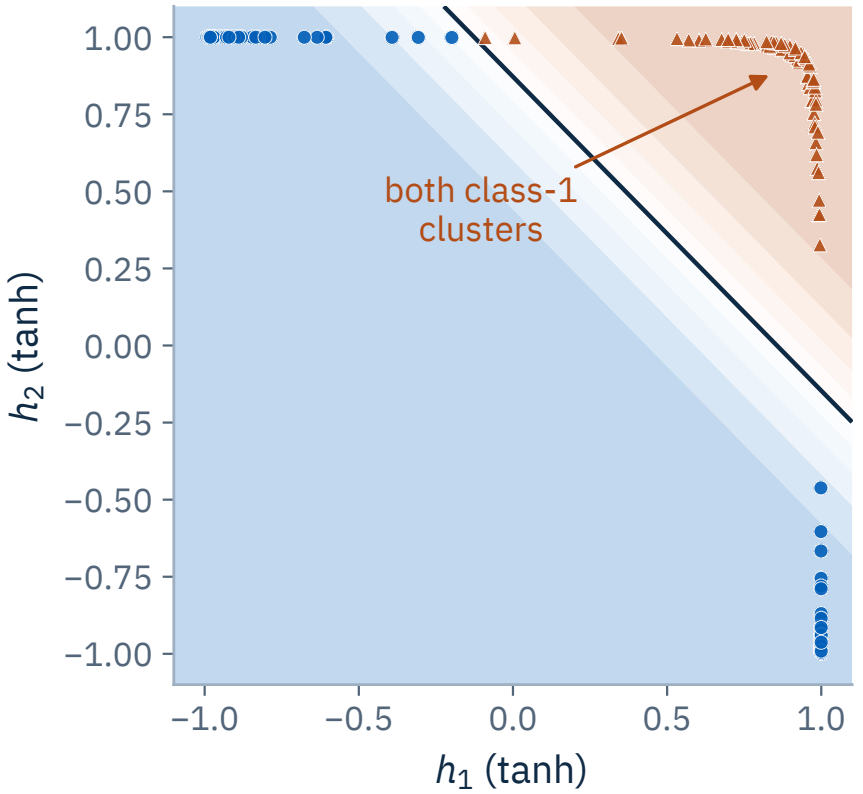
Exact hand-set weights with step activations. Hidden unit 1 is the OR line, hidden unit 2 the AND line; the output fires in the band between them.

The hidden layer re-maps inputs so that a line suffices

Input space: two hidden lines form a band



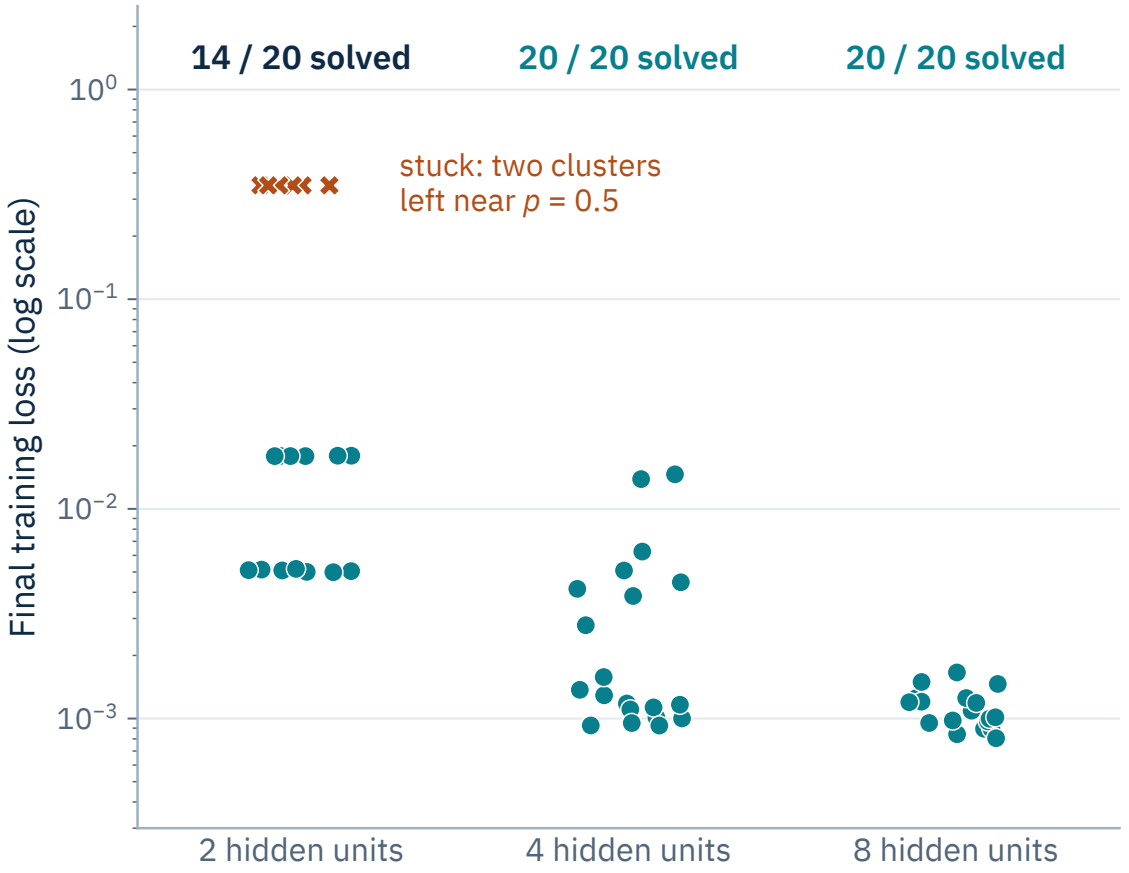
Hidden space: a straight line suffices



Computed: noisy XOR (4 × 40 points, seed 0); 2–2–1 tanh network, initialization seed 2, 300 full-batch gradient steps. Solid lines: decision boundary (p = 0.5); dashed: where each hidden unit is zero.

Gradient descent solves XOR, but not from every start

- Same data and training recipe; only the initial weights differ.
- 2 hidden units: 14 of 20 starts solve XOR.
- The others stall with two clusters left at $p \approx 0.5$.
- 4 or 8 hidden units: all 20 starts solve it.



Computed on the same noisy XOR data: 20 initialization seeds per width, 4000 full-batch steps, learning rate 1.0.

PRACTICE

3 min · Pairs

Which of these can a single perceptron represent?

Inputs are 0 or 1 unless stated; a unit fires when $w \cdot x + b > 0$.

1. NAND of two inputs
2. XNOR: 1 when both inputs are equal
3. Majority vote of three inputs
4. "$x_1 > x_2$" for real-valued inputs

PART 4 OF 7

04

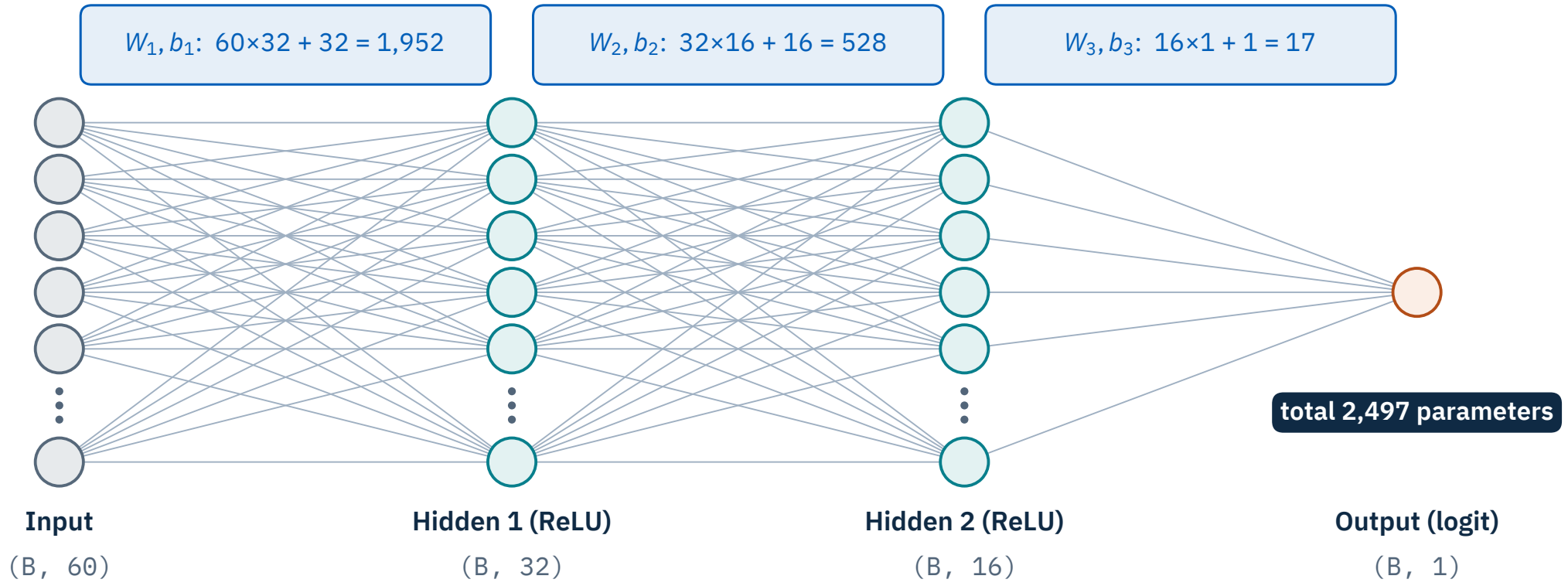
Multilayer networks

Stacked weighted sums and nonlinearities bend to the data.

- Layers as matrix products
- Why activations must be nonlinear
- Activation functions and their derivatives
- Universal approximation, width, and depth

A multilayer perceptron is a chain of dense layers

Each layer maps (B, in) to (B, out) with a weight matrix (in, out) and one bias per output unit.

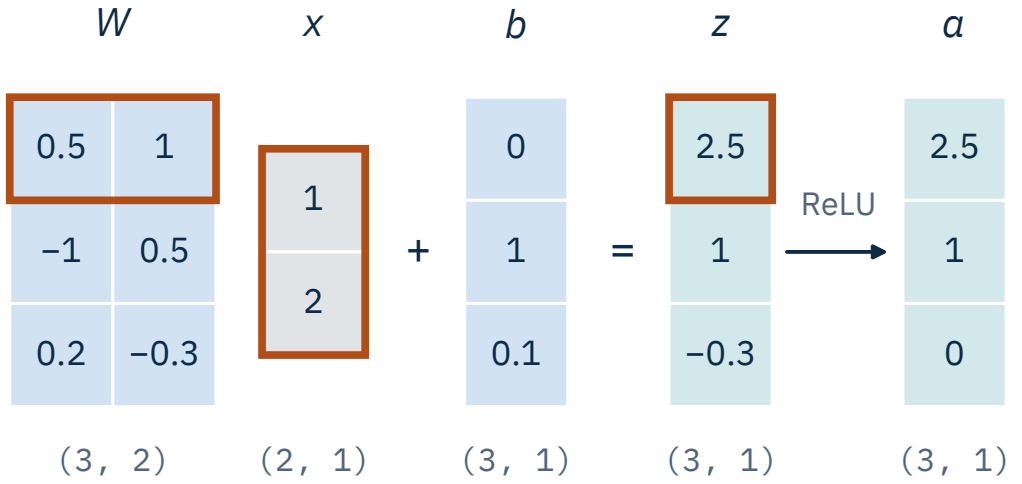


Schematic of a 60–32–16–1 network for Sonar-sized input; parameter counts are exact.

A dense layer is one matrix product plus a bias

$$z = Wx + b, \qquad a = \mathrm{ReLU}(z)$$

- One example in as a column vector *x*, one vector *a* out.
- Row *j* of *W* holds the weights of unit *j*.
- A batch stacks examples as rows: $Z = XW^{\top} + b$.



$z_1 = 0.5 \cdot 1 + 1.0 \cdot 2 + 0 = 2.5$

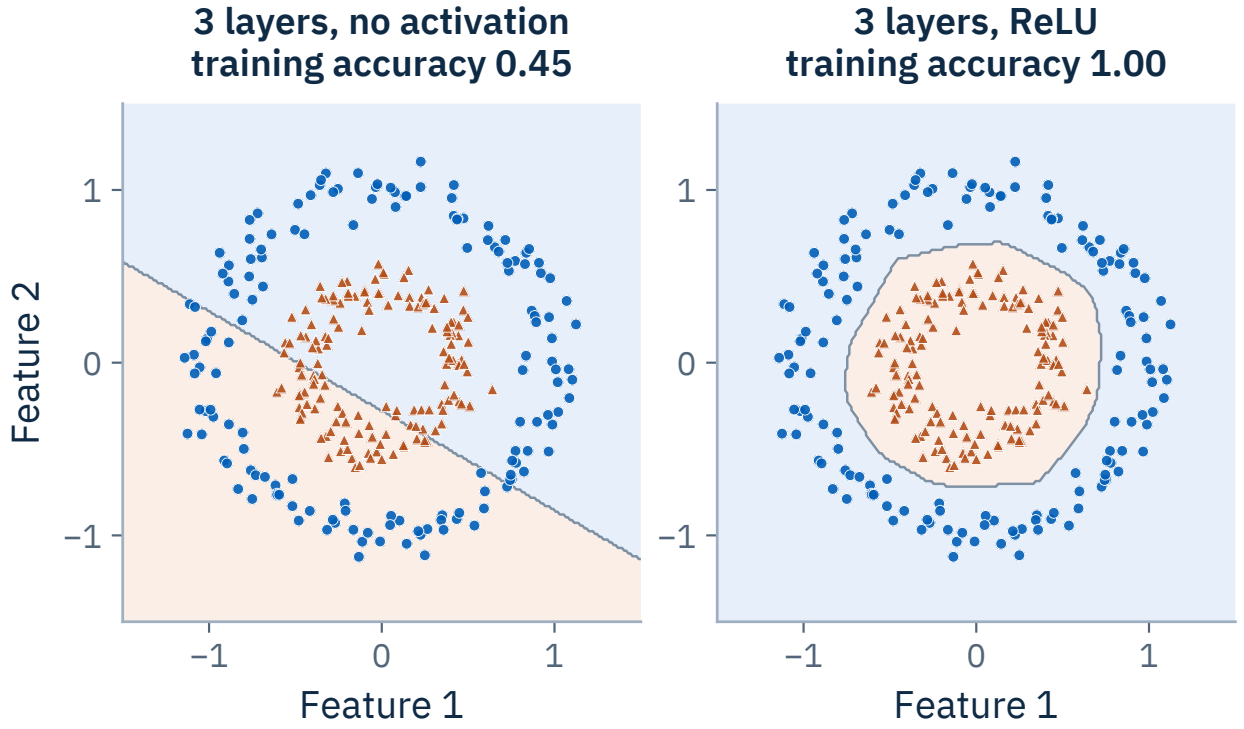
row *j* of *W* holds the weights of unit *j*

Exact arithmetic: z = (2.5, 1.0, −0.3), so a = (2.5, 1.0, 0).

Stacking linear layers is still linear

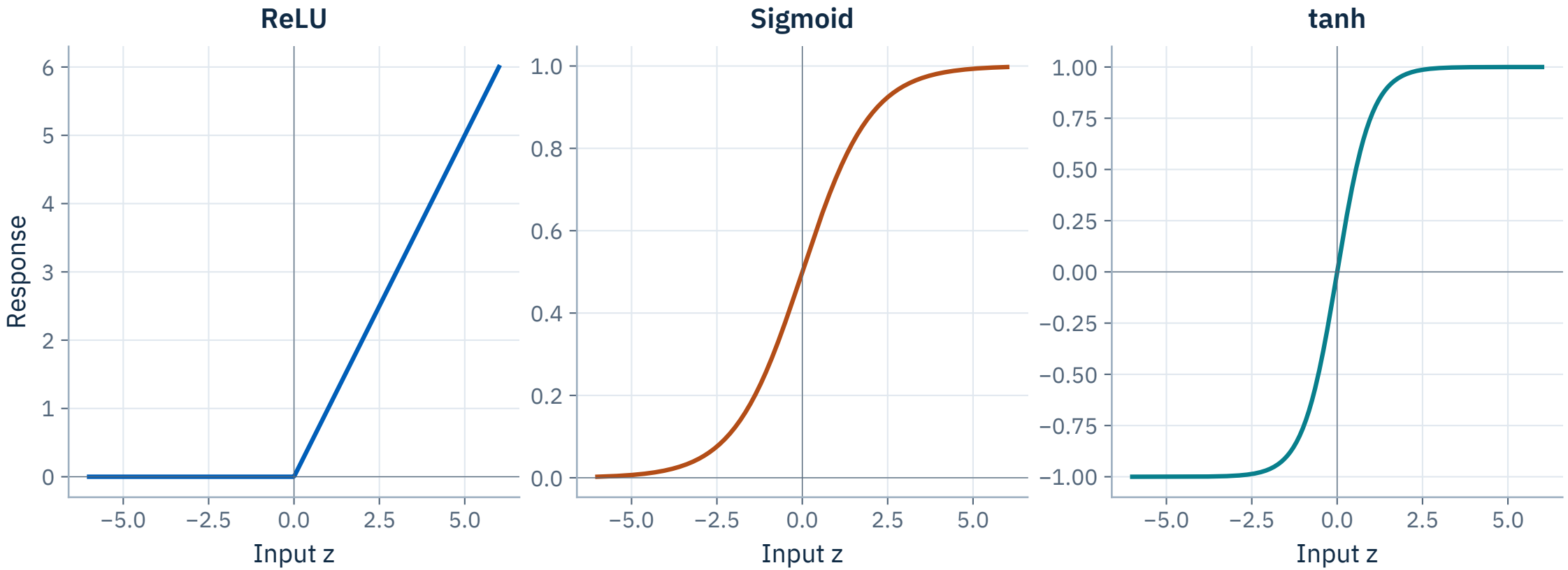
$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

- Without activations, any stack of layers collapses into one.
- Nonlinear activations let hidden units bend the boundary.
- Same data, same three layers: only the activation differs.



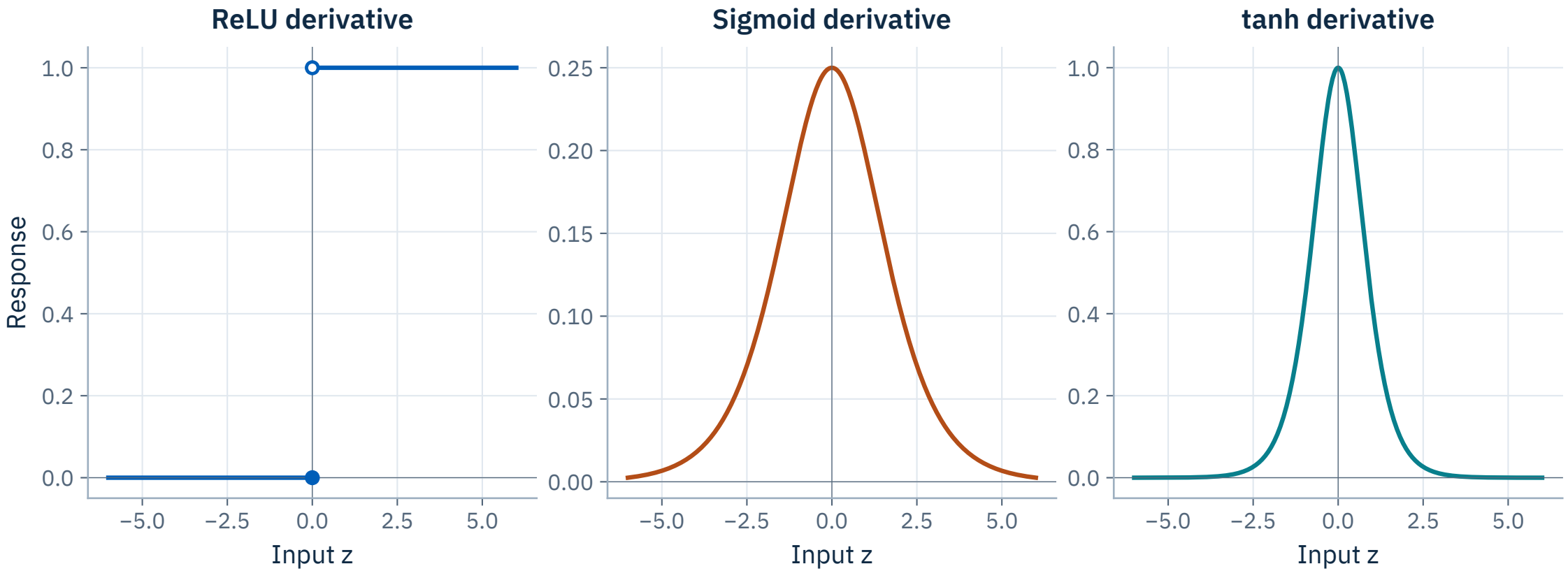
Computed: synthetic circles (seed 0); scikit-learn MLPClassifier with three hidden layers of 16 units.

Activation functions respond differently



Exact activation functions over a common input range. ReLU, sigmoid, and tanh have different ranges and saturation behaviour.

Derivatives explain saturation

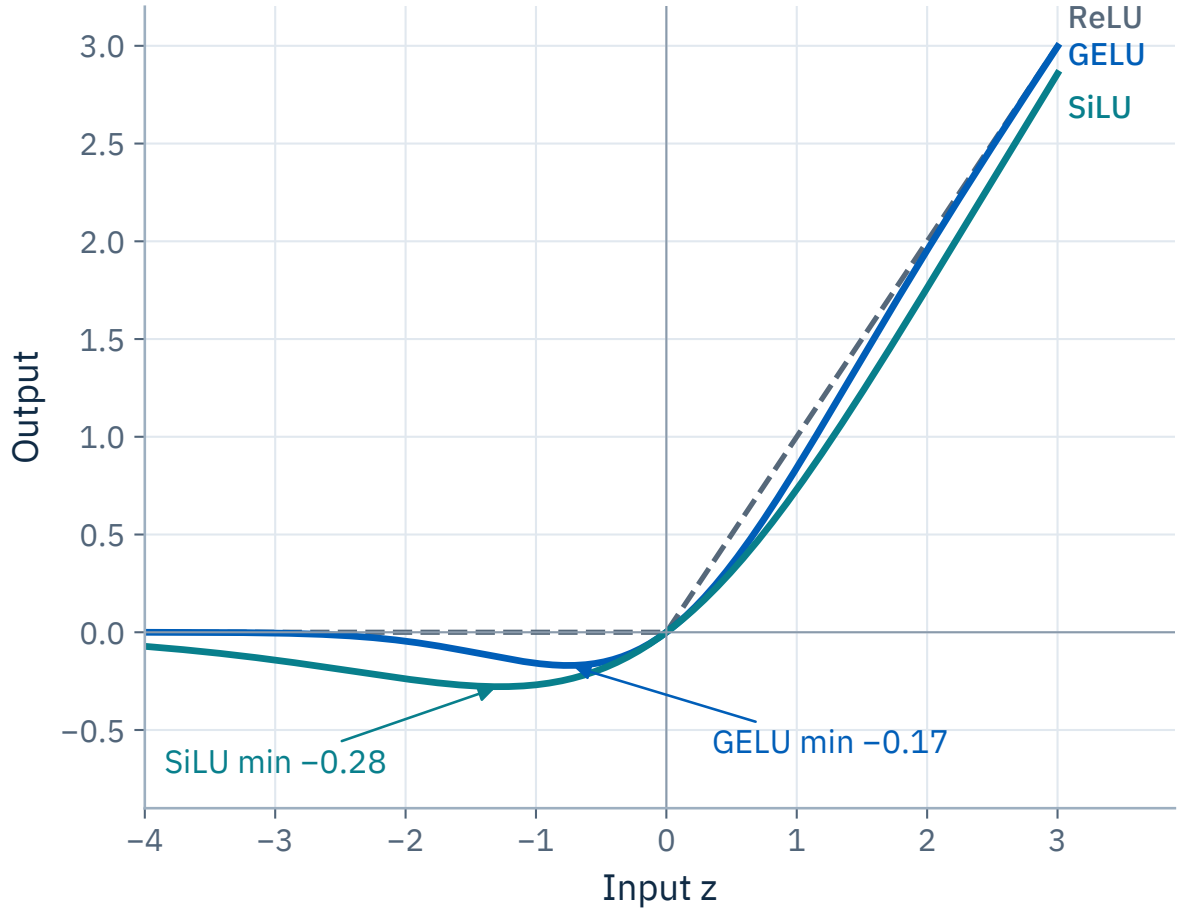


Analytical derivatives. ReLU's derivative at zero is undefined mathematically; software selects a convention.

GELU and SiLU provide smooth alternatives

$$\mathrm{GELU}(z) = z\,\Phi(z), \qquad \mathrm{SiLU}(z) = z\,\sigma(z)$$

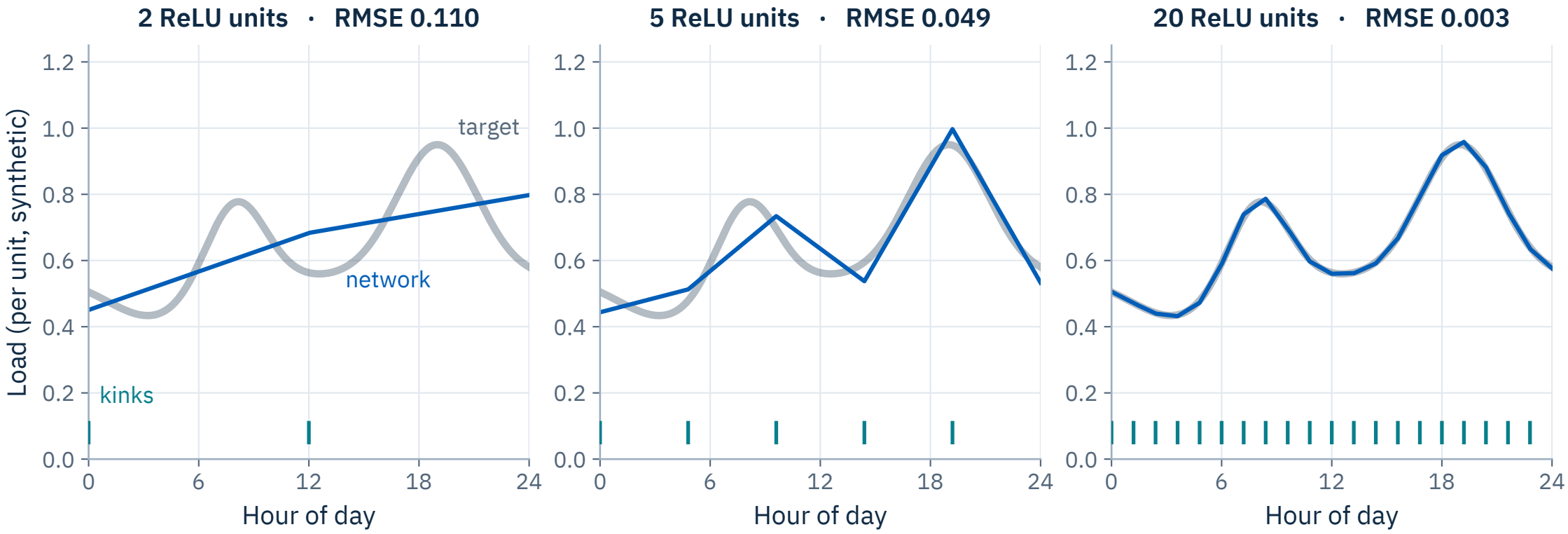
- GELU weights z by the standard-normal CDF Φ(z).
- SiLU, also called Swish, gates z with a sigmoid.
- Both are smooth and dip slightly below zero.
- Choose within a tested architecture, not in isolation.



Exact functions. Minima: GELU −0.17, SiLU −0.28.

Enough ReLU units can approximate a smooth curve

Each ReLU unit adds one kink; the output layer weights and sums them.



Computed: synthetic daily-load-shaped curve, not measured data. One hidden layer, evenly spaced kinks, output weights by least squares.

Universal approximation promises existence, not success

✓ What the theorem says

- One hidden layer with enough units
- Approximates any continuous function on a bounded domain
- Cybenko (1989); Hornik (1991)

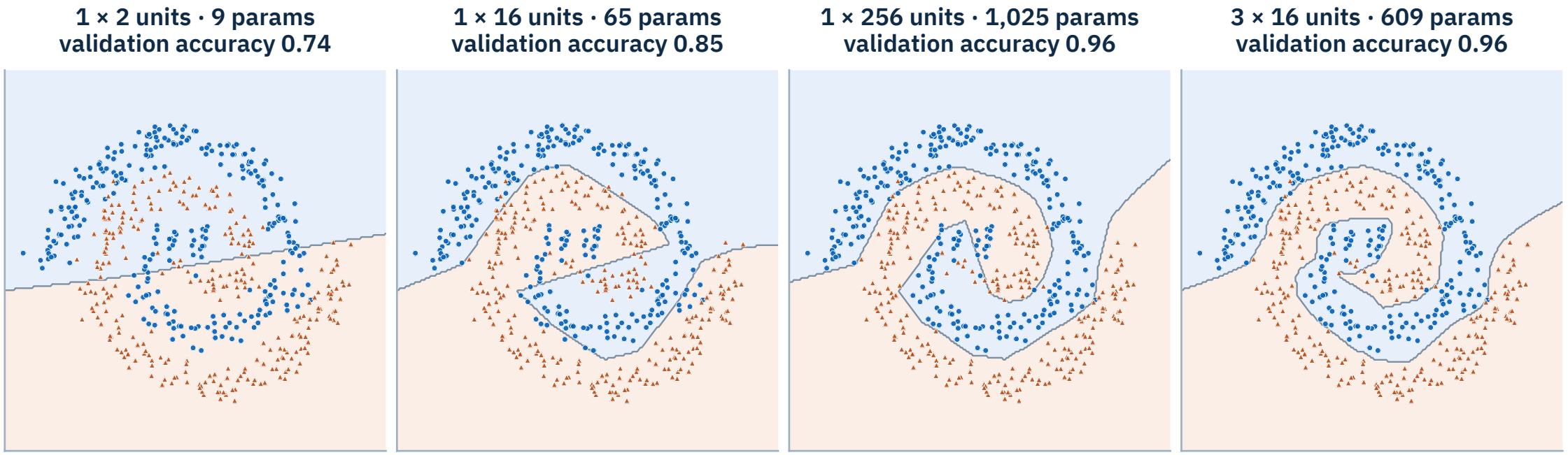
vs

✗ What it does not say

- How many units you will need
- Whether gradient descent will find the weights
- Whether the fit generalizes beyond the data

◆ Whether a network can represent your target is rarely the question; data, training, and validation are.

Width and depth both add capacity to the boundary



Computed: synthetic two spirals (seed 0, 70/30 split); scikit-learn MLPClassifier defaults (ReLU, Adam), random_state 0. Dots: training points; accuracy on the 30% validation points.

PRACTICE

3 min · Individual

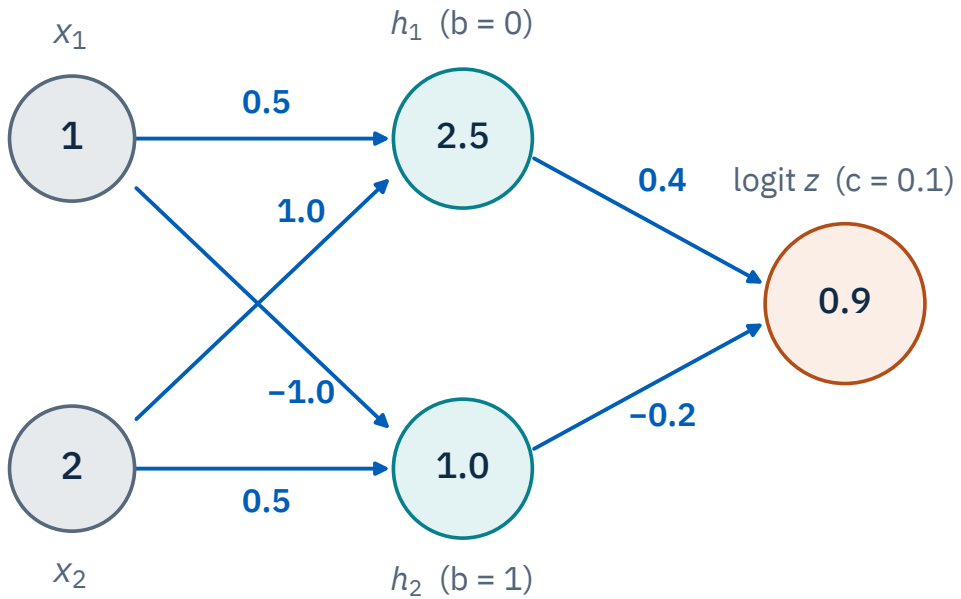
Dimensions determine parameter count

A dense network maps 6 inputs to 8 hidden units, then to 1 output.

1. Count the weights and biases in each layer.
2. State the output shape for a batch of 32 observations.
3. How does doubling the batch size change the parameter count?

A two-hidden-unit forward pass by hand

- $h_1 = \text{ReLU}(0.5 \cdot 1 + 1.0 \cdot 2 + 0) = 2.5$
- $h_2 = \text{ReLU}(-1.0 \cdot 1 + 0.5 \cdot 2 + 1) = 1.0$
- $z = 0.4 \cdot 2.5 - 0.2 \cdot 1.0 + 0.1 = 0.9$
- $p = \sigma(0.9) \approx 0.711$



$p = \sigma(0.9) = 0.711$ · ReLU keeps both hidden values

◆ Write these numbers down: Part 6 sends gradients back through this exact network.

Exact arithmetic. The weights are chosen for clear arithmetic, not fitted to data.

PRACTICE

4 min · Individual

Run the same network on a new input

Use the weights from the previous slide with $x = (3, 1)$.

1. Compute h_1, h_2, the logit z, and p.
2. Which hidden unit is inactive, and why?
3. Which weights receive zero gradient from this example?

PART 5 OF 7

05

Losses and gradient descent

The loss scores predictions; its gradient says how to improve.

- Output layers and matching losses
- Regression losses: MSE, MAE, and Huber
- Cross-entropy and softmax
- Gradient descent, batches, and the learning rate

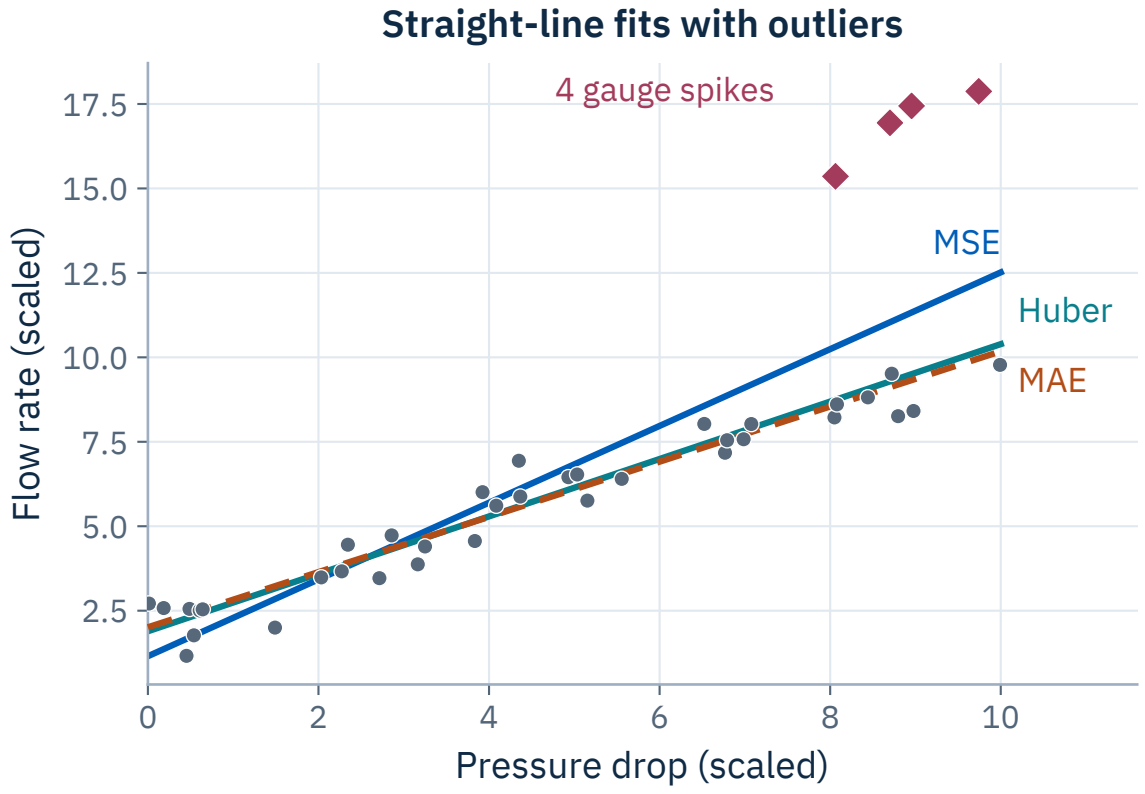
Output and loss must agree

Task	Model output	Training target and loss
Regression	Unconstrained numeric output	Numeric target, MSE or Huber
Binary classification	One raw logit per example	Float 0/1, BCEWithLogitsLoss
Multiclass classification	C raw logits per example	Integer class index, CrossEntropyLoss
Multilabel classification	One logit per independent label	Multiple 0/1 targets, binary losses

- Do not apply sigmoid or softmax before a loss that expects raw logits.

MSE, MAE, and Huber treat large errors differently





Left: exact loss functions. Right: synthetic data (seed 5), true slope 0.8, with 4 added spikes. Fitted slopes: MSE 1.14, MAE 0.82, Huber 0.85.

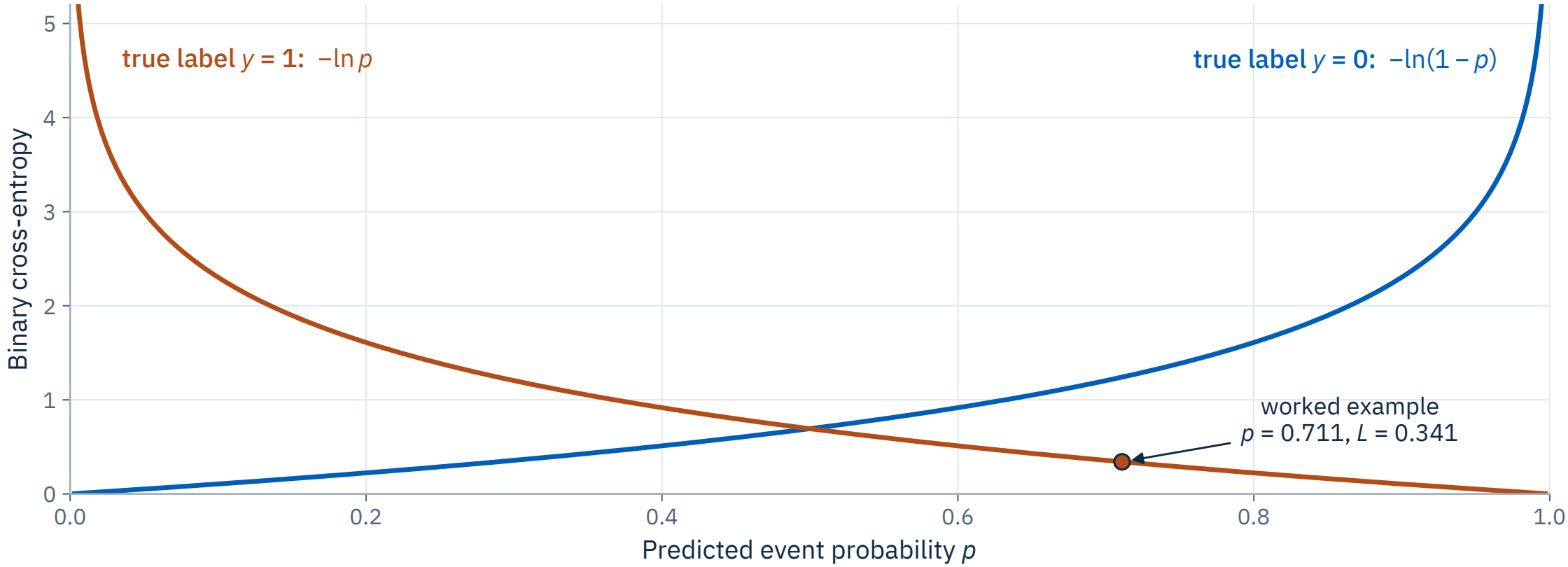
Binary cross-entropy rewards calibrated confidence

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

- For y = 1, a low predicted probability incurs a high loss.
- For y = 0, a high predicted probability incurs a high loss.
- Stable implementations combine this with the sigmoid on raw logits.

◆ Worked example: y = 1 and p = 0.711 give L = −ln 0.711 ≈ 0.341.

Confident wrong answers cost the most



Exact binary cross-entropy for each possible label; the marked point is the worked example. Confident wrong probabilities incur much larger loss.

Softmax couples mutually exclusive classes

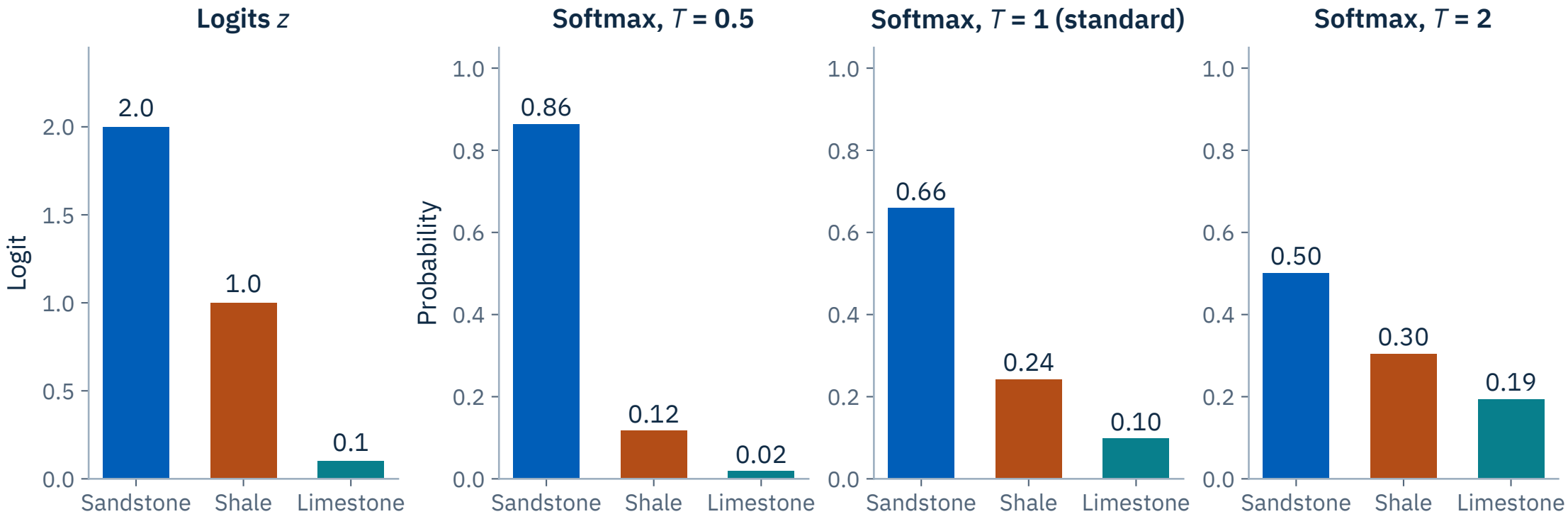
$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad L = -\log p_y$$

- The probabilities are positive and sum to one across classes.
- The target index *y* selects the correct class's probability.
- Adding the same constant to every logit changes nothing.
- `CrossEntropyLoss` applies the softmax internally, stably.

◆ Libraries subtract the largest logit before exponentiating; the probabilities are unchanged.

Temperature sharpens or flattens the softmax

Divide the logits by T before the softmax: $T < 1$ sharpens, $T > 1$ flattens, and the ranking never changes.



Exact softmax of illustrative facies logits (2.0, 1.0, 0.1) at three temperatures.

PRACTICE

4 min · Pairs

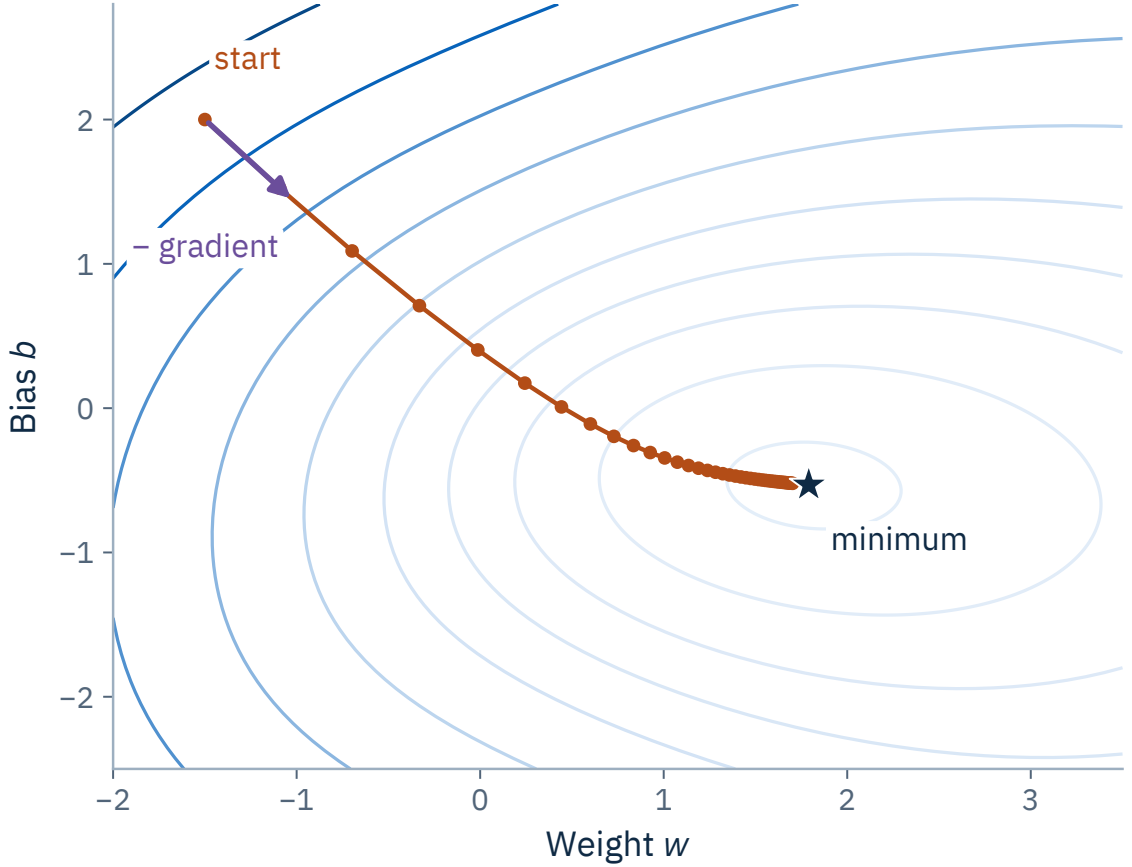
Compute a softmax and its loss

1. Logits (0, 0, 0): what are the probabilities, and the loss for any target?
2. Logits (2.0, 1.0, 0.1) with target Shale (index 1): what is the loss?
3. Add 5 to every logit: what changes?

Gradient descent steps against the gradient

$$w_{t+1} = w_t - \eta \nabla_w L(w_t)$$

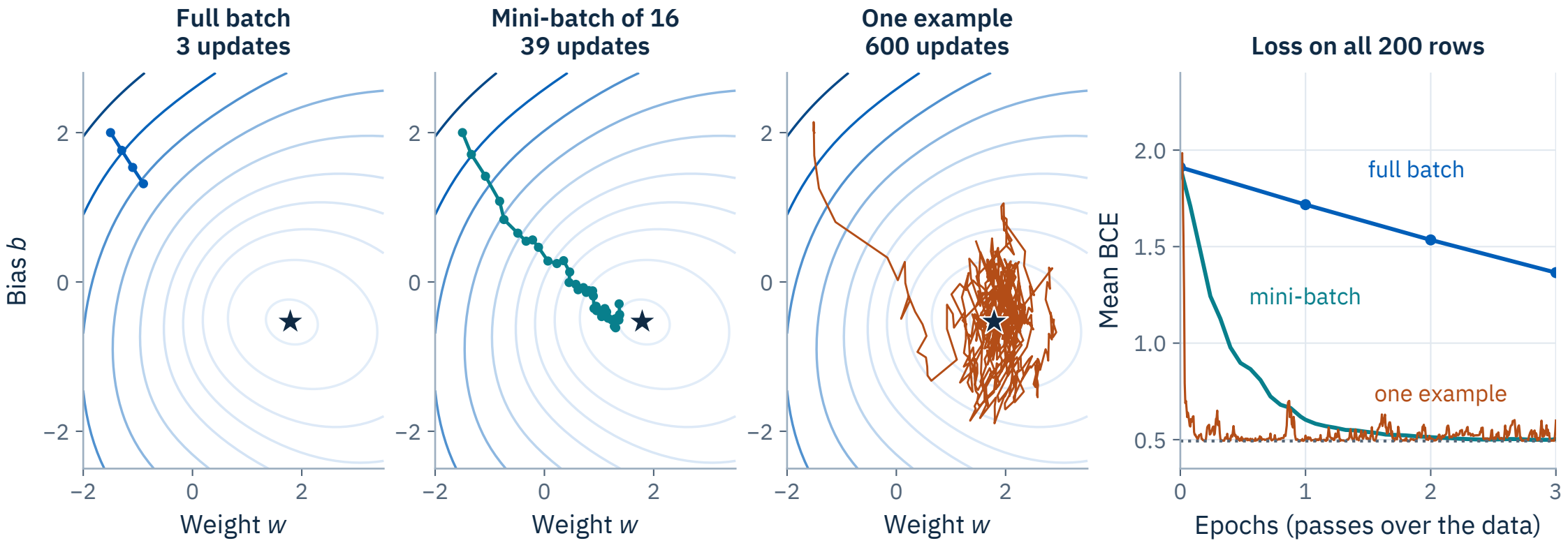
- The gradient points uphill; step the other way.
- The learning rate η sets the step size.
- Steps shrink as the surface flattens near the minimum.



Computed: loss surface of a one-feature logistic model on synthetic data (seed 7); 40 full-batch steps, learning rate 1.0.

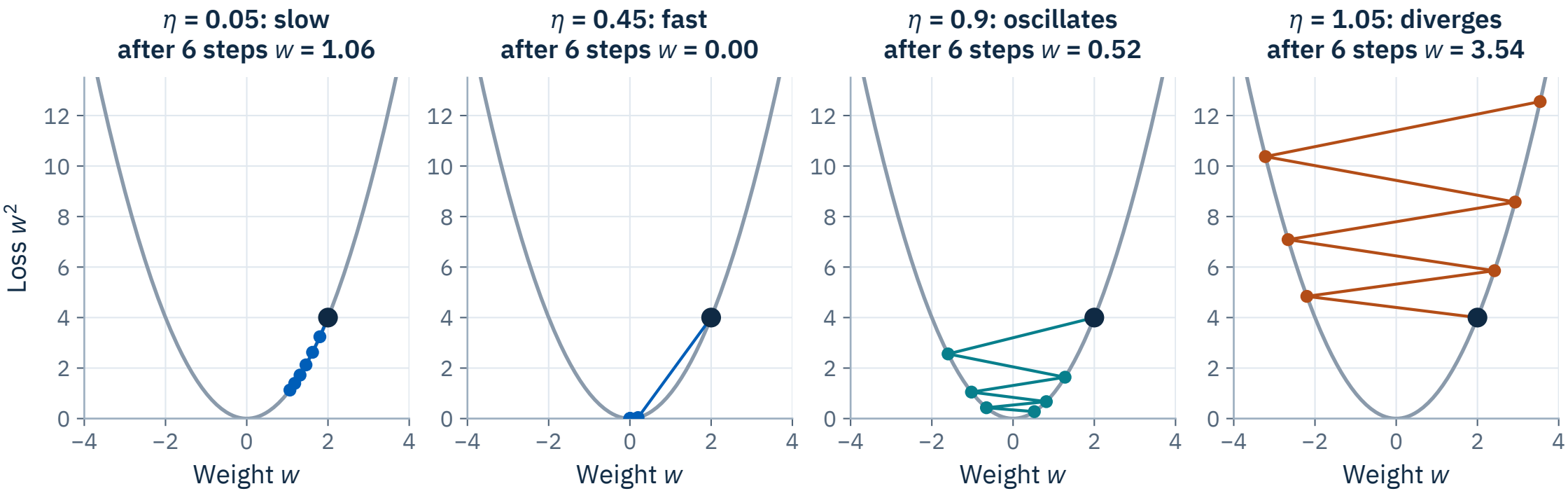
Batch size trades gradient noise for update count

Same data, start, learning rate, and 3 epochs: smaller batches make more, noisier updates.



Computed on the synthetic logistic problem: full batch (3 updates), mini-batches of 16 (39), single examples (600). Dotted line: minimum loss.

The learning rate decides whether descent converges



Exact gradient descent on L(w) = w² from w = 2; each step multiplies w by (1 − 2η). Schedules and adaptive optimizers follow on Day 2.

PRACTICE

4 min · Pairs

Match each output layer to its loss

For a batch of 64, give the output shape and the PyTorch loss for each task.

1. Daily gas rate of a well
2. Gas lock: yes or no
3. Facies: one of five classes
4. Five alarms that can fire together

PART 6 OF 7

06

Backpropagation

The chain rule, run in reverse, gives every gradient at once.

- The chain rule on a computational graph
- A full backward pass on the worked network
- Reverse mode versus finite differences
- Checking gradients numerically

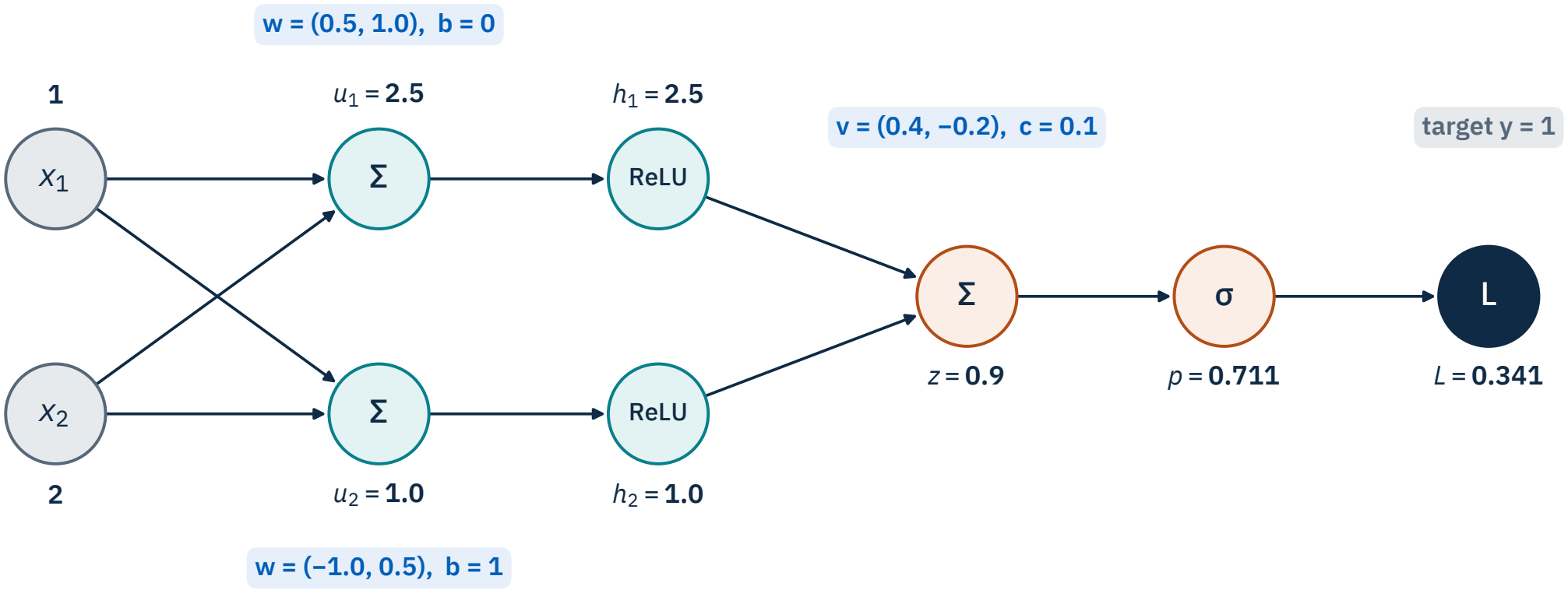
Backpropagation applies the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\,\frac{\partial z}{\partial w}$$

- Forward pass: compute and store every intermediate value.
- Backward pass: propagate sensitivities from the loss to the parameters.
- An optimizer then uses the gradients to update the parameters.

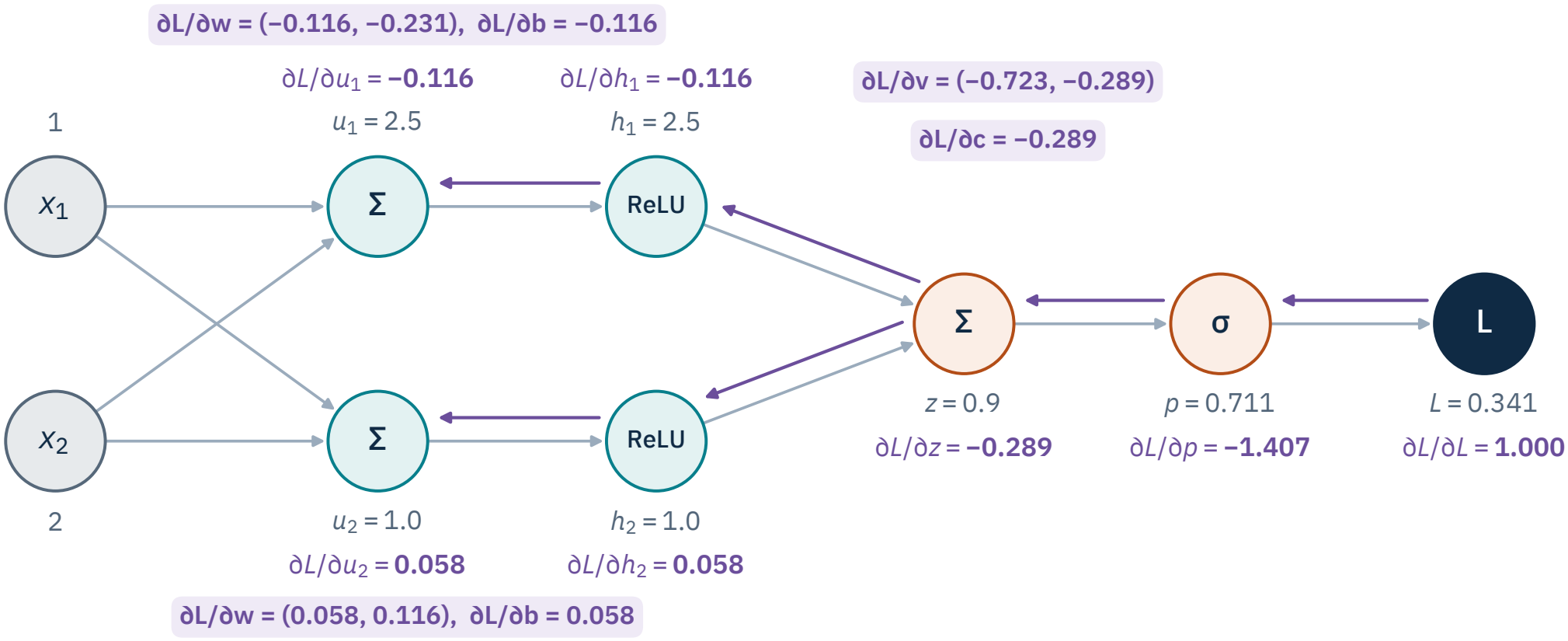
◆ For $z = v \cdot h + c$, the local derivative $\partial z / \partial v_1$ is simply h_1, a value stored by the forward pass.

A computational graph records the forward pass



Exact values for the worked network with x = (1, 2) and target y = 1. Σ: weighted sum; σ: sigmoid; L: binary cross-entropy.

Gradients flow backward through the same graph



Exact reverse-mode gradients. Each purple value is the upstream gradient times the node's local derivative.

Backpropagation is reverse-mode differentiation

✗ Finite differences

- Perturb one parameter, rerun the forward pass
- Two passes per parameter: millions of passes
- Approximate and sensitive to the step size

vs

✓ Reverse mode

- One forward pass and one backward pass
- All gradients for a small multiple of the forward cost
- Exact up to floating-point rounding

◆ PyTorch's autograd records the graph in the forward pass and runs reverse mode on `loss.backward()`.

One backward calculation updates one weight

$$\frac{\partial L}{\partial z} = p - y$$

- In the worked example, p = 0.711 and y = 1, so ∂L/∂z = −0.289.
- The output weight on h_1 has gradient −0.289 × 2.5 = −0.723.
- A learning rate of 0.1 moves that weight from 0.4 to about 0.472.

◆ Sigmoid with cross-entropy, like softmax with cross-entropy, gives "prediction minus target".

PRACTICE

5 min · Pairs

Push the gradient one layer further

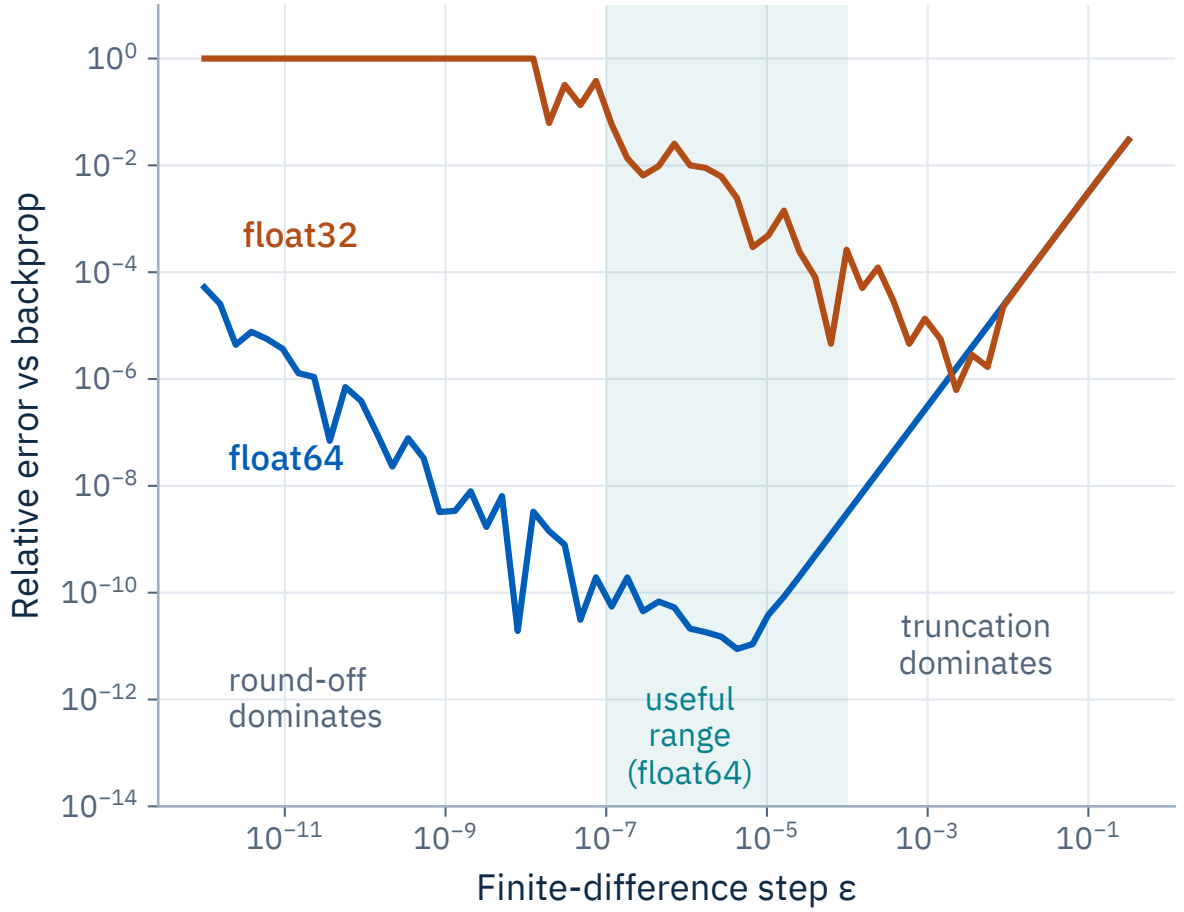
Worked network, $x = (1, 2)$, $y = 1$, and $p - y = -0.289$.

1. Find ∂L/∂w for the weight from x_2 to h_1 (value 1.0).
2. Update it with learning rate 0.1.
3. Repeat both steps for the weight from x_2 to h_2 (value 0.5).

A numerical gradient check catches silent errors

$$g_{\text{num}} = \frac{L(w+\epsilon) - L(w-\epsilon)}{2\epsilon}$$

- Compare with backpropagation using a relative error.
- Use float64 and ε around 10^{-5} to 10^{-6}.
- Too large ε: truncation error; too small: round-off.
- Avoid kinks, such as ReLU exactly at zero.



Computed for ∂L/∂v₁ of the worked network: relative error against the step ε.

A gradient check for the Lab 2 network

```python
def grad_check(loss_fn, params, grads, key, idx, eps=1e-5):
    old = params[key][idx]
    params[key][idx] = old + eps
    up = loss_fn(params)
    params[key][idx] = old - eps
    down = loss_fn(params)
    params[key][idx] = old                          # restore the weight
    num, ana = (up - down) / (2 * eps), grads["d" + key][idx]
    return abs(num - ana) / max(abs(num), abs(ana), 1e-12)

loss_fn = lambda p: binary_cross_entropy(y_train, forward(X_train, p)["a2"])
grads = backward(X_train, y_train, params, forward(X_train, params))
for key, idx in [("W1", (0, 0)), ("b1", (0, 3)), ("W2", (5, 0))]:
    print(key, idx, f"{grad_check(loss_fn, params, grads, key, idx):.1e}")
```

OUTPUT

```
W1 (0, 0) 3.8e-10
b1 (0, 3) 1.1e-08
W2 (5, 0) 2.5e-11
```

PART 7 OF 7

07

Training networks in code

Repeat forward, loss, backward, and step until validation says stop.

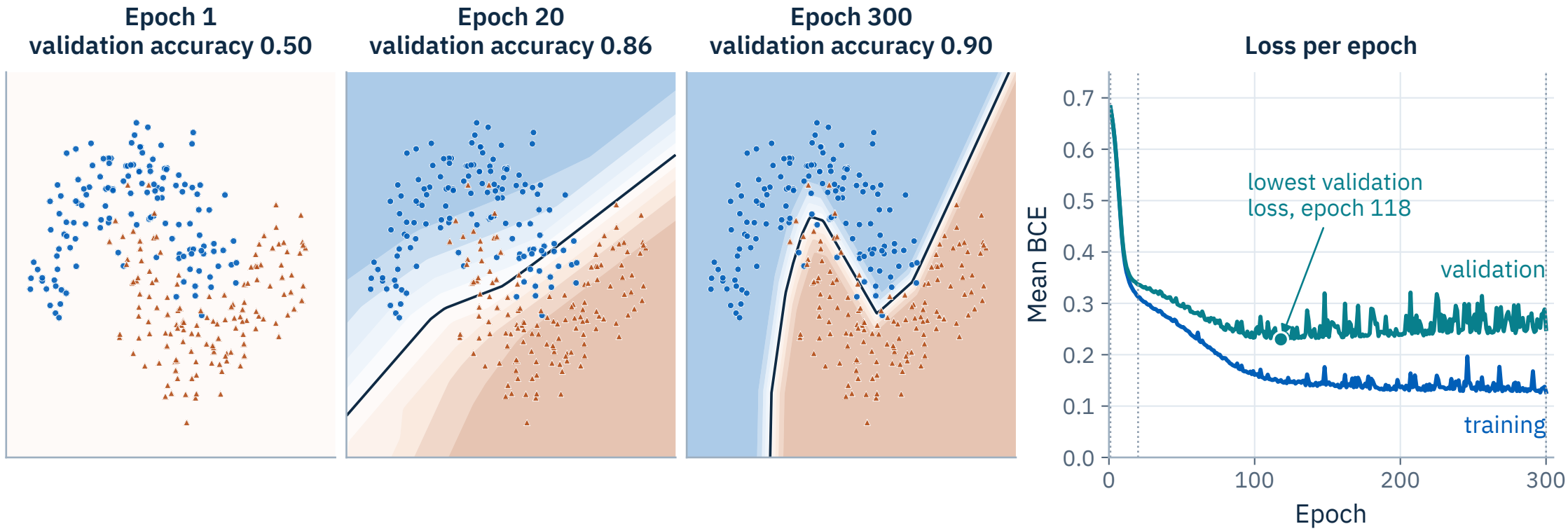
- The training loop and what each line does
- Watching the boundary and the losses evolve
- A first network in scikit-learn and in PyTorch
- Shape checks that catch first-day bugs

The training loop has four responsibilities

```
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()        # 1 clear
        logits = model(X_batch)      # 2 forward
        loss = criterion(logits, y_batch)
        loss.backward()              # 3 gradients
        optimizer.step()             # 4 update
    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val), y_val)
```

- Clear old gradients: PyTorch accumulates them.
- Forward pass and loss on one mini-batch.
- `backward()` fills every parameter's `.grad`.
- `step()` applies the update rule.
- Validate without building a graph.

The decision boundary takes shape during training



Computed: synthetic moons (seed 1); PyTorch 2–16–16–1 ReLU network, SGD with learning rate 0.1 and batch 32. Dotted lines: the three snapshot epochs. Validation loss is lowest at epoch 118.

Training loss is not the final score

- Monitor a separate validation set during development.
- Compare with the same baseline under the same split.
- Inspect learning curves and errors, not only the last epoch.
- Evaluate the chosen model once on the final test data.

◆ A model selected by looking at the test set has no unbiased score left.

A first network in scikit-learn

```
from sklearn.neural_network import MLPClassifier

mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(32,), alpha=1e-3,
                  max_iter=2000, random_state=0),
)
scores = cross_val_score(mlp, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.843 +/- 0.052
```

Same folds as the logistic baseline (0.753 ± 0.061): compare fold by fold before celebrating.

A first network in PyTorch

```python
import torch
from torch import nn

class SonarNet(nn.Module):
    def __init__(self, n_in=60, n_hidden=32):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_in, n_hidden), nn.ReLU(),
            nn.Linear(n_hidden, 1))              # one raw logit

    def forward(self, x):                        # x: (B, 60) float32
        return self.layers(x)                    # (B, 1) logits

model = SonarNet()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
1985
```

Autograd reproduces the gradients computed by hand

```python
import torch

x = torch.tensor([1.0, 2.0])
W1 = torch.tensor([[0.5, 1.0], [-1.0, 0.5]], requires_grad=True)
b1 = torch.tensor([0.0, 1.0], requires_grad=True)
v = torch.tensor([0.4, -0.2], requires_grad=True)
c = torch.tensor(0.1, requires_grad=True)
h = torch.relu(W1 @ x + b1)                    # (2.5, 1.0)
z = v @ h + c                                  # logit 0.9
loss = torch.nn.functional.binary_cross_entropy_with_logits(
    z, torch.tensor(1.0))
loss.backward()
print(round(loss.item(), 4), v.grad, W1.grad, sep="\n")
```

OUTPUT

```
0.3412
tensor([-0.7226, -0.2891])
tensor([[-0.1156, -0.2312],
        [ 0.0578,  0.1156]])
```

Shape checks turn silent bugs into loud ones

```
X_t = torch.as_tensor(X_train, dtype=torch.float32)   # (166, 60)
y_t = torch.as_tensor(y_train, dtype=torch.float32)   # (166,)
criterion = nn.BCEWithLogitsLoss()
logits = model(X_t)                                   # (166, 1)
assert X_t.shape[1] == 60 and torch.isfinite(X_t).all()
print(logits.shape, y_t.shape)
loss = criterion(logits, y_t)          # raises: the shapes differ
```

OUTPUT

```
torch.Size([166, 1]) torch.Size([166])
ValueError: Target size (torch.Size([166])) must be the same as
input size (torch.Size([166, 1]))
```

Fix with `y_t = y_t.reshape(-1, 1)`. `MSELoss` would only warn, then broadcast to (166, 166).

Choose scikit-learn for speed, PyTorch for control

Aspect	scikit-learn MLPClassifier	PyTorch
Architectures	Dense layers only	Any differentiable model
Training loop	Built into `fit`	You write it
Losses and outputs	Fixed by the estimator	You choose and check them
Hardware	CPU	CPU or GPU
Best use	Quick tabular baseline	Custom models, Days 2 to 5

◆ Lab 2 builds the same network a third way, by hand in NumPy, to see every step.

LABORATORY

45 min

Lab 2: a NumPy network you can explain

- ☐ Implement sigmoid, initialization, forward pass, BCE, and backward pass.
- ☐ Assert that every gradient has the shape of its parameter.
- ☐ Add the gradient check and report three relative errors.
- ☐ Train, plot the loss, and evaluate once on the test rows.
- ☐ Compare with the Lab 1 baseline; sections 12–13 are extensions.

NOTEBOOK

`exercises/02_nn_from_scratch/NN_From_Scratch_Student.ipynb`

EVIDENCE TO SUBMIT

- Shapes of W1, b1, W2, b2 and of their gradients
- Gradient-check relative errors in float64
- Loss curve, and train and test accuracy against the baseline

DISCUSSION

Lab 2 debrief: the loss fell, the test score is uncertain

Solution settings: 32 sigmoid units, learning rate 0.01, 1000 full-batch epochs, seed 42.

0.689 → 0.448

training loss, first to last epoch

0.843

training accuracy, 140 of 166

0.833

test accuracy, 35 of 42

2.4 points

the weight of one test echo

◆ Section 12 compares settings on validation rows; the test score belongs to the fixed settings.

DISCUSSION

Lab 2 debrief: what the small dataset cannot establish

Model ranking

- Test gaps are two echoes at most
- Rank models on the same CV folds
- Report spread, not only means

Chosen settings

- Fixed before the single test
- Compared on validation rows only
- A re-tuned model needs a fresh test

Deployment claims

- One lab dataset, no acquisition metadata
- No evidence about new sites or sensors
- Treat results as a teaching benchmark

Key takeaways

- ✓ A network composes weighted sums and nonlinearities; shapes are its contract.
- ✓ One line cannot separate XOR; hidden units re-map inputs until one can.
- ✓ Choose the output activation, target encoding, and loss together.
- ✓ Backpropagation is the chain rule in reverse; check it numerically once.
- ✓ Training loss falls by design; only held-out data show generalization.
- ✓ A network must beat a baseline on the same split to earn its place.

LOOKING AHEAD

Tomorrow: training neural networks reliably

Diagnose

- Initial loss, one-batch fit, learning curves

Initialize and normalize

- Keep signals and gradients at a trainable scale

A loop you can trust

- Checkpoints, seeds, logs, and the Keras equivalent

Optimize

- Momentum, AdamW, schedules, batch size

Regularize

- Weight decay, dropout, early stopping, augmentation

Autoencoders

- Reconstruction error as an anomaly score

◆ Every Day 2 tool plugs into today's loop: forward, loss, backward, step.

PRACTICE

5 min | Individual

Exit check

1. Why do dense layers need nonlinear activations?
2. Why must the raw-logit loss and the output activation agree?
3. Which shape error can pass silently, and how do you catch it?
4. How can a gradient check reveal a silent implementation error?

References and further study

- Goodfellow, Bengio, Courville (2016). *Deep Learning*. MIT Press. Chapters 6 and 8.
- Rumelhart, Hinton, Williams (1986). Learning representations by back-propagating errors. *Nature* 323.
- Rosenblatt (1958). The perceptron. *Psychological Review* 65.
- Minsky, Papert (1969). *Perceptrons*. MIT Press.
- Cybenko (1989); Hornik (1991): universal approximation theorems.
- PyTorch tutorials: tensors, autograd, and the training loop. scikit-learn User Guide: neural network models.
- UCI Machine Learning Repository: Connectionist Bench (Sonar, Mines vs. Rocks).

NEURAL NETWORKS / DAY 2

Training neural networks reliably

Diagnostics, initialization, normalization, optimization, regularization, and framework practice

Full day · 09:00–17:00

5 parts · 2 laboratories

PyTorch core · Keras alternative

Today's plan

09:00	Welcome, then Part 1: diagnose before you tune	Lecture
09:50	Part 2: signal propagation and normalization	Lecture
10:35	Break	Break
10:50	Part 3: a PyTorch training loop you can trust	Lecture
11:30	Lab 1: a reproducible PyTorch pipeline	Lab
12:30	Lunch	Lunch
13:30	Lab 1 debrief, then Part 4: optimizers, schedules, and tuning	Lecture
14:35	Break	Break
14:50	Part 5: regularization and autoencoders	Lecture
15:35	Lab 2, debrief, and exit check	Lab

LEARNING OUTCOMES

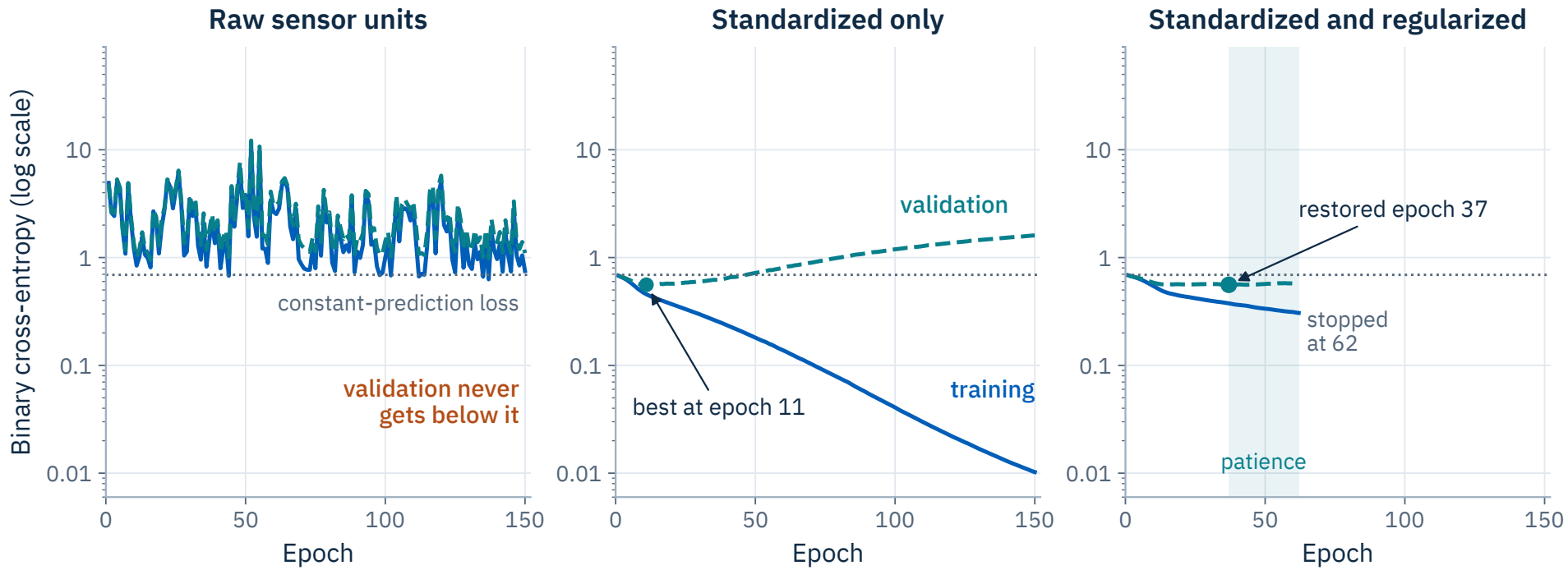
What you will be able to do

By the end of the day you will be able to:

1. Run a fixed diagnostic sequence: initial loss, one-batch fit, gradient flow, learning curves.
2. Explain how initialization, normalization, and skip connections keep signals at a trainable scale.
3. Write a PyTorch loop with correct train and eval modes, a copied checkpoint, and a reload check.
4. Choose an optimizer, learning rate, schedule, and batch size from evidence, not habit.
5. Match weight decay, dropout, early stopping, and augmentation to the symptom they treat.
6. Explain how an autoencoder turns reconstruction error into an anomaly score, and its limits.

Same network, same data, three different outcomes

A 12-input MLP learns a pump-failure flag from a synthetic ESP table with 240 training rows. Only the training setup changes.



Computed: synthetic ESP-style data, seeds 1 and 2 (raw pressures in kPa next to vibration in g), MLP 12-64-64-1, AdamW, torch seed 0; the dotted line is the loss of always predicting the training base rate.

PRACTICE

5 min · Pairs

Warm-up: which run would you trust, and why?

Look at the three runs on the previous slide.

1. Which run would you hand to an operations team, and what evidence would you ask for first?
2. The middle run reaches a training loss of 0.01. What does that number prove?
3. Name the first setting you would check in the left-hand run.

PART 1 OF 5

01

Diagnose before you tune

Most failed runs are pipeline bugs, not architecture problems.

- A fixed order of checks
- Initial loss and one-batch memorization
- Gradient flow, layer by layer
- Reading learning curves

A diagnostic sequence saves time

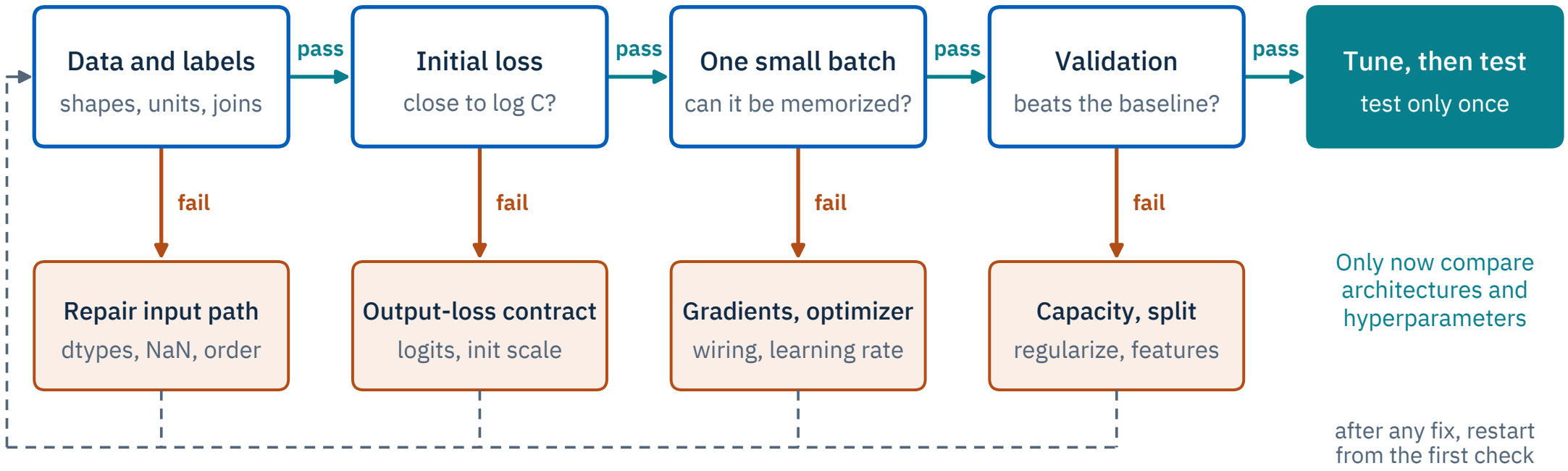
Check	Expected evidence	If it fails
Data and labels	Correct shapes, units, and alignment	Repair the input path
One forward pass	Finite outputs and plausible loss	Inspect output-loss contract
Tiny batch fit	Training loss falls substantially	Inspect gradients and optimizer
Validation fit	Generalization beyond fitted rows	Review capacity, split, and regularization

Run the checks in this order; each one assumes the previous one passed.

◆ A failure early in the list makes every later result meaningless.

Debug in a fixed order: data, contract, fit, generalize

Each check is cheap; skipping one usually costs hours later.

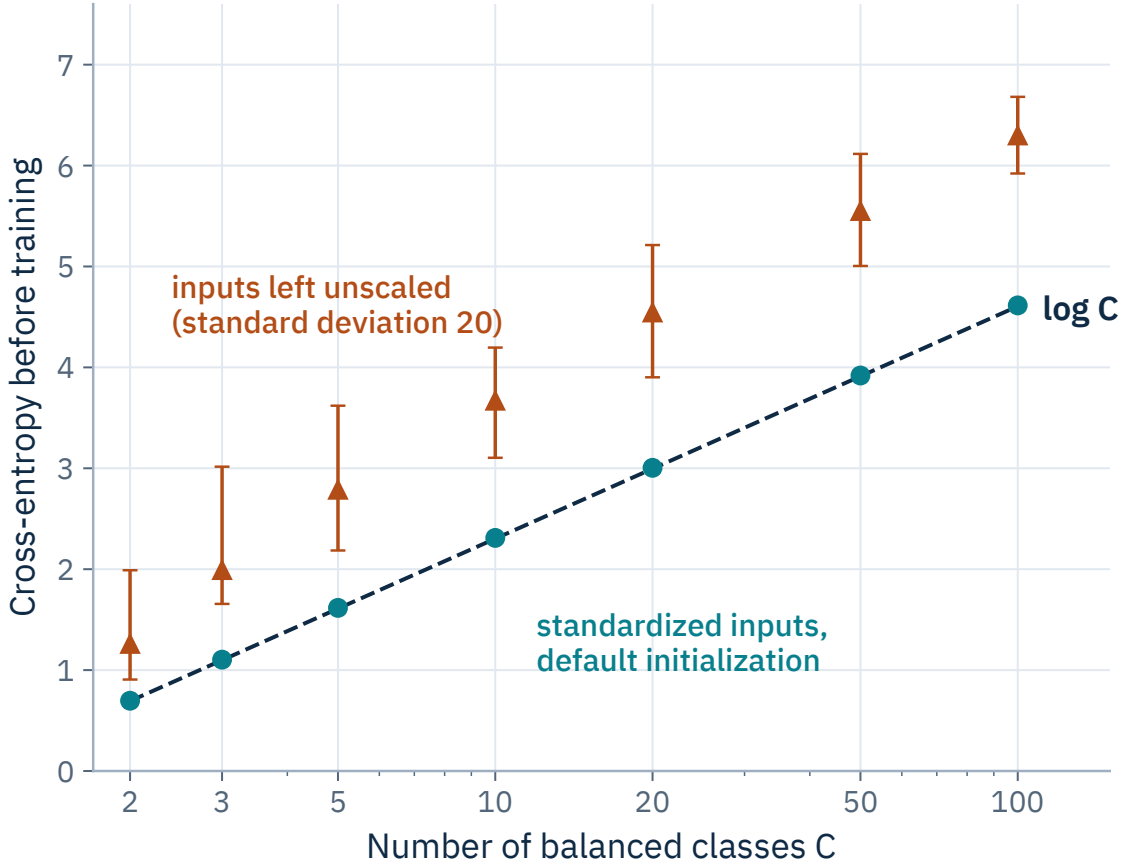


Schematic. A failed check sends you to its repair box; after any fix, restart from the first check.

Initial loss should be close to log C

$$L_0 = -\log \frac{1}{C} = \log C$$

- Untrained logits are near zero: predictions near uniform
- Binary: log 2 = 0.693; ten classes: log 10 = 2.303
- Far higher: logits too large, or a wrong loss contract
- Imbalanced classes: use the entropy of the class prior



Computed: 20 seeds per point, PyTorch default initialization; the orange points use the same inputs multiplied by 20.

Put the sanity checks in code, not in memory

```python
import math

xb, yb = next(iter(train_loader))
assert xb.shape == (16, 60) and yb.shape == (16, 1)
assert torch.isfinite(xb).all()
assert set(yb.unique().tolist()) <= {0.0, 1.0}

model.eval()
with torch.no_grad():
    loss0 = criterion(model(xb), yb).item()
print(f"initial loss {loss0:.3f}   log 2 = {math.log(2):.3f}")
```

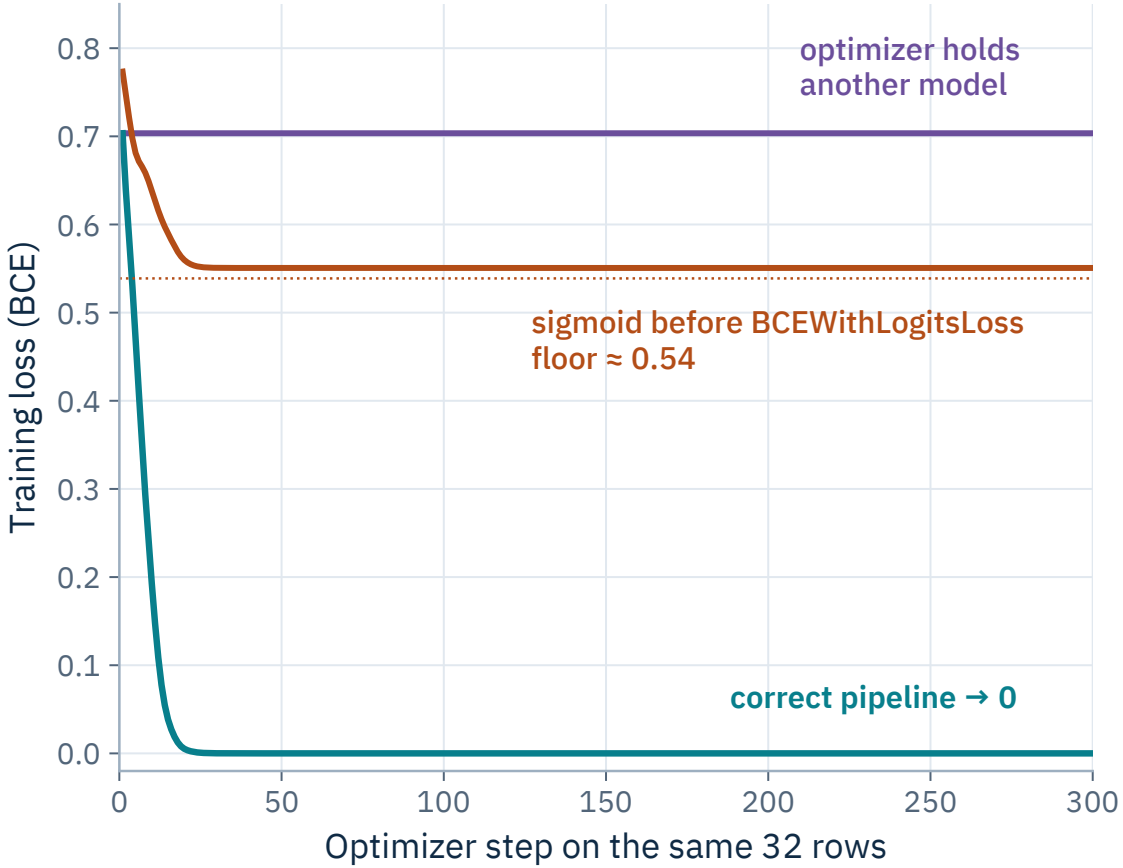
OUTPUT

```
initial loss 0.703   log 2 = 0.693
```

Lab 1 data and untrained model (seed 42); one batch of 16 rows varies by a few hundredths.

A healthy pipeline can memorize one small batch

- Train on the same 32 rows for a few hundred steps
- Switch off dropout and weight decay for this test
- Loss near zero: forward, loss, backward, and step are wired correctly
- A floor or a flat line points to a contract or optimizer bug



◆ Memorizing one batch proves the plumbing, not generalization.

Computed: 32 rows of the synthetic ESP table (seed 1), MLP 12-64-64-1, Adam lr 0.01, torch seed 0; both bugs injected on purpose.

Check that every layer receives a gradient

```
loss = criterion(model(xb), yb)
loss.backward()
for name, p in model.named_parameters():
    g = "None" if p.grad is None else f"{p.grad.norm():.4f}"
    print(f"{name:11s} {str(tuple(p.shape)):9s} {g}")
```

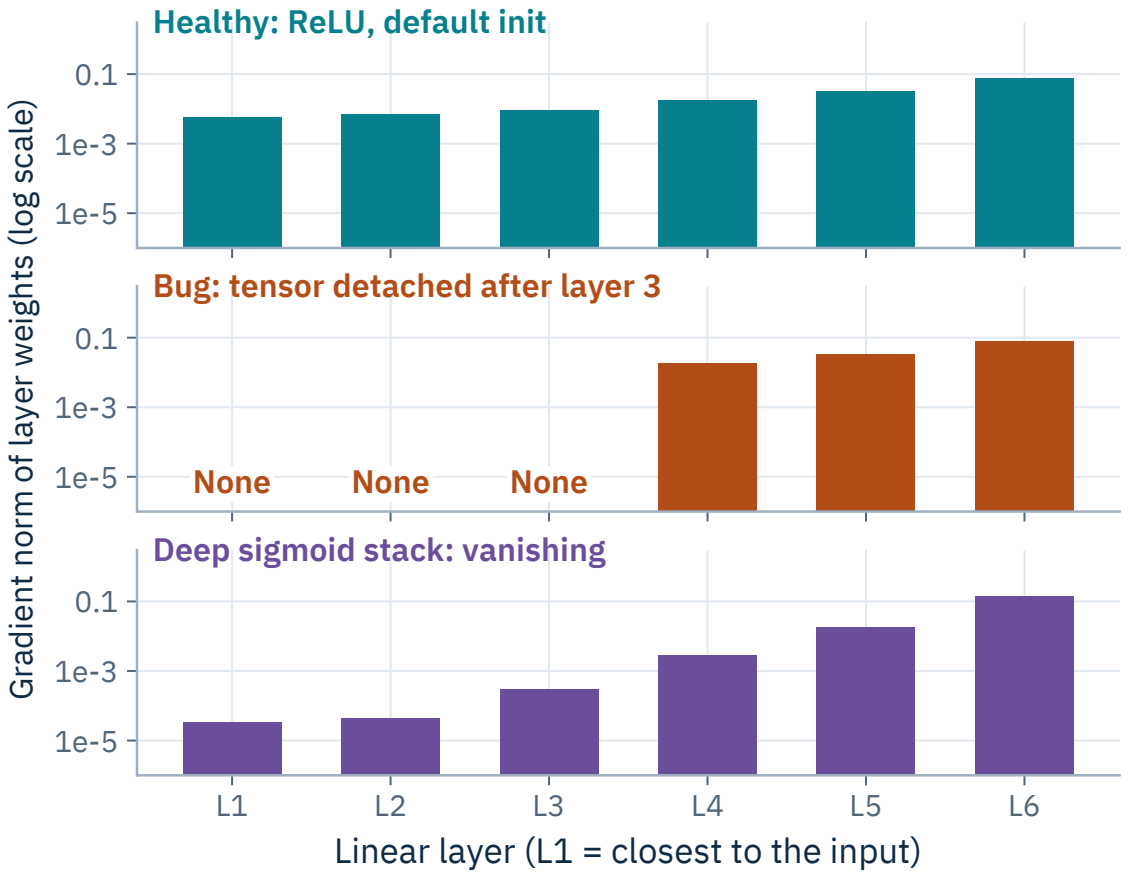
OUTPUT

```
fc1.weight  (32, 60)  0.5140
fc1.bias    (32,)     0.0632
fc2.weight  (32, 32)  None
fc2.bias    (32,)     None
out.weight  (1, 32)   0.3493
out.bias    (1,)      0.1227
```

Here `fc2` is defined in `__init__` but never called in `forward()`, so it can never learn.

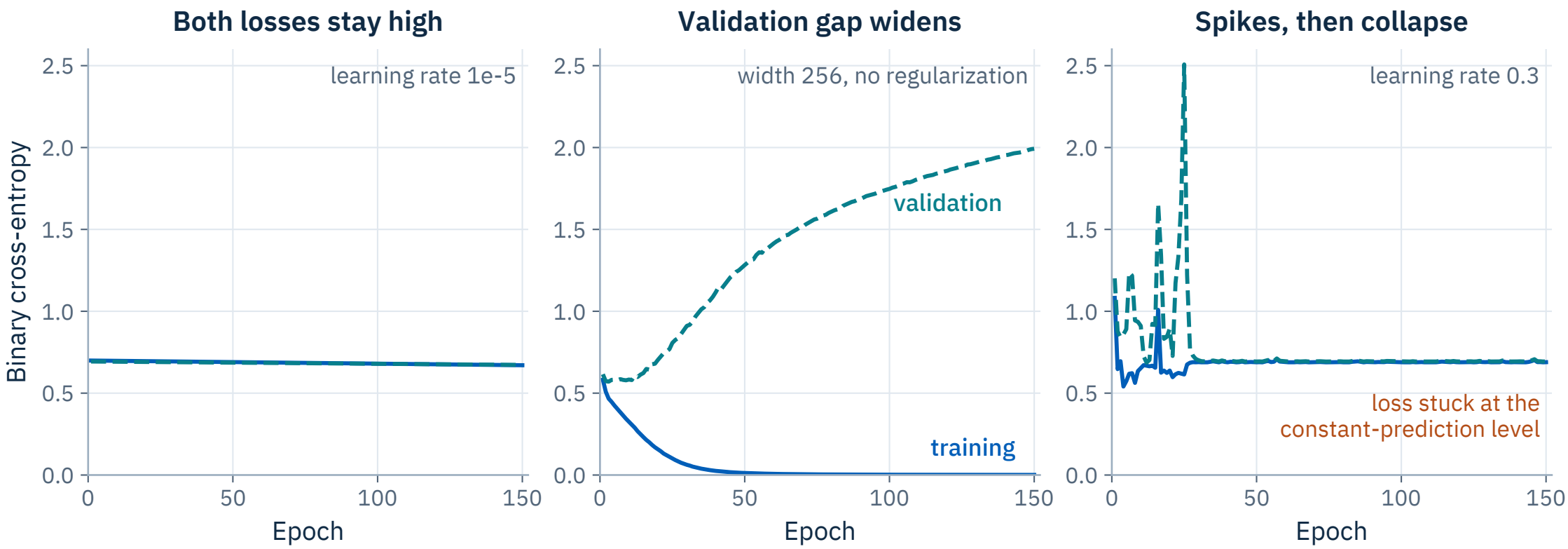
Missing or zero gradients point to a cut or dead path

- Compare gradient norms layer by layer after one backward pass
- None: detached, frozen, or unused; exactly zero: dead units
- Tiny near the input, large near the output: vanishing gradients
- Log these norms during training, not only when debugging



Computed: 6-layer MLPs of width 32, one backward pass on 256 random rows, torch seed 0; "None" marks layers that received no gradient.

Learning curves distinguish several problems



Computed: synthetic ESP table (seeds 1 and 2), two hidden layers, AdamW, torch seed 0; one setting changed per panel. Similar shapes can have other causes, so treat them as leads, not diagnoses.

DIAGNOSE BEFORE YOU TUNE

Read the curves before changing the model

Both losses high

- Check learning rate and input scaling first
- Then capacity and feature content

Validation gap widens

- Regularize or stop earlier
- Check the split for distribution shift

Spikes or NaN

- Lower the learning rate; clip gradients
- Look for bad rows, log(0), division by zero

Validation below training

- Dropout is active in the training loss
- Easier or leaked validation rows?

◆ Change one thing, rerun, and compare on the same validation rows.

PRACTICE

6 min · Pairs

Debugging from observations

Training loss remains flat near 0.69. Predictions are almost constant.

1. List two checks you would run before adding more layers.
2. What experiment would separate a data bug from an optimization problem?
3. The initial loss was 0.69 in one run and 7.5 in another. What does each suggest?

DISCUSSION

Check the path before the architecture

Data and labels

- Targets still aligned after joins and sorts
- Float targets of shape [batch, 1]

Output-loss contract

- Raw logits into BCEWithLogitsLoss
- No sigmoid inside the model

Gradients and optimizer

- Optimizer built from this model's parameters
- zero_grad, backward, and step all run

Only then capacity

- Overfit one batch first
- Then widen or deepen in controlled steps

PART 2 OF 5

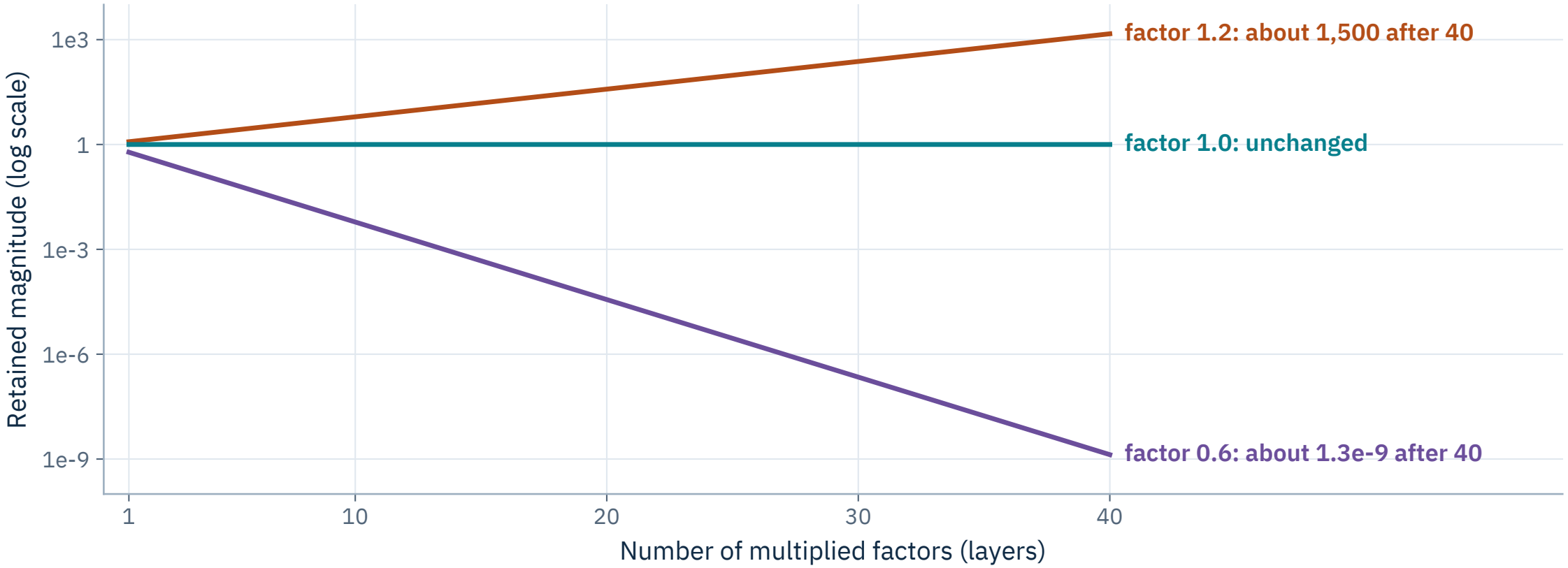
02

Signal propagation and normalization

Deep networks train when signals keep a stable scale.

- Why signals vanish or explode
- Initialization matched to the activation
- Batch, layer, and group normalization
- Residual connections as a gradient highway

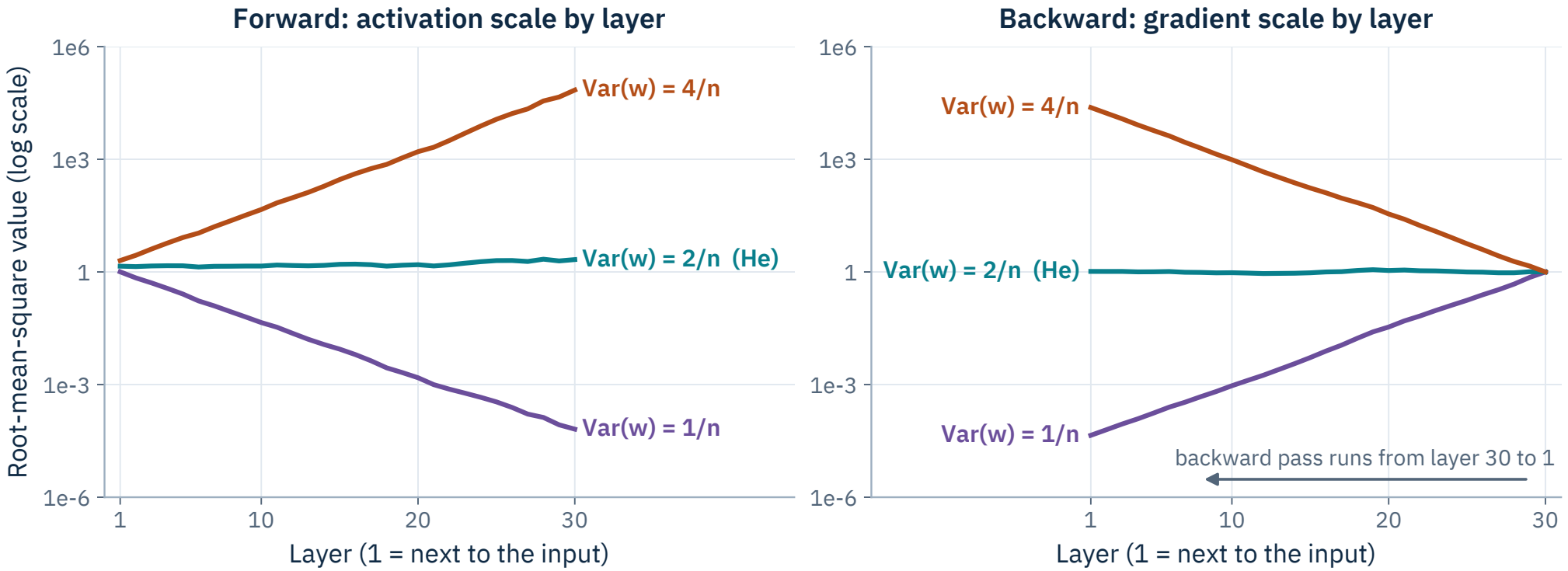
Gradient products can shrink or explode



Computed: analytical products of a repeated per-layer factor. The curves illustrate a mechanism, not measured gradients from a trained network.

Weight scale decides whether signals vanish or explode

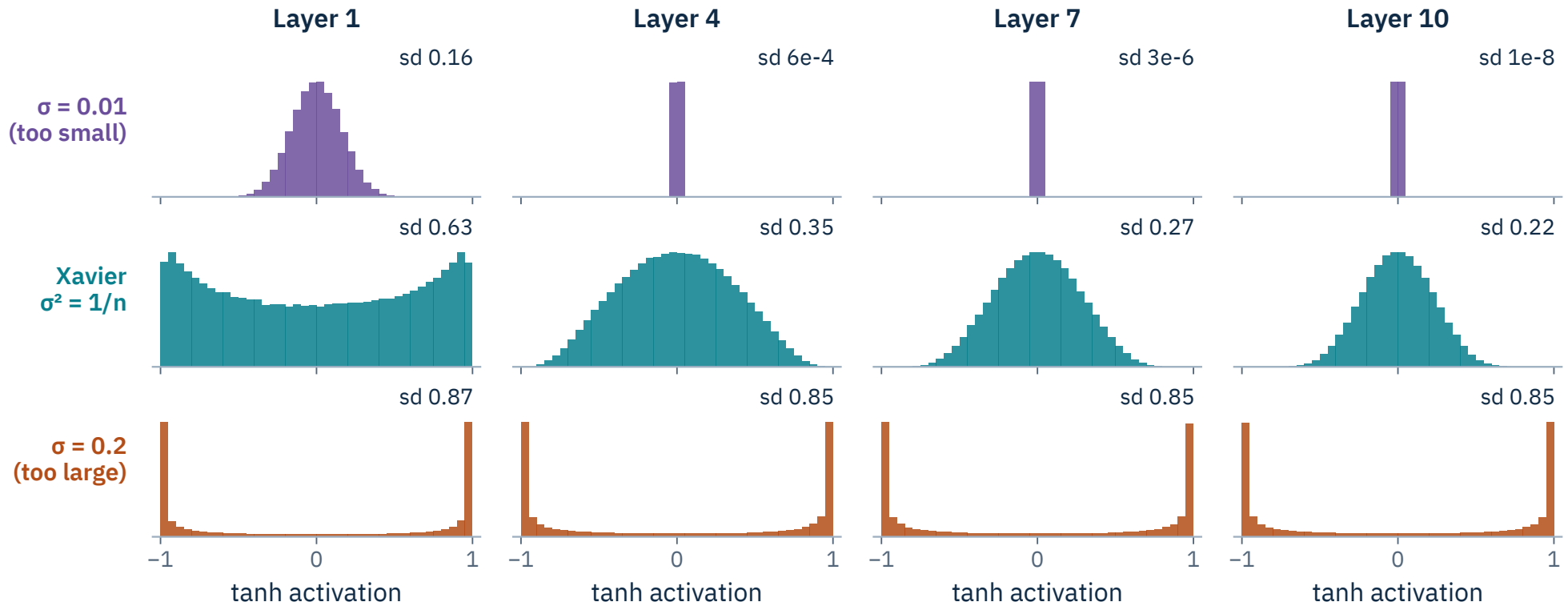
Each ReLU layer multiplies the signal's variance by about $n \cdot \mathrm{Var}(w)/2$; over 30 layers small differences compound.



Computed: 30-layer ReLU network, width n = 256, Gaussian weights, no biases or normalization; numpy seed 0.

Activation histograms reveal a bad initialization

A 10-layer tanh network: too small collapses to zero, too large saturates at ±1.



Computed: tanh network of width 256, Gaussian weights with the stated standard deviation, no biases, numpy seed 0; histograms of all activations at four depths.

Record activation statistics with forward hooks

```
stats = {}
def record(name):
    def hook(module, inputs, output):
        stats[name] = output.std().item()
    return hook

for name, m in model.named_modules():
    if isinstance(m, nn.Tanh):
        m.register_forward_hook(record(name))
with torch.no_grad():
    model(xb)
print({k: f"{v:.1e}" for k, v in stats.items()})
```

OUTPUT

```
{'1': '1.6e-01', '3': '2.5e-02', '5': '4.0e-03', '7': '6.4e-04'}
```

Four tanh layers of width 256, weights drawn with standard deviation 0.01, zero biases: the spread shrinks about 6× per layer.

Initialization controls the first signal scale

✗ Identical weights

- Every hidden unit computes the same function
- Gradients stay identical, so symmetry never breaks

Too small

- Activations and gradients shrink layer by layer
- Deep stacks stop learning

Too large

- ReLU signals explode; tanh and sigmoid saturate
- Loss spikes or NaN from the first steps

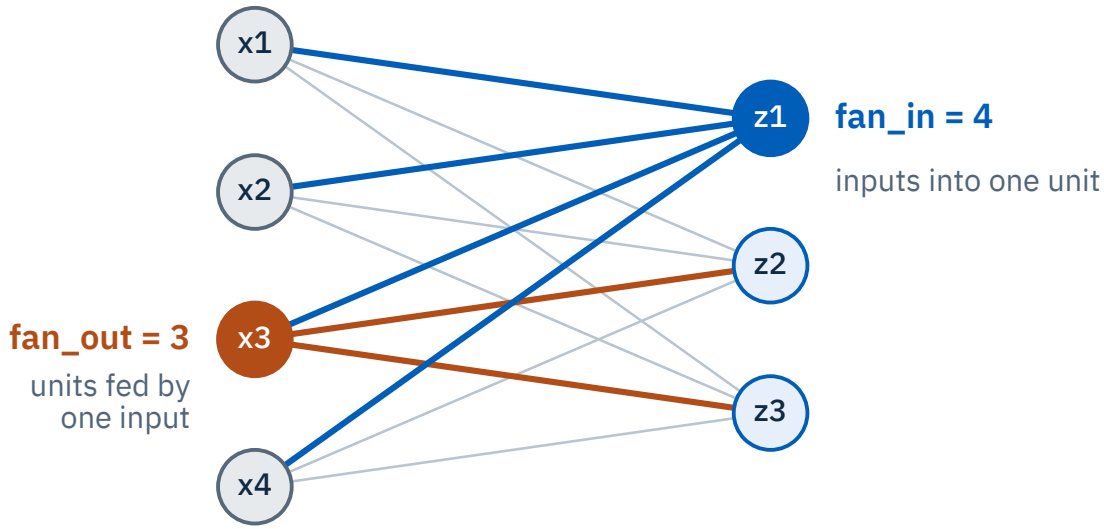
✓ Variance-preserving

- Xavier for tanh-like units, He (Kaiming) for ReLU
- Check both forward and backward scale

Initialization variance scales with fan-in

$$\mathrm{Var}(w) \approx \frac{2}{\mathrm{fan}_{\mathrm{in}} + \mathrm{fan}_{\mathrm{out}}} \quad \text{or} \quad \frac{2}{\mathrm{fan}_{\mathrm{in}}}$$

- Xavier (Glorot) averages fan-in and fan-out: suits tanh and sigmoid
- He (Kaiming) uses 2 / fan-in: compensates for ReLU zeroing half its inputs
- Distribution (normal or uniform) and gain change the exact bounds



$$\mathrm{Var}(z) = \mathrm{fan}_{\mathrm{in}} \cdot \mathrm{Var}(w) \cdot \mathrm{E}[x^2]$$

keep Var(z) steady → Var(w) ≈ 1 / fan_in (× 2 for ReLU)

Schematic: one dense layer with fan-in 4 and fan-out 3.

PRACTICE

5 min · Individual

How fast does a bad scale compound?

A 10-layer ReLU network has fan-in 256 in every layer; weights are Gaussian with standard deviation σ.

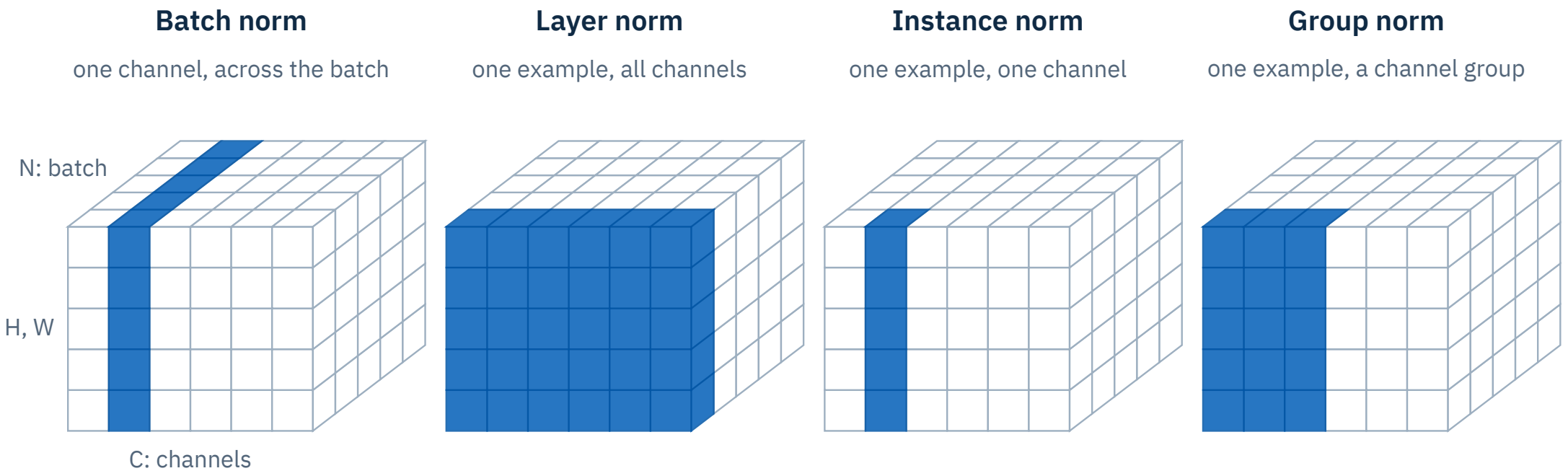
1. Compute the per-layer variance factor $256\,\sigma^2/2$ for $\sigma = 0.01$, $\sigma = 0.1$, and He initialization.
2. By what factor has the activation standard deviation changed after 10 layers in each case?
3. Which σ does He initialization give here?

Normalization uses different axes

Method	Statistics used	Training and inference issue
Input standardization	Training-set feature statistics	Keep fitted values fixed
Batch normalization	Batch statistics during training	Running estimates during evaluation
Layer normalization	Features within each example	No batch running-average dependency
Group normalization	Channel groups within each example	Independent of batch size

Batch, layer, and group normalization all add a learned scale and shift after normalizing.

Batch, layer, and group norm share statistics differently

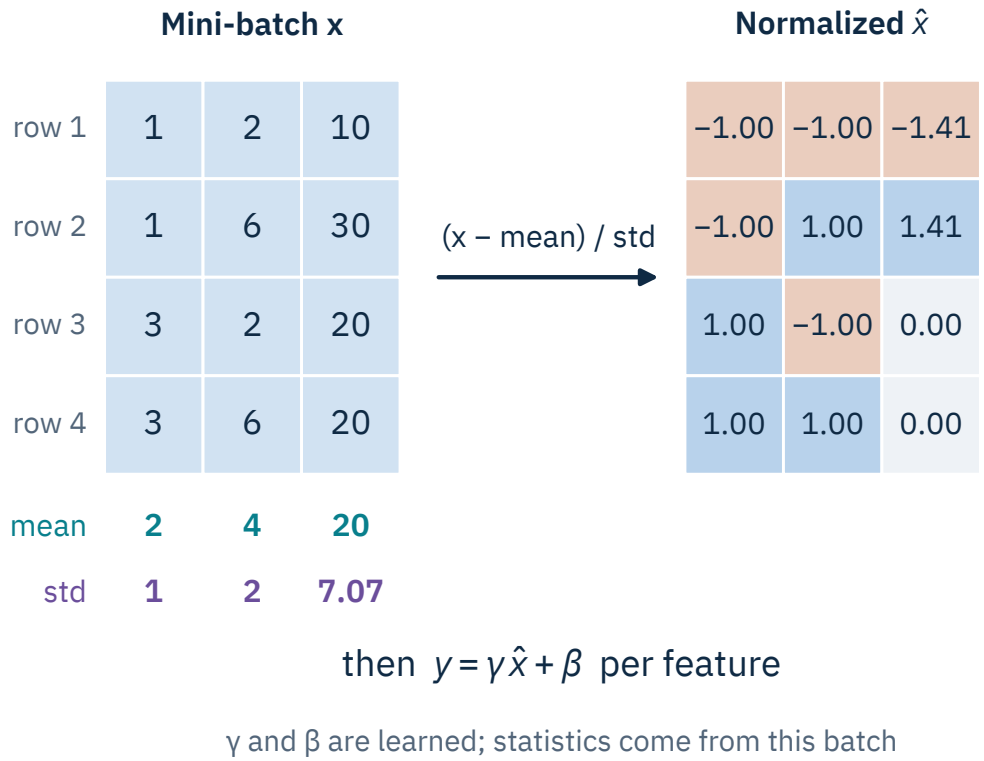


Schematic after Wu and He (2018). For a dense layer H, W = 1: batch norm normalizes each feature over the batch; layer norm normalizes each example over its features.

Batch normalization includes learned scale and shift

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta$$

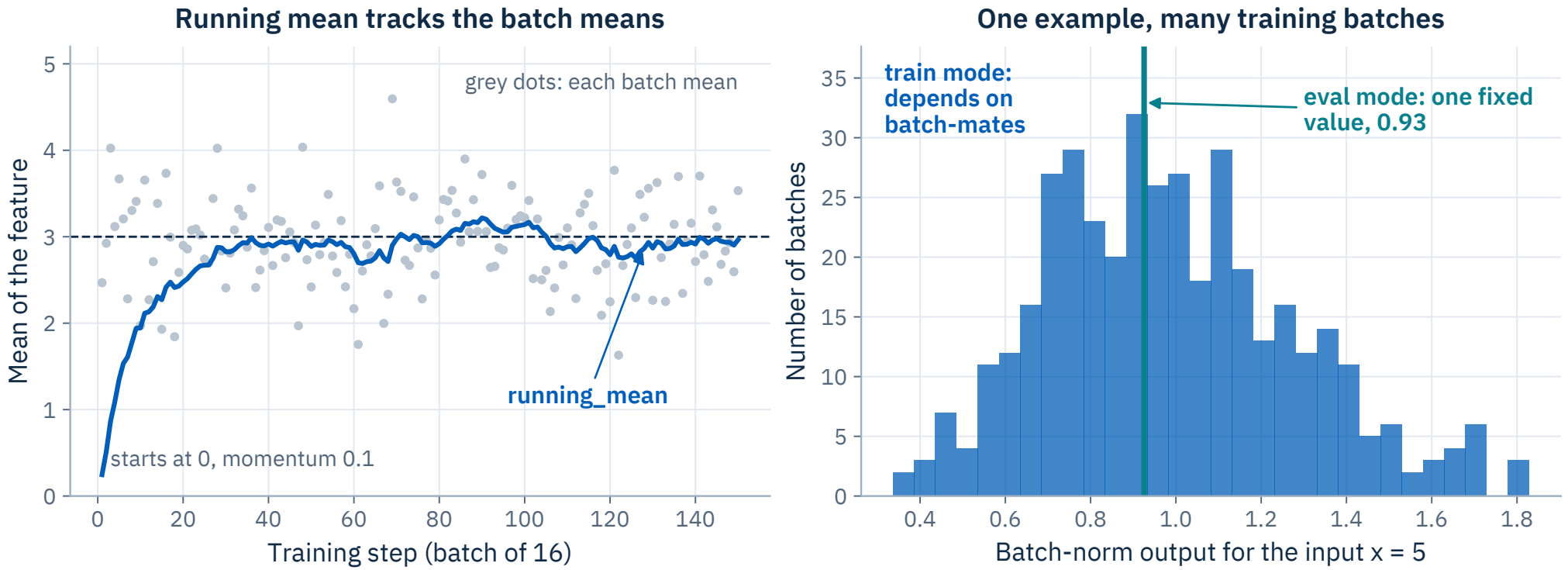
- μ_B and σ_B^2 are computed over the batch for each feature
- γ and β are learned, so the layer can undo the scaling
- Evaluation uses running estimates instead



Exact arithmetic with torch.nn.BatchNorm1d in training mode (affine off); population standard deviation over the batch.

BatchNorm uses batch statistics only in training mode

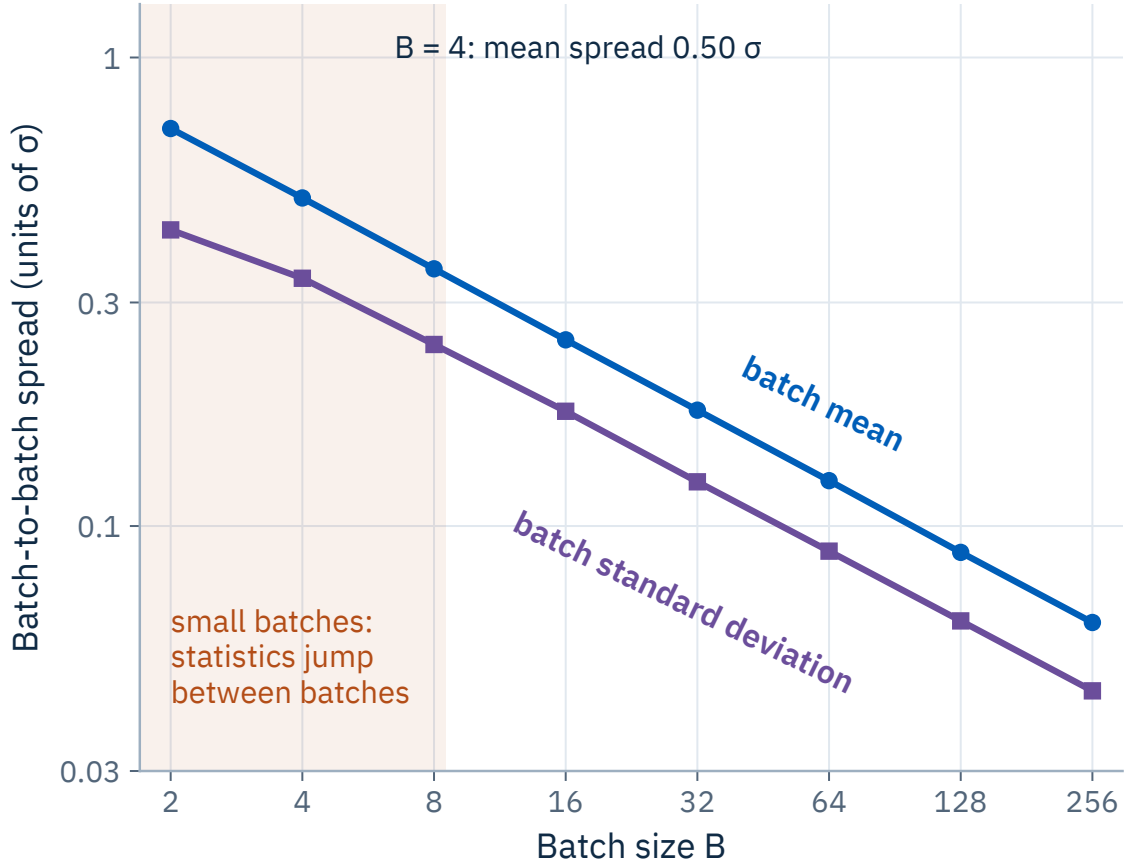
Running averages are updated in training and used after `model.eval()`; forget `eval()` and predictions depend on batch-mates.



Computed with torch.nn.BatchNorm1d (momentum 0.1) on a synthetic feature with mean 3 and standard deviation 2; batches of 16; numpy seed 0.

Batch statistics are noisy when batches are small

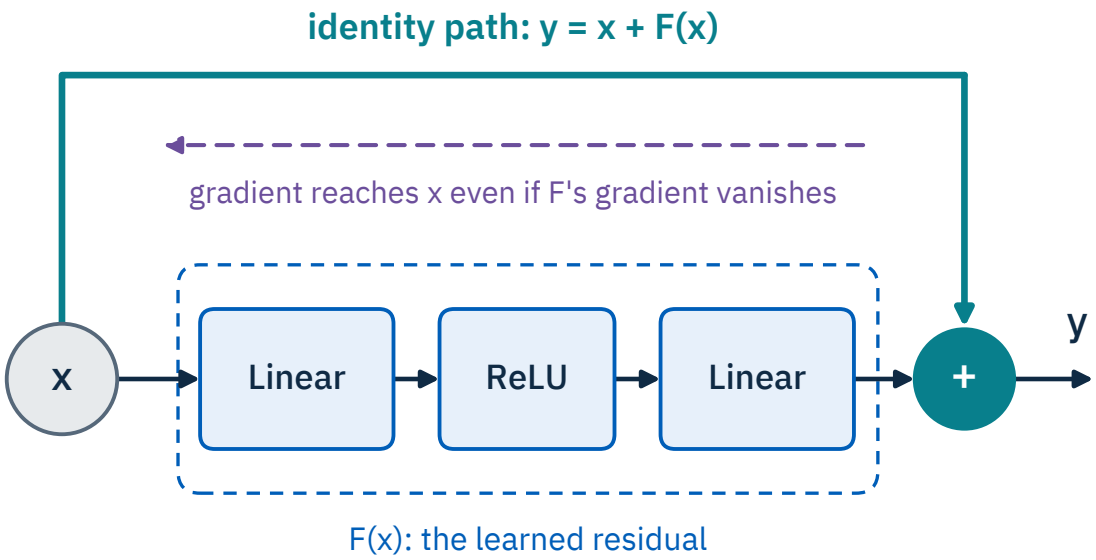
- Batch norm needs large, representative batches (common in Day 4 CNNs)
- With 2 to 8 rows per batch, the mean and variance jump between batches
- Layer and group norm use one example: fine for small batches and Day 5 sequences
- None of them replaces input standardization fitted on training rows



Computed: 20,000 random batches per size from a standard-normal feature, numpy seed 0; spread across batches in units of σ.

Residual connections give gradients a highway

- The block learns a correction $F(x)$ added to its input
- The identity term gives early layers a direct gradient path
- Very deep networks become trainable; Day 4's ResNets build on this
- x and $F(x)$ must have the same shape, or a projection is added



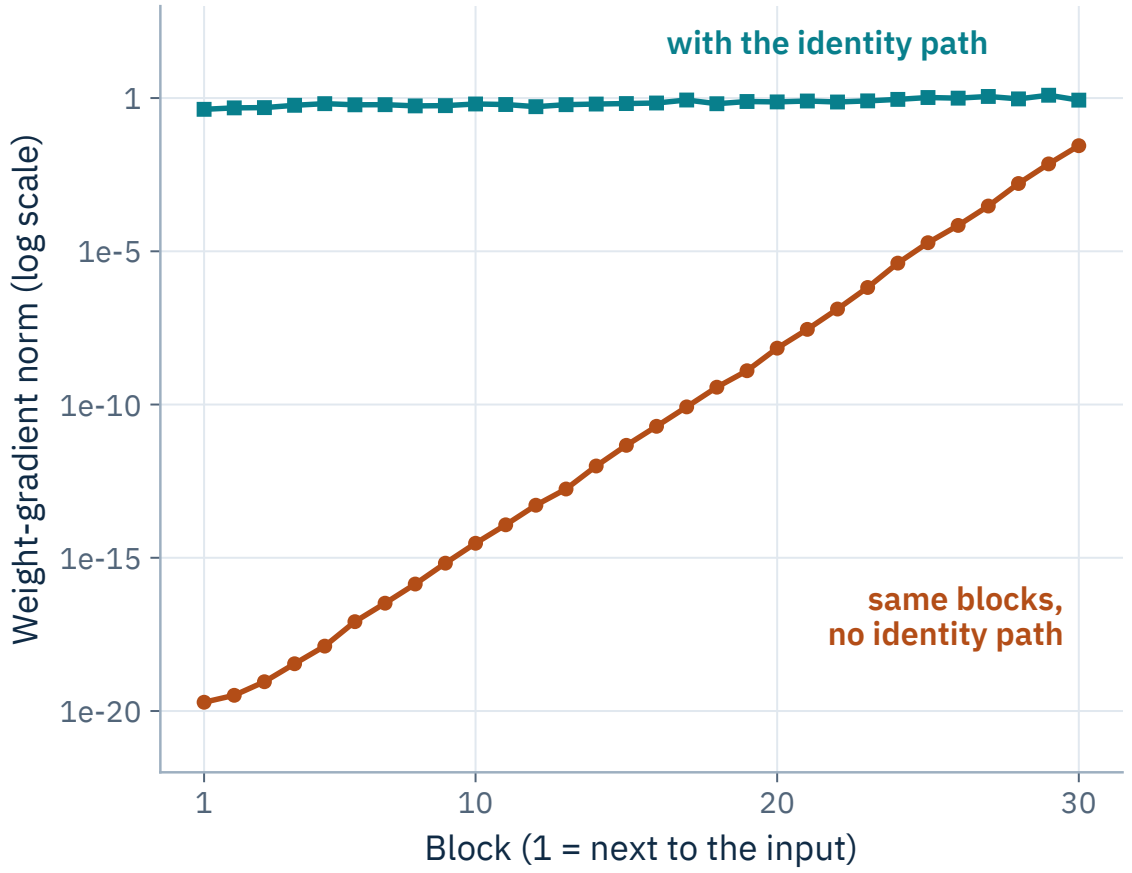
$$\partial y/\partial x = I + \partial F/\partial x$$

Day 4 uses this block in residual CNNs

Schematic residual block: y = x + F(x).

Skip paths keep gradients alive in deep stacks

- Same 30 blocks and identical weights; only the identity path differs
- Without it, the first block's gradient is about 10^{-20}
- With it, every block receives a gradient of similar size



Computed: 30 Linear-ReLU-Linear blocks of width 64, PyTorch default initialization, one backward pass on 256 random rows; torch seeds 0 and 1.

PRACTICE

5 min · Pairs

Which fix would you try first?

For each observation, name the first change you would test and the evidence that would confirm it.

1. Activation histograms collapse to a spike at zero by layer 6.
2. A BatchNorm model gives different predictions for the same well-log sample depending on its batch.
3. A 40-layer plain MLP trains no better than a 4-layer one.

PART 3 OF 5

03

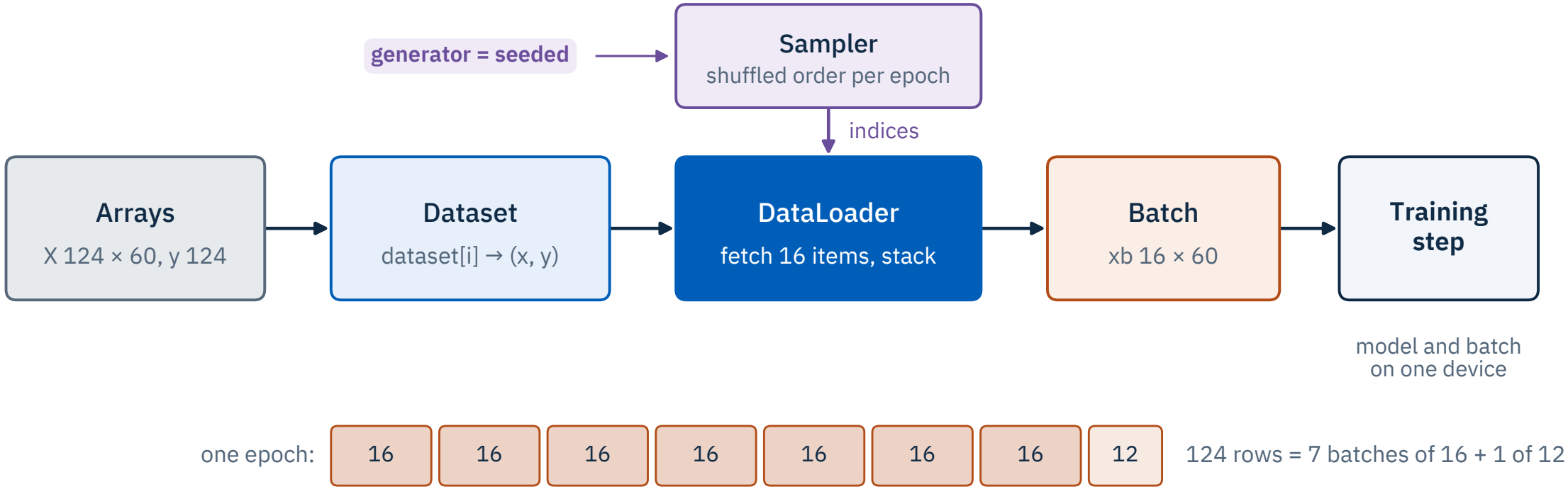
A PyTorch training loop you can trust

Silent errors live in the loop; make each step explicit.

- Datasets, loaders, and devices
- The training step, evaluation mode, and gradients
- Checkpoints, saving, and reloading
- Reproducibility, experiment logs, and the Keras equivalent

Dataset and DataLoader turn arrays into batches

The Dataset answers "give me row i"; the DataLoader decides order, batching, and collation.



Schematic. Sizes follow the Lab 1 Sonar training partition: 124 rows, batch size 16.

Build loaders with an explicit, seeded shuffle

```
from torch.utils.data import (
    TensorDataset, DataLoader)

train_ds = TensorDataset(Xt, yt)
g = torch.Generator().manual_seed(42)
train_loader = DataLoader(
    train_ds, batch_size=16,
    shuffle=True, generator=g)
val_loader = DataLoader(
    TensorDataset(Xv, yv),
    batch_size=64, shuffle=False)
xb, yb = next(iter(train_loader))
print(xb.shape, yb.shape, len(train_loader))
```

OUTPUT

```
torch.Size([16, 60]) torch.Size([16, 1]) 8
```

- Shuffle training batches, never validation
- A seeded generator repeats the batch order
- `len(loader)` is the number of batches per epoch

Model and batches must share one device

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")

model = model.to(device)   # before the optimizer
for xb, yb in train_loader:
    xb = xb.to(device)
    yb = yb.to(device)
```

- Move the model once, before building the optimizer
- Move every batch inside the loop
- Use `.cpu()` before converting results to NumPy
- Small tabular models are often fastest on CPU

A small binary model in PyTorch

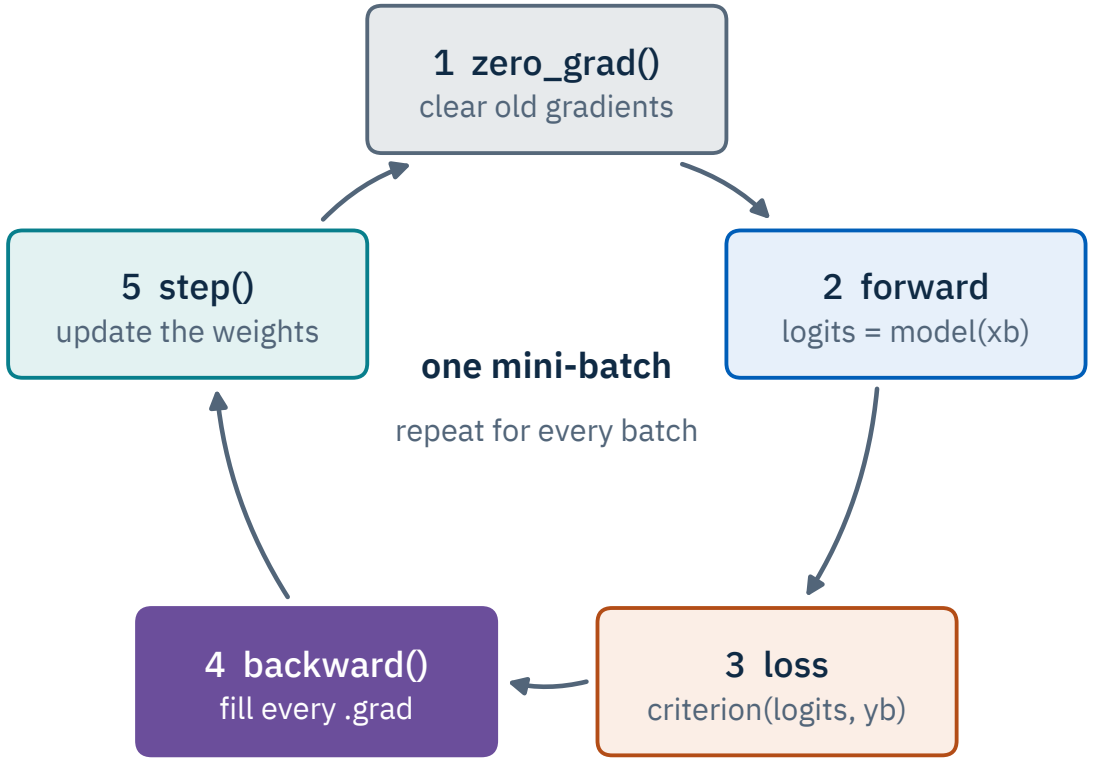
```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(n_features, 32), nn.ReLU(),
    nn.Dropout(0.2), nn.Linear(32, 1)
)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.AdamW(
    model.parameters(), lr=1e-3, weight_decay=1e-3
)
```

The final layer emits one raw logit per row; the loss applies the sigmoid internally.

Every training step repeats five operations

- `zero_grad` clears gradients left from the previous batch
- Forward and loss build the graph on the current batch
- `backward` fills `.grad`; `step` applies the optimizer rule
- Skip or reorder one and training fails quietly



Schematic of one PyTorch optimization step.

The training loop orders zero_grad, backward, and step

```
model.train()
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(xb)                          # (B, 1)
    loss = criterion(logits, yb)                # yb: (B, 1) floats
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
```

Clipping is optional; if used, it belongs after `backward()` and before `step()`.

Evaluation mode and gradient tracking are separate

```
model.eval()
with torch.inference_mode():
    logits = model(X_val.to(device))                    # (n, 1)
    val_loss = criterion(logits, y_val.to(device))      # y_val: (n, 1)
    probabilities = torch.sigmoid(logits).squeeze(-1).cpu().numpy()
```

- `eval()` changes dropout and normalization behaviour
- `inference_mode()` switches off autograd bookkeeping for inference

Mode switches and autograd switches do different jobs

Call	Dropout	BatchNorm statistics	Gradient tracking
`train()`	active	batch; running averages updated	no effect
`eval()`	off	running averages used	no effect
`no_grad()`	no effect	no effect	off
`inference_mode()`	no effect	no effect	off; stricter, faster

Evaluation needs both: `eval()` for layer behaviour, and a no-grad context for speed and memory.

A reference to state_dict is not a snapshot

```python
import copy

best_ref = model.state_dict()                    # live references
best_copy = copy.deepcopy(model.state_dict())    # snapshot
w0 = best_copy["0.weight"][0, 0].item()

train_one_epoch(model, train_loader, optimizer)   # weights change

print(f"{w0:.4f}")
print(f"{best_ref['0.weight'][0, 0].item():.4f}")
print(f"{best_copy['0.weight'][0, 0].item():.4f}")
```

OUTPUT

```
0.0987
0.0919
0.0987
```

Early stopping needs a copied checkpoint

```
import copy

best_loss, stale, patience = float("inf"), 0, 10
for epoch in range(1, max_epochs + 1):
    train_one_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)       # eval mode inside
    if val_loss < best_loss:
        best_loss, best_epoch, stale = val_loss, epoch, 0
        best_state = copy.deepcopy(model.state_dict())
    else:
        stale += 1
        if stale >= patience:
            break

model.load_state_dict(best_state)                # restore, then
model.eval()                                     # evaluate or save
```

Save the state dict with everything needed to use it

```
torch.save(model.state_dict(), run_dir / "weights.pt")
joblib.dump(scaler, run_dir / "scaler.joblib")
(run_dir / "run.json").write_text(json.dumps(metadata, indent=2))

restored = build_model(config["width"], config["dropout"])
state = torch.load(run_dir / "weights.pt",
                   map_location="cpu", weights_only=True)
restored.load_state_dict(state)
restored.eval()
check = torch.sigmoid(restored(X_val_scaled)).detach().numpy().ravel()
np.testing.assert_allclose(check, val_prob, rtol=1e-6, atol=1e-6)
```

The reload check proves the saved files reproduce the predictions you validated.

Save the whole prediction path, not only the weights

A weights file without its scaler, feature order, threshold, and split is not a model anyone can use.

Operations will run your model months later on new data; everything that turns raw sensor rows into a decision must travel with it.

After training: restore, verify, save, test once

Five steps separate the last epoch from a result you can report.

1. **Restore** — load the best validation checkpoint
2. **Verify** — eval mode and a reload prediction check
3. **Save** — weights, scaler, config, threshold, split
4. **Test once** — locked final assessment, no tuning
5. **Report** — metrics with budget, seeds, and limits

◆ If a test result changes your settings, the test set has become a validation set.

Mixed precision is a GPU speed option, not a default

```
use_amp = device.type == "cuda"
for xb, yb in train_loader:
    xb, yb = xb.to(device), yb.to(device)
    optimizer.zero_grad(set_to_none=True)
    with torch.autocast(
            device_type=device.type,
            dtype=torch.bfloat16,
            enabled=use_amp):
        loss = criterion(model(xb), yb)
    loss.backward()
    optimizer.step()
```

- Autocast runs matrix multiplies in 16-bit where safe
- bfloat16 needs no loss scaling; float16 needs a GradScaler
- Compare losses and metrics with a float32 run
- Little benefit for small tabular models

Log enough to rerun and audit every experiment

Configuration

- Architecture, optimizer, learning rate, schedule
- Batch size, epochs, patience, seeds

Data

- Dataset version and split indices
- Feature list and fitted preprocessing

Curves

- Training and validation loss per epoch
- Learning rate and gradient norm per step

Environment

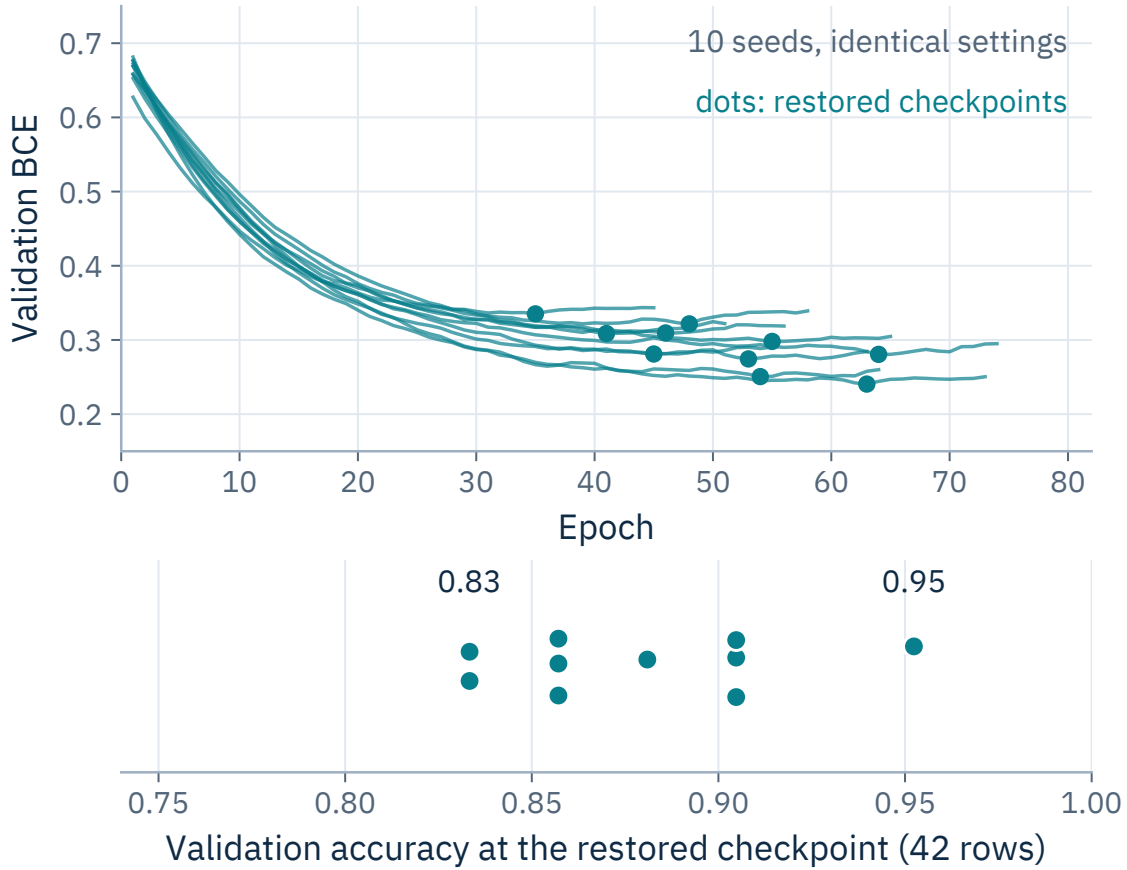
- Library versions and device
- Code version and run time

Outcome

- Selected epoch and checkpoint path
- Decision and reason, even for failed runs

Reproducibility includes the experiment definition

- Record split indices, seeds, dataset version, and feature schema
- Save preprocessing, configuration, weights, and threshold
- Log library versions and hardware
- Repeat promising settings over several seeds



◆ Ten seeds, one configuration: validation accuracy from 0.83 to 0.95 on 42 rows.

Computed from the course file sonar.all-data.csv with the Lab 1 split and training loop; torch seeds 0–9.

The same contract in Keras

```
import keras

model = keras.Sequential([
    keras.layers.Input(shape=(n_features,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2), keras.layers.Dense(1)
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss=keras.losses.BinaryCrossentropy(from_logits=True)
)
```

Same architecture, same raw-logit output, same loss; `fit()` hides the loop you just wrote.

Explicit validation is easier to audit

```
stop = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10,
    restore_best_weights=True
)
model.fit(
    X_train_scaled, y_train,
    validation_data=(X_val_scaled, y_val),
    epochs=100, batch_size=32, callbacks=[stop]
)
```

Prefer `validation_data` over `validation_split`: you choose the rows, and the scaler never sees them.

LABORATORY

60 min

Laboratory 1: a reproducible PyTorch pipeline

- ☐ Assert that the scaler was fitted on the 124 training rows only
- ☐ Predict the parameter count for width 32, then check it
- ☐ Mark the five training-step operations, the train/eval switch, and the copy
- ☐ Change one setting, keep split and budget fixed, compare with the baseline
- ☐ Save, reload, pass the prediction check; keep `RUN_FINAL = False`

NOTEBOOK

`exercises/01_pytorch_pipeline/PyTorch_Pipeline_Student.ipynb`

EVIDENCE TO SUBMIT

- Learning curves with the selected epoch
- Config, checkpoint, and reload evidence
- Baseline comparison and one error case

◆ Keras track: 02_tensorflow_pipeline, only where TensorFlow is installed.

DISCUSSION

Lab 1 debrief: what the evidence supports

Reference run: seed 42 on CPU; identical in PyTorch 2.3 and 2.14, may vary elsewhere.

1,985

parameters for width 32

epoch 44

restored checkpoint (stopped at 54)

0.86 vs 0.69

validation accuracy, network vs logistic baseline

- 42 validation rows: one row moves accuracy by 2.4 points, and the checkpoint was chosen on them.

PART 4 OF 5

04

Optimizers, schedules, and tuning

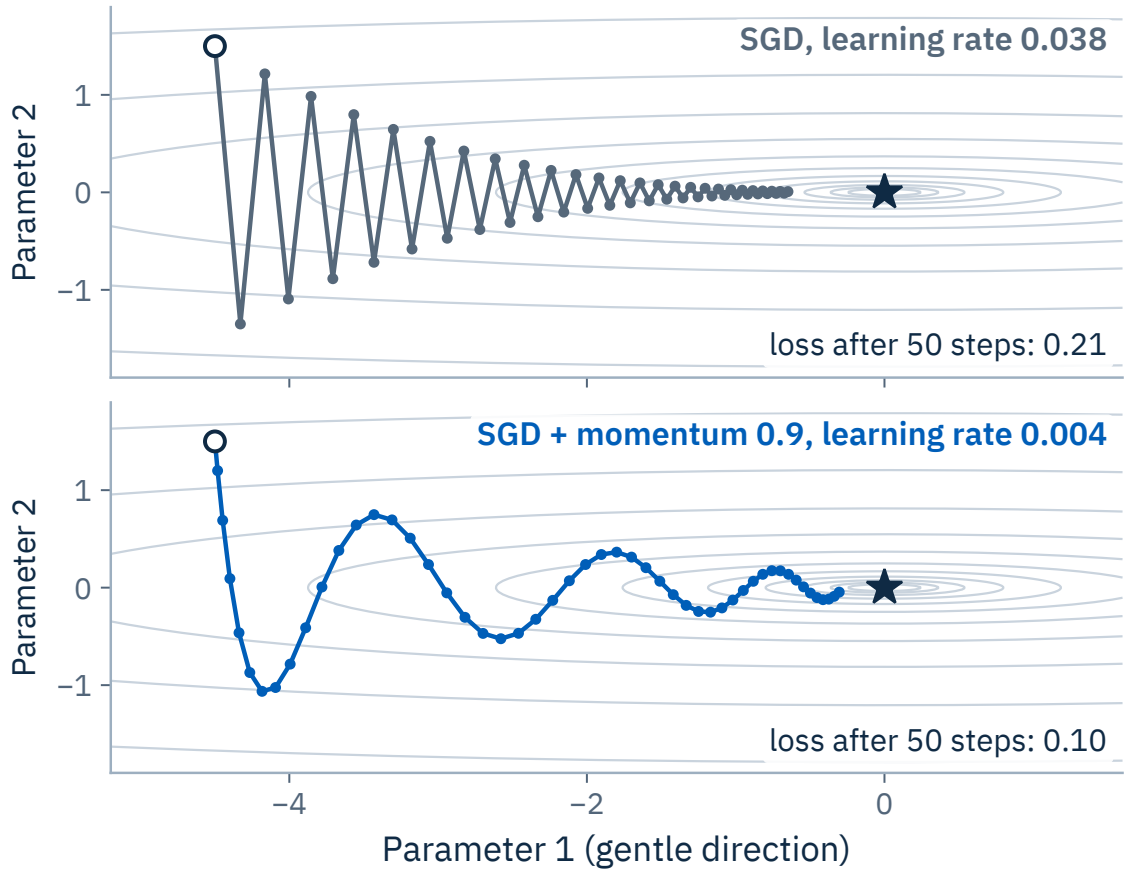
Learning rate often matters more than a large architecture search.

- Momentum and adaptive optimizers
- AdamW and decoupled weight decay
- Learning-rate range tests and schedules
- Batch size, clipping, and a tuning budget

SGD with momentum accumulates direction

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

- *v* is a running sum of past gradients, weighted by β
- Zig-zags across the valley cancel; steady progress along it adds up
- The learning rate still sets the overall step scale



Computed with torch.optim on L = (w1² + 50 w2²)/2, 50 steps from (−4.5, 1.5); learning rates chosen for illustration.

Adaptive optimizers track gradient scale

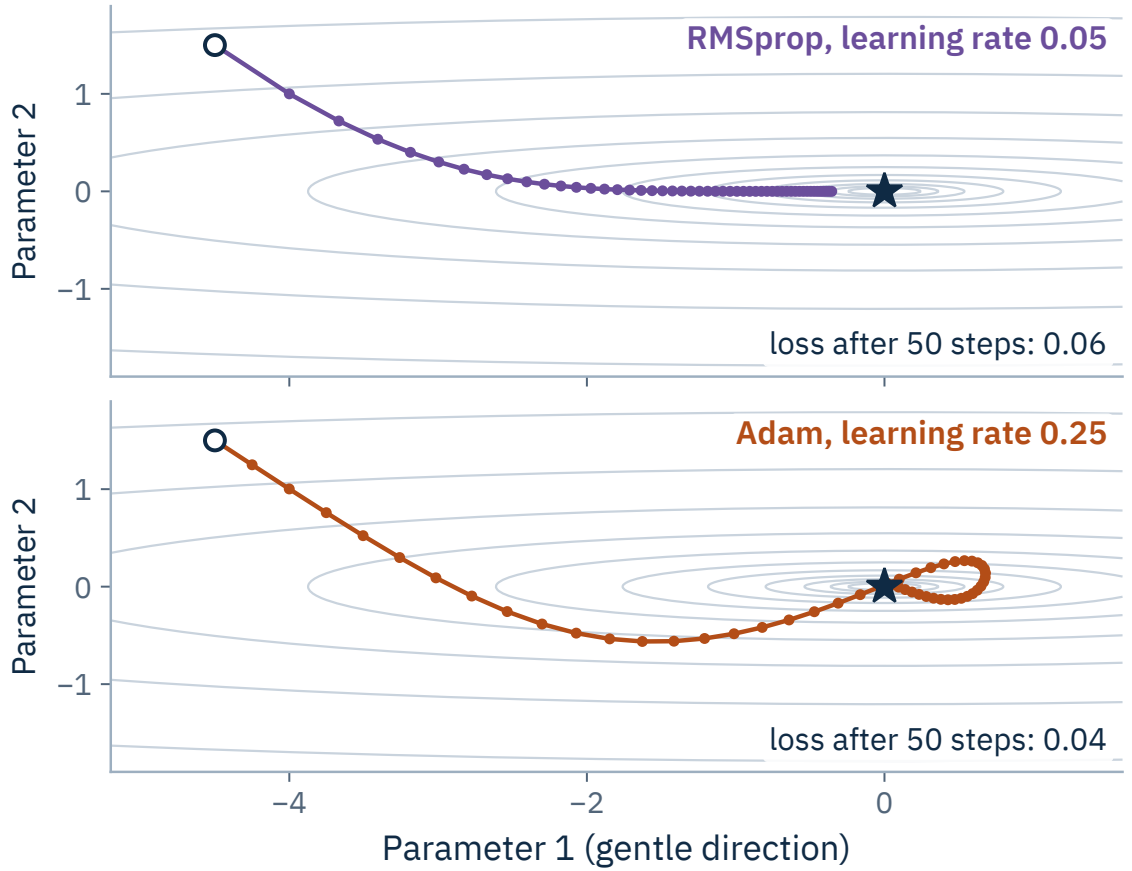
Optimizer	Running information	Practical consideration
RMSprop	Squared gradients	Adapts step sizes per parameter
Adam	Gradients and squared gradients	Bias-corrected moment estimates
AdamW	Adam-style moments plus decoupled decay	Separates decay from adaptive gradient scaling

Each still has a learning rate that must be tuned; defaults are starting points.

Adam's update uses two moment estimates

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- *m* tracks direction and *v* tracks squared size, per parameter
- Hats mark the correction for averages that start at zero
- Early steps are about η per parameter, whatever the gradient size
- ε keeps the denominator away from zero



Computed with torch.optim (default betas and alpha) on the same bowl; learning rates chosen for illustration.

A fair comparison tunes each optimizer's learning rate

An optimizer run at someone else's learning rate is not a fair competitor.

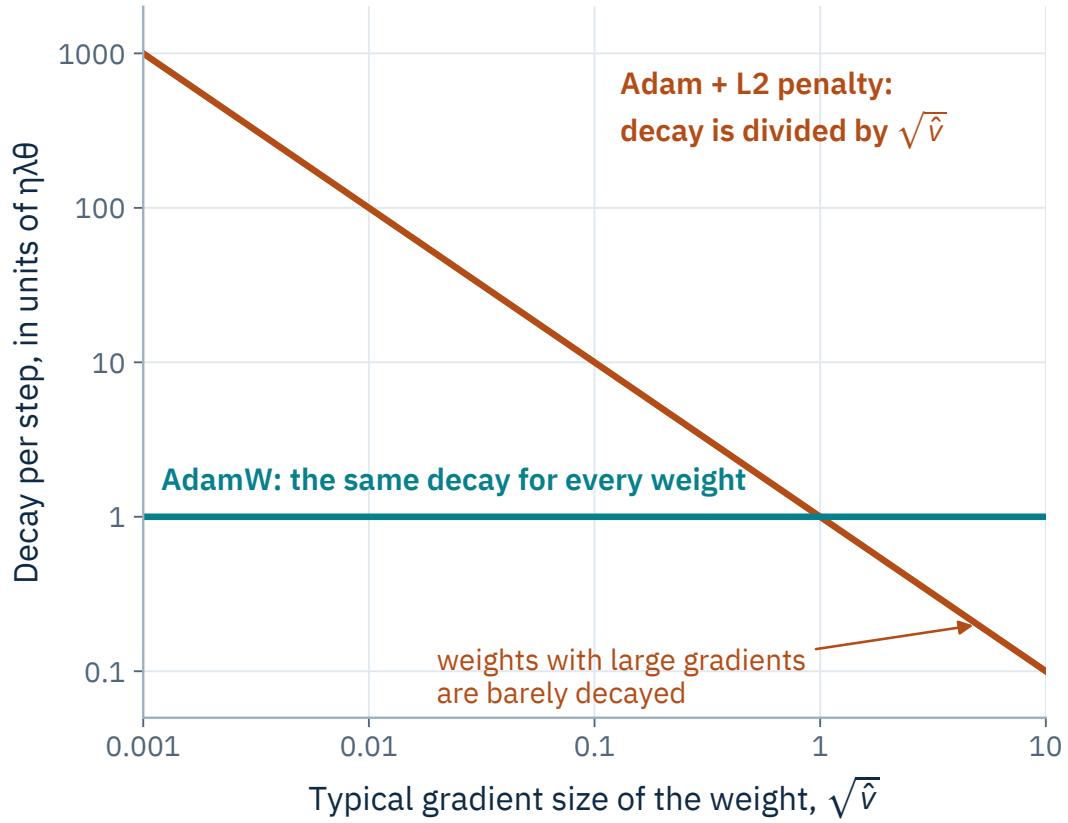
Give every candidate the same budget, the same split, several seeds, and its own small learning-rate search.

AdamW decouples weight decay from the adaptive step

$$\text{Adam} + \text{L2}: \; g_t \leftarrow g_t + \lambda\theta_t \text{ before scaling}$$

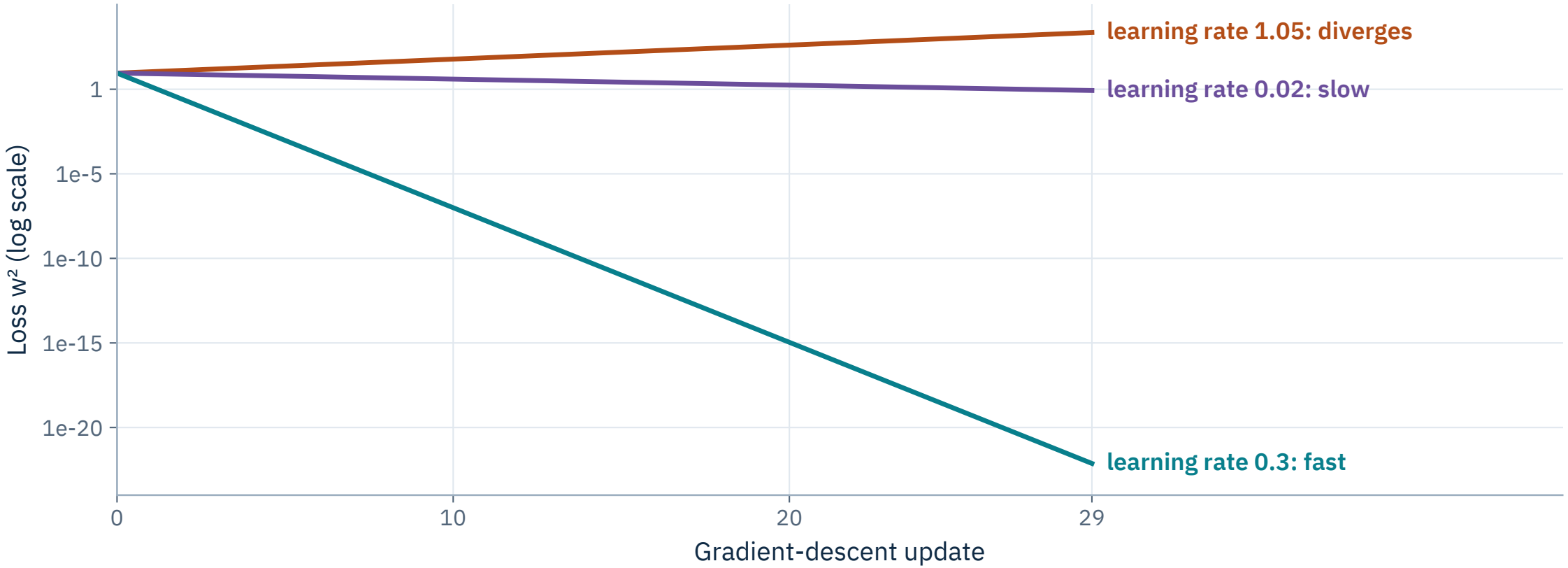
$$\text{AdamW}: \; \theta_{t+1} = \theta_t - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\theta_t\right)$$

- With L2 inside Adam, the adaptive denominator divides the penalty too
- Weights with large gradients are then barely regularized
- `Adam(weight_decay=...)` is the L2 form; `AdamW` is decoupled



Analytical approximation, valid while the decay term is small relative to the gradient.

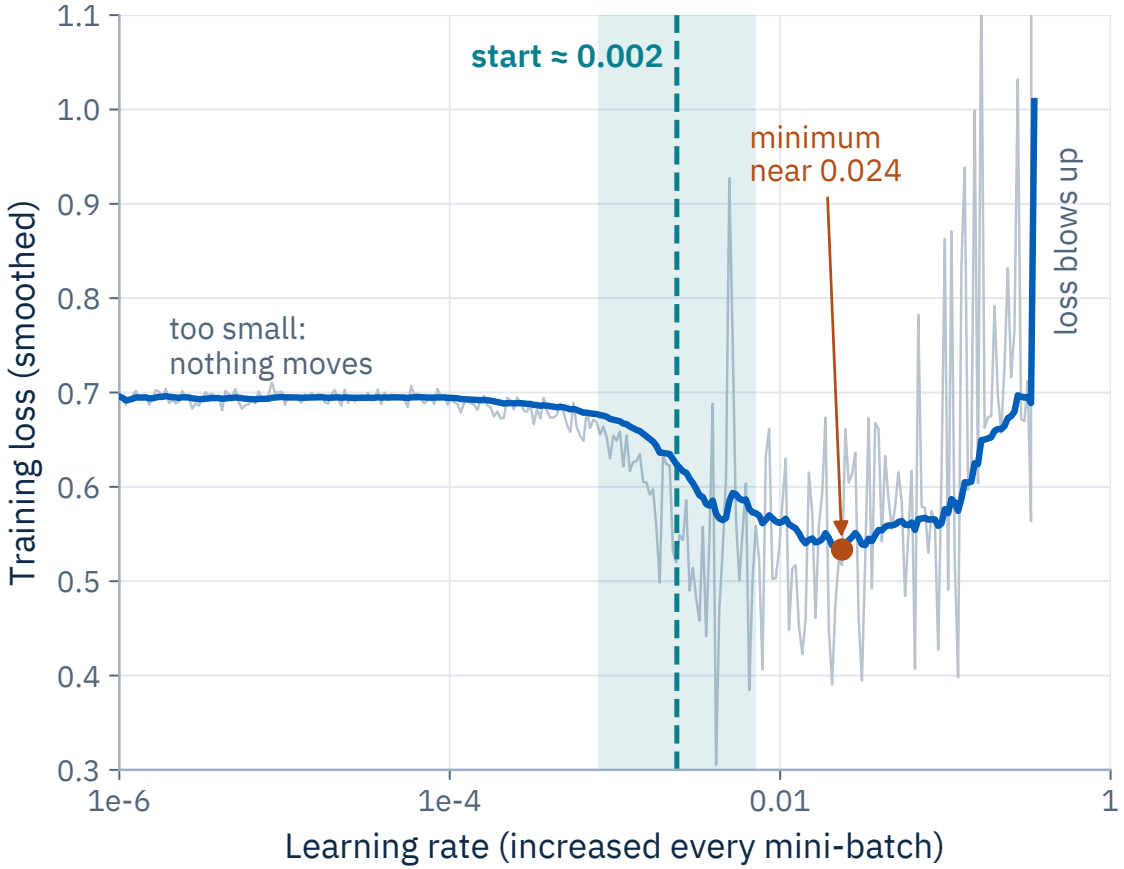
Step size changes convergence



Computed: exact gradient descent on the loss w^2 from $w = 3$ with three learning rates. Large steps can diverge even in a simple problem.

A learning-rate range test finds the usable window

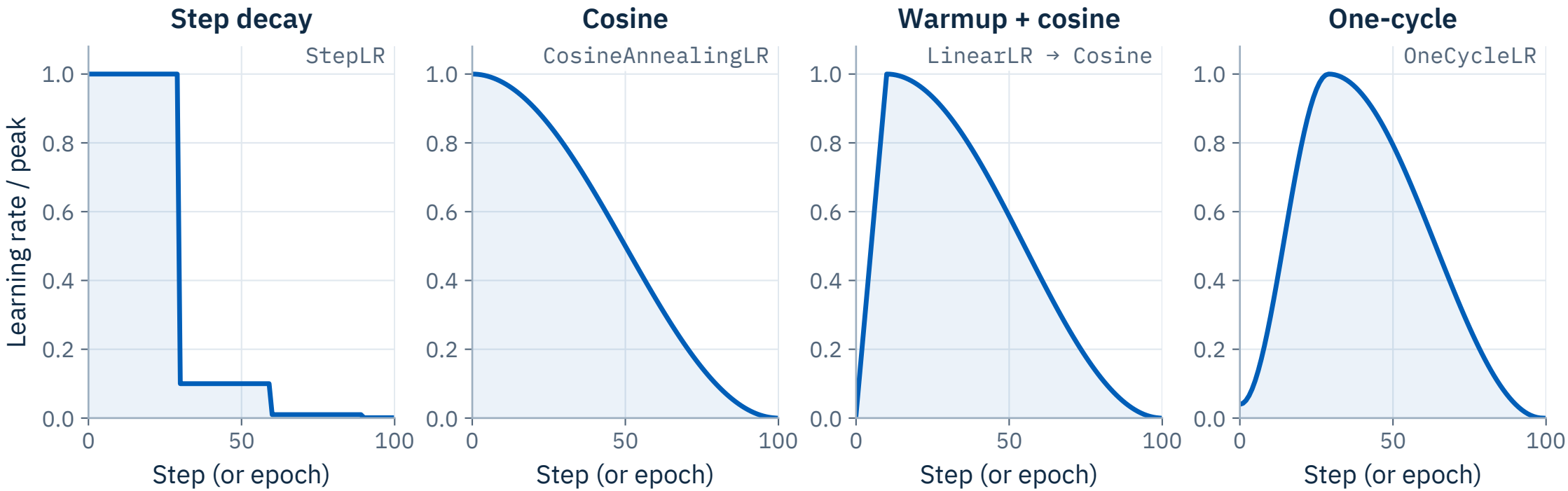
- Raise the rate every mini-batch, starting from 1e-6
- Stop when the smoothed loss climbs well above its start
- A common heuristic: start about 10× below the loss minimum
- Rerun after changing batch size, model, or optimizer



Computed: synthetic ESP table (4,000 rows, seed 3), MLP 12-64-64-1, AdamW, batch 32, torch seed 0; grey = raw batch loss, blue = moving average.

Schedules are part of the experiment

Step, cosine, warmup, and one-cycle schedules shape the same peak rate over a fixed budget.



Computed by stepping the named torch.optim.lr_scheduler classes for 100 steps; peak rate scaled to 1.

Schedulers step on a declared clock

```python
from torch.optim import lr_scheduler as S

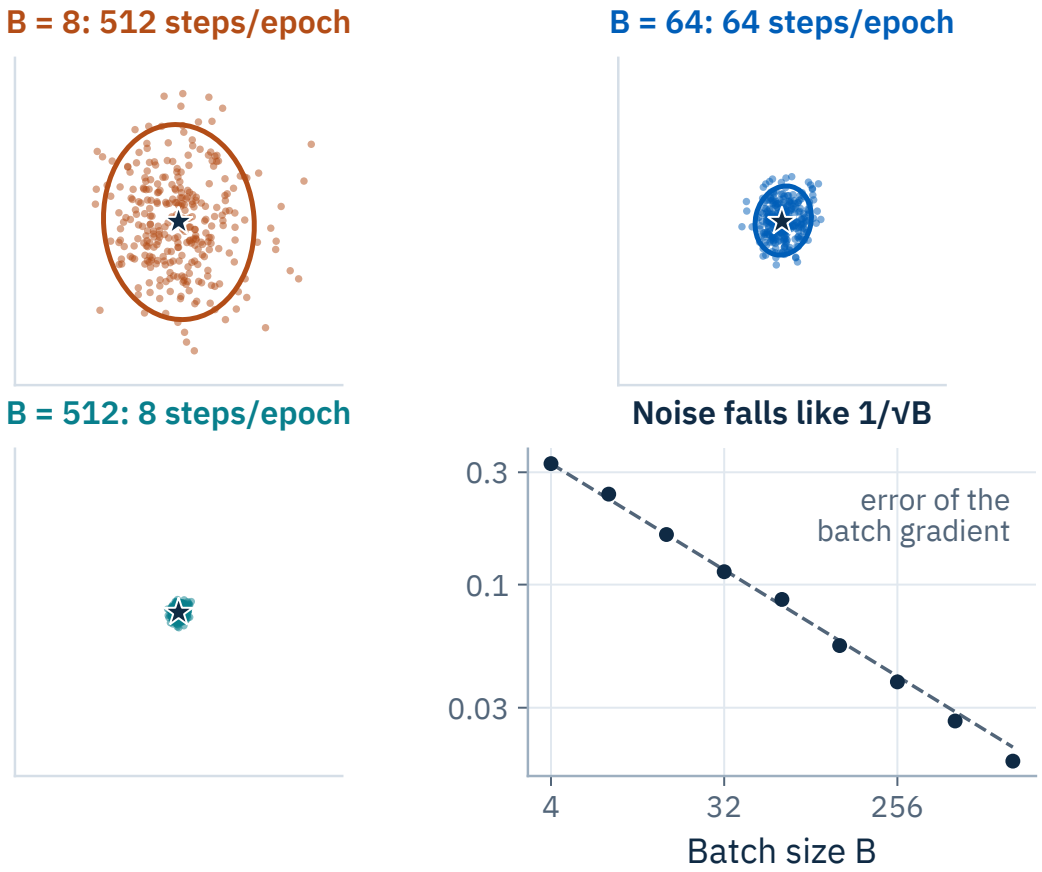
steps = epochs * len(train_loader)
warm = steps // 10
warmup = S.LinearLR(optimizer, start_factor=0.01, total_iters=warm)
cosine = S.CosineAnnealingLR(optimizer, T_max=steps - warm)
sched = S.SequentialLR(optimizer, [warmup, cosine], milestones=[warm])

for epoch in range(epochs):
    for xb, yb in train_loader:
        ...                              # zero_grad, forward, backward
        optimizer.step()
        sched.step()                     # once per batch: the clock
```

`ReduceLROnPlateau` is the exception: call `sched.step(val_loss)` once per epoch.

Batch size changes both compute and learning

- Larger batches may improve hardware throughput
- Smaller batches add gradient noise and more updates per epoch
- Memory depends on activations as well as parameters
- Compare update counts or compute budgets, not epochs alone



Computed: logistic-regression gradients at one fixed point, 4,096 synthetic rows (numpy seed 0), 300 random mini-batches per size.

Bigger batches mean fewer updates per epoch

If all 3,243 rows of the Day 3 ESP training file were used for training, one epoch would give:

203

updates at batch size 16

51

updates at batch size 64

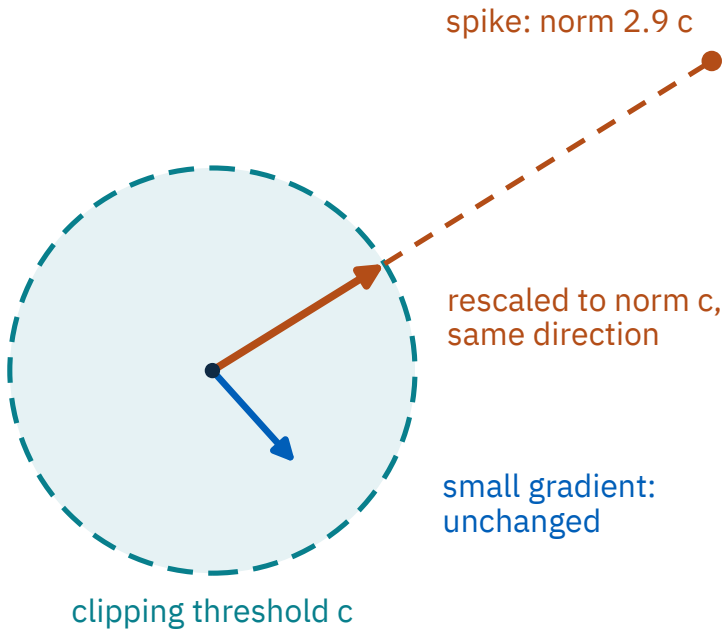
13

updates at batch size 256

- Doubling the batch halves the updates per epoch; compare runs at matched updates or compute.

Gradient clipping caps rare, oversized steps

- Rescales the whole gradient when its global norm exceeds c
- Direction is kept; only the step length is capped
- Call after `backward()`, before `optimizer.step()`
- Log how often it triggers; constant clipping hides a bad rate



$$g \leftarrow g \cdot \min(1,\ c / \|g\|)$$

one global norm over all parameters

Schematic of gradient-norm clipping in two dimensions.

PRACTICE

6 min · Pairs

Plan a fair optimizer comparison

You will compare SGD with momentum against AdamW on the ESP table (3,243 training rows).

1. With batch size 32 for 50 epochs, how many optimizer updates is that?
2. A colleague switches to batch size 64 but keeps 50 epochs. What changed besides the batch?
3. What must be tuned separately for each optimizer before comparing them?

Tune the settings that matter most first

1 · Learning rate and schedule

- Search on a log scale first
- Often the most influential setting

2 · Batch size with the rate

- Change them together
- Match the update budget

3 · Regularization strength

- Weight decay, dropout, patience
- Tune once the rate is sane

4 · Width and depth

- Only if training loss stays high
- Grow in steps of about 2×

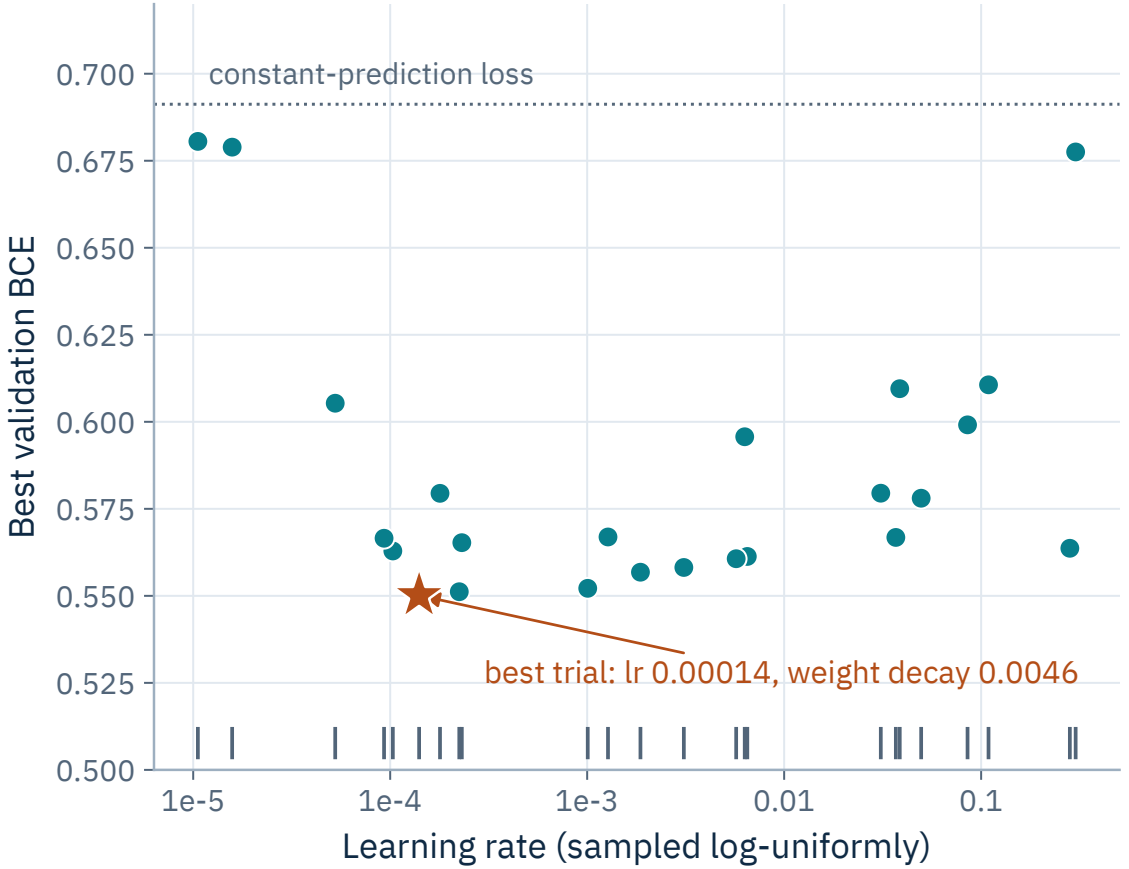
5 · Everything else

- Optimizer betas, epsilon, init details
- Usually fine at defaults

◆ Search strategies (grid, random, successive halving, Bayesian) are covered in ML Module 5.

Sample learning rates on a log scale

- Draw log10 of the rate uniformly: each decade gets equal attention
- A wide first pass finds the usable window; then narrow it
- Losses within about 0.01 may be seed noise: repeat the best few
- Keep split, budget, and seed policy fixed across trials



Computed: 24 random trials of learning rate and weight decay (numpy seed 7) on the synthetic ESP table; best validation loss with patience 15.

PRACTICE

6 min · Groups

A small search can answer a useful question

You can afford twelve training runs on the Lab 1 Sonar pipeline.

1. Define the comparison you want to learn from those runs.
2. Choose at most two settings to vary.
3. Specify what remains fixed and how you will select the final checkpoint.

DISCUSSION

A twelve-run plan that answers one question

Runs	What varies	Seeds	Decision rule
1–4	learning rate 3e-4, 1e-3, 3e-3, 1e-2	0	keep the best two
5–12	the best two rates	1–4 each	compare mean and spread
after	final assessment of the chosen setting	—	once, no changes

Fixed throughout: split, width 32, dropout 0.2, weight decay 1e-3, batch 16, 80 epochs, patience 10.

PART 5 OF 5

05

Regularization and autoencoders

Constrain what a network can memorize; then learn without labels.

- Weight decay, dropout, and early stopping
- Augmentation for tabular and sensor data
- Label smoothing and the double-descent curve
- Autoencoders and reconstruction scores

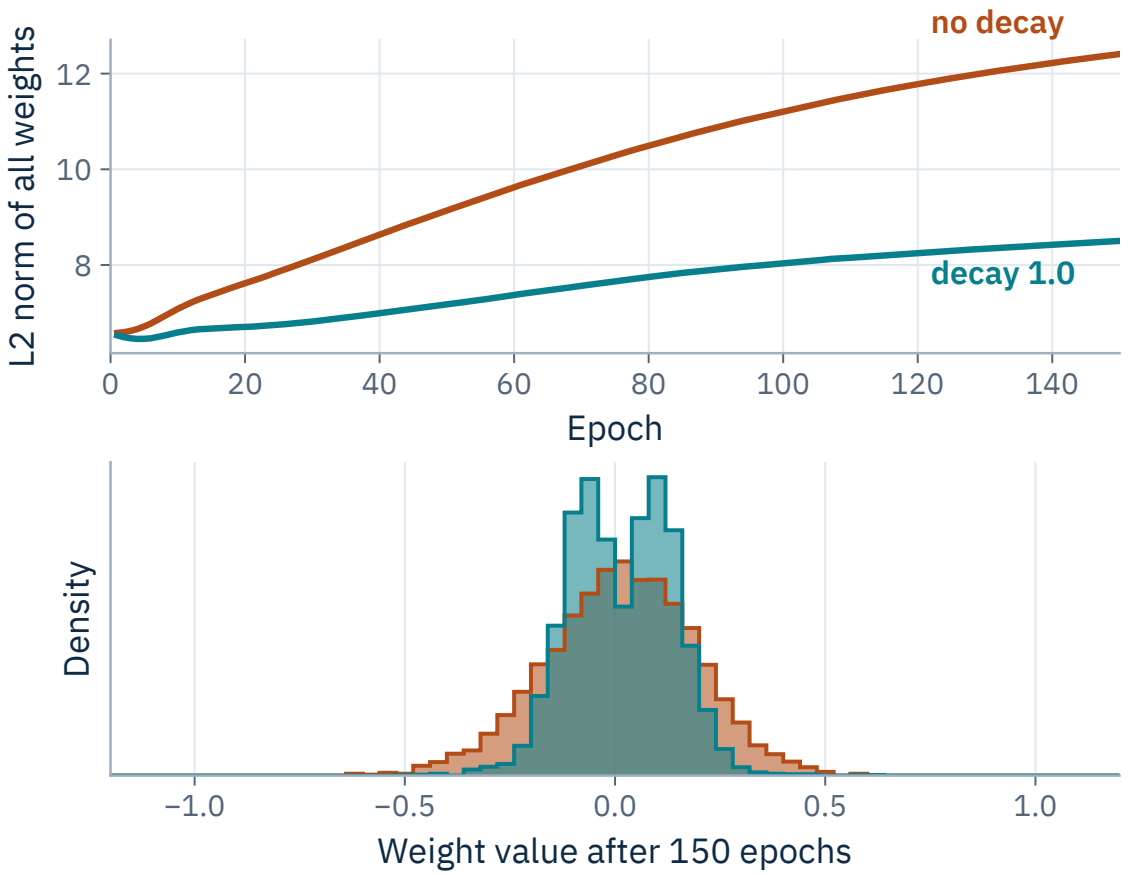
Regularization controls different mechanisms

Technique	Mechanism	What to tune
Weight penalty / decay	Discourages large parameter values	Penalty strength
Dropout	Perturbs hidden activations	Drop probability
Early stopping	Chooses a checkpoint before validation worsens	Patience and monitored metric
Data augmentation	Adds valid training variations	Label-preserving transforms
Label smoothing	Softens one-hot targets	Smoothing amount ε

Weight decay pulls every weight toward zero

$$\tilde{L} = L + \tfrac{\lambda}{2}\|w\|^2 \quad \Rightarrow \quad \nabla\tilde{L} = \nabla L + \lambda w$$

- The penalty adds λ *w* to each gradient; AdamW shrinks *w* directly
- Smaller weights make smoother functions of the inputs
- Here decay 1.0 limited the weight norm to 8.5 instead of 12.4

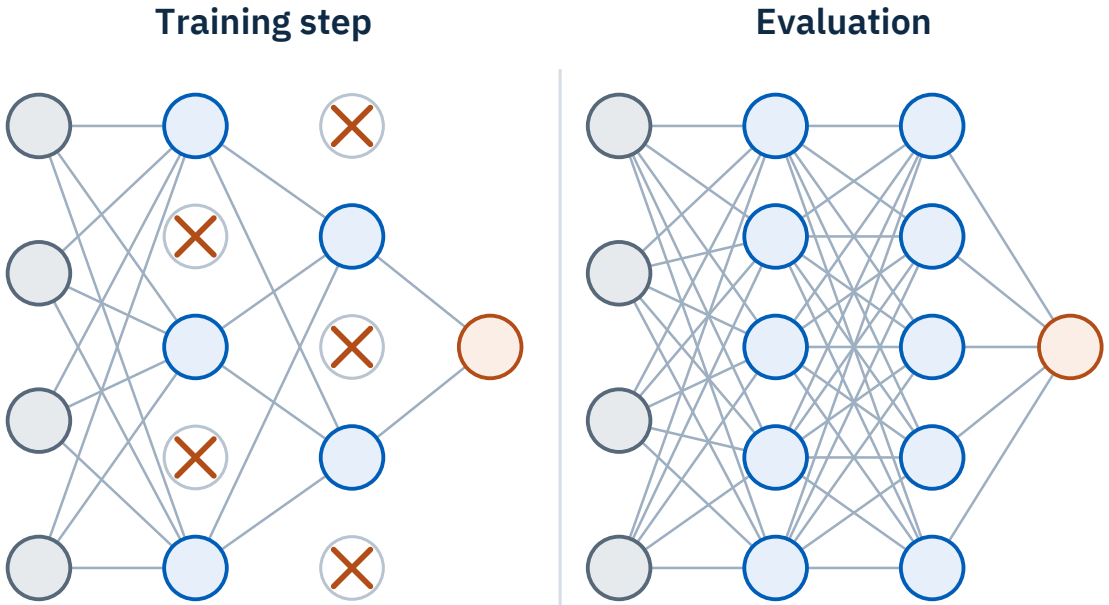


Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW lr 1e-3 with weight_decay 0 or 1.0, 150 epochs, torch seed 0.

Dropout changes training-time activations

$$\tilde{a} = \frac{m \odot a}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p)$$

- During training, each activation is kept with probability 1–*p*
- Inverted dropout rescales kept activations to preserve their expectation
- During evaluation, dropout is disabled

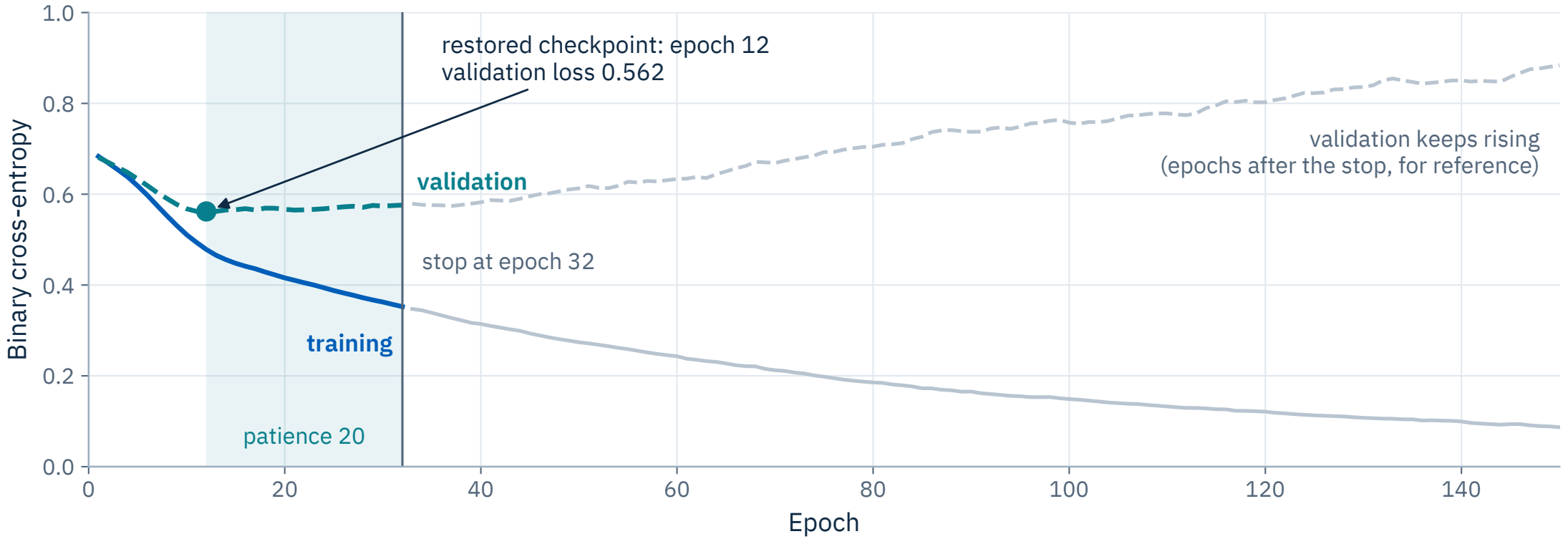


a new random mask is drawn for every mini-batch

Schematic of inverted dropout with drop probability p = 0.5.

Early stopping keeps the best validation checkpoint

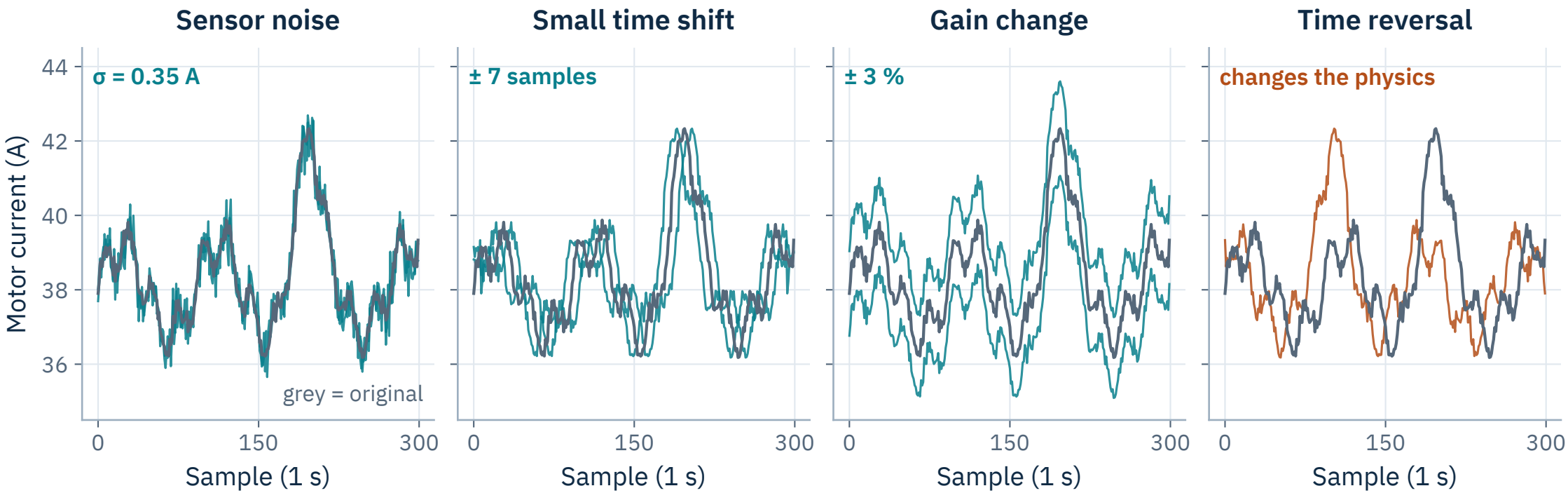
The patience rule decides when to stop; the copied checkpoint decides what you keep.



Computed: synthetic ESP table (seeds 1 and 2), MLP 12-64-64-1, AdamW (weight decay 0.1, dropout 0.2), torch seed 0; patience 20 applied to the recorded curve; grey = epochs after the stop.

Sensor augmentation must preserve the label

Add only variation the real sensors could produce under the same operating condition.



Computed: synthetic ESP motor-current trace (numpy seeds 4 and 1); noise level, shift, and gain are illustrative, not sensor specifications.

Augment training rows only, within physical limits

✓ Reasonable

- Noise at the sensor's stated precision
- Small shifts that keep the event in the window
- Gain changes within calibration tolerance
- Dropping a channel, if real outages occur

vs

✗ Risky

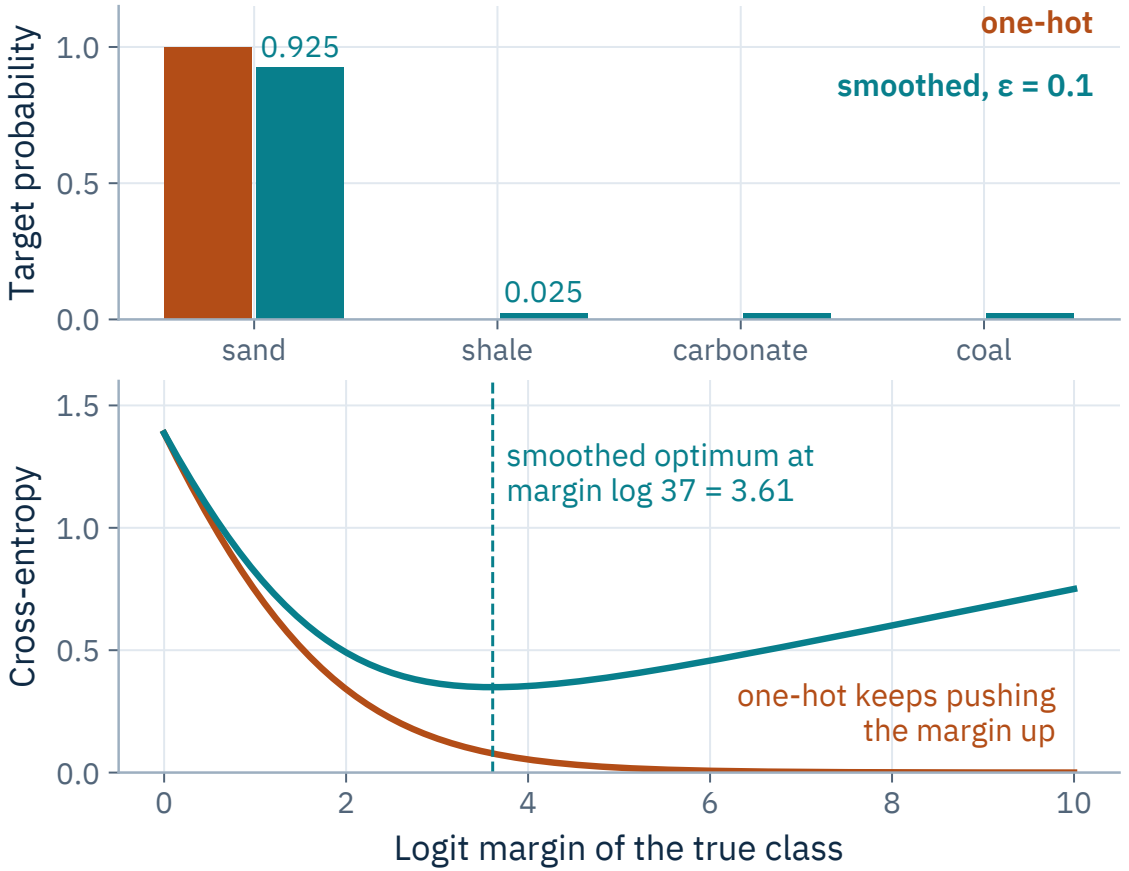
- Augmenting before the split: copies leak
- Values outside physical limits
- Time reversal; mixing wells or regimes
- Augmenting validation or test rows

◆ Augmented copies of a row must stay in the same fold as the original row.

Label smoothing softens overconfident targets

$$y_k^{\mathrm{LS}} = (1-\varepsilon)\, y_k + \frac{\varepsilon}{K}$$

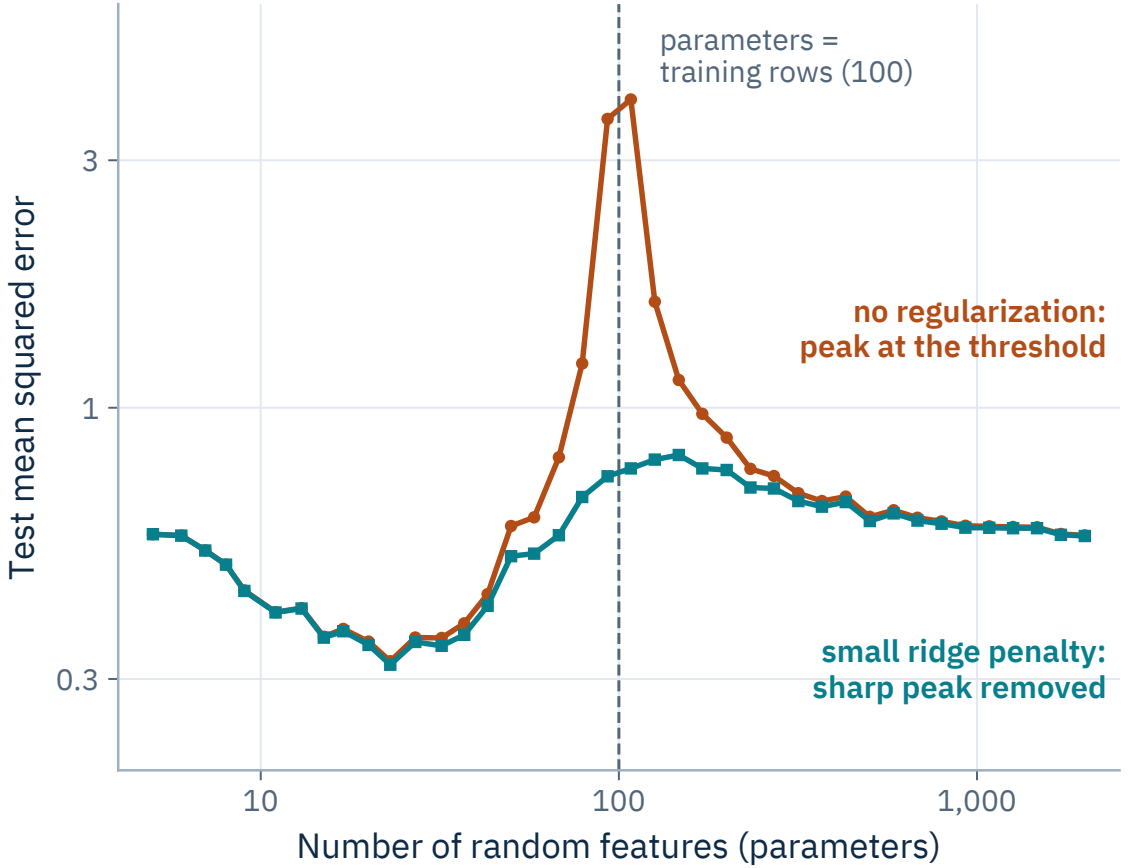
- For K = 4 and ε = 0.1 the targets become 0.925 and 0.025
- The loss no longer rewards ever-larger logits
- PyTorch: `label_smoothing=0.1` in the loss
- Check calibration on validation data; probabilities shift



Exact targets and cross-entropy for four classes with smoothing 0.1; other logits held equal.

Test error can peak where parameters equal rows

- Classical U-curve: error falls, then rises with capacity
- At the threshold, the fit is forced through every noisy row
- Beyond it, error can fall again; a small penalty removes the spike
- Shown for random-feature regression; not a guarantee for your model



Computed: random ReLU features fitted by minimum-norm least squares (orange) or ridge 0.1 (teal) to 100 noisy synthetic rows; numpy seed 0.

PRACTICE

5 min · Pairs

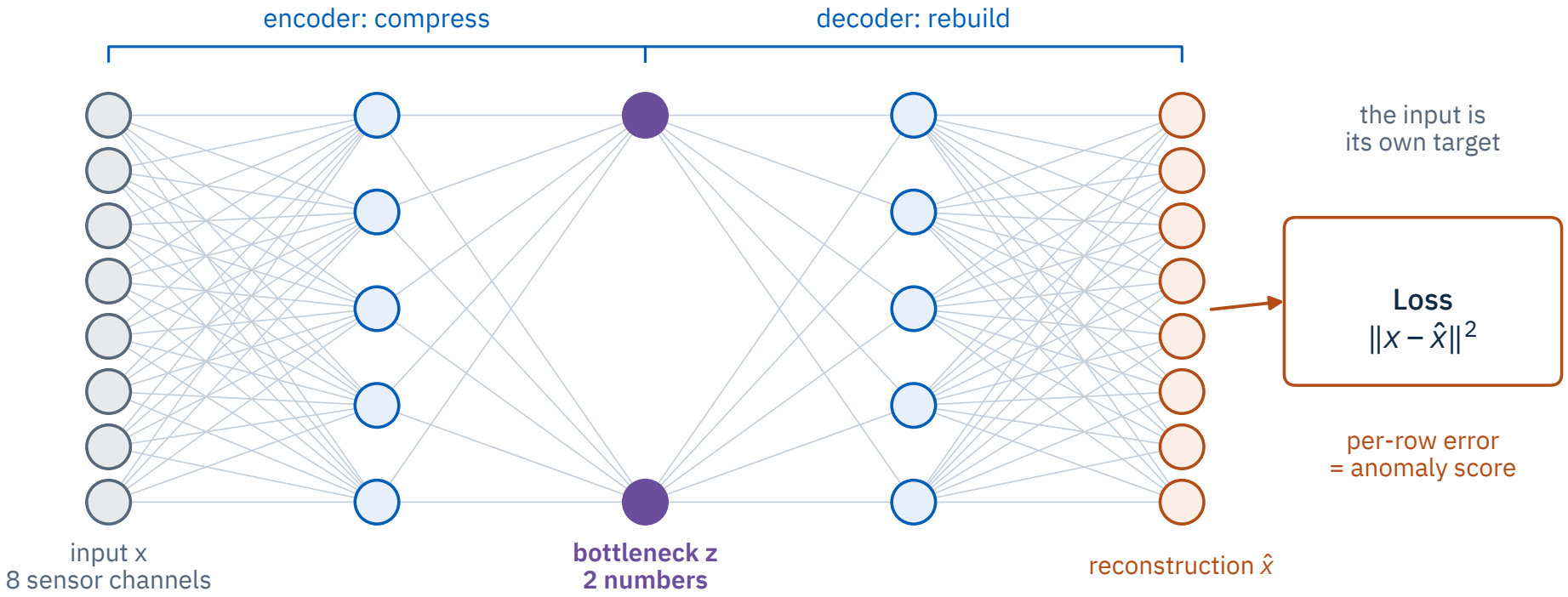
Match the regularizer to the symptom

For each case, choose a first remedy and the evidence you would check afterwards.

1. Training loss 0.05, validation loss rising after epoch 15, only 300 rows.
2. Validation accuracy is fine, but wrong predictions come with probabilities of 0.999.
3. A model trained on one field fails on sensors with a 2% gain offset.

An autoencoder squeezes data through a bottleneck

Trained to reproduce its own input, the network must keep only the dominant structure of the data.



Schematic of a dense autoencoder (8-5-2-5-8); the loss compares the reconstruction with the input.

An autoencoder is an encoder, a decoder, and an MSE loss

```
class AE(nn.Module):
    def __init__(self, n_in=8, k=2):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(n_in, 16), nn.Tanh(), nn.Linear(16, k))
        self.decoder = nn.Sequential(
            nn.Linear(k, 16), nn.Tanh(), nn.Linear(16, n_in))

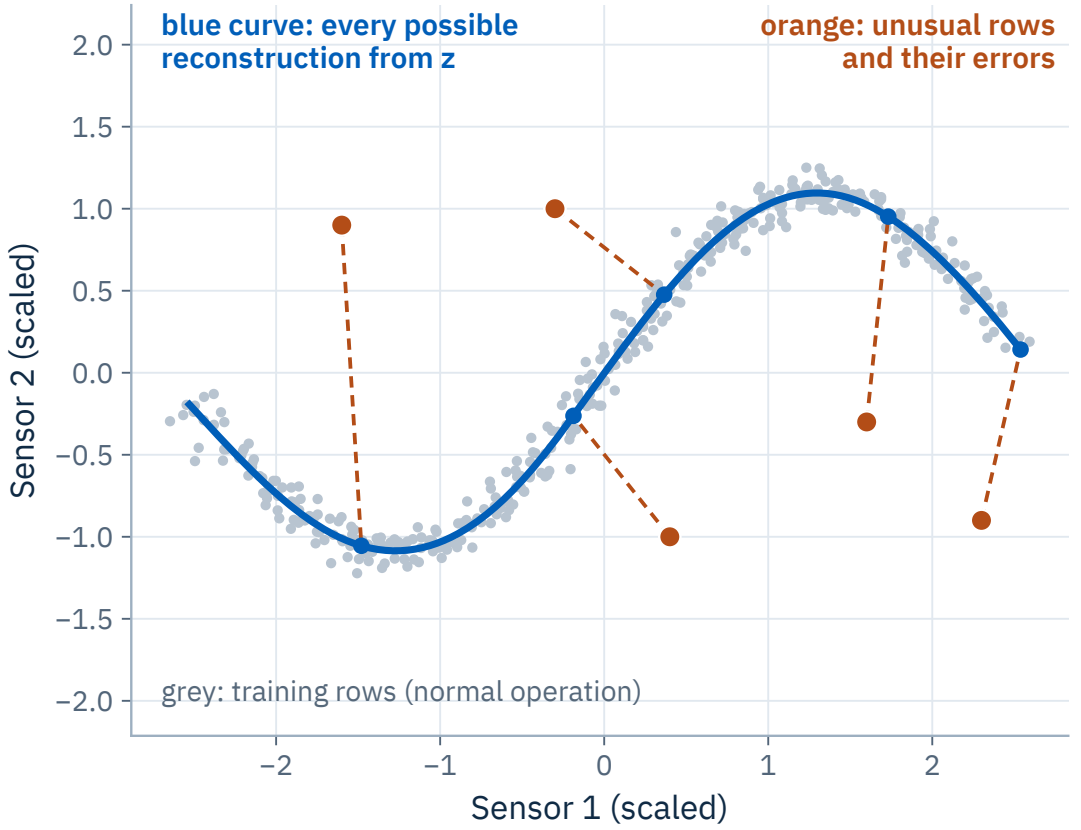
    def forward(self, x):
        return self.decoder(self.encoder(x))

model = AE()
loss = nn.functional.mse_loss(model(xb), xb)    # target = input
score = ((model(x) - x) ** 2).mean(dim=1)       # per-row error
```

Train on standardized rows from normal operation only; score new rows in `eval()` mode.

The bottleneck learns the shape of normal data

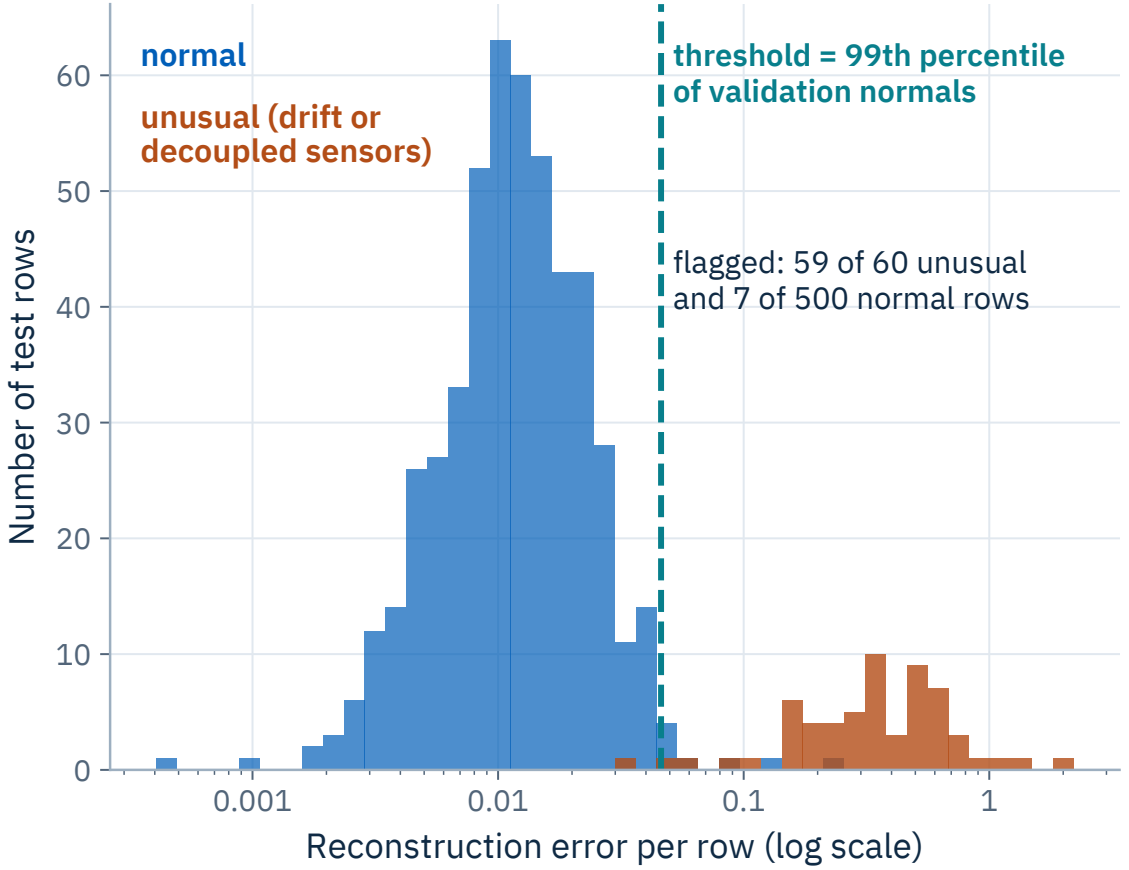
- With a one-number bottleneck, every output lies on one learned curve
- Normal rows sit near the curve and reconstruct well
- Unusual rows are pulled onto the curve: their error is large



Computed: 400 synthetic two-sensor rows near a curve (numpy seed 3); autoencoder 2-32-32-1-32-32-2 (tanh), torch seed 0; five hand-placed unusual rows.

Reconstruction error becomes an anomaly score

- Train on normal operation; score each row by its mean squared error
- Set the threshold on validation normals, for example their 99th percentile
- Here 59 of 60 unusual rows and 7 of 500 normal rows were flagged
- Evaluate events with precision-recall (ML Module 5)



Computed: synthetic 8-channel sensor rows driven by two hidden factors (numpy seed 11); autoencoder 8-16-2-16-8 trained on 2,000 normal rows, torch seed 0.

Reconstruction scores need careful interpretation

Threshold choice

- A percentile sets the alarm rate, not accuracy
- Choose it on validation, report on test

Too much capacity

- A wide bottleneck can reconstruct anomalies too
- Compare with a PCA reconstruction baseline

Contaminated training

- Unlabelled faults in "normal" data raise the bar
- Audit the training window first

Where next

- ML Module 4: autoencoders versus PCA
- Days 4 and 5: convolutional and sequence versions

LABORATORY

55 min

Laboratory 2: regularization or autoencoder

- Track A: compare baseline, dropout, batch-norm, and weight-decay MLPs
- Track A: hold out 10,000 training images for validation; test once
- Track B (04_nn_unsupervised): autoencoder with a one-number bottleneck
- Track B: set the threshold on validation rows, then inspect flagged points
- Both: change one thing at a time and record the budget and seeds

NOTEBOOK

`exercises/03_advanced_nn/Advanced_NN_MNIST_Student.ipynb`

EVIDENCE TO SUBMIT

- A: table of validation accuracy and train-validation gap
- B: error histogram, threshold, flagged points
- One sentence on what the result cannot show

Track A needs torchvision and MNIST downloaded before class; Track B runs offline.

DISCUSSION

Lab 2 debrief: evidence over impressions

Track A

- Was validation separate from test?
- Is the gap larger than seed-to-seed noise?

Track B

- Was the threshold fitted on validation rows?
- Where do the flagged points lie, and why?

Both tracks

- Budget and seeds reported
- Claims limited to this data

◆ Every claim carries its budget, its seeds, and its limits.

Key takeaways

- ✓ Diagnose in a fixed order: data, initial loss, one batch, gradients, validation.
- ✓ Initialization, normalization, and skip paths keep signals at a trainable scale.
- ✓ `eval()` and no-grad do different jobs; copy the best checkpoint and verify a reload.
- ✓ Learning rate first: range test, log-scale search, and a schedule on a declared clock.
- ✓ Regularize for the symptom you see, and validate every change over several seeds.
- ✓ Reconstruction error is a score; its threshold is a decision made on validation data.

PRACTICE

10 min · Individual

Exit check

1. What is the difference between `model.eval()` and `torch.no_grad()`?
2. Why does early stopping use validation rather than test data?
3. Why can adding L2 to Adam differ from AdamW weight decay?
4. Your initial loss on a balanced 5-class problem is 9.8. What do you check first?
5. Name one augmentation that would be invalid for your own sensor data, and why.

References and further study

- Goodfellow, Bengio, and Courville (2016), *Deep Learning*, chapters 7, 8, and 11; Karpathy (2019), "A recipe for training neural networks".
- Glorot and Bengio (2010) on initialization; He et al. (2015) on rectifier initialization; He et al. (2016), deep residual learning.
- Ioffe and Szegedy (2015), batch normalization; Ba, Kiros, and Hinton (2016), layer normalization; Wu and He (2018), group normalization.
- Kingma and Ba (2015), Adam; Loshchilov and Hutter (2019), AdamW; Smith (2017), cyclical learning rates; Srivastava et al. (2014), dropout.
- Bergstra and Bengio (2012), random search; Belkin et al. (2019), double descent; Hinton and Salakhutdinov (2006), autoencoders; Google Research, *Deep Learning Tuning Playbook*; PyTorch documentation.

NEURAL NETWORKS / DAY 3

Energy modelling capstone

A reproducible baseline, a neural candidate, and a defensible conclusion

Full day · 09:00–17:00 | 3 supplied projects | 4 milestone checkpoints | 5-minute team presentations

Today's plan

09:00	Welcome and the brief	Lecture
09:35	ESP gas lock briefing	Lecture
09:55	PE log curve briefing	Lecture
10:15	Break	Break
10:30	Sensor briefing	Lecture
10:45	Sprint 1: data contract	Project work
11:20	Checkpoint 1	Check
11:30	Tabular-NN playbook	Lecture
11:50	Sprint 2: baselines	Project work
12:30	Checkpoint 2	Check
12:40	Lunch	Lunch
13:25	Experiment design	Lecture
13:40	Sprint 3: experiment	Project work
14:20	Checkpoint 3	Check
14:30	Error analysis	Lecture
14:45	Sprint 4: final test	Project work
15:25	Checkpoint 4	Check
15:35	Break	Break
15:45	Presenting evidence	Lecture
16:00	Sprint 5: handoff	Project work
16:15	Final presentations	Presentation
16:50	Takeaways, exit check	Discussion

LEARNING OUTCOMES

What you will be able to do

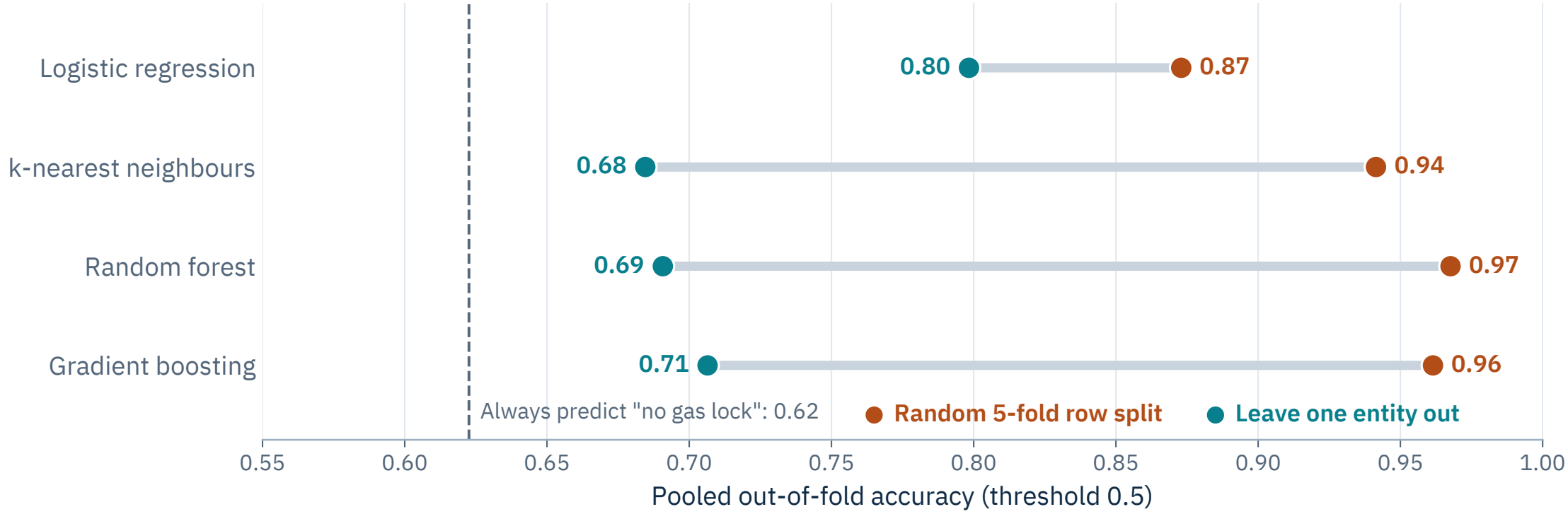
By 17:00 you will be able to:

1. Write a data contract and an audit for a supplied energy dataset.
2. Choose a validation design that matches what the data can establish.
3. Compare a baseline and a small neural network on identical rows and preprocessing.
4. Run one controlled experiment, then lock the configuration before the final test.
5. Diagnose errors by group, class and calibration, and state their limits.
6. Present the evidence in five minutes with a run that others can reproduce.

MOTIVATING CASE

Random row splits flattered every ESP model

Same file, same four models, two validation designs. Which number answers "will it work on the next pump"?



Computed from esp_train.csv (3,243 rows, 7 entities): pooled out-of-fold accuracy at threshold 0.5, preprocessing fitted inside each fold.

PART 1 OF 7

01

The capstone brief

Sound evidence, not a target score

- The deliverable and the project lifecycle
- The day plan and four milestone checkpoints
- Three roles for data and a one-page data contract
- An audit checklist applied to all three files

The deliverable is evidence, not a winning model

Define

- The target and the row unit
- What the evaluation can establish

Compare

- A simple baseline first
- One small network on the same split

Reproduce

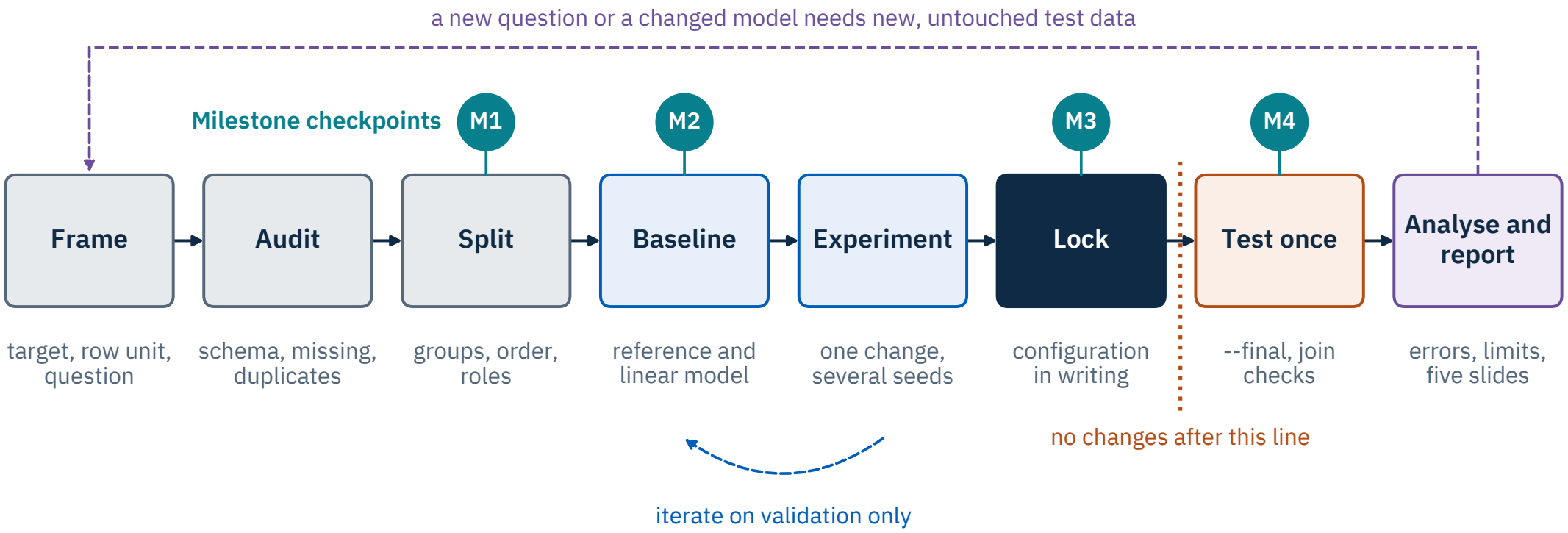
- One command from a fresh process
- Score table and saved artifacts

Present

- Five minutes, five slides
- Limitations stated explicitly

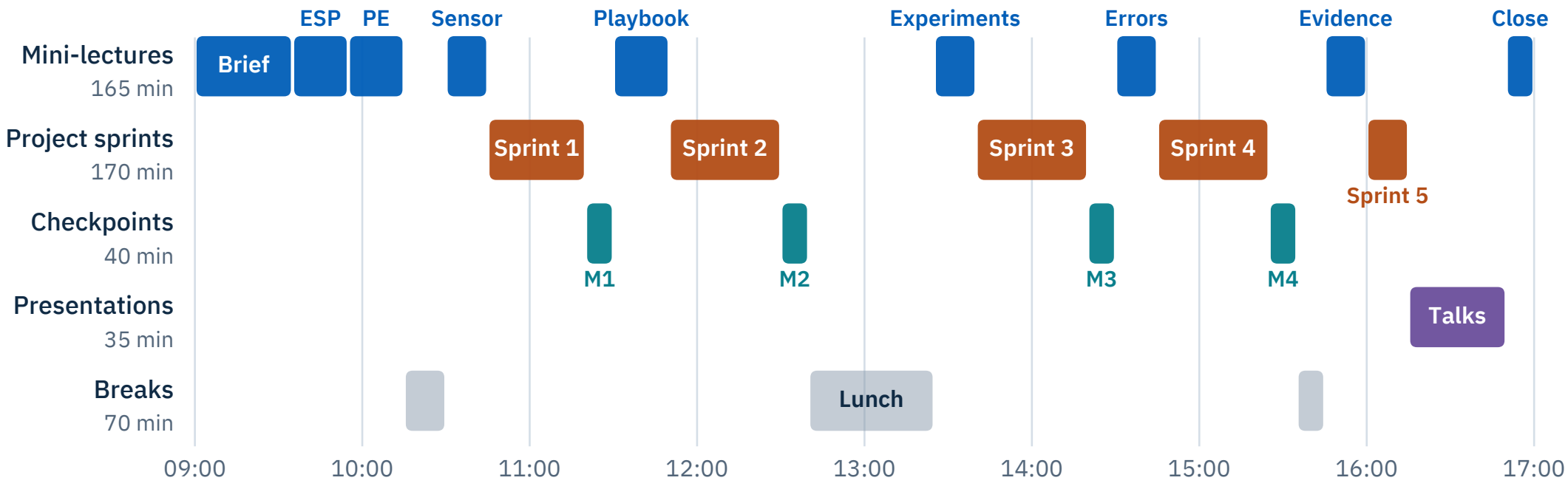
◆ A failed or inconclusive neural model can earn full marks with sound evidence.

Every project follows the same eight-stage lifecycle



Schematic. Teal badges mark the four milestone checkpoints; after the lock only analysis and reporting change.

Short lectures alternate with five project sprints



Schematic of today's plan: 165 min of lectures including the close, 170 min of sprints, four 10-min checkpoints, 35 min of presentations and 70 min of breaks.

Each team chooses one of three supplied projects

Project	Actual target	Supplied training data	Validation fallback
ESP gas lock	GAS_LOCK, values 0 and 1	3,243 rows, 7 entities	Hold out 2 entities
PE log curve	PE, numeric	2,922 rows, 6 input columns	Last 20% of source rows
Sensor signals	target, values −1 and 1	3,580 rows, 10 inputs plus uuid	Stratified 20% of rows

Counts describe the supplied course files.

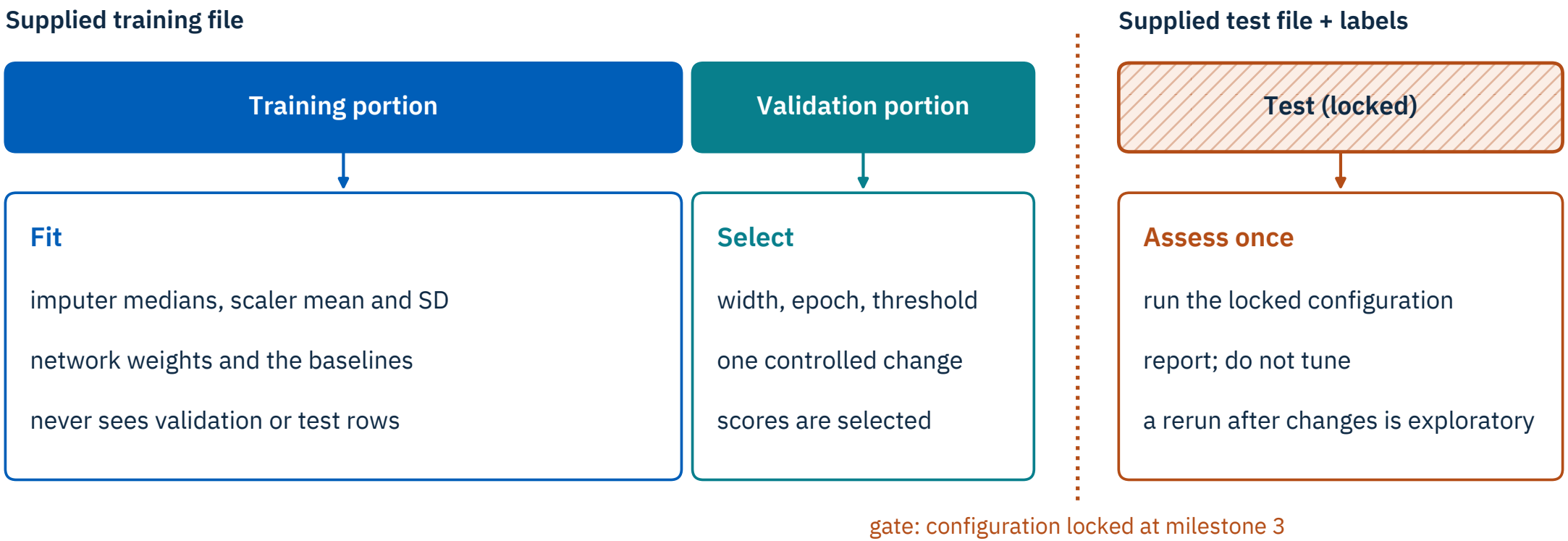
◆ Each fallback split has a stated limit; a team may propose a better split at checkpoint 1.

Four milestones gate the day

Milestone	Due	Evidence to show
M1 Data contract	11:20	Schema, target, audit findings, split proposal
M2 Baseline	12:30	Reproducible validation result and learning curve
M3 Controlled experiment	14:20	Hypothesis, seed spread, locked configuration
M4 Locked final assessment	15:25	Final scores, join checks, one error plot
Presentations	16:15	Five slides, one command, predictions with ids

◆ A team that misses a gate keeps working on it; the gate moves, the evaluation standard does not.

Each portion of the data has one job



Schematic. Validation is carved from the supplied training file; the supplied test file and its labels are opened once, after the configuration is locked at milestone 3.

A data contract fits on one page

Row unit and target

- What one row represents
- Target column and its values

Predictors

- Columns available at prediction time
- Units, or "undocumented"

Excluded metadata

- IDs, keys and timestamps kept out
- The reason for each exclusion

Groups and order

- Entity, depth or record structure
- The split that respects it

Quality

- Missing, duplicate and label counts
- A decision for each finding

Limits

- What the final score cannot show
- Metadata that would remove the limit

◆ Define the split before fitting any preprocessing.

The audit checklist flags issues in every file

	ESP gas lock	PE log curve	Sensor signals
Shape and schema	✓ 3,243 rows × 9 columns	✓ 2,922 rows × 7 columns	✓ 3,580 rows × 12 columns
Target and balance	✓ GAS_LOCK = 1 in 37.7% of rows	✓ PE 0.20–8.09, median 3.51	! labels −1/1 (57.6% = 1): map to 0/1
Missing values	! 300 cells, all in 50 esp_39 rows	✓ none	✓ none
Duplicates	! 2 identical rows; 6 repeat stamps	! 15 rows; a 12-row block repeats	✗ 2,484 rows repeat a feature vector
Keys and joins	✓ uuid unique in test; join on uuid	! no key: labels align by position	✓ uuid unique; labels join on uuid
Groups and order	✓ 7 entities, hourly TimeSlot	✗ no well ID; 7 depth resets	! no groups; uuid order has long runs
Units and meaning	! units undocumented	! units undocumented	! meaning and units undocumented
Test file inputs	✓ 3 unseen entities, same dates	✓ 310 rows, Depth 2964.5–3122.5	✗ 646 of 895 repeat a training row

Computed from the supplied training files and the test input files; no test labels were read. Tick: fine. Gold: needs a documented decision. Orange cross: limits what the evaluation can show.

PRACTICE

4 min · Pairs

Which data may each step use?

For each step, answer: training portion, validation portion, test file, or none.

1. Computing the medians used for imputation
2. Choosing the epoch at which training stops
3. Picking the decision threshold for gas-lock alerts
4. Confirming that test uuids match the label file one-to-one
5. Choosing width 32 over 128 after seeing both test scores

PART 2 OF 7

02

Project 1: ESP gas lock

Classifying labelled operating observations on held-out entities

- The actual schema and label prevalence
- Traces, class distributions and missing rows
- The validation question and a grouped split
- Baseline, candidate and the final join

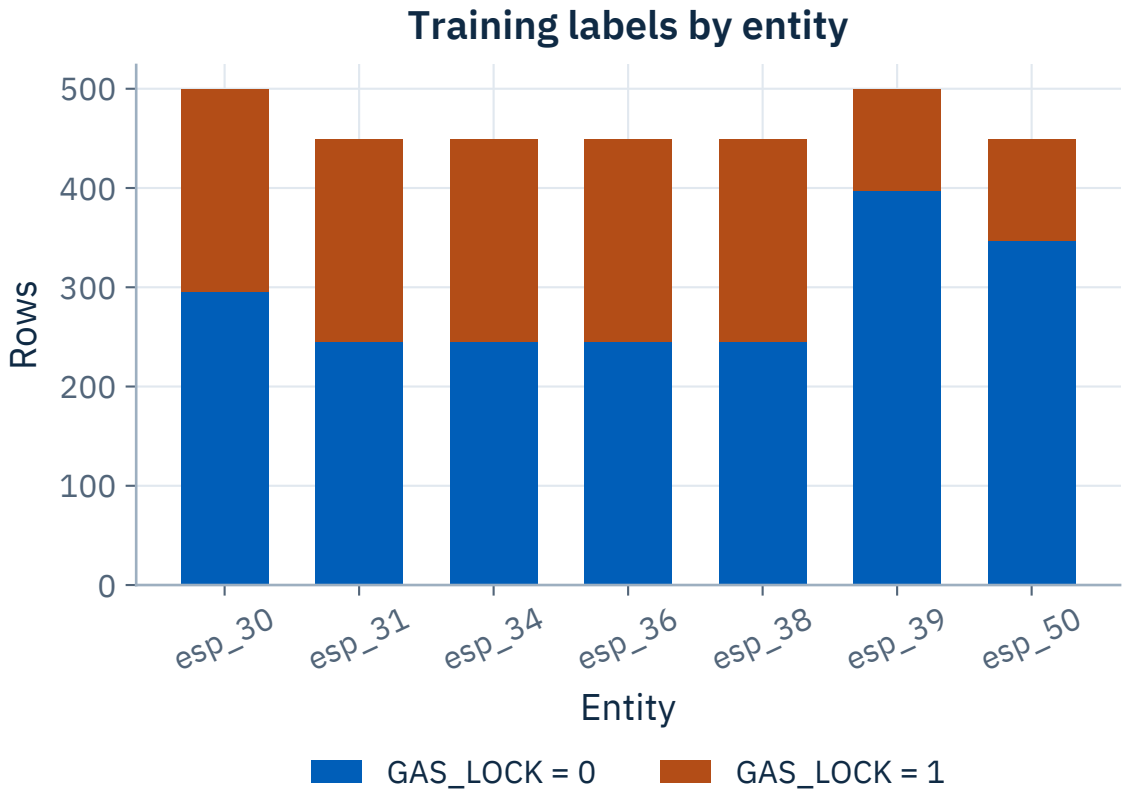
Six sensors, an entity and a timestamp per row

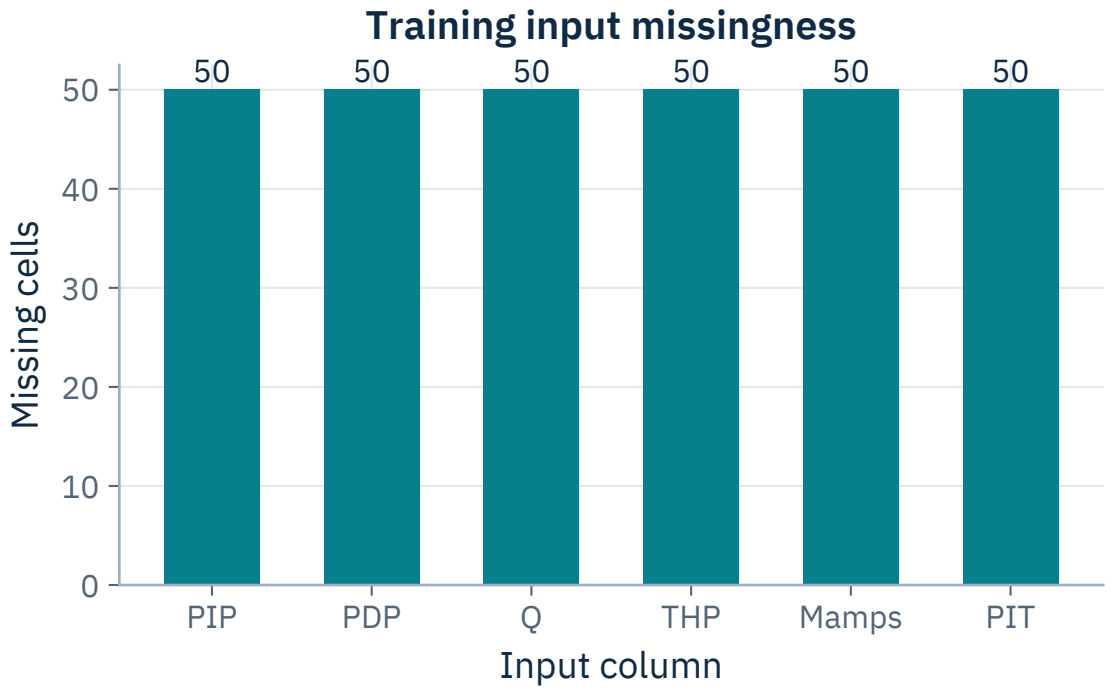
Role	Columns	Treatment
Numeric measurements	PIP, PDP, Q, THP, Mamps, PIT	Check missingness and units
Asset grouping	ENTITY	Hold entities together in validation
Ordering	TimeSlot	Parse timestamps, inspect chronology
Target / test key	GAS_LOCK / uuid	Target in train, join key in test

No vibration channels are present in the supplied files.

◆ ENTITY and TimeSlot define the split and the order; they are not model inputs.

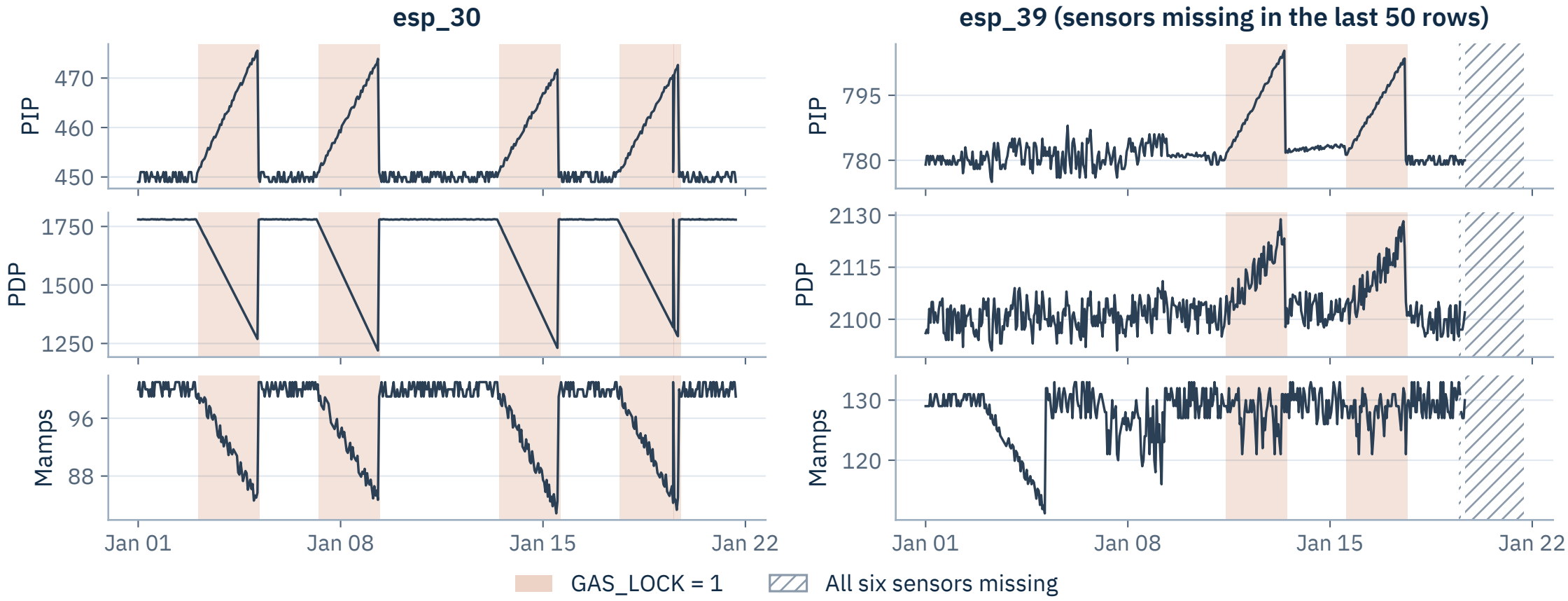
Label prevalence is measured, not assumed





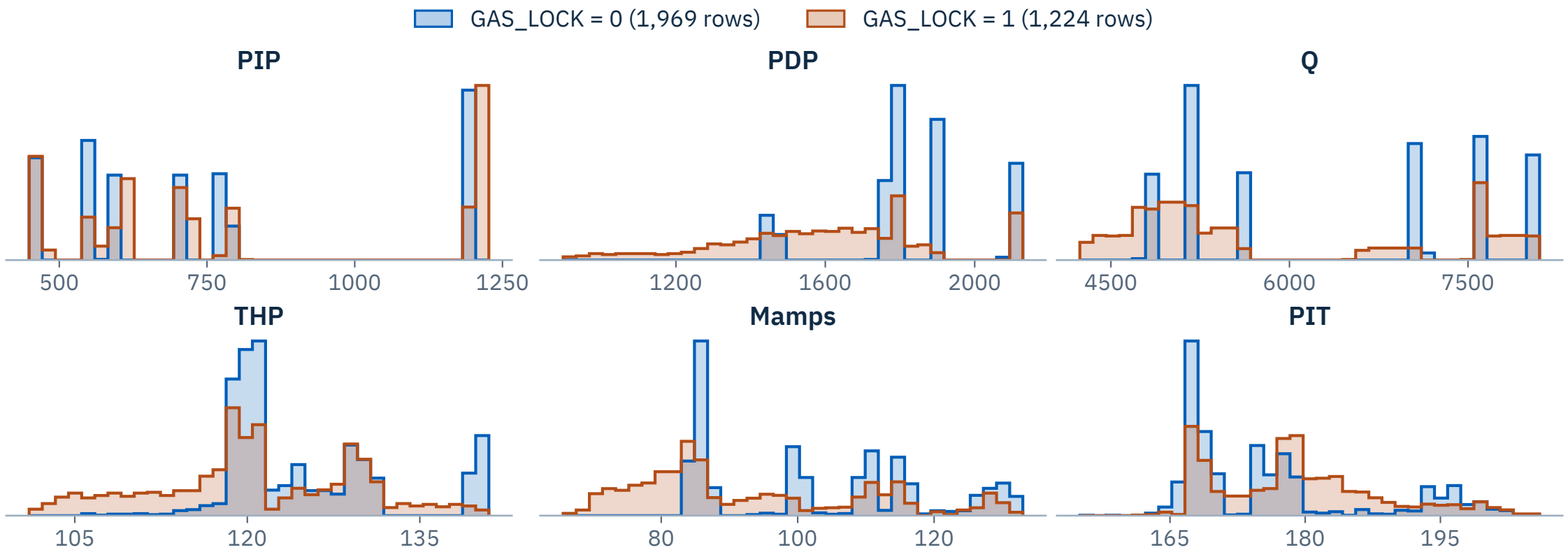
Supplied esp_train.csv. Counts are per row, not independent failure events. Missing sensor values require an explicit training-only treatment.

Gas-lock periods show up as ramps in the traces



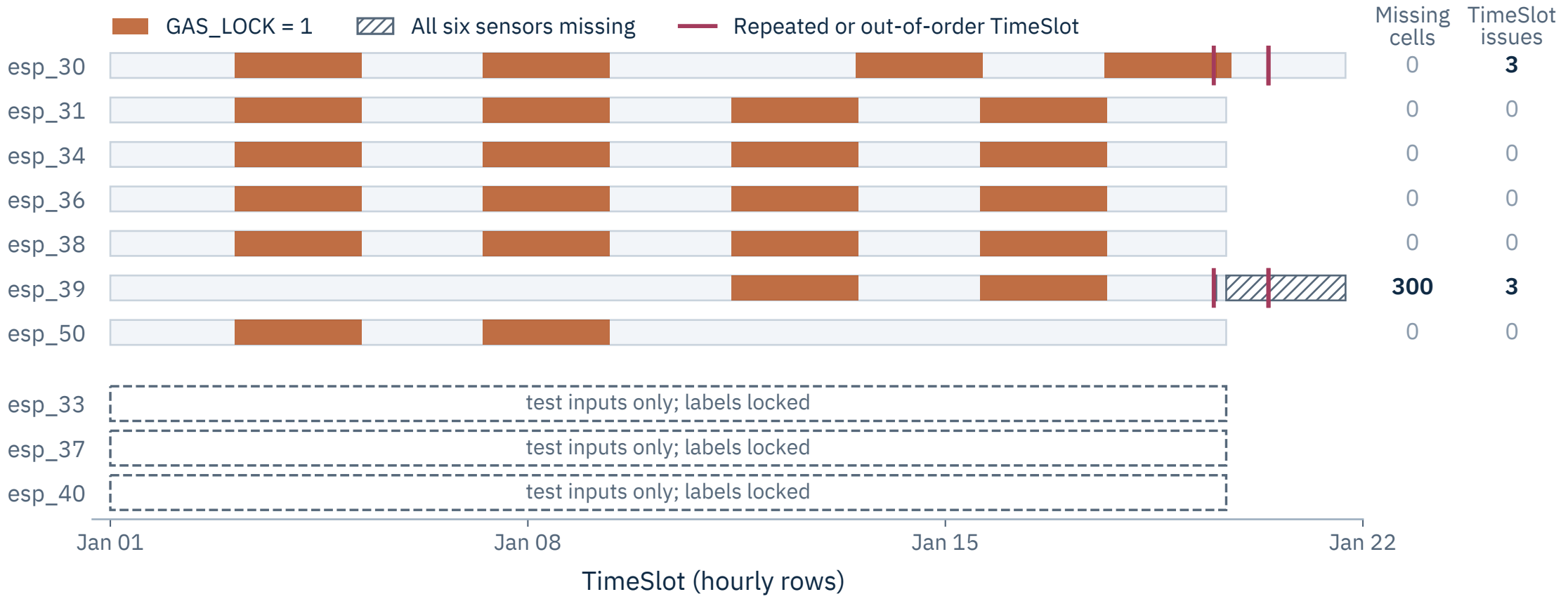
Computed from esp_train.csv: entities esp_30 and esp_39 in TimeSlot order; values as supplied, units undocumented.

Pooled histograms mix seven operating set-points



Computed from esp_train.csv: 3,193 rows with all six sensors present, pooled over seven entities.

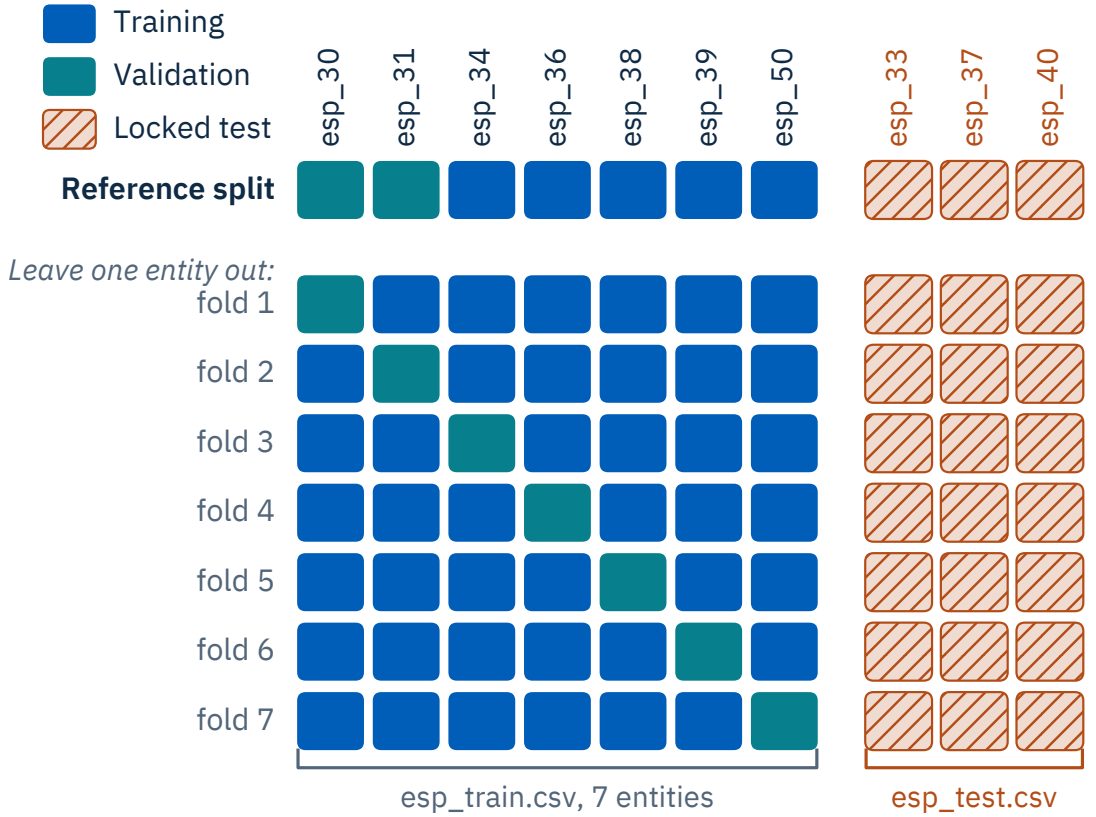
Missing rows and odd timestamps sit in two entities



Computed from esp_train.csv and the ENTITY and TimeSlot columns of esp_test.csv; the test label file was not read.

The ESP question is about unseen entities

- Can the model classify rows from an entity it has never seen?
- Group validation by ENTITY within the seven training entities
- Keep ENTITY and TimeSlot out of the numeric inputs
- Early warning is a separate task that needs a future target



Computed: GroupShuffleSplit(test_size=2, random_state=42) holds out esp_30 and esp_31; leave-one-entity-out repeats the question seven times.

Split by entity before fitting any preprocessing

```python
from sklearn.model_selection import GroupShuffleSplit

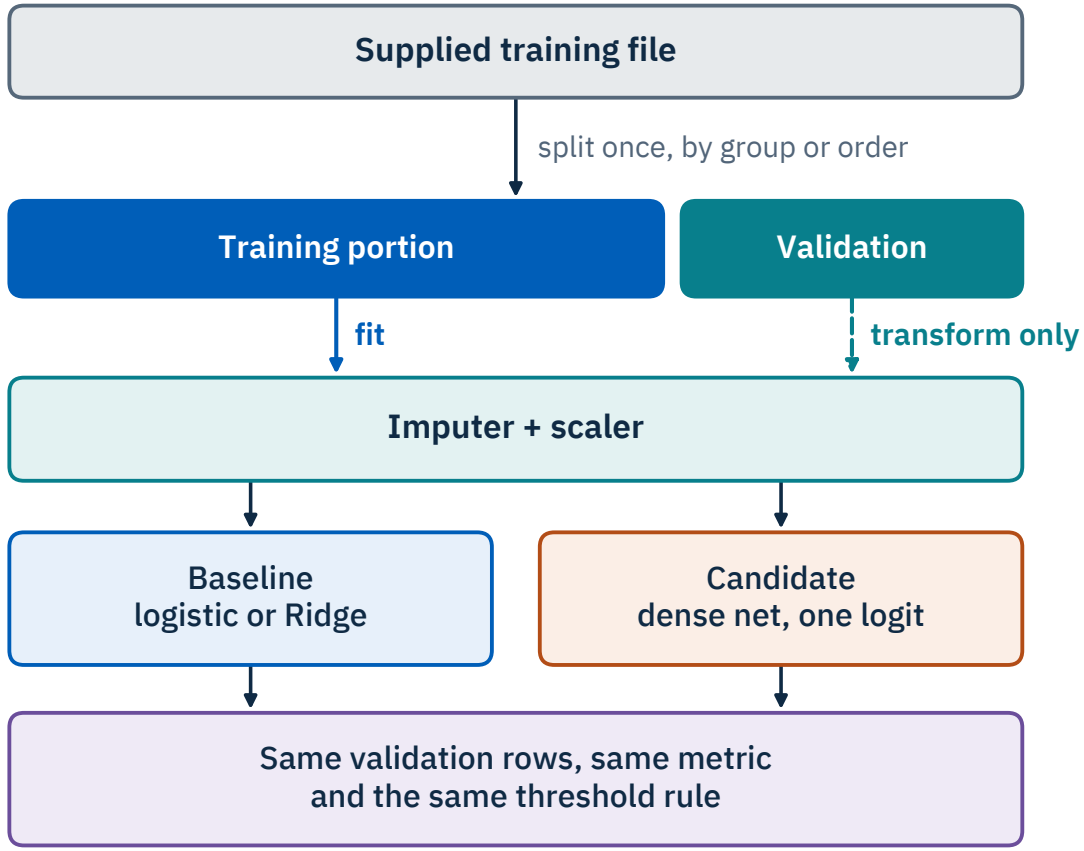
features = ["PIP", "PDP", "Q", "THP", "Mamps", "PIT"]
splitter = GroupShuffleSplit(
    n_splits=1, test_size=2, random_state=42
)
train_idx, val_idx = next(splitter.split(
    train_df[features], train_df["GAS_LOCK"],
    groups=train_df["ENTITY"]
))
val_ents = set(train_df["ENTITY"].iloc[val_idx])
assert not val_ents & set(train_df["ENTITY"].iloc[train_idx])
print(sorted(val_ents), len(train_idx), len(val_idx))
```

OUTPUT

```
['esp_30', 'esp_31'] 2295 948
```

Baseline and candidate share every other choice

- Baseline: median imputation, scaling, logistic regression
- Candidate: one hidden layer and a raw-logit loss
- Report gas-lock precision, recall and per-entity results
- Add average precision as a threshold-free check



Schematic. Split and fitted preprocessing are shared, so only the model differs.

The ESP final join must be checked

```
scored = test_df[["uuid"]].copy()
scored["probability"] = probabilities
scored = scored.merge(
    actual_df, on="uuid", how="left", validate="one_to_one"
)
assert len(scored) == len(test_df)
assert scored["GAS_LOCK"].notna().all()
```

Run this once, at milestone 4, after the configuration is locked.

⚠ A join that leaves missing labels means the files do not correspond: stop, do not drop rows.

PART 3 OF 7

03

Project 2: PE log-curve prediction

Regression with limited grouping metadata

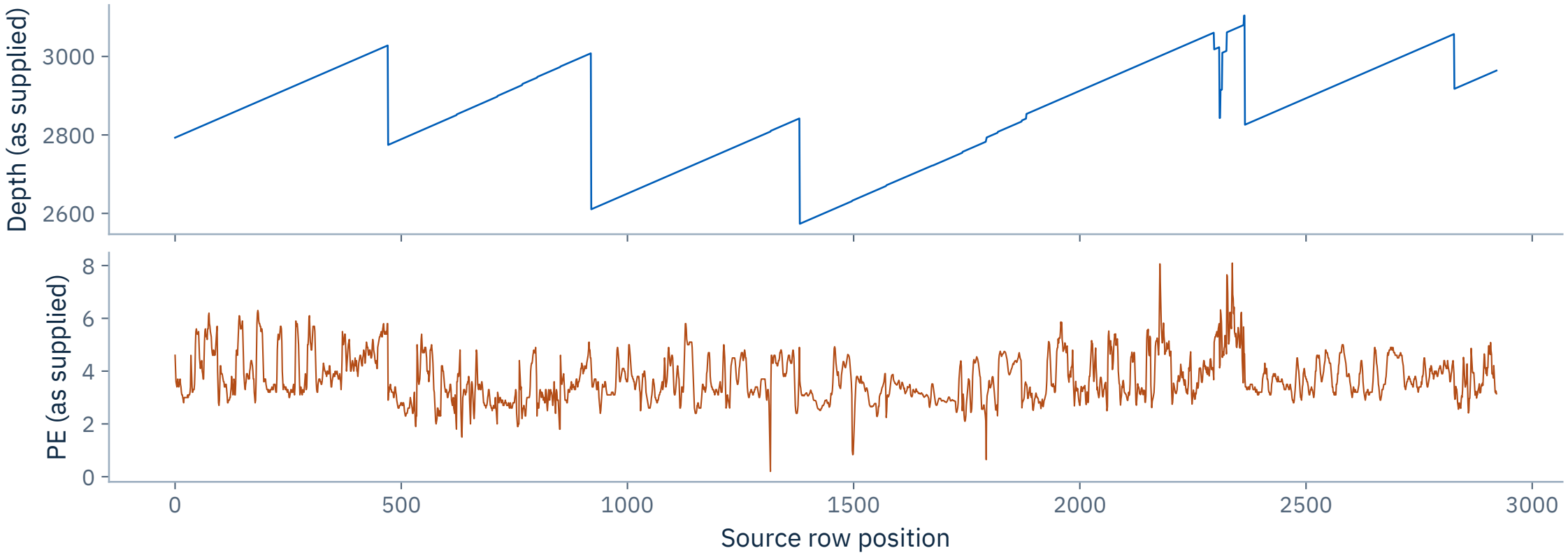
- Schema, depth resets and duplicated rows
- A well-log view of one depth block
- Correlations and the target distribution
- A contiguous-block validation fallback

Six inputs and PE, but no well identifier

Role	Columns	Known limitation
Predictors	Depth, GR, ILD_log10	Units and acquisition grouping need documentation
Predictors	DeltaPHI, PHIND, RELPOS	Use provided definitions when available
Training target	PE	Numeric regression target
Supplied evaluation	310 input rows and 310 PE labels	Label alignment has no explicit key

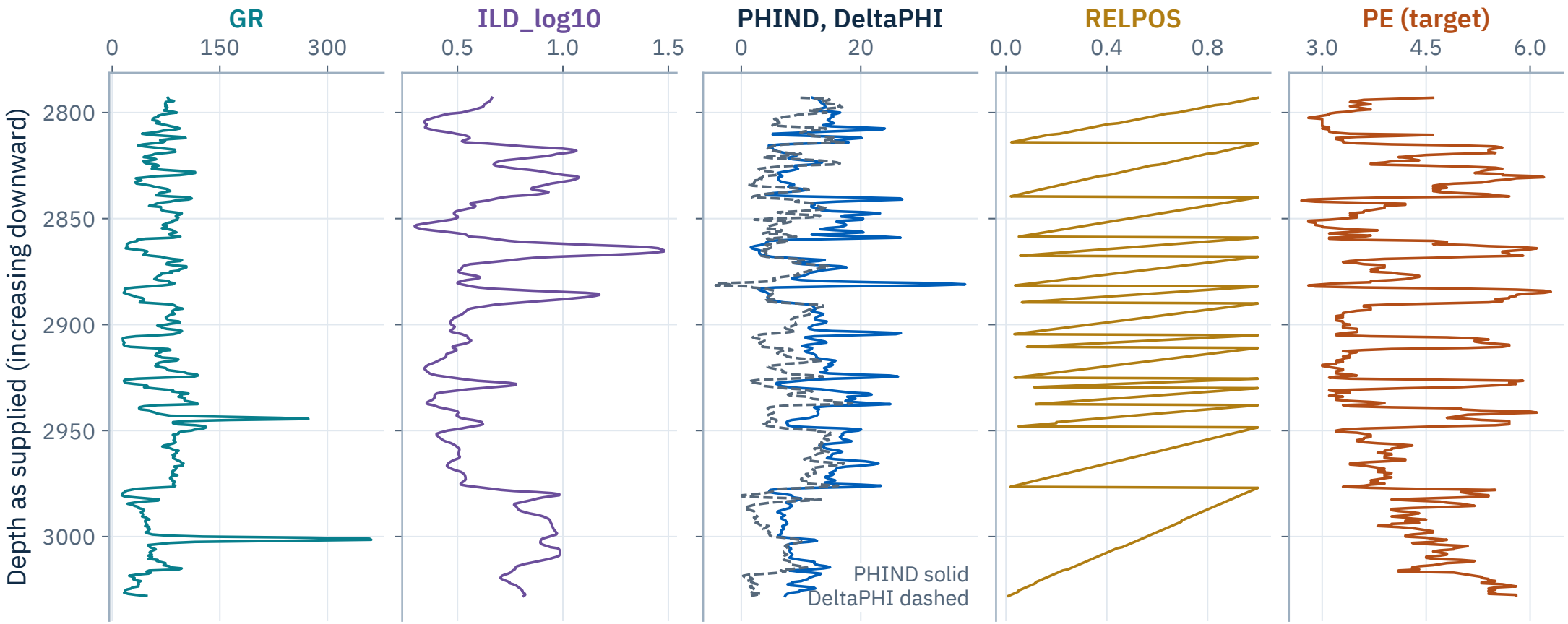
- Depth is supplied as a column, not proven to be measured depth in a single well.

Depth resets split the file into eight blocks



Supplied log training data, plotted in source-row order. The file lacks well identifiers, so changes in depth cannot alone establish well boundaries.

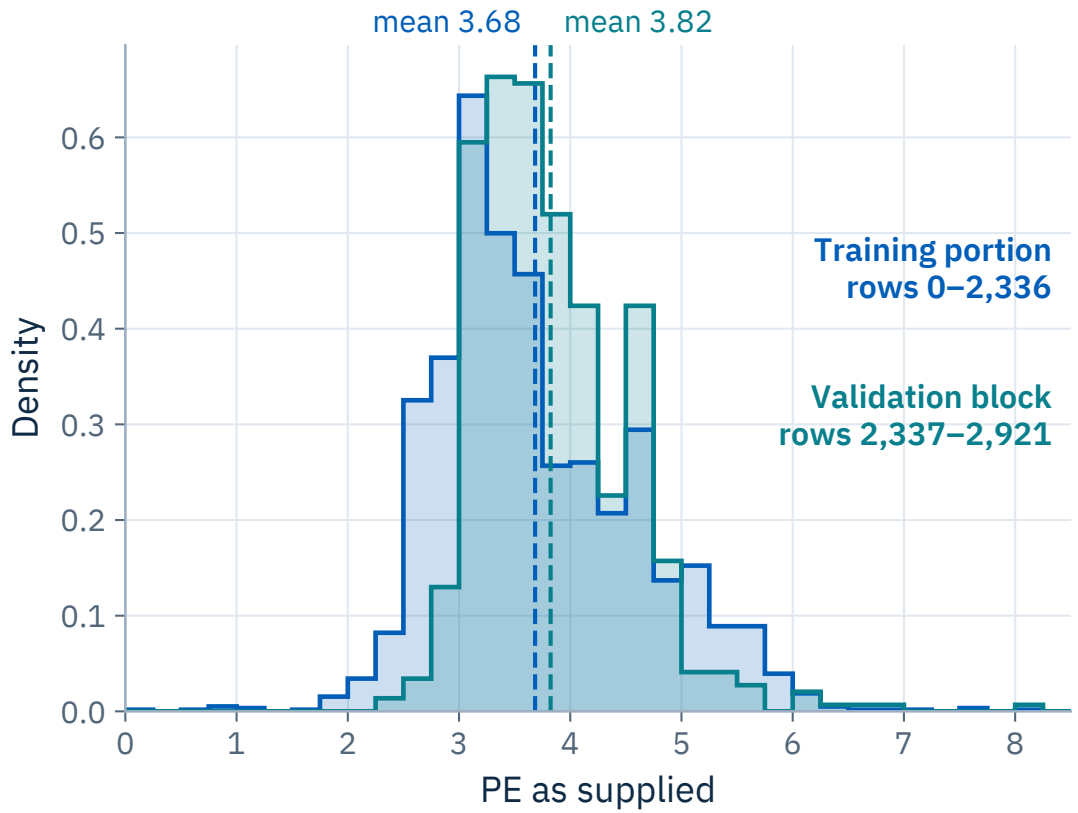
Read one depth block as a well log first



Computed from Log curve - Train.csv: source rows 0–470, the first block of increasing Depth; curves as supplied, units undocumented; well identity is not given.

PE follows PHIND and ILD; validation PE runs higher

	Depth	GR	ILD	DeltaPHI	PHIND	RELPOS	PE
Depth		−.07	.15	−.04	−.13	−.02	**.31**
GR	−.07		−.16	.20	.27	−.14	**−.27**
ILD	.15	−.16		−.11	−.56	.09	**.45**
DeltaPHI	−.04	.20	−.11		−.23	.05	**.01**
PHIND	−.13	.27	−.56	−.23		−.05	**−.59**
RELPOS	−.02	−.14	.09	.05	−.05		**.01**
PE	**.31**	**−.27**	**.45**	**.01**	**−.59**	**.01**	



Computed from Log curve - Train.csv. Left: Pearson correlations over all 2,922 rows (ILD = ILD_log10; descriptive, rows are not independent). Right: PE in the training portion and the validation block of the reference split.

A contiguous source-row block is the honest fallback

- Hold out a contiguous block in source-row order
- Fit input and target scalers on the remaining rows only
- Compare with the training mean and a Ridge baseline
- Asset-level generalization stays unverified without well IDs



Computed from Log curve - Train.csv: the reference split keeps rows 0–2,336 for training and holds out rows 2,337–2,921.

Regression evaluation stays in target units

Three habits turn one PE score into evidence about where the model works.

Invert any scaling

- Report MAE and RMSE in PE units
- Target scaler fitted on training rows

Plot against the truth

- Observed and predicted with an identity line
- Residuals against depth and key inputs

Find where errors live

- Depth blocks, ranges, sparse regions
- R^2 can be negative

◆ Predicting the training mean gives $R^2 = -0.04$ on the validation block, whose mean PE is 3.82, not 3.68.

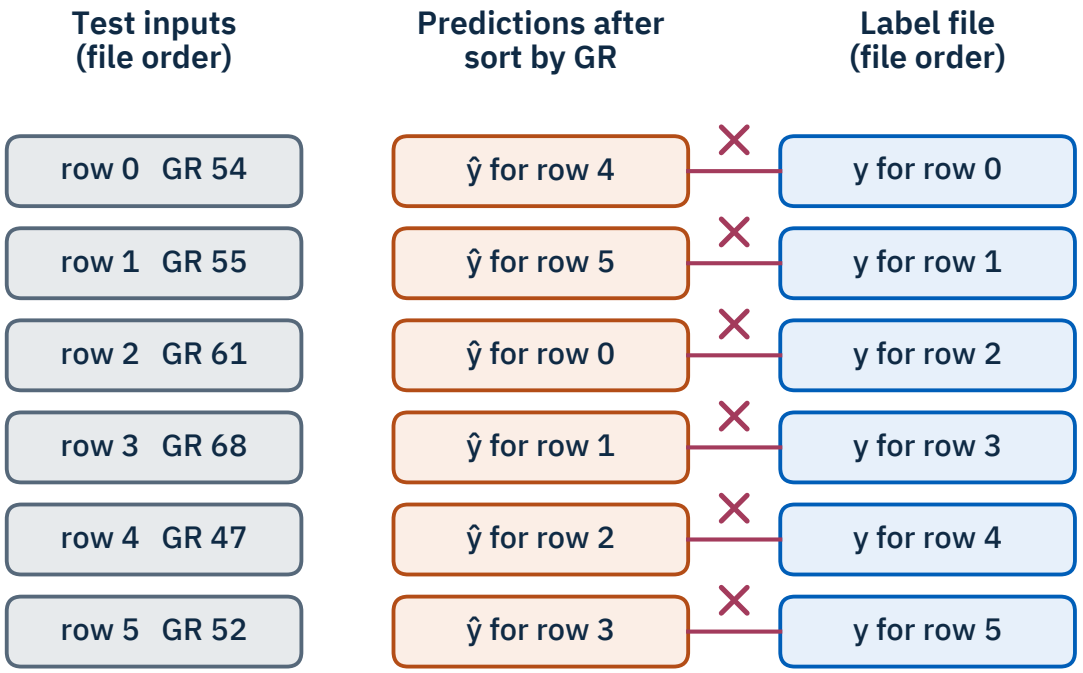
PRACTICE

4 min · Pairs

Positional labels require row-order discipline

You sort the PE test inputs by GR to inspect outliers, predict, and score against the unchanged label file.

1. What happens to the score?
2. Why does the row-count check still pass?
3. How should the evaluation preserve alignment?



Row counts match (310 = 310), so the length check passes,

yet every prediction is scored against another row's label.

Schematic with illustrative GR values.

PART 4 OF 7

04

Project 3: sensor-signal classification

Anonymous numeric inputs with explicit record keys

- Schema and the label convention
- Profiles, class ranges and near-copy columns
- Duplicates, uuid order and split design
- Modelling and evaluation with stated limits

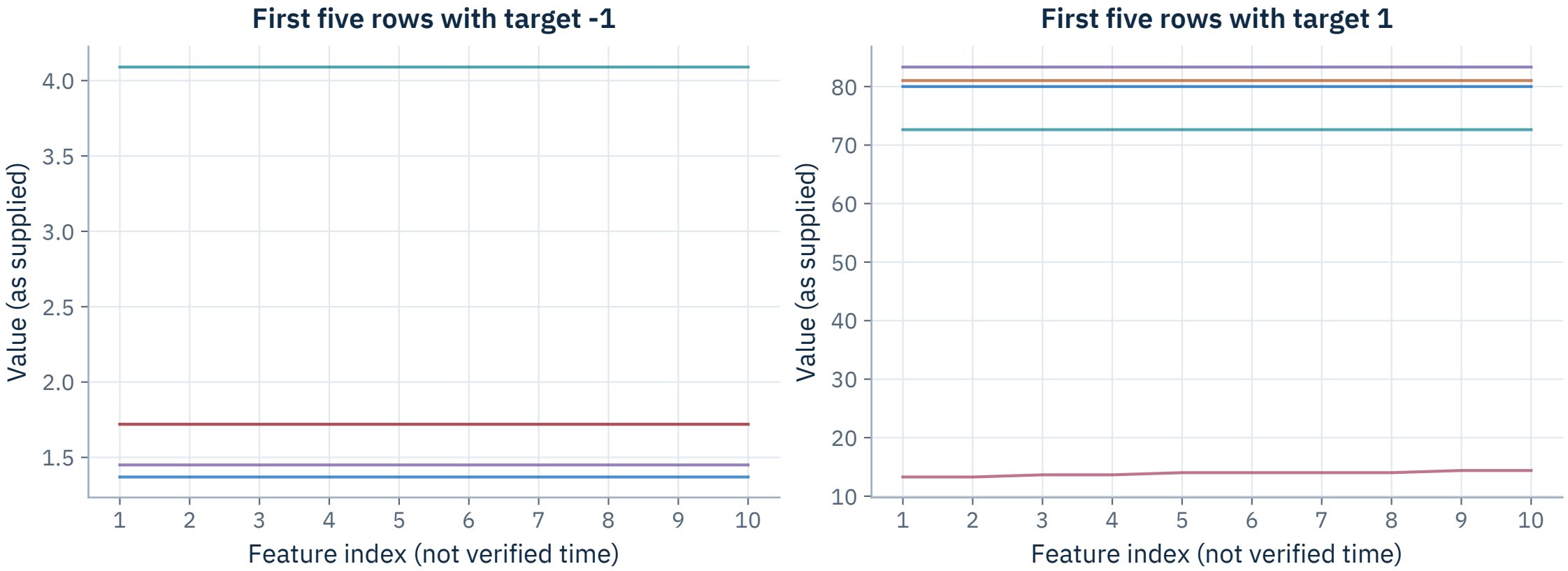
Ten anonymous columns, a key and ±1 labels

```
target = train_df["target"]
assert set(target) == {-1, 1}
y = (target == 1).astype(int)
X = train_df[[f"col{i}" for i in range(1, 11)]]
assert train_df["uuid"].is_unique
```

- Ten predictors: col1 through col10
- uuid identifies records and joins test labels
- Labels are −1 and 1, not 0 and 1
- Map them explicitly before a binary-logit loss

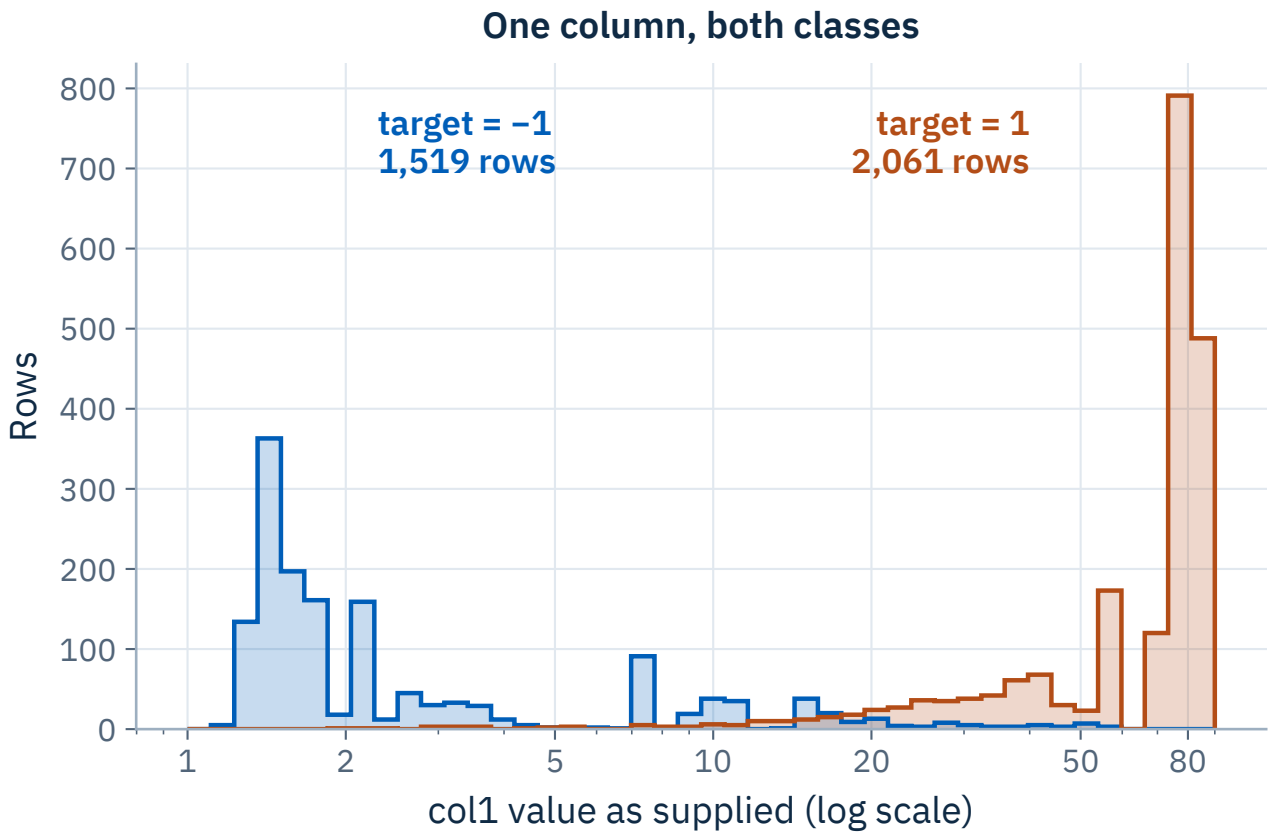
⚠ BCEWithLogitsLoss accepts a −1 target without error and silently optimizes the wrong objective.

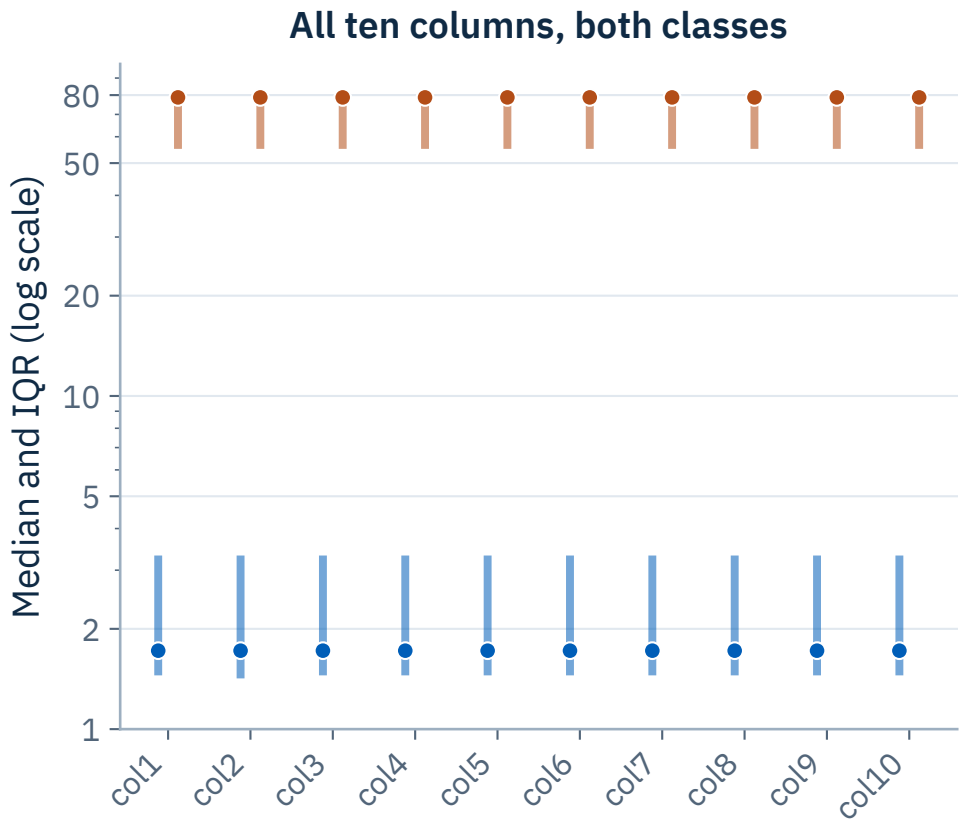
Inspect the profiles before choosing an architecture



Selected rows from the supplied sensor training file. The horizontal axis is feature index, not verified elapsed time.

The two classes occupy different value ranges

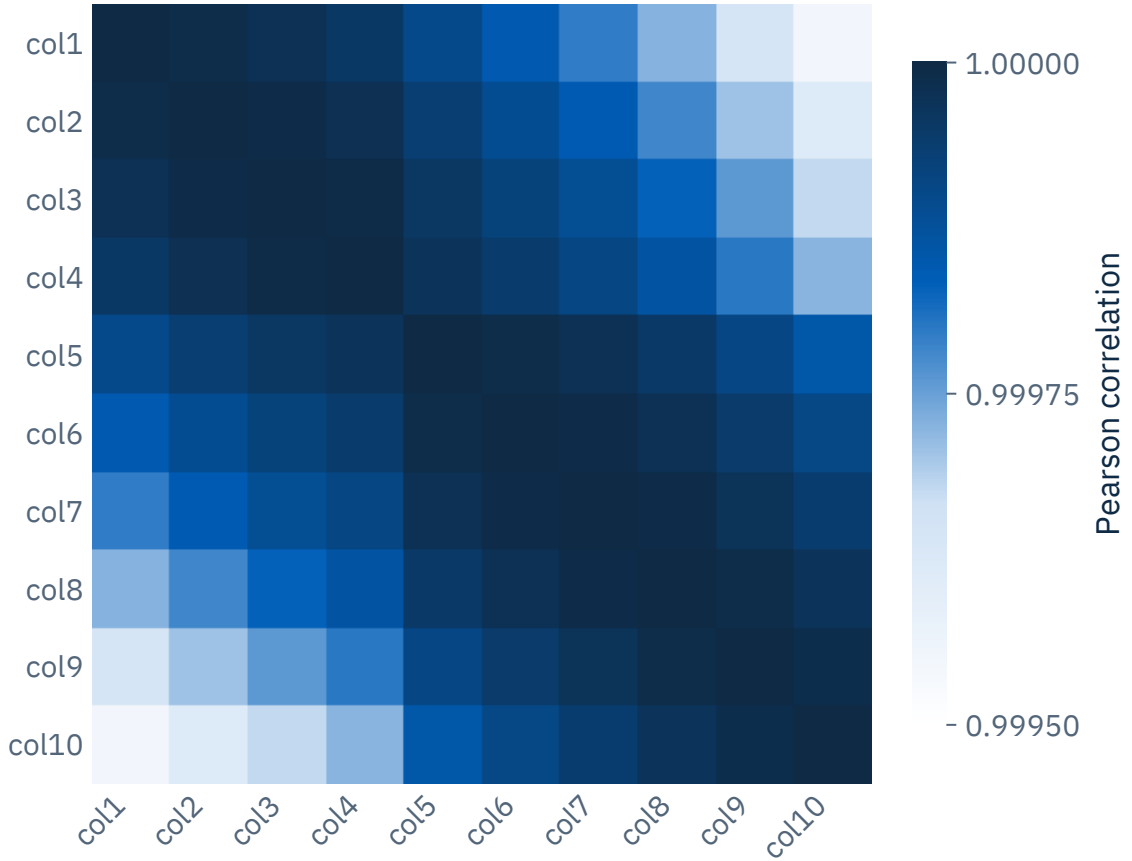




Computed from Sensor_train.csv (3,580 rows). Column index is a name, not a verified time axis.

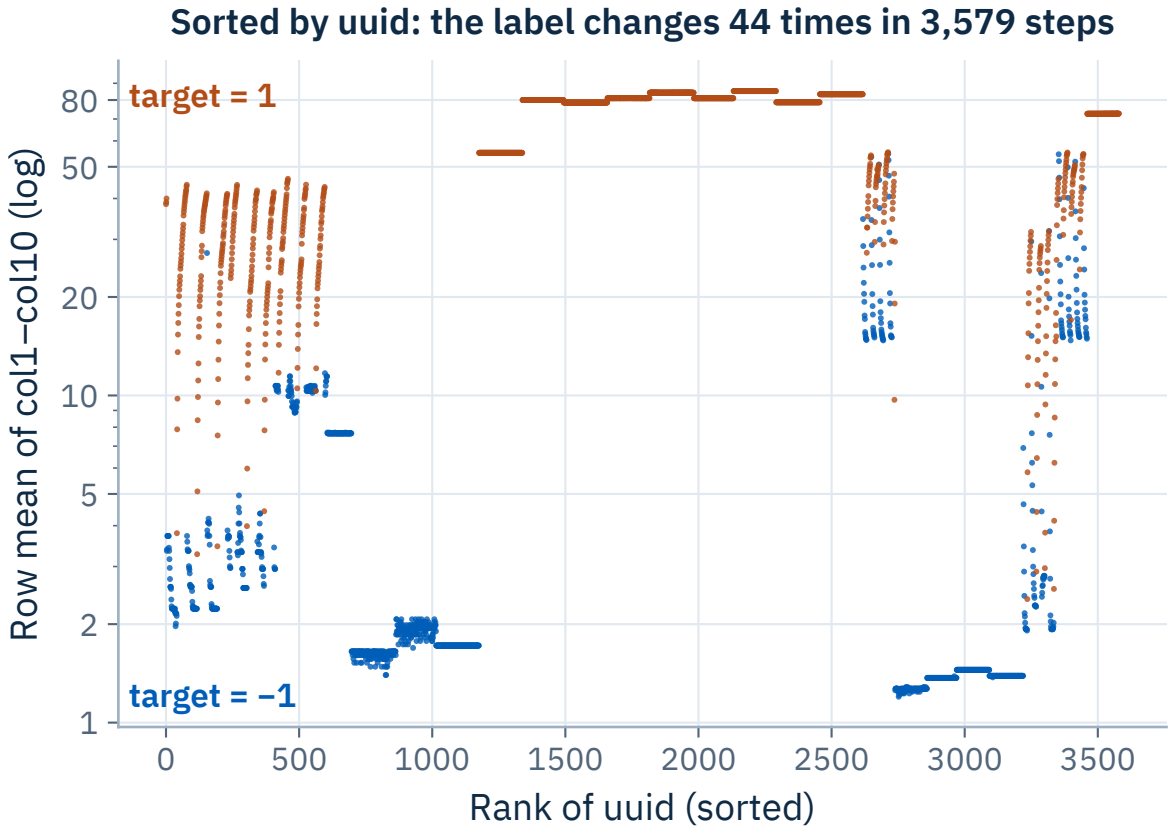
The ten columns are near-copies of one another

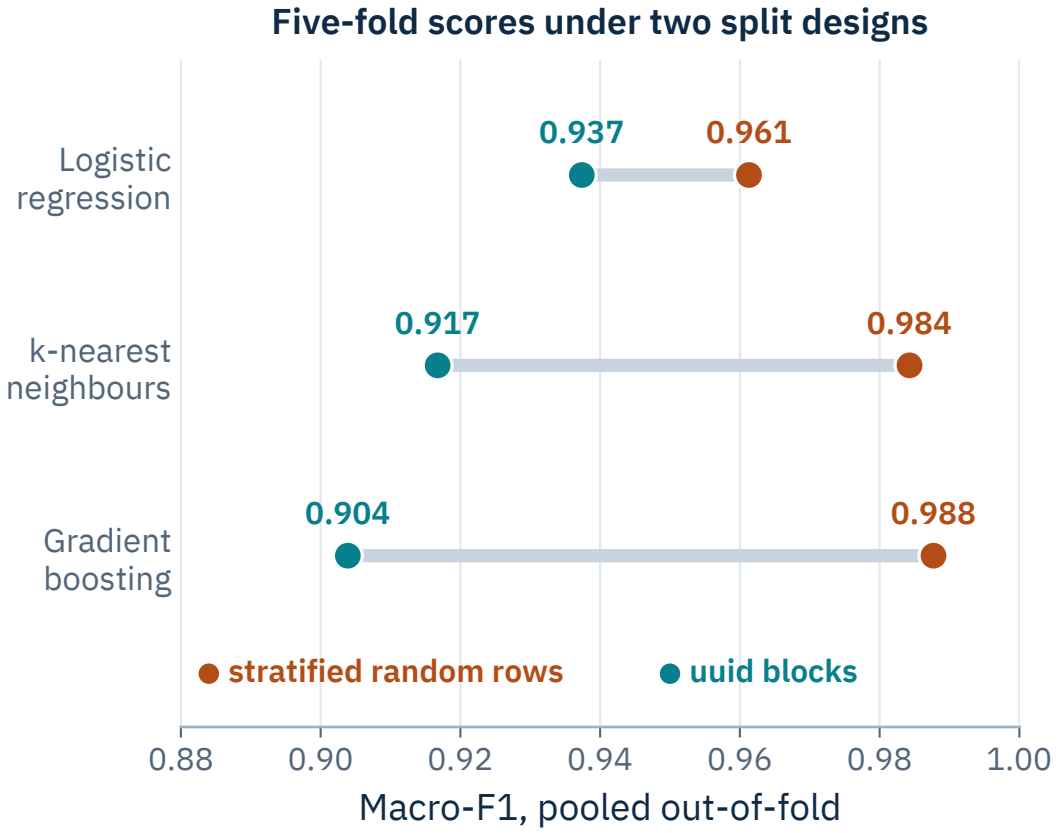
- Every pairwise correlation is at least 0.9995
- 65% of rows hold ten identical values, in both classes
- Treat the columns as one signal plus small deviations
- No data dictionary supports reading the index as time



Computed from Sensor_train.csv: Pearson correlations between col1–col10 over 3,580 rows.

Neighbouring uuids hold near-identical records





Computed from Sensor_train.csv: row means in uuid order; macro-F1 from stratified 5-fold versus five contiguous uuid blocks, scaling fitted inside folds, threshold 0.5.

Model the sensor data with its caveats stated

Baseline

- Train-only scaling, logistic regression
- Threshold chosen on validation

Candidate

- Small dense network, one raw logit
- Same split and preprocessing

Evaluation

- Macro-F1 and per-class metrics
- Stratified split as a documented fallback

Robustness

- Repeat with contiguous uuid blocks
- Report both splits side by side

◆ The supplied test is spread through the training uuid range, so it cannot show performance on a new range.

LABORATORY

35 min

Sprint 1: write the data contract and audit

- ☐ Form a team of two or three; open your project's student notebook
- ☐ Notebook sections 1–2: inspect the file and write the prediction contract
- ☐ Audit missing values, duplicates, keys, groups and order with the checklist
- ☐ Sections 3–4: check the split and the training-only preprocessing

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- One-page data contract
- Audit table with a decision per finding
- Split policy and what it cannot show

MILESTONE 1 OF 4 · 11:20

Checkpoint 1: the data contract

Show

- Target, row unit, excluded columns
- Audit findings with a decision each
- The split policy in one sentence

Instructor checks

- No test labels opened
- The split respects groups or order
- Limits written down, not implied

✓ Proceed when

- The split is fixed and saved
- Preprocessing fits training rows only

◆ About two minutes per team: start with the split sentence, it reveals most problems.

PART 5 OF 7

05

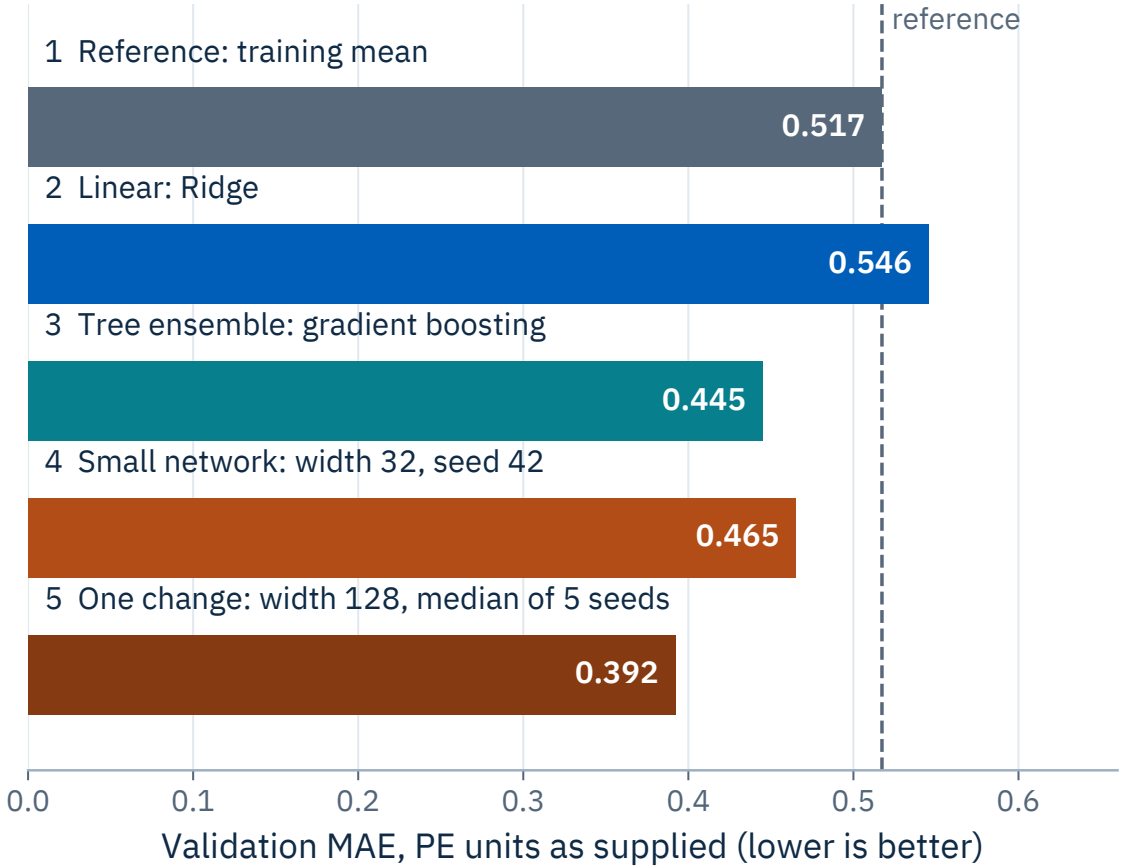
Baselines and experiments

A tabular-NN playbook for a few thousand rows

- Baselines first, including a tree ensemble
- Network size, scaling, missing values, early stopping
- After lunch: one controlled change over several seeds
- Candidate changes: class weights, loss, width

The playbook starts with the cheapest honest model

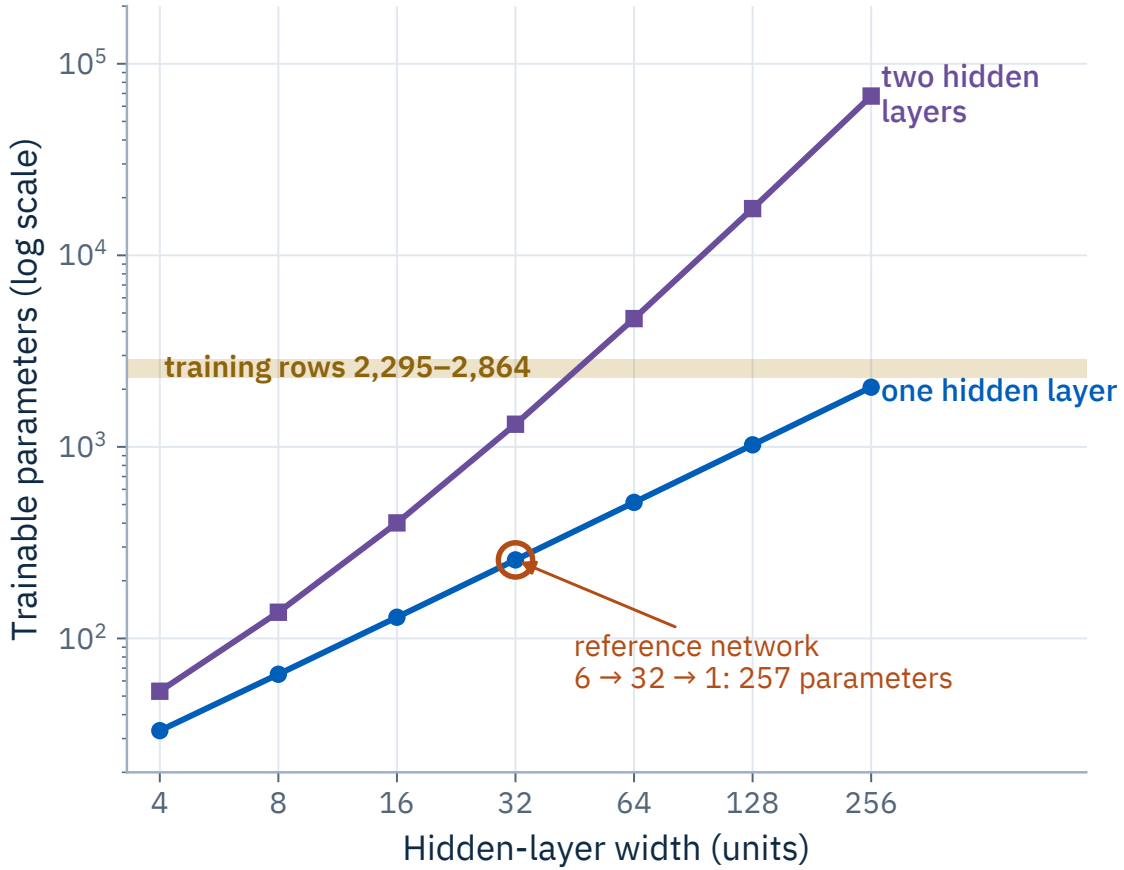
1. **Reference**: training mean or class prevalence
2. **Linear**: logistic or Ridge regression
3. **Tree ensemble**: gradient boosting
4. **Small network**: one hidden layer
5. **One change**: a controlled experiment



Computed on the PE reference split (source rows 0–2,336 train, 2,337–2,921 validation).

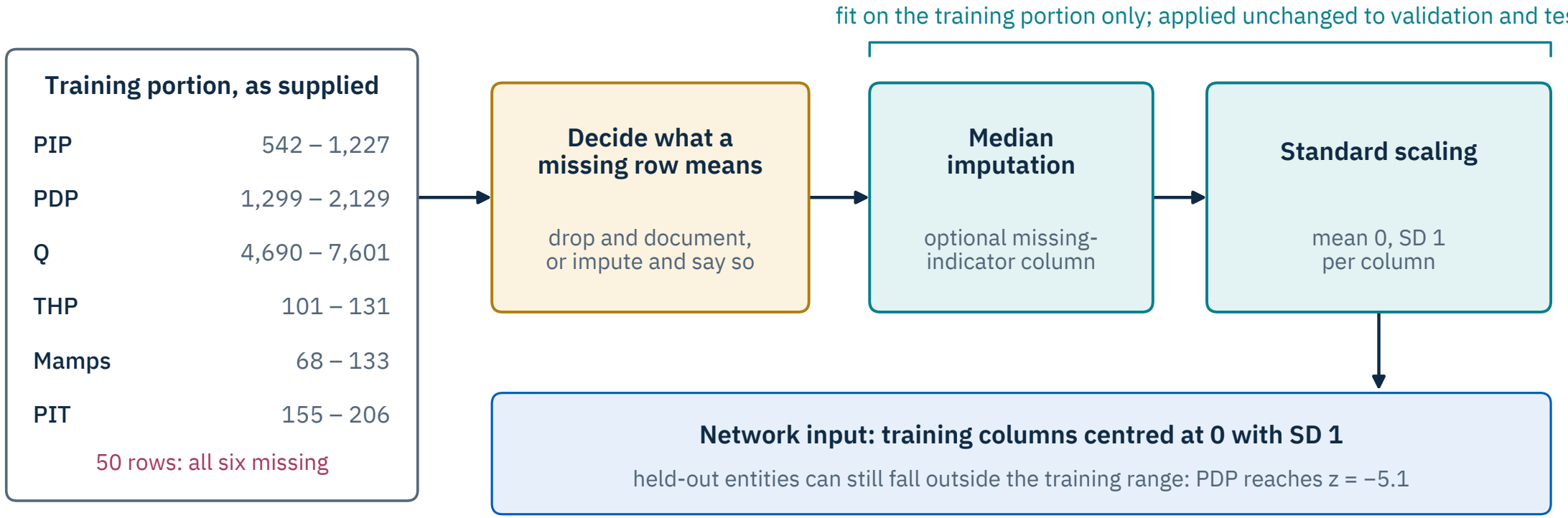
A few thousand rows call for a small network

- Start with one hidden layer of 16–64 units
- 6 → 32 → 1 has 257 parameters; two layers of 64 have 4,673
- Parameter count is not a hard limit; validation decides
- Width is cheap to test; depth adds tuning burden



Exact parameter counts for dense networks with 6 inputs and one output; the band spans the training rows of the three reference splits.

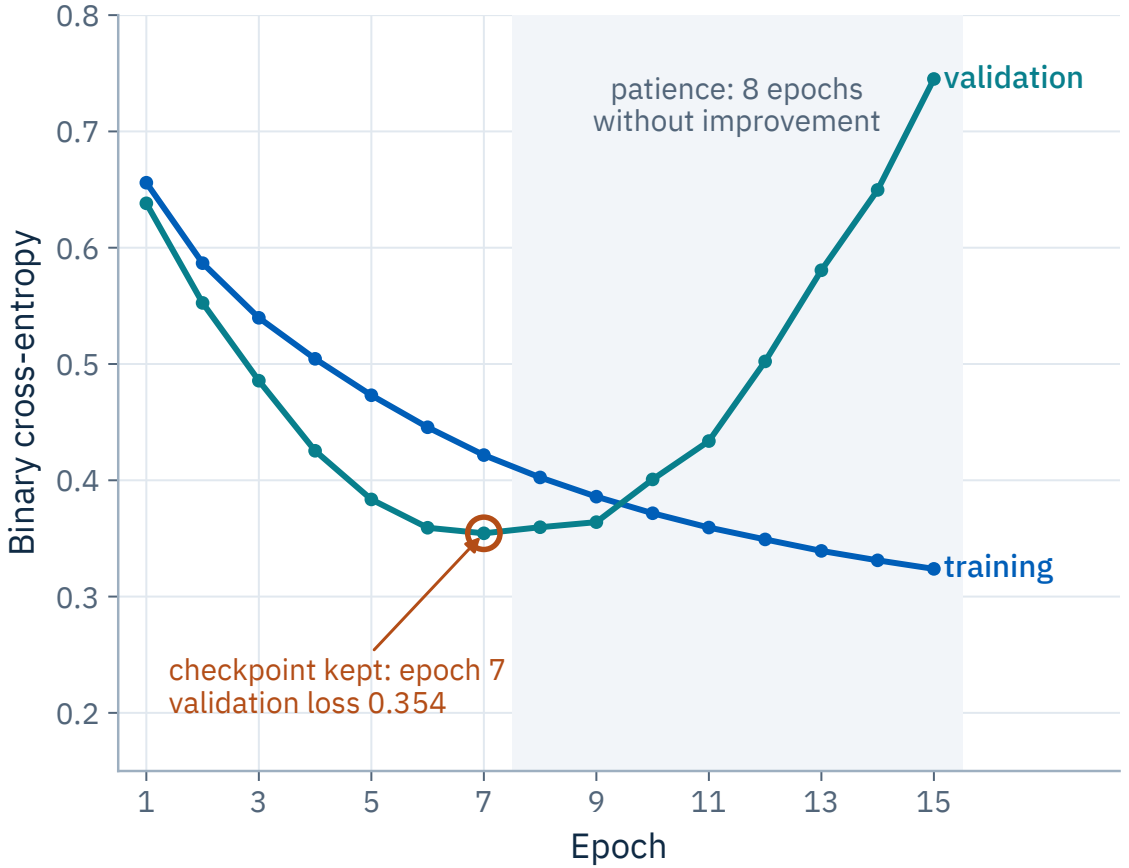
Scale inputs and decide what a missing row means



Schematic; value ranges computed from the training portion of the reference ESP split in esp_train.csv.

Early stopping keeps the best validation checkpoint

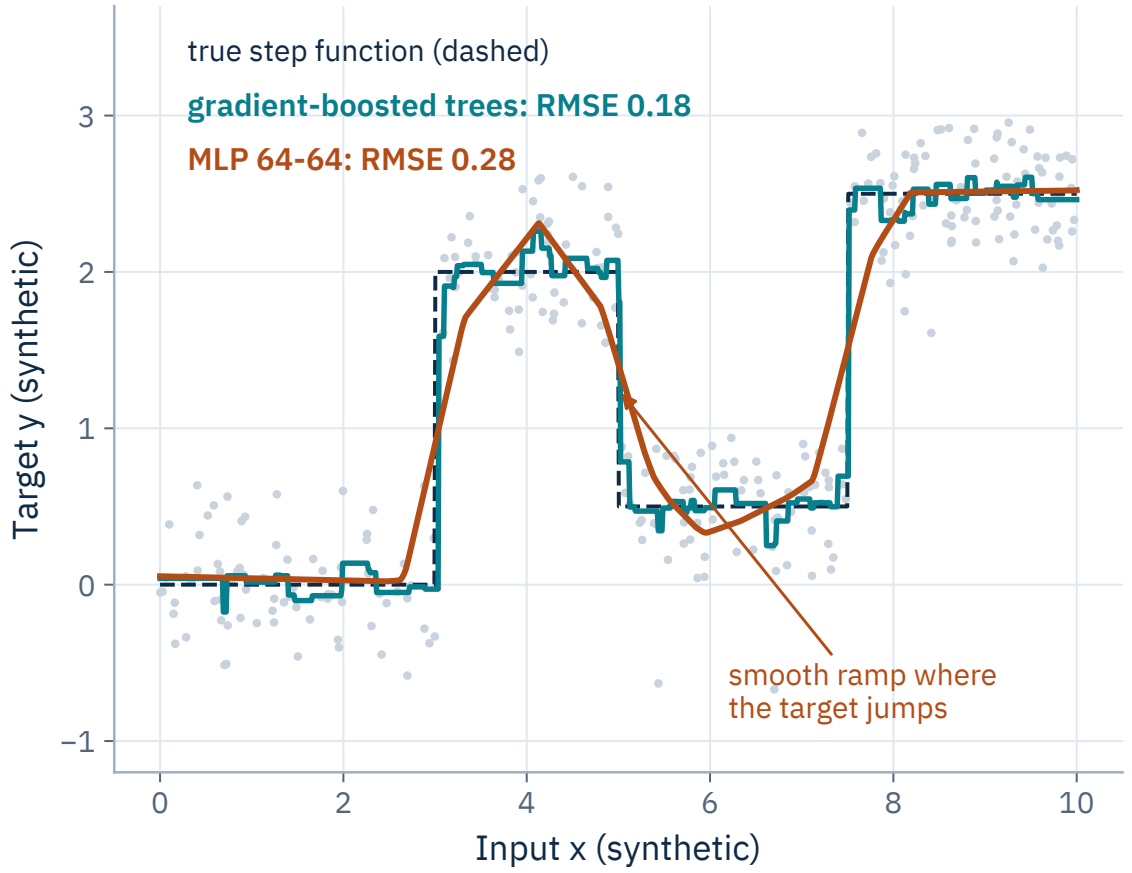
- Validation loss is lowest at epoch 7, then climbs
- The loop copies that state; patience is 8 epochs
- Restore the copy before evaluating or saving
- Training loss keeps falling: the gap is the warning



Computed: curriculum/capstone.py on the ESP project (width 32, AdamW, lr 0.001, seed 42); validation = esp_30 and esp_31.

Tree ensembles are often strong on tabular data

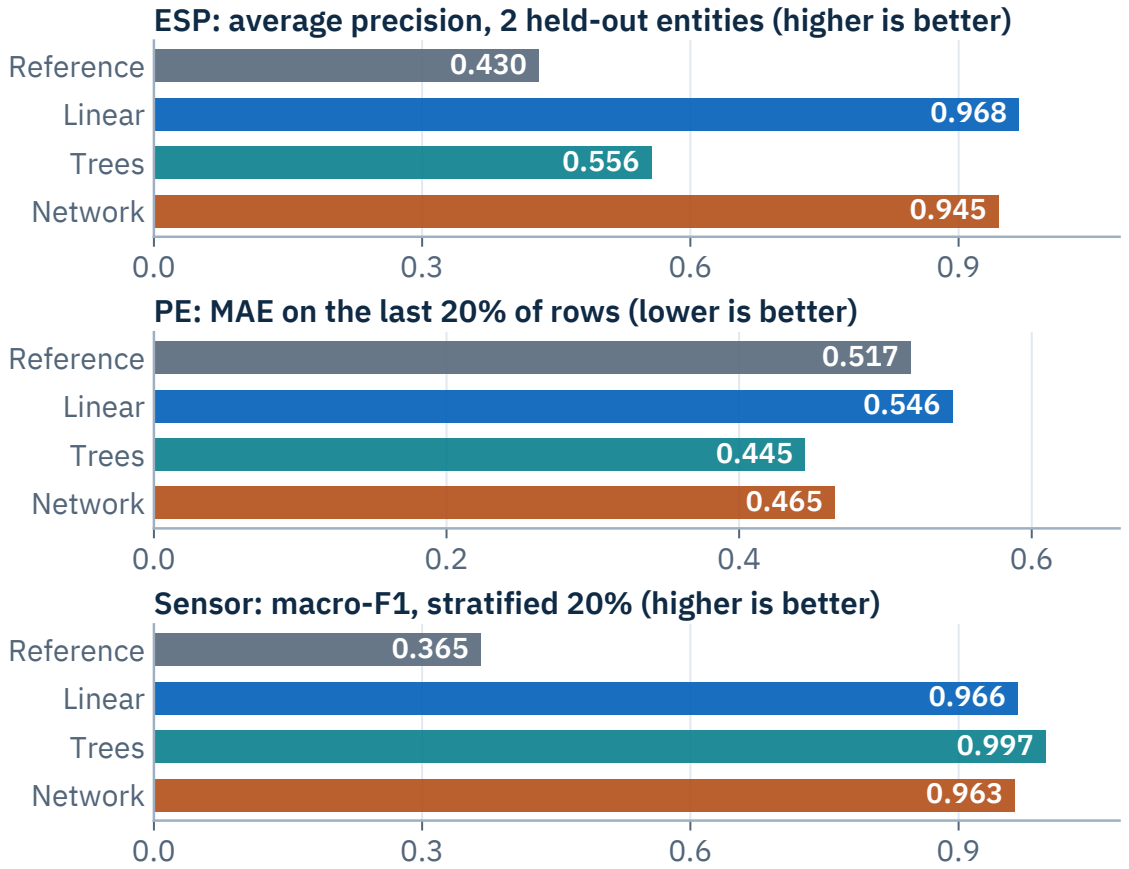
- A 45-dataset benchmark found trees ahead on medium-sized tables
- Networks favour smooth functions; trees fit jumps readily
- Uninformative columns hurt MLPs more than trees
- Columns carry meaning; trees split along them



Synthetic example, seed 0: 300 noisy samples of a step function; RMSE against the true function.

Treat a tree ensemble as a second baseline

- Fit gradient-boosted trees on the same rows; they accept missing values
- Trees win on PE and on sensor rows, but fail on held-out ESP pumps
- Rank models per project and per split, not by reputation
- The network does not need to win; a clear negative result counts



Computed on the reference splits (training files only). Network: width 32, seed 42. Sensor thresholds chosen on validation.

LABORATORY

40 min

Sprint 2: fit the baselines and a first network

- [ ] Section 5: run the reference baseline and network on your split
- [ ] Add a no-skill reference and a tree ensemble on the same rows
- [ ] Section 6: read the learning curve and note the selected epoch
- [ ] Save the run folder; record the command, seed and versions

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table: reference, linear, tree, network
- Learning curve with the selected epoch
- The exact command that reproduced it

MILESTONE 2 OF 4 · 12:30

Checkpoint 2: a reproducible baseline

Show

- Score table, reference first
- Learning curve and the selected epoch
- The command that reproduced it

Instructor checks

- Same validation rows for every model
- Medians fitted on training rows only
- Threshold chosen on validation

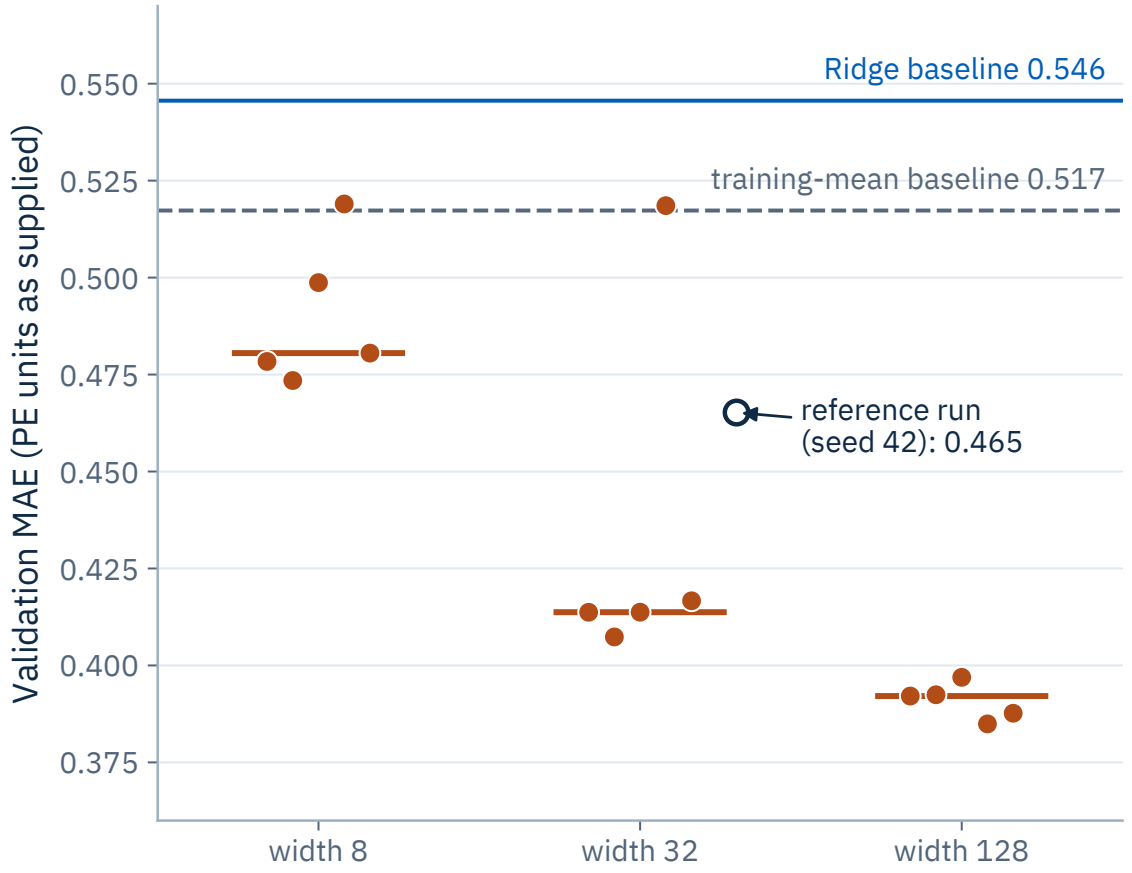
✓ Proceed when

- A fresh process gives the same numbers
- The team can say which model leads, by how much

◆ A solid baseline without a network passes this gate; a network without a baseline does not.

A controlled experiment changes exactly one thing

- Write the hypothesis and expected direction first
- Hold split, preprocessing, budget and seeds fixed
- Repeat each setting over at least three seeds
- Here, one seed at width 32 scored like width 8

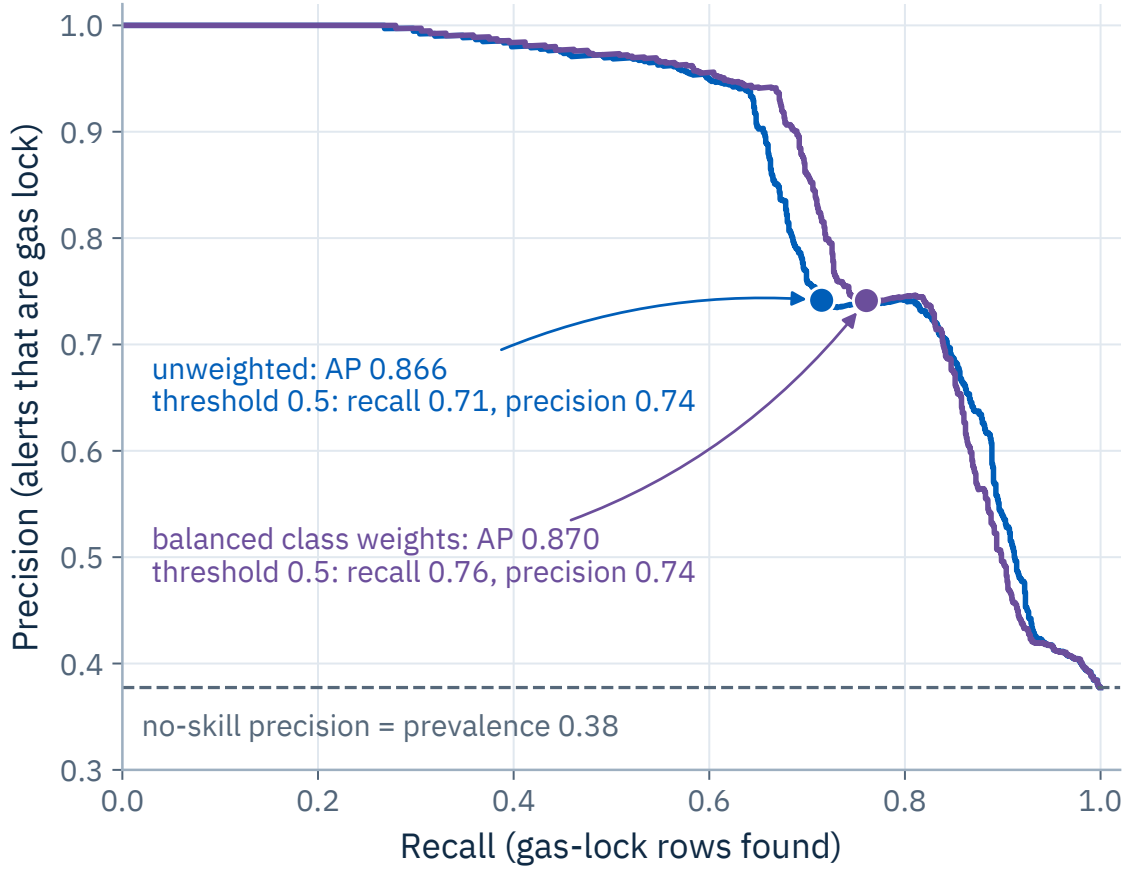


Computed: reference training loop on the PE project, seeds 0–4 per width (bars mark medians); baselines fitted on the same training rows.

Class weights move the operating point, not the ranking

$$L = -\frac{1}{n}\sum_i [w_1\, y_i \log p_i + w_0\,(1-y_i)\log(1-p_i)]$$

- Balanced weights set $w_c = n/(2n_c)$ for each class c
- ESP: AP 0.866 → 0.870; recall at 0.5 rises 0.71 → 0.76
- A threshold chosen on validation does a similar job



Computed from esp_train.csv: logistic regression, leave-one-entity-out predictions pooled over seven entities.

Each project needs a matching loss and metric

Project	Output	Loss	Evaluation
ESP	One logit	Binary cross-entropy with logits	PR metrics, confusion counts
PE	One numeric value	MSE or Huber	MAE, RMSE, residuals
Sensor	One logit	Binary cross-entropy with logits	Macro-F1, per-class metrics

Train on the loss, select on validation loss or metric, and report the metric in target units.

- Probabilities and thresholds belong to the decision step, not to the training loss.

Good first experiments for each project

ESP

- Class weights on versus off
- All seven entity folds, not one split

PE

- Width 32 versus 128, five seeds
- MSE versus Huber loss

Sensor

- Stratified versus uuid-block split
- One row-mean feature versus ten columns

◆ One change, a written hypothesis, and a decision rule set before the runs.

LABORATORY

40 min

Sprint 3: run one controlled experiment

- [ ] Write the hypothesis, the change and the decision rule first
- [ ] Run both settings over at least three seeds; change nothing else
- [ ] Compare the difference with the seed spread
- [ ] Decide, then write the locked configuration down

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Hypothesis, result and decision in three sentences
- Seed-by-seed table for both settings
- Locked configuration: width, lr, seed, epochs, threshold rule

MILESTONE 3 OF 4 · 14:20

Checkpoint 3: hypothesis, result, decision

Show

- The hypothesis written before the runs
- Both settings over the same seeds
- The locked configuration in writing

Instructor checks

- Exactly one change between settings
- The difference is compared with the spread
- The test file is still closed

✓ Proceed when

- Configuration and command are frozen
- A null result is recorded as a result

◆ After this gate only analysis and presentation may change.

PART 6 OF 7

06

Error analysis

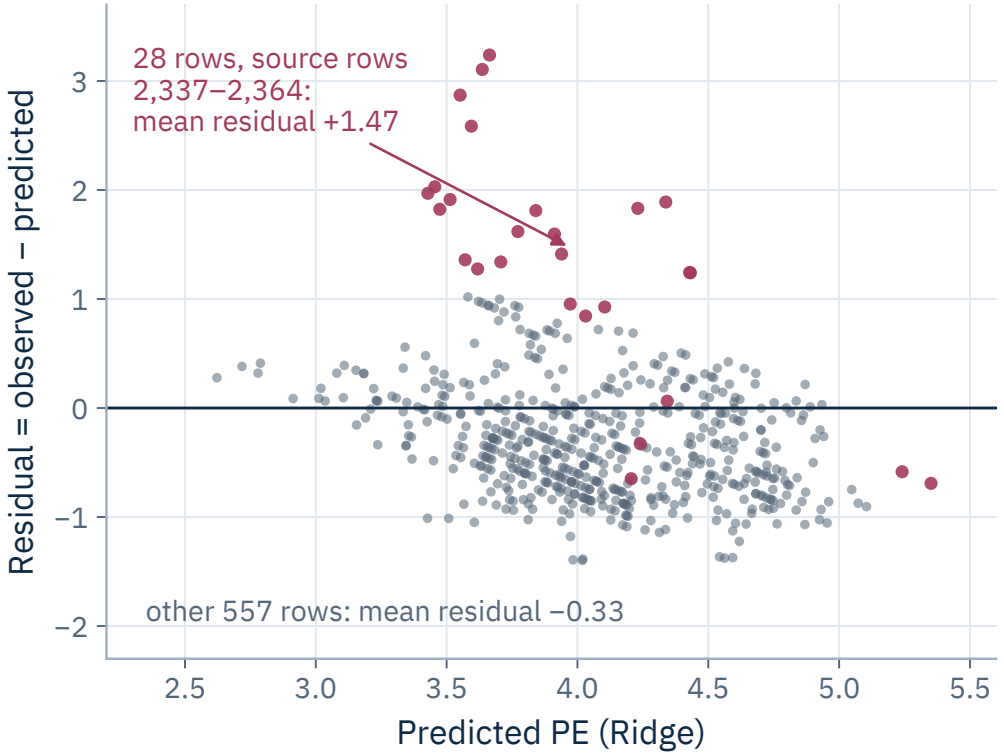
What the score hides, and how to find it

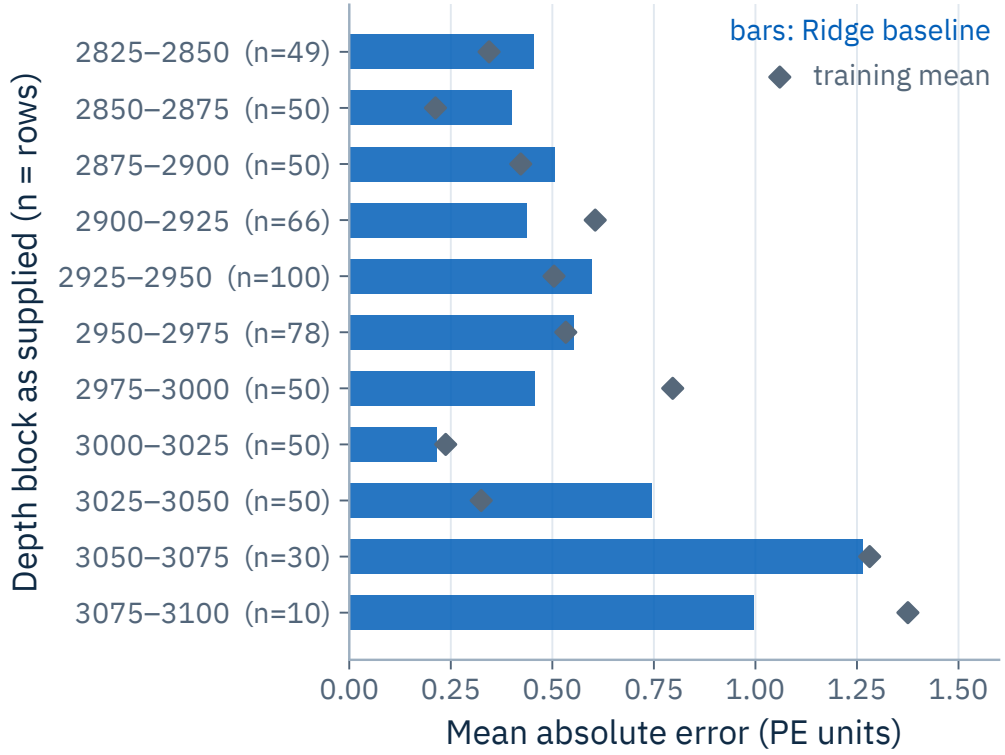
- Residuals against predictions and by depth block
- Errors by held-out entity
- Confusion by class
- Calibration of predicted probabilities

ERROR ANALYSIS

Residuals reveal a biased 28-row block in PE

Ridge baseline fitted on source rows 0–2,336 and scored on the validation block.

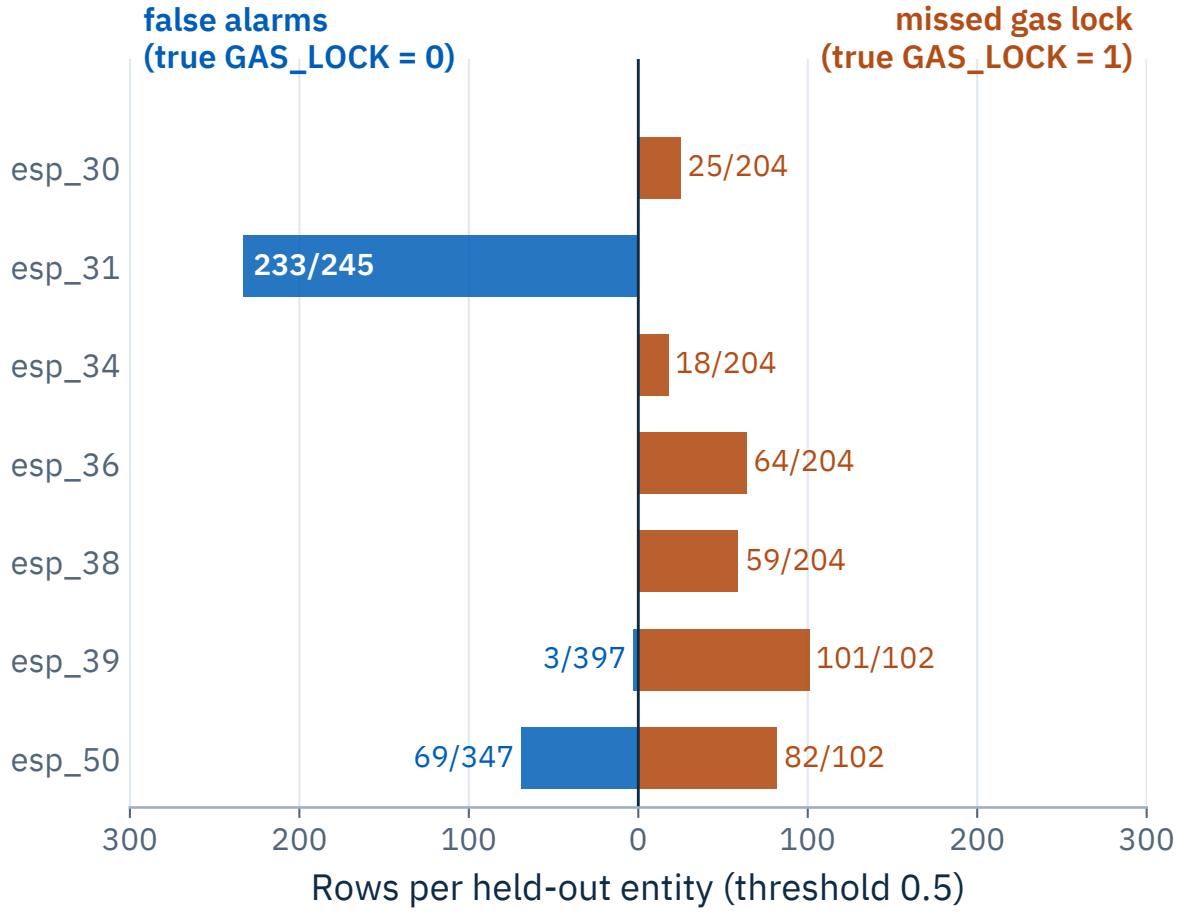




Computed from Log curve - Train.csv. Left: residuals against predictions. Right: mean absolute error by 25-unit Depth block (bars: Ridge; diamonds: training mean).

Two held-out entities fail in two different ways

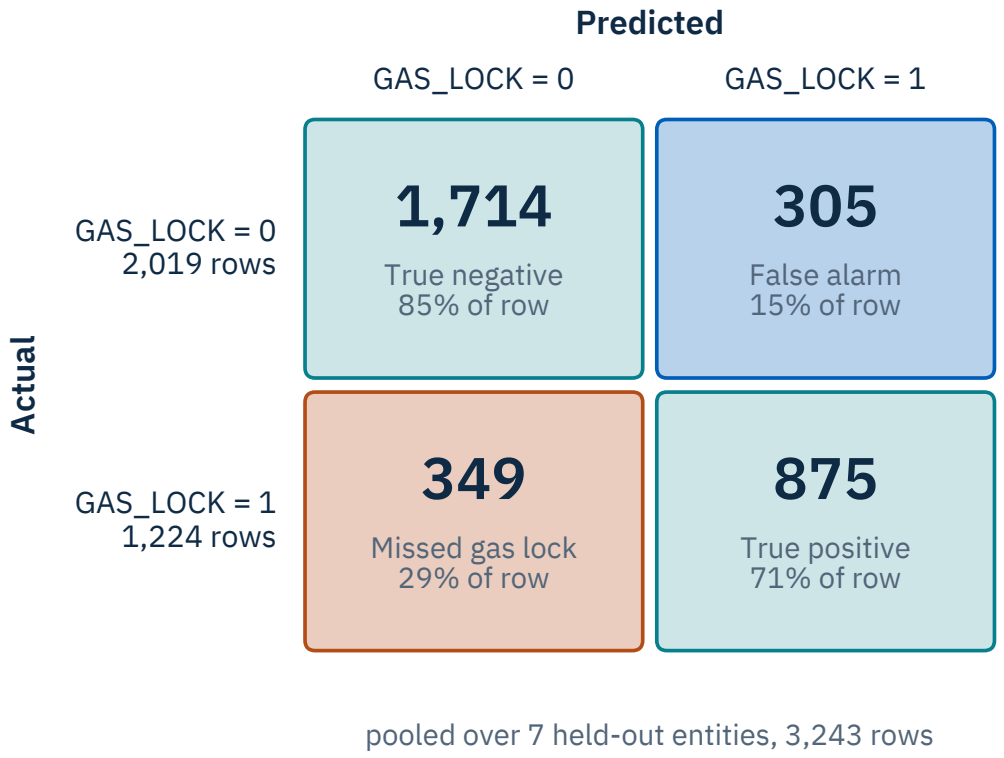
- esp_31: every gas-lock row found, 233 of 245 normal rows flagged
- esp_39 and esp_50: most gas-lock rows missed
- Pooled, these failures average into one number
- Each failure suggests a different next test



Computed from esp_train.csv: logistic baseline, leave-one-entity-out, threshold 0.5.

A confusion matrix separates misses from false alarms

- Rows are actual classes; columns are predictions
- 349 of 1,224 gas-lock rows missed (29%)
- 305 of 2,019 normal rows flagged (15%)
- Decide which error costs more before moving the threshold

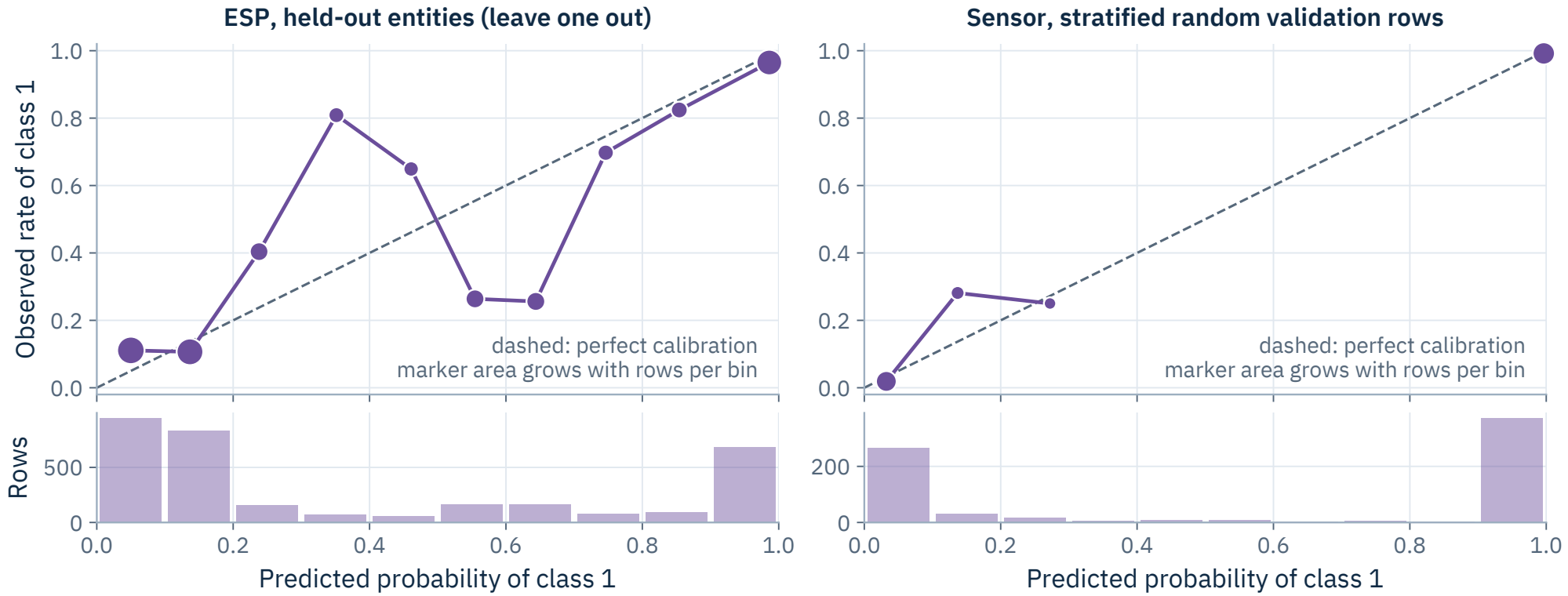


Computed from esp_train.csv: logistic baseline, leave-one-entity-out predictions pooled over seven entities, threshold 0.5.

ERROR ANALYSIS

Calibration shows whether probabilities can be trusted

Group predictions into bins and compare the average prediction with the observed rate.



Computed: logistic baselines. Left: esp_train.csv, leave-one-entity-out (3,243 rows). Right: Sensor_train.csv, reference stratified validation rows (716). Bins with fewer than 10 rows omitted.

PRACTICE

10 min · Teams

An error review asks what the score hides

1. Choose one failed example or slice and inspect its inputs, target and prediction
2. Decide: label noise, missing information, model limitation, or shift?
3. Propose one experiment that could tell the explanations apart

◆ Evidence, not a story: give the slice's size, its error rate and the comparison with the rest.

LABORATORY

40 min

Sprint 4: run the final test once, then analyse errors

- Section 8: run once with the locked configuration and final=True
- Confirm the uuid join or the positional row count
- Make one error-analysis figure by entity, depth block or class
- Draft slides 1–4 while the numbers are fresh

NOTEBOOKS

`exercises/01_esp_gas_lock/ESP_Gas_Lock_Student.ipynb`

`exercises/02_log_curves/Log_Curve_Prediction_Student.ipynb`

`exercises/03_sensor_signals/Sensor_Signals_Student.ipynb`

EVIDENCE TO SUBMIT

- Final score table: baseline and candidate side by side
- One error-analysis figure with a caption
- Draft slides 1–4

MILESTONE 4 OF 4 · 15:25

Checkpoint 4: the locked final assessment

Show

- The final command and its configuration
- Join or alignment checks that passed
- Final baseline and candidate scores

Instructor checks

- Configuration matches checkpoint 3
- No changes after the final run
- Validation and test numbers kept apart

✓ Proceed when

- One error figure is ready
- Limitations are written down

◆ A final score below the validation score is common, not a failure: selection makes validation optimistic.

PART 7 OF 7

07

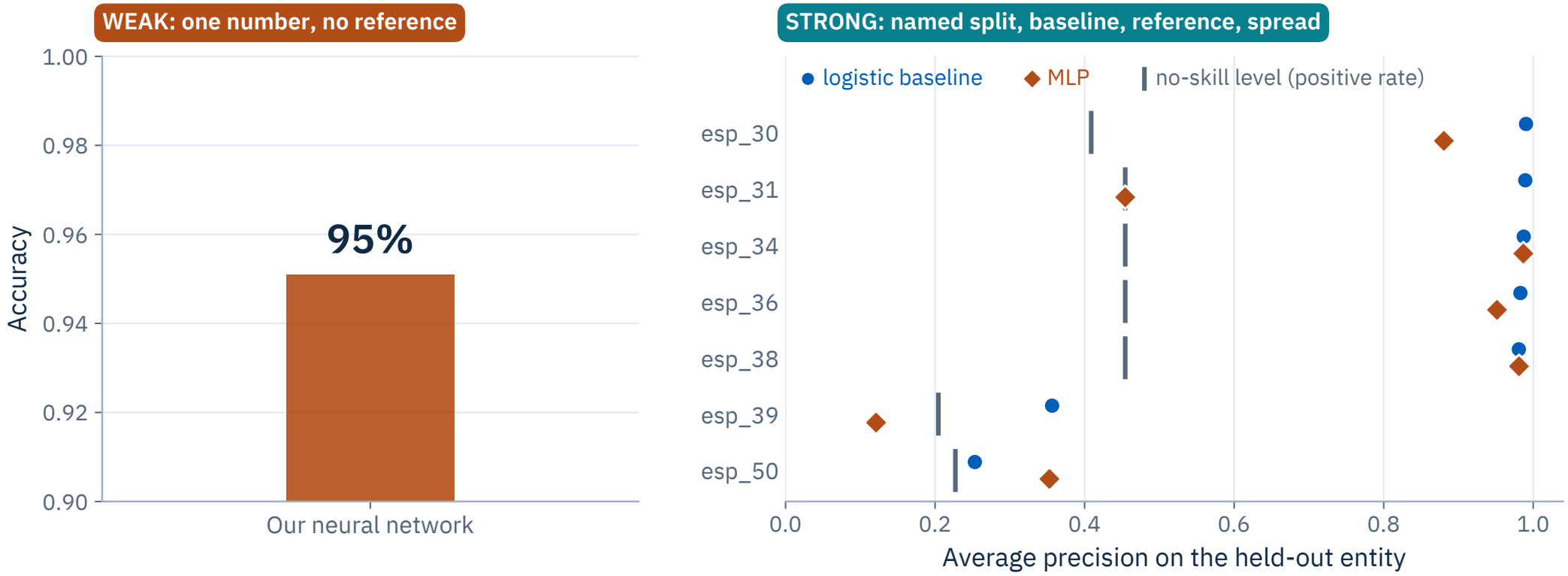
Communicating evidence

Make the claim no stronger than the evidence

- Weak and strong evidence figures
- A five-minute structure
- A model card and a reproducibility checklist
- The rubric and the handoff

Strong evidence names the split, baseline and spread

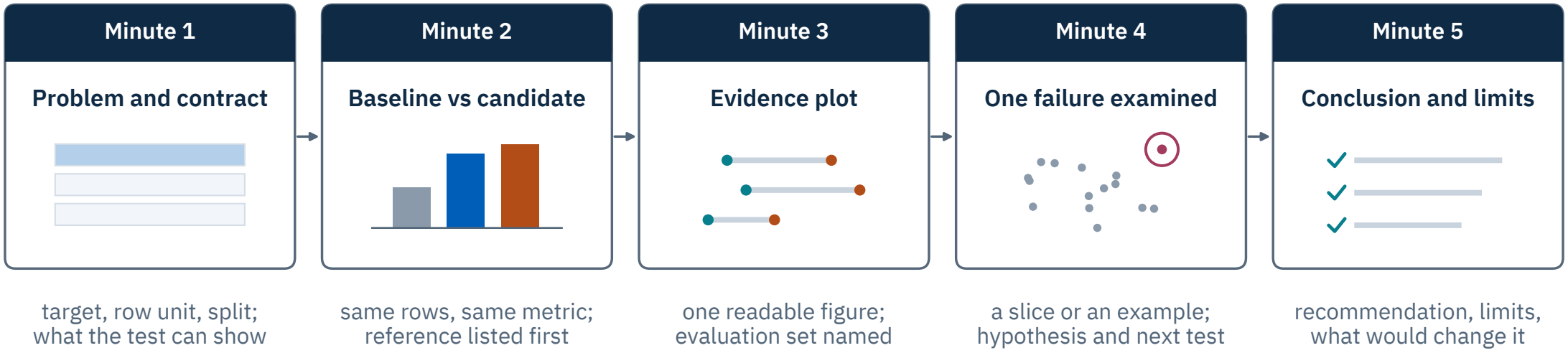
Both panels come from the same file and the same kind of network.



Computed from esp_train.csv. Left: MLP with a random 5-fold row split, pooled accuracy. Right: leave-one-entity-out average precision per entity; ticks mark each entity's positive rate.

Five minutes, five slides, one conclusion

Readable figures with axis labels, and the evaluation set named on every chart.



Schematic storyboard; two minutes of questions follow each talk.

A model card states use, evidence and limits

Model card — ESP gas-lock logistic baseline · teaching example, not a field model

Intended use

label hourly rows as gas lock or not

for entities unseen in training

Not intended for

early warning: no lead time was tested

inputs outside the training ranges

Data and split

esp_train.csv: 3,243 rows, 7 entities

leave one entity out; test entities locked

Evaluation

average precision by entity 0.25–0.99

weakest: esp_50 and esp_39

Limitations

units undocumented; 300 missing cells

labelled periods look highly regular

Reproduce

LeaveOneGroupOut on ENTITY, 7 folds

baseline pipeline of curriculum/capstone.py

Template filled with values computed from esp_train.csv (logistic baseline, leave-one-entity-out).

A reproducible run survives a fresh process

Checklist: fresh process, pinned environment, fixed seeds, saved artifacts, reload check.

fresh process

```
$ python3 curriculum/capstone.py log
    --epochs 40 --width 32
    --lr 0.001 --seed 42
    --out runs/pe_w32_s42
```

one command, pinned environment, unchanged data

Saved artifacts

- `split.csv`
- `learning_curve.csv`
- `validation_predictions.csv`
- `results.json`
- `preprocessing_and_baseline.joblib`
- `neural_weights.pt`

Checks

- ✓ reload gives identical predictions
- ✓ seed, width, lr and epochs recorded in results.json
- ✓ split ids let others rescore the run
- ! final: same flags plus --final, run once

Schematic; command-line options and artifact names from curriculum/capstone.py.

The rubric weights evaluation above accuracy

Criterion	Weight	Full-credit evidence
Problem and data contract	20%	Correct target, schema, and timing
Evaluation design	30%	Appropriate split; preprocessing fitted on training rows only
Modelling and reproducibility	20%	Baseline, candidate, saved run
Error analysis	20%	Relevant metrics and investigated failures
Communication	10%	Clear, supported conclusion and limitations

◆ Leakage or misaligned labels invalidate the affected assessment, whatever the score.

15 min

Handoff checklist: a run others can repeat

- [ ] Re-run the notebook or script from a fresh process
- [ ] Include split identifiers, preprocessing, configuration and model state
- [ ] Export predictions with identifiers or original row positions
- [ ] Submit the five-slide report and one reproducible command

EVIDENCE TO SUBMIT

- Five slides
- One command and its run folder
- Predictions with uuids or row positions

DISCUSSION

35 min

Final presentations: five minutes per team

Presenters

- Five minutes, five slides
- Evaluation set named on every chart

Audience

- One question about the evidence
- One question about a limit

Instructor

- Score with the rubric
- Note one strength and one fix

Honest negative results and clear limits are praised in front of the room.

Key takeaways

- ✓ The split decides what a score can claim; name it on every chart
- ✓ Every supplied file hid a limitation that only the audit revealed
- ✓ Baselines, including a tree ensemble, make improvement measurable
- ✓ One change over several seeds, then lock before the final test
- ✓ Errors by group, class and calibration beat one pooled number

PRACTICE

5 min · Individual

Exit check

1. A team reports 95% accuracy for ESP from a random row split. What two things do you ask for?
2. Why is a PE validation MAE not evidence of cross-well performance?
3. A network beats logistic regression by 0.003 macro-F1 on one seed. What do you conclude?
4. Name one thing you may not do after the final test run.

References and further study

- Grinsztajn, Oyallon and Varoquaux (2022). *Why do tree-based models still outperform deep learning on typical tabular data?* NeurIPS Datasets and Benchmarks Track.
- Mitchell and colleagues (2019). *Model Cards for Model Reporting.* Conference on Fairness, Accountability, and Transparency (FAT*).
- scikit-learn User Guide: cross-validation for grouped data, probability calibration, common pitfalls.
- PyTorch documentation: BCEWithLogitsLoss (pos_weight), saving and loading models.
- Course files: curriculum/capstone.py and the capstone section of INSTRUCTOR_GUIDE.md.

NEURAL NETWORKS / DAY 4

Convolutional neural networks

Images as tensors, convolution, architectures, transfer learning, detection, segmentation and reconstruction

Full day · 09:00–17:00 | 7 parts · 2 laboratories | PyTorch

Today's plan

09:00	Welcome, casing-inspection case, images as tensors	Lecture
09:45	Convolution mechanics	Lecture
10:30	Break	Break
10:45	Feature maps, pooling and a compact CNN	Lecture
11:30	Lab 1: a checked CNN on CIFAR-10	Lab
12:30	Lunch	Lunch
13:30	Lab 1 debrief, architecture ideas, training and transfer learning	Lecture
14:50	Break	Break
15:05	Beyond classification; autoencoders and 1-D signals	Lecture
15:50	Lab 2: autoencoder or casing defects, then debrief and exit check	Lab

LEARNING OUTCOMES

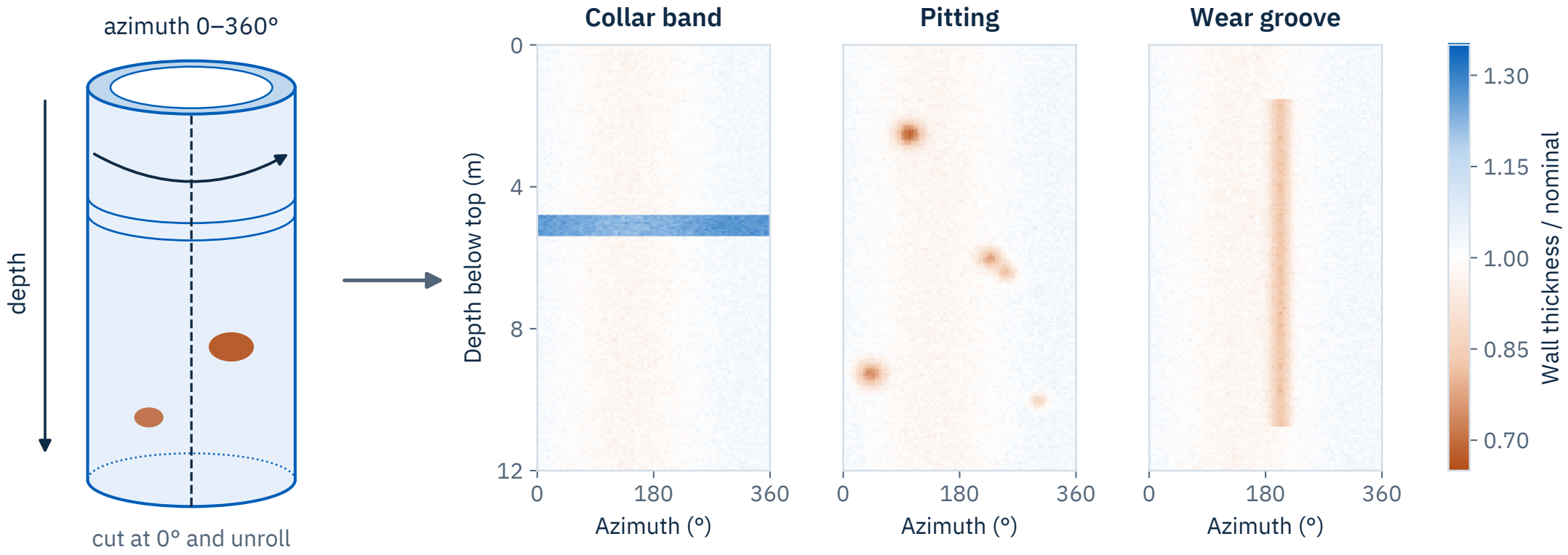
What you will be able to do

By the end of the day you will be able to:

1. Represent images and energy imaging data as tensors with the right shape and meaning.
2. Compute convolution outputs, output sizes, parameter counts and receptive fields by hand.
3. Build, shape-check and train a compact CNN in PyTorch.
4. Adapt a pretrained backbone with label-preserving augmentation and grouped evaluation.
5. Choose between classification, detection and segmentation, and test what a model uses.
6. Use convolutional autoencoders and 1-D convolutions without overstating what they show.

A casing inspection log is already an image

Ultrasonic imagers record wall thickness around the pipe at each depth: a depth × azimuth grid.



Synthetic thickness maps (seeds 4–6) with 0.05 m rows and 5° sectors, drawn to show the geometry; not tool data.

PRACTICE

4 min · Pairs

Decide what the model should return

For the casing maps on the previous slide:

1. Should the model return one label per image, per depth window, or per pixel?
2. Which neighbouring pixels would you inspect to judge whether one pixel is a pit?
3. Two windows from one well overlap by half. May one be in training and the other in test?

PART 1 OF 7

01

Images as tensors and imaging data

Know exactly which numbers the network sees.

- Greyscale grids and colour channels
- Batches in PyTorch: B × C × H × W
- Why dense layers struggle with pixels
- Seismic, borehole, core and inspection images

A greyscale image is a grid of numbers

- A scikit-learn digit is an 8 × 8 array of intensities from 0 to 16.
- Row and column indices carry meaning: neighbours are physically adjacent.
- Flattening to 64 numbers keeps the values but hides which ones are neighbours.

	0	1	2	3	4	5	6	7
0	0	0	7	15	13	1	0	0
1	0	8	13	6	15	4	0	0
2	0	2	1	13	13	0	0	0
3	0	0	2	15	11	1	0	0
4	0	0	0	1	12	12	1	0
5	0	0	0	0	1	10	8	0
6	0	0	8	4	5	14	9	0
7	0	0	7	13	13	9	0	0

rendered



8 × 8 array

64 intensities

values 0–16

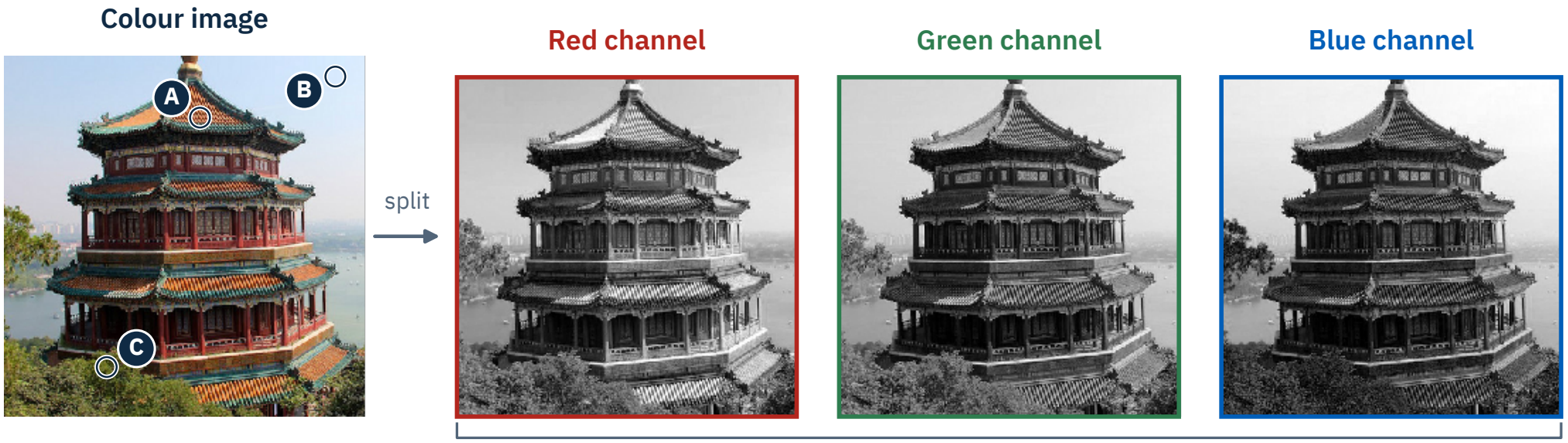
x[1, 4] = 15

row 1, column 4

scikit-learn bundled digit (index 3); exact pixel values.

A colour image stacks three channel arrays

Each pixel is a vector of three numbers; each channel on its own is a greyscale image.



A roof: (R, G, B) = (209, 121, 81)

B sky: (R, G, B) = (216, 234, 254)

C foliage: (R, G, B) = (116, 125, 42)

NumPy / PIL: H × W × C = 360 × 360 × 3, integers 0 (dark) to 255 (bright)

PyTorch: C × H × W = 3 × 360 × 360, usually rescaled to floats in [0, 1]

Crop of scikit-learn's sample image china.jpg (CC BY 2.0, danielbuechele on Flickr); exact 8-bit values.

PyTorch expects batches shaped B × C × H × W

```python
import torch
from sklearn.datasets import load_sample_image

img = load_sample_image("china.jpg")        # NumPy array: H x W x C
print(img.shape, img.dtype)

x = torch.tensor(img).permute(2, 0, 1)      # C x H x W (a copy)
x = x.float().div(255).unsqueeze(0)         # B x C x H x W, values 0-1
print(x.shape, x.dtype)
```

OUTPUT

```
(427, 640, 3) uint8
torch.Size([1, 3, 427, 640]) torch.float32
```

Keras and many image libraries default to channels-last (B × H × W × C): check before you reshape.

PRACTICE

4 min · Individual

Locality reduces the parameter burden

An RGB image has shape $32 \times 32 \times 3$.

1. How many weights connect every pixel value to 100 dense units?
2. How many weights define 16 filters of size 3×3 over the same input channels?
3. Which of the two counts changes if the image grows to 224×224?

DISCUSSION

Dense weight counts grow with every pixel

Weights (biases excluded) for 100 dense units versus 16 convolution filters of size 3 × 3:

307,200

dense layer, 32 × 32 × 3 input

15,052,800

dense layer, 224 × 224 × 3 input

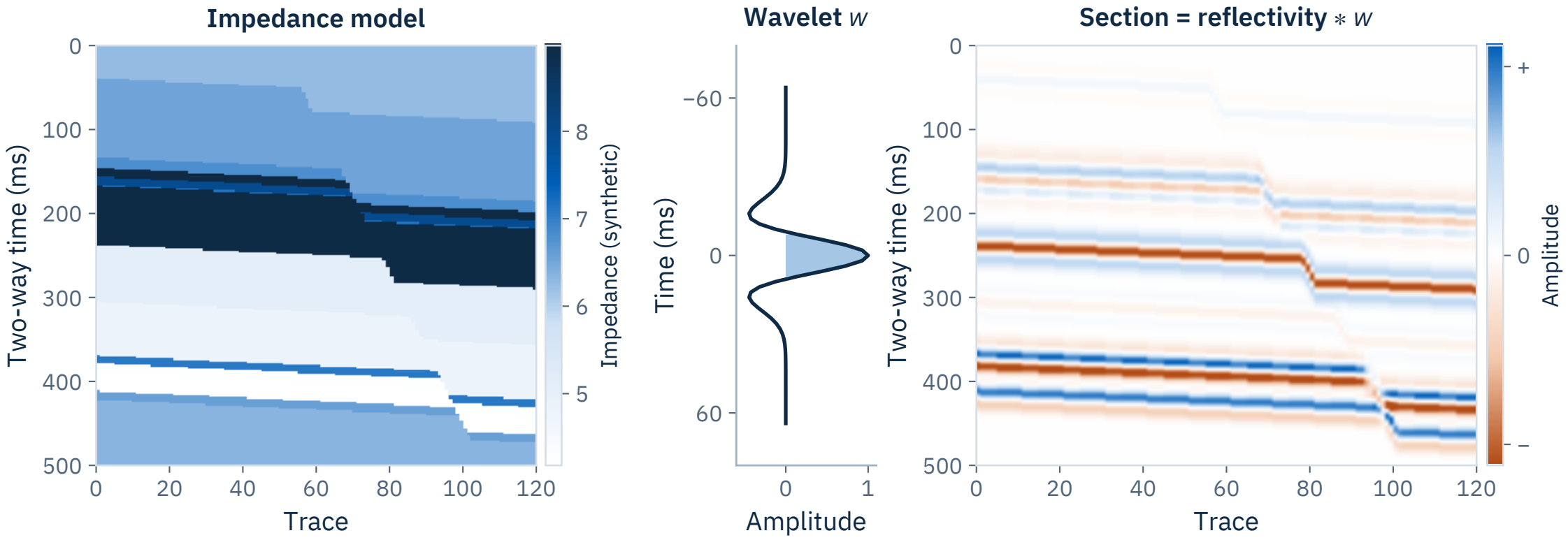
432

16 conv filters, any image size

◆ Convolution saves weights by assuming local patterns that recur; compute still grows with the pixel count.

Seismic sections are images made by convolution

Each trace is reflectivity convolved with a wavelet; faults offset the events.



Synthetic impedance model with a normal fault (seed 7), 25 Hz Ricker wavelet, 2 ms sampling; not field data.

Energy imaging comes in many modalities

Seismic

- Amplitude sections and volumes
- Faults, horizons, salt, facies

Borehole images

- Resistivity or acoustic wall images
- Fractures, bedding, breakouts

Core photographs

- Slabbed core in white or UV light
- Lithology, facies, fractures

Thin sections

- Microscope images of rock slices
- Grains, pores, minerals

Casing inspection

- Ultrasonic thickness by depth and azimuth
- Metal loss, pitting, wear

Drone and solar inspection

- RGB and thermal images of assets
- Hot spots, cracks, soiling

◆ Record pixel spacing, units, tool and acquisition run with every image: they decide augmentation and splits.

PRACTICE

4 min · Pairs

Every tensor axis has a physical meaning

A batch holds 16 casing patches of 32 depth rows × 36 azimuth sectors of 10°, one thickness channel; each row spans 0.1 m.

1. Write the PyTorch shape of the batch.
2. What depth interval and azimuth range does one patch cover?
3. Which axis wraps around, and what does that mean at the patch edges?

PART 2 OF 7

02

Convolution mechanics

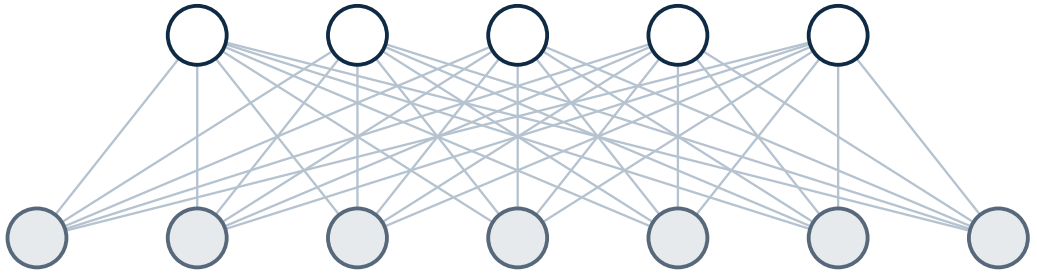
A small kernel, applied everywhere, with a few size settings.

- Weight sharing and sliding windows
- Padding, stride and dilation
- Channels and 1 × 1 convolutions
- Output sizes and parameter counts

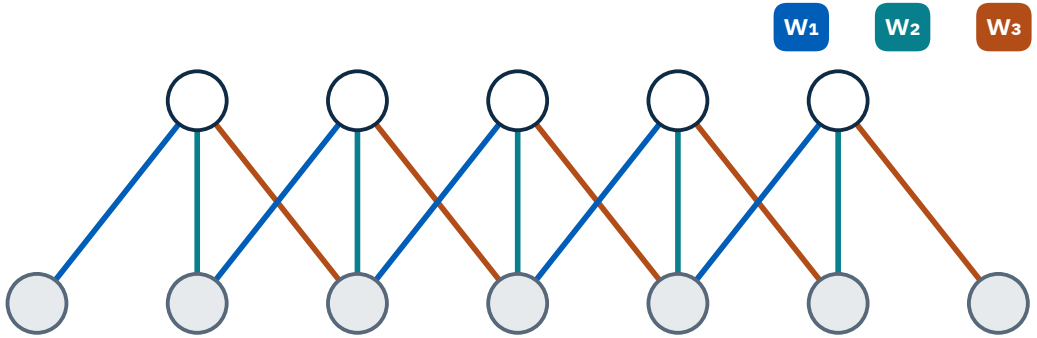
A filter applies the same rule across positions

- Each output combines a local neighbourhood of the input with learned weights.
- The same weights are reused at every position: **weight sharing**.
- A filter spans every input channel; each filter makes one output channel.
- Sharing encodes an assumption: a useful local pattern can recur anywhere.

Dense: every output sees every input — 35 weights



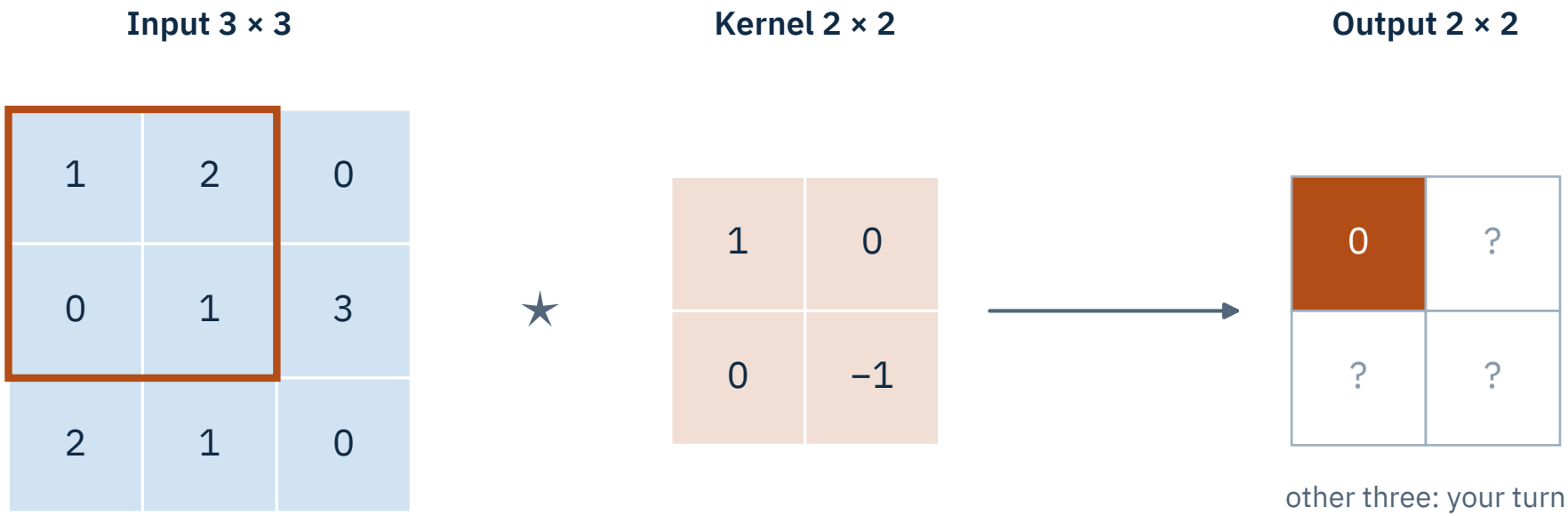
Convolution: 3 shared weights, reused — 3 weights



Schematic: seven inputs, five outputs, kernel size 3, no padding.

One output value is a weighted sum of a patch

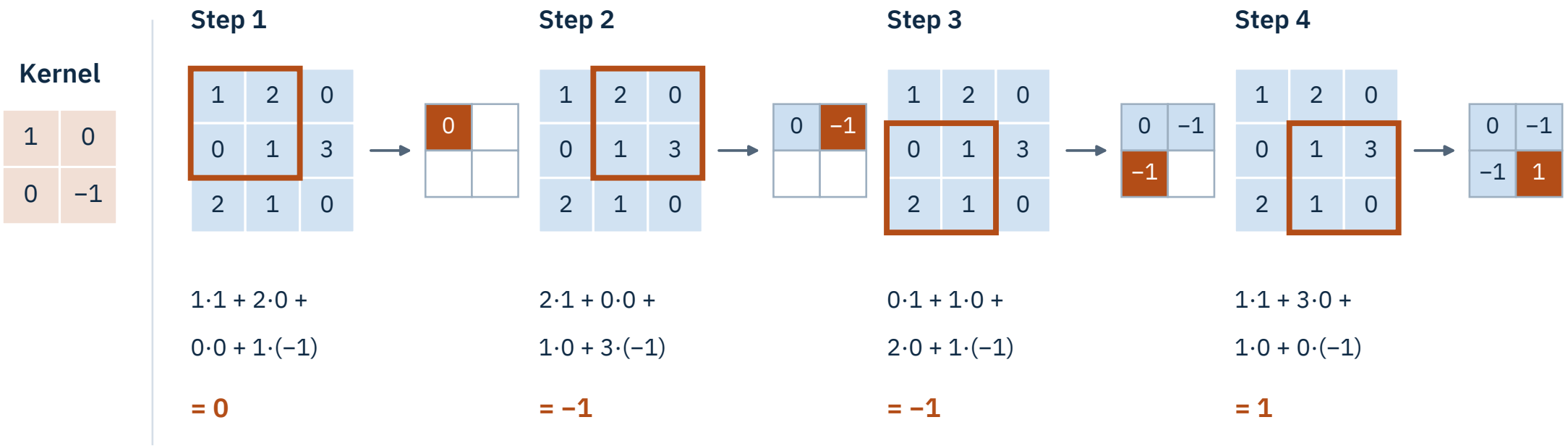
Multiply the highlighted patch by the kernel element-wise and add: 1·1 + 2·0 + 0·0 + 1·(−1) = 0.



Exact cross-correlation (scipy.signal.correlate2d) with one 2 × 2 kernel, stride 1, no padding; three outputs left for the room.

The kernel slides to fill the output map

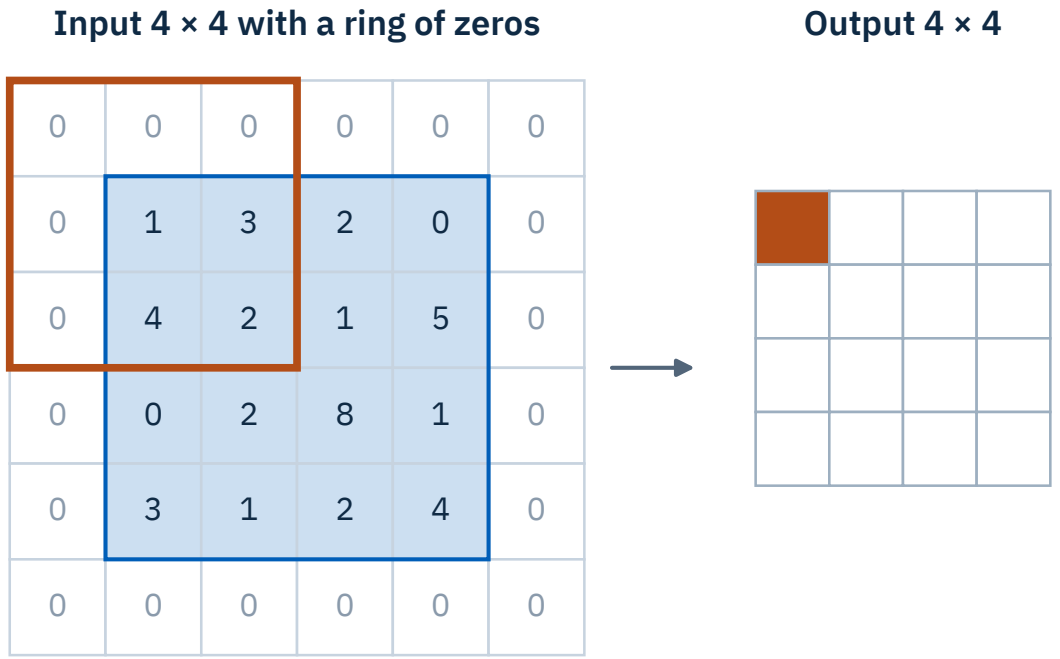
Frameworks compute cross-correlation: the kernel slides without being flipped.



Exact arithmetic on the same numbers as the previous slide; stride 1, no padding.

Zero padding keeps border pixels in play

- **Padding** adds a border so windows can centre on edge pixels.
- With K = 3, P = 1 and stride 1, the output keeps the input size.
- Border outputs mix real values with artificial zeros.



K = 3, P = 1, S = 1: $(4 + 2\cdot1 - 3)/1 + 1 = 4$

The corner window holds 5 padding zeros and 4 real values

Schematic with exact output-size arithmetic; padding width 1.

Azimuth wraps around, depth does not

```python
import torch
import torch.nn.functional as F
from torch import nn

x = torch.randn(16, 1, 32, 36)  # B, C, depth, azimuth
x = F.pad(x, (1, 1, 0, 0), mode="circular")
conv = nn.Conv2d(1, 8, kernel_size=3, padding=(1, 0))
print(x.shape, conv(x).shape)
```

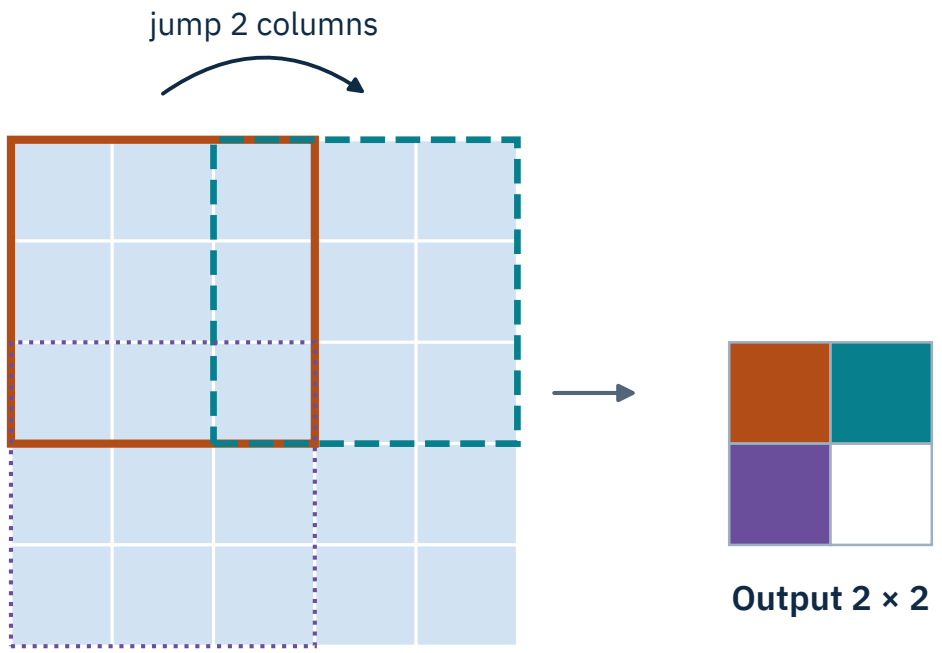
OUTPUT

```
torch.Size([16, 1, 32, 38]) torch.Size([16, 8, 32, 36])
```

- `F.pad` wraps only the last axis, azimuth.
- `padding=(1, 0)` zero-pads depth only.
- `padding_mode="circular"` would wrap depth too.

Stride skips positions and shrinks the map

- **Stride** S moves the window S pixels at each step.
- S = 2 roughly halves height and width.
- Neighbouring outputs now sit S input pixels apart.

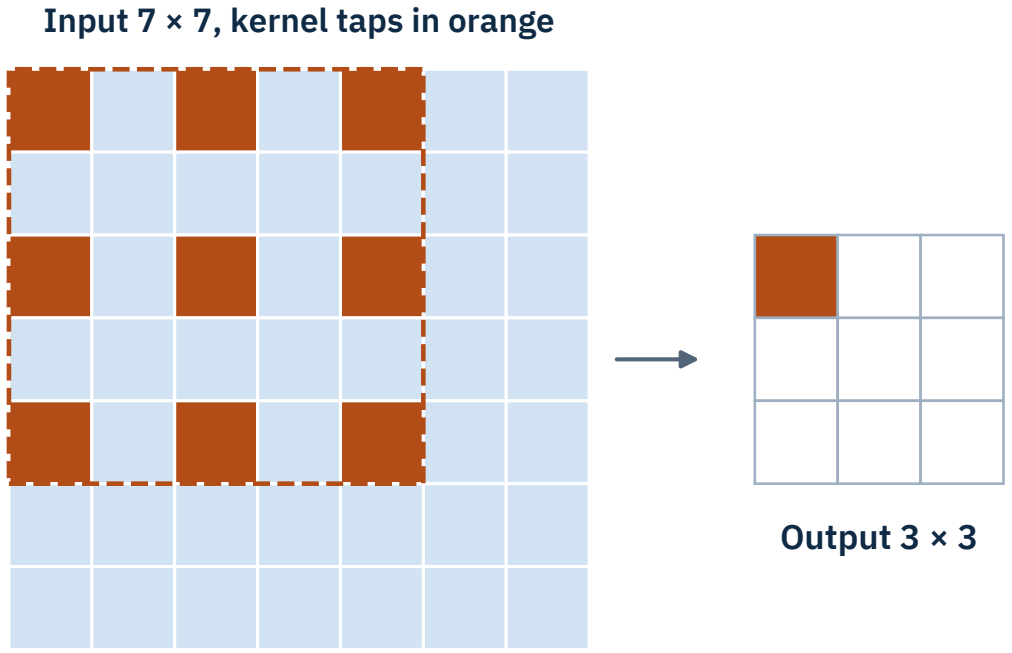


K = 3, S = 2, P = 0: $\lfloor (5 - 3)/2 \rfloor + 1 = 2$

Schematic; three of the four window positions shown.

Dilation widens a kernel's reach, not its weights

- **Dilation** D spaces the kernel taps D pixels apart.
- Nine weights cover a 5 × 5 area when D = 2.
- It adds context without downsampling: segmentation, seismic, 1-D signals.



K = 3, D = 2: taps span D(K – 1) + 1 = 5 pixels

Still 9 weights; (7 – 5)/1 + 1 = 3 outputs per row

Schematic; dilation 2, no padding, stride 1.

Output size follows kernel, stride, padding and dilation

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - D(K-1) - 1}{S} + 1 \right\rfloor$$

- K kernel size, S stride, P padding, D dilation; the same rule gives the width.
- H_{in} = 32: K = 3, P = 1 keeps 32; adding S = 2 gives 16; K = 5, P = 0 gives 28.
- The number of output channels is the number of filters.

PRACTICE

4 min · Individual

Predict the shape before you run the layer

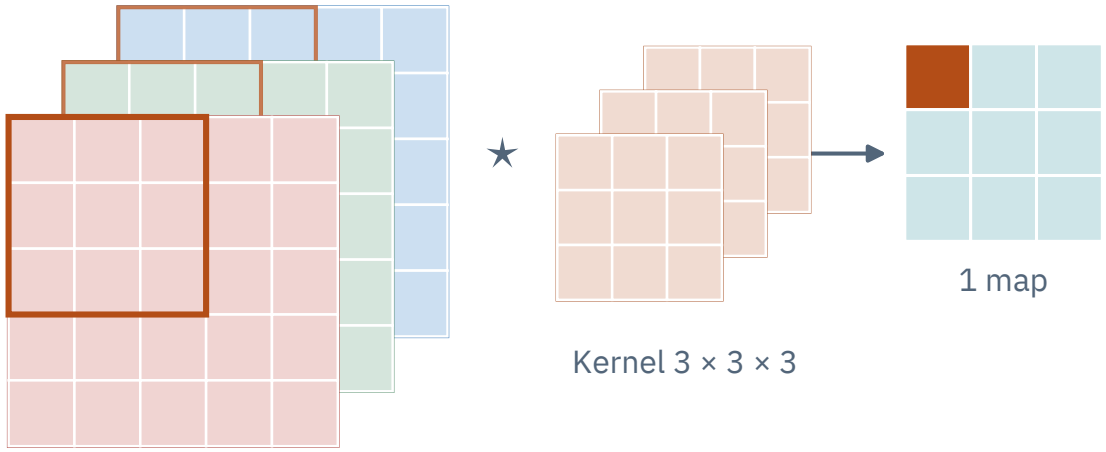
Input: B × 3 × 32 × 32 in PyTorch. Layer: `Conv2d(3, 16, kernel_size=3, stride=2, padding=1)`.

1. State the output shape.
2. Count the trainable parameters, including biases.
3. What changes, and what does not, if the input becomes 64 × 64?

Each kernel spans all input channels and makes one map

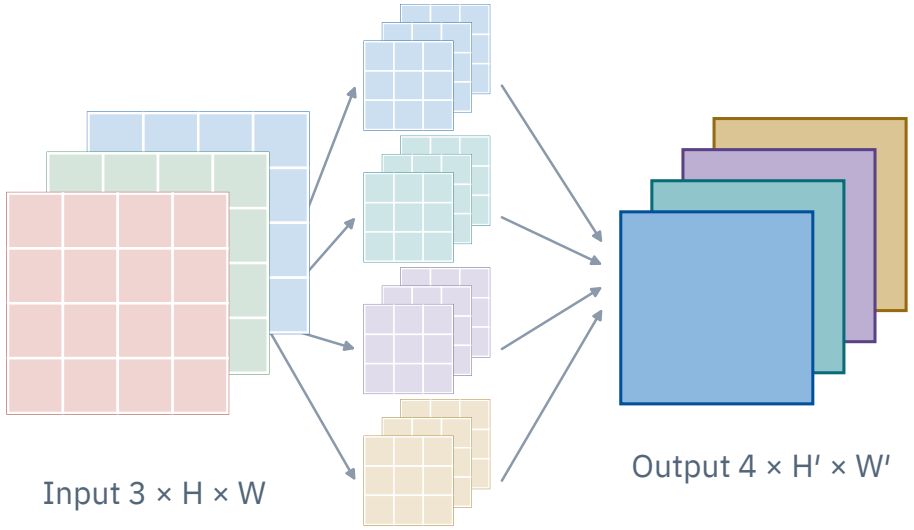
A Conv2d weight tensor has shape (C_{out}, C_{in}, K, K) and $C_{out}\,(C_{in}\,K^2 + 1)$ parameters.

One kernel spans every input channel



27 weights + 1 bias, summed over all channels

C_{out} kernels make C_{out} maps



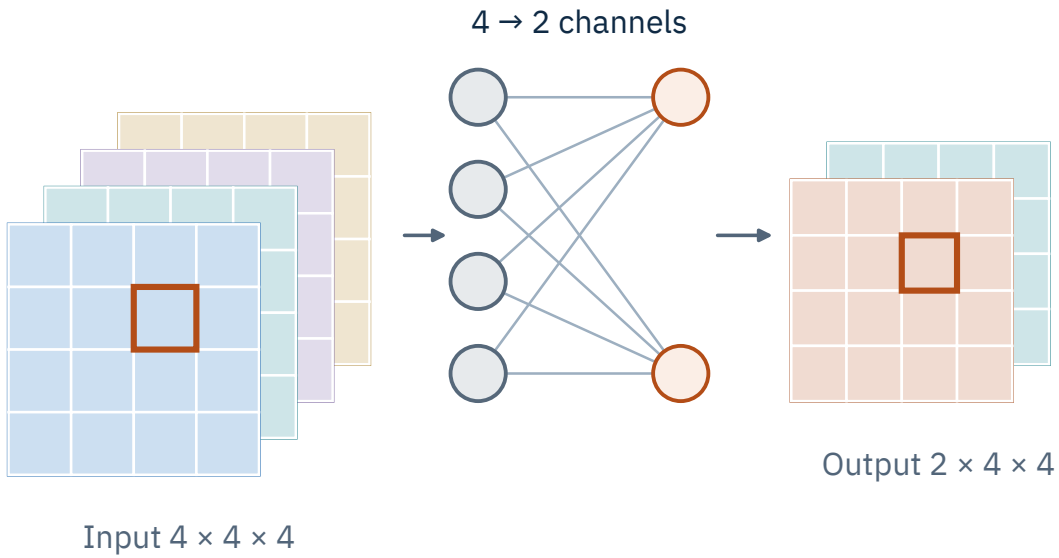
weight shape $(C_{out}, C_{in}, K, K) = (4, 3, 3, 3)$

Schematic; one bias per output channel.

A 1 × 1 convolution mixes channels at each pixel

- A **1 × 1 convolution** is a dense layer applied at every pixel.
- It changes the channel count cheaply: C_{out} (C_{in} + 1) parameters.
- Bottleneck blocks use it before costly 3 × 3 layers.

The same small dense layer at every pixel



Parameters: 2 × (4 + 1) = 10; H and W unchanged

Schematic: four channels mapped to two at every pixel.

PRACTICE

5 min · Pairs

Count the parameters before you build

Count weights and biases for each layer:

1. `Conv2d(64, 128, kernel_size=3, padding=1)`
2. `Conv2d(64, 16, kernel_size=1)`
3. Two stacked 3 × 3 layers (64 → 64 channels) versus one 5 × 5 layer (64 → 64). Which sees further?

DISCUSSION

Two 3 × 3 layers see as far as one 5 × 5 layer

Answers for 64 input and 64 output channels, biases included:

73,856

two stacked 3 × 3 layers

102,464

one 5 × 5 layer

5 × 5

receptive field of both

◆ The stack also puts a ReLU between its layers: more nonlinearity for fewer weights.

Convolution is a dense layer with two assumptions

Nearby values belong together, and useful patterns recur across positions.

Right for images and ordered signals; wrong for shuffled table columns.

PART 3 OF 7

03

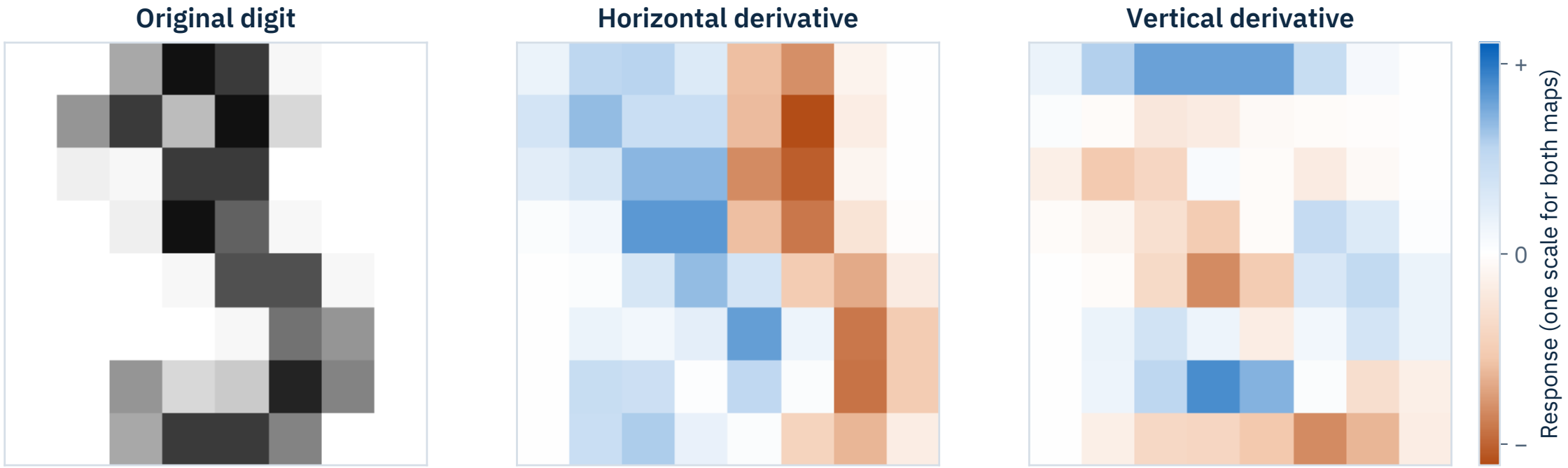
Feature maps and a compact CNN

Stacked filters see wider context; a ledger checks every shape.

- Hand-set and learned filters
- Receptive fields
- Pooling, strided convolution, global averaging
- Equivariance, invariance and the Lab 1 model

Filters respond to local structure

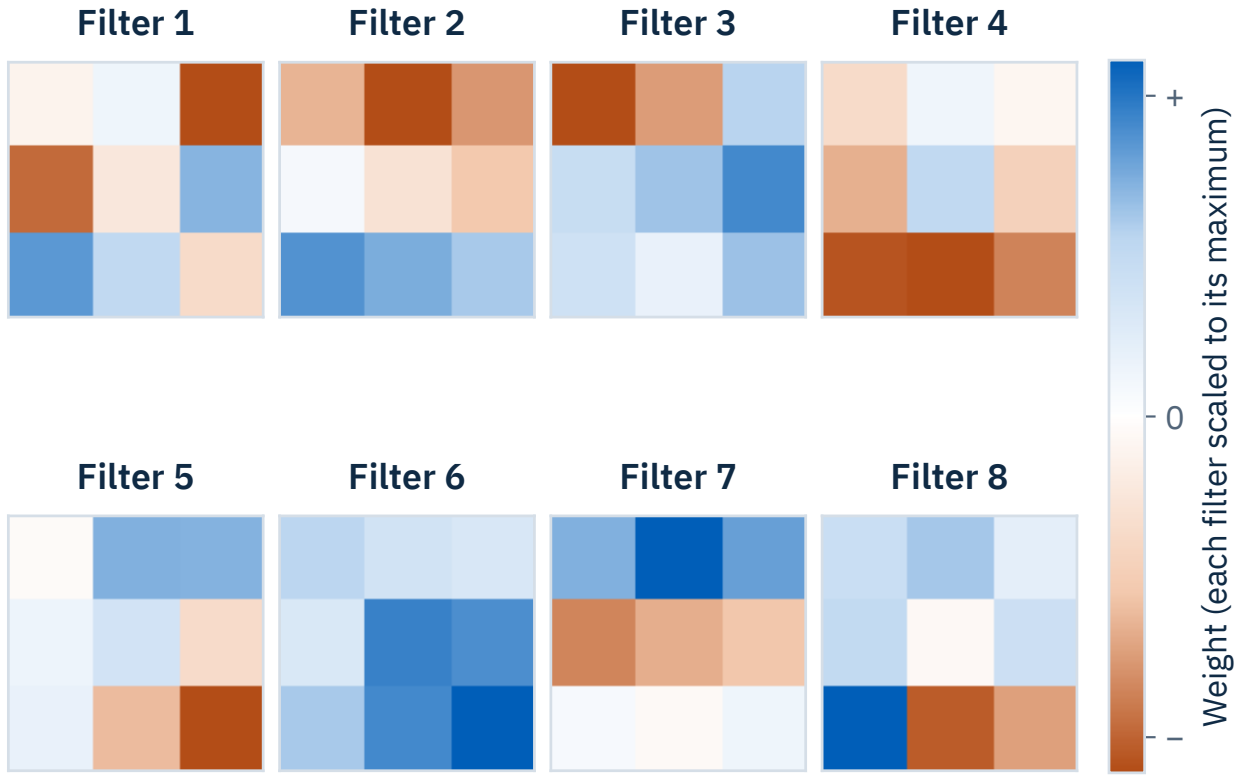
Sobel kernels respond where intensity changes; blue and orange mark opposite signs.



Fixed Sobel kernels applied to scikit-learn digit 3 with zero padding; hand-specified, not learned; one scale centred on zero.

Learned first-layer filters act as local detectors

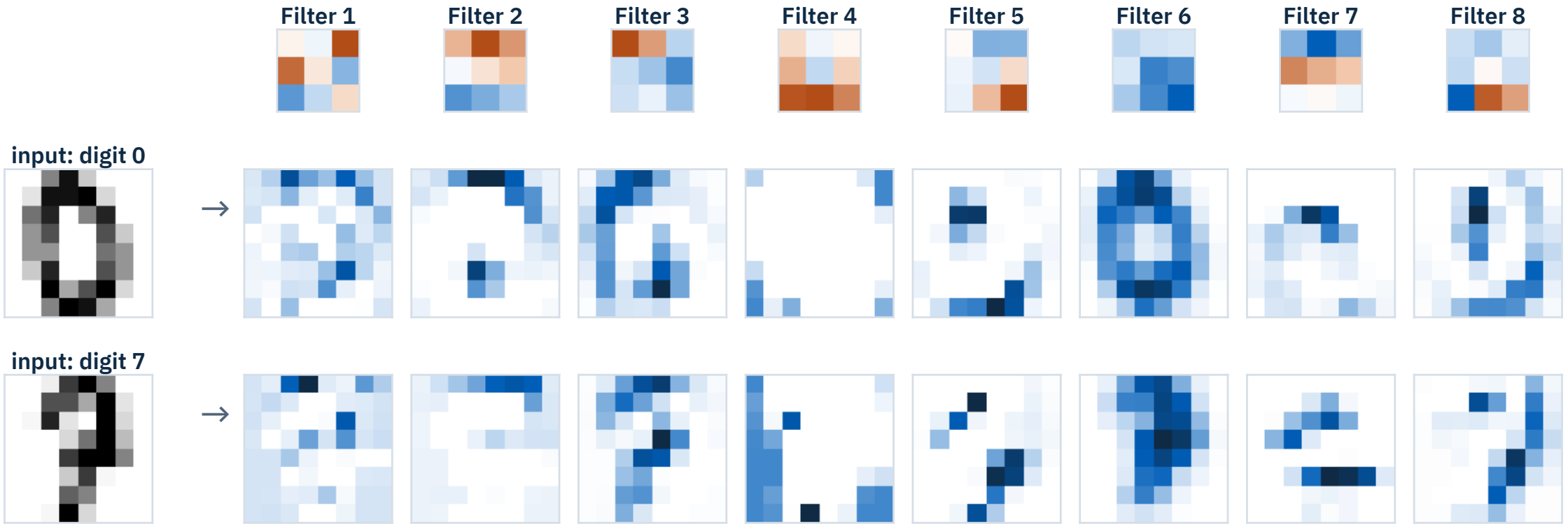
- A tiny CNN trained on 1,437 digits learned these eight 3 × 3 kernels.
- Several resemble oriented edge or stroke detectors.
- Not every filter has a human name; another seed gives different filters.



Tiny CNN (1,418 parameters) trained on scikit-learn digits, seed 0, CPU; 95.0% accuracy on 360 held-out digits.

Feature maps show where each filter fires

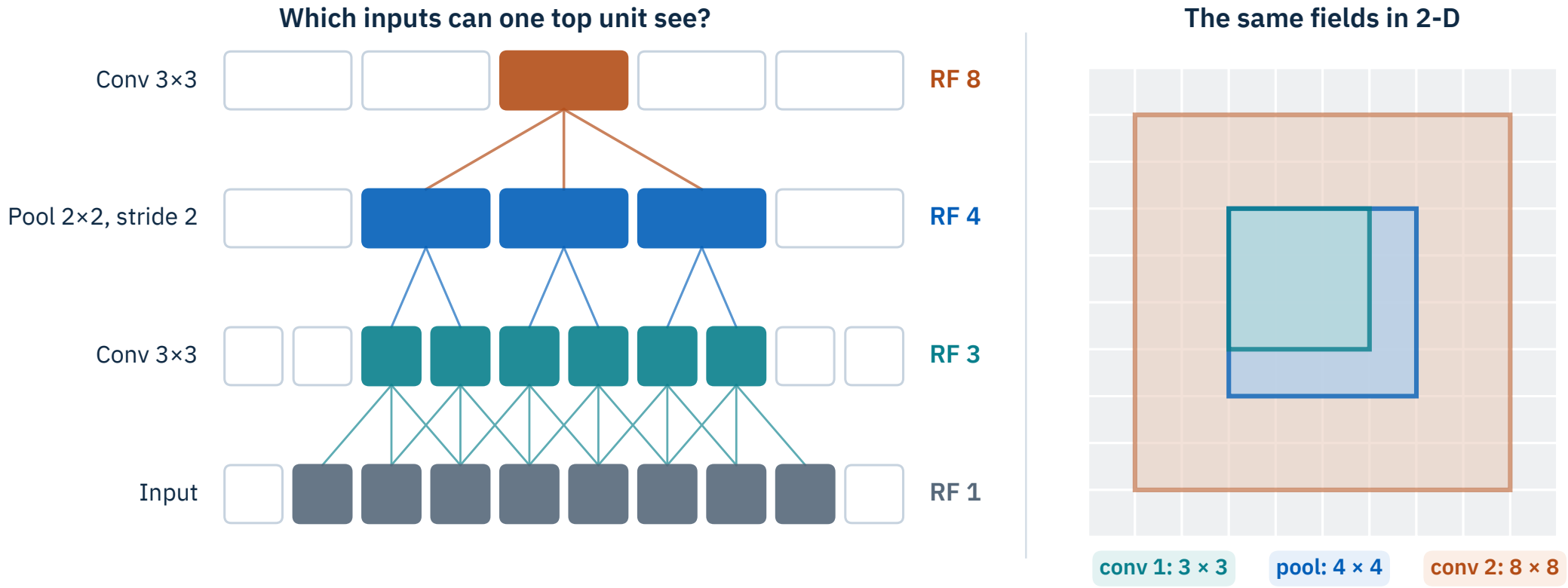
Maps keep the image layout: dark cells show where each filter's pattern appears.



First-layer ReLU maps of the tiny CNN for two held-out digits; each column scaled to its own maximum.

Receptive fields grow with depth

After conv 3 × 3, pool 2 × 2 and conv 3 × 3, one unit depends on an 8 × 8 patch of the input.



Exact receptive-field bookkeeping for stride-1 convolutions and stride-2 pooling.

A receptive-field ledger tracks jump and size

Layer	Kernel / stride	Jump j	Receptive field r
Input	–	1	1
Conv 1	3 / 1	1	3
Pool	2 / 2	2	4
Conv 2	3 / 1	2	8

Each layer applies $r \leftarrow r + (K - 1)\, j$, then $j \leftarrow j \cdot S$, with kernel K and stride S.

PRACTICE

4 min · Pairs

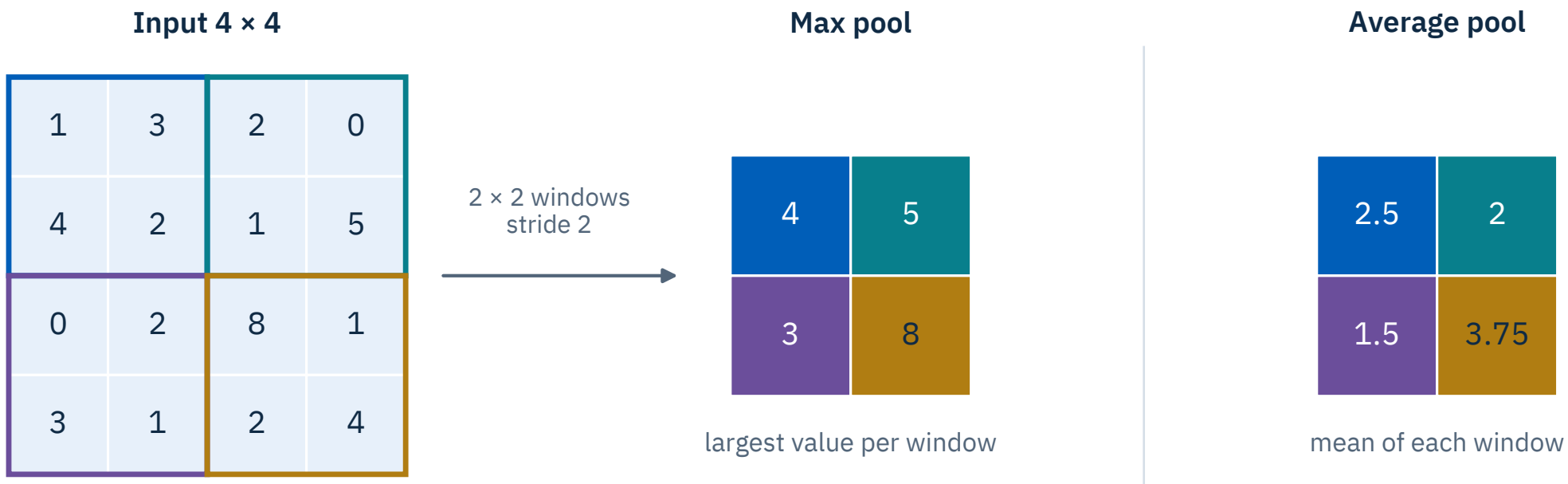
Extend the receptive-field ledger

Continue the table after Conv 2:

1. Add a pool 2 / 2, then a conv 3 / 1. Give j and r after each.
2. How many stride-1 3 × 3 convolutions alone would cover at least the same field?
3. Each depth row spans 0.1 m. How tall is that field in metres?

Pooling summarizes neighbourhoods

Max pooling keeps the strongest response in each window; average pooling keeps the mean.



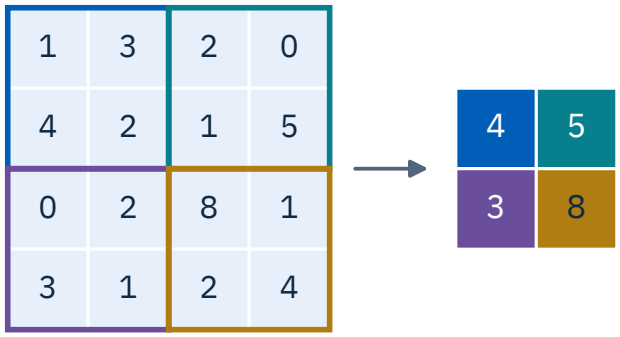
Exact max and average pooling on one 4 × 4 matrix, 2 × 2 windows, stride 2; colours link each window to its output.

Strided convolution and global averaging also downsample

Modern CNNs often use strided convolutions and end with global average pooling.

Max pool 2 × 2, stride 2

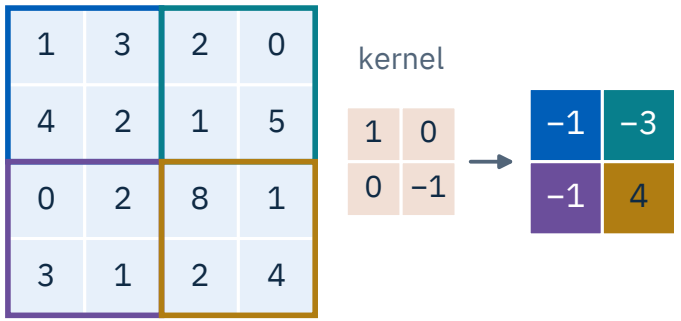
fixed rule, 0 parameters



keeps the largest value per window

Strided convolution 2 × 2, stride 2

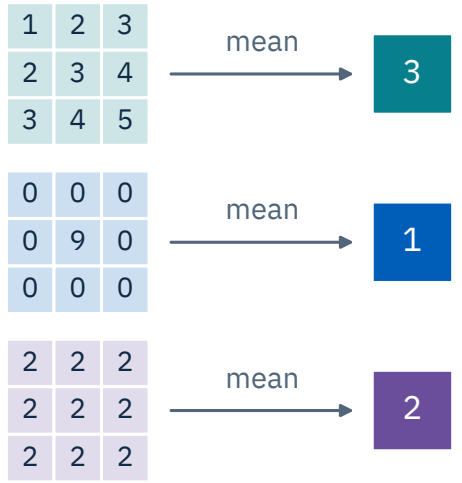
learned: 4 weights + 1 bias



weighted sum per window, learned

Global average pooling

one number per channel

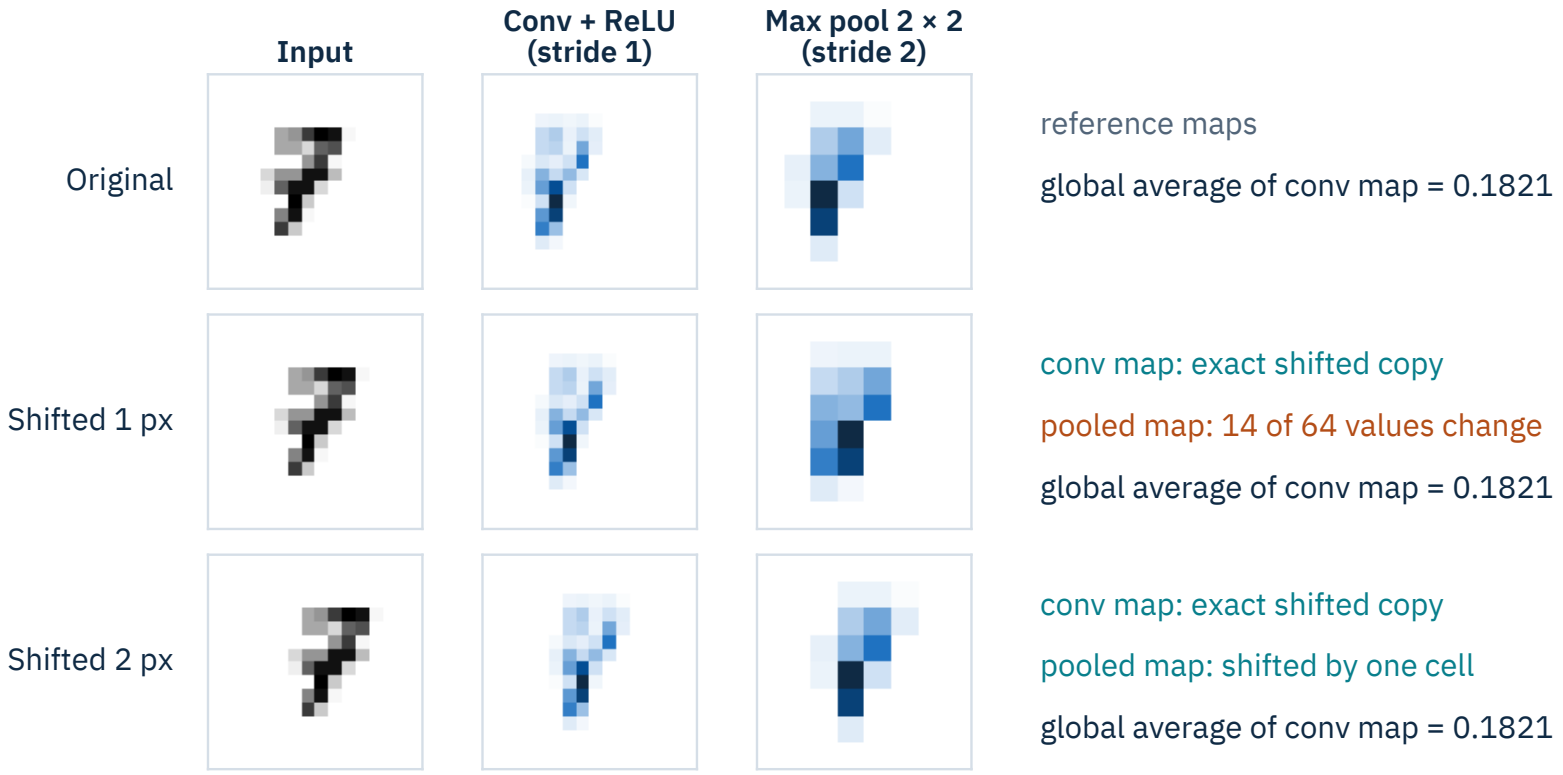


C × H × W → C, for any H × W

Exact arithmetic; the strided convolution uses kernel [[1, 0], [0, −1]] and zero bias.

Equivariance and invariance are different

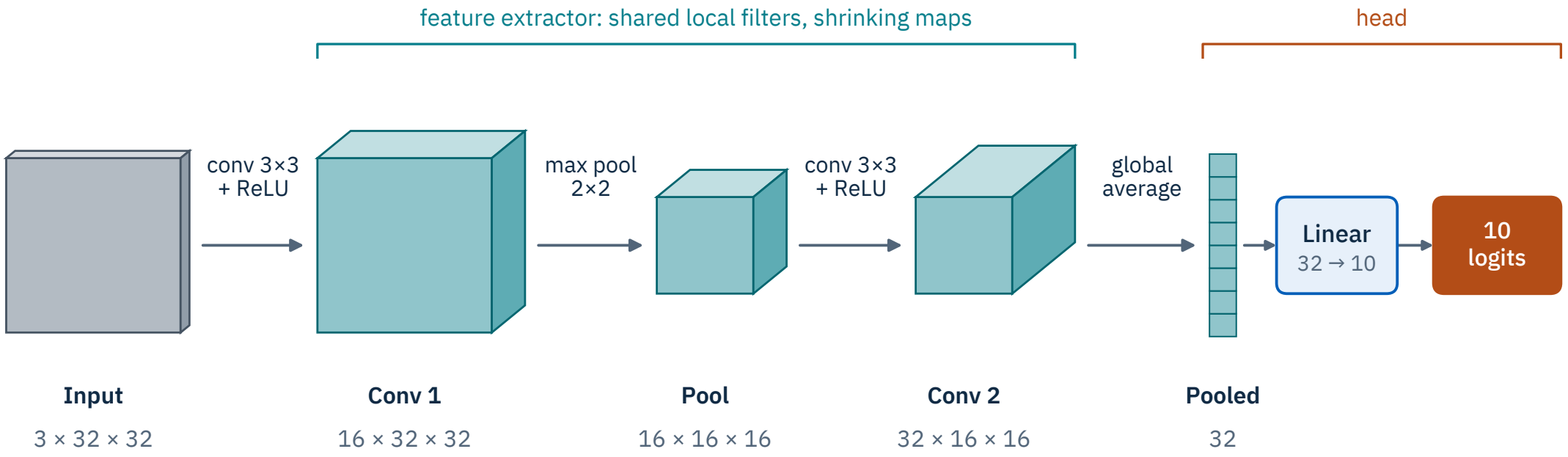
Shifted input, shifted map: **equivariance**. Same average: **invariance**.



Exact computation: a digit on a 16 × 16 zero canvas, fixed Sobel kernel with zero padding, ReLU, 2 × 2 max pooling.

A CNN alternates feature extraction and downsampling

Spatial size shrinks while channels grow; a small head turns pooled features into class logits.



Schematic of the compact CNN on the next slides; shapes are C × H × W for one image.

A shape ledger records every tensor in the model

Operation	Output for one batch	Purpose
Input	B × 3 × 32 × 32	RGB images
Conv 3 × 3, 16 filters, padding 1	B × 16 × 32 × 32	Local features
ReLU, max pool 2 × 2	B × 16 × 16 × 16	Nonlinearity, downsampling
Conv 3 × 3, 32 filters, padding 1, ReLU	B × 32 × 16 × 16	Feature combinations
Global average pool, linear	B × 10	Class logits

The compact CNN fits in one Sequential block

```
from torch import nn

model = nn.Sequential(
    nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(), nn.Linear(32, 10),
)
criterion = nn.CrossEntropyLoss()
```

No softmax before `CrossEntropyLoss`: it takes raw logits and integer class labels 0 to 9.

Run a dummy batch before any training

```python
import torch

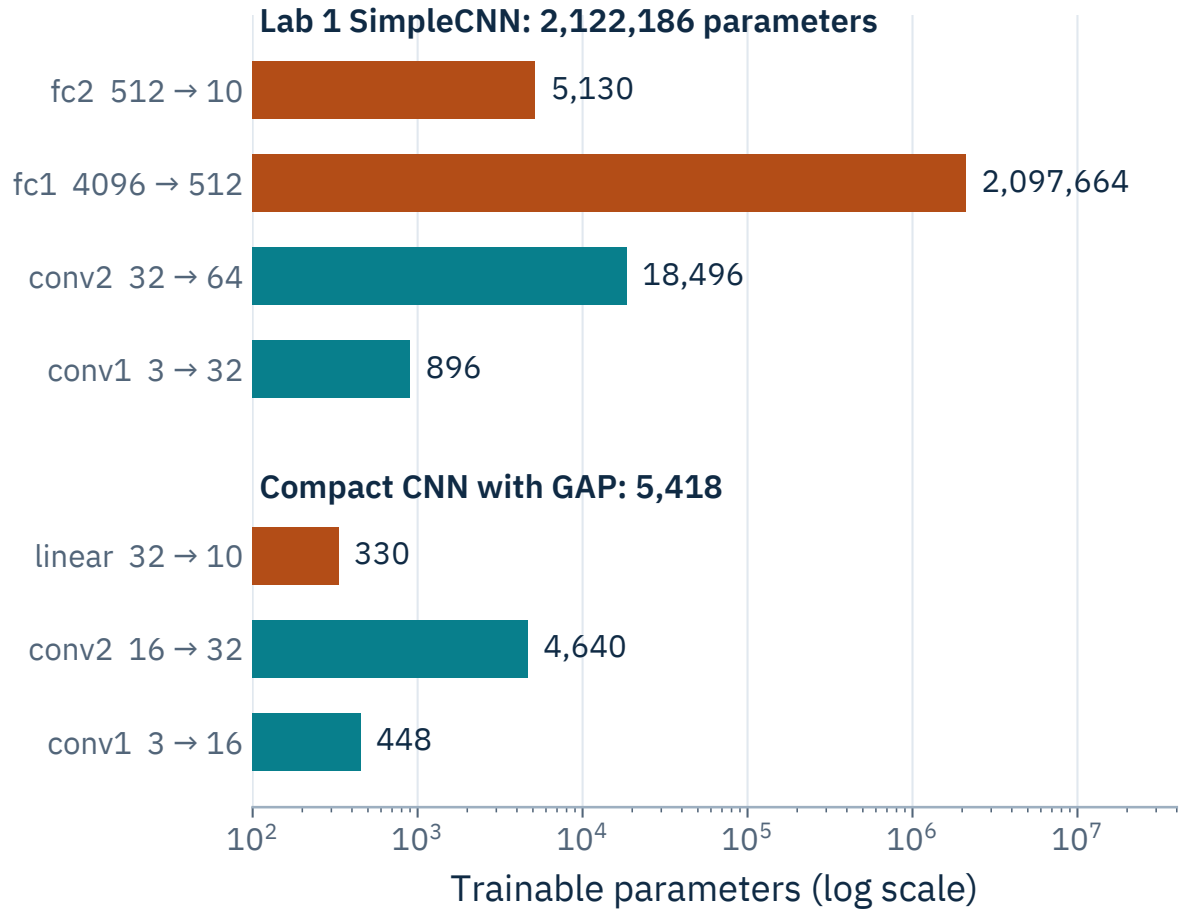
x = torch.randn(8, 3, 32, 32)
for layer in model:
    x = layer(x)
    print(f"{layer.__class__.__name__:<18} {tuple(x.shape)}")
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
Conv2d             (8, 16, 32, 32)
ReLU               (8, 16, 32, 32)
MaxPool2d          (8, 16, 16, 16)
Conv2d             (8, 32, 16, 16)
ReLU               (8, 32, 16, 16)
AdaptiveAvgPool2d  (8, 32, 1, 1)
Flatten            (8, 32)
Linear             (8, 10)
5418
```

Parameters concentrate in dense heads

- Lab 1's SimpleCNN has 2,122,186 parameters; 98.8% sit in one dense layer.
- Global average pooling shrinks the head to 330 parameters.
- Fewer parameters is not automatically better: compare on validation data.



Exact counts (weights and biases), checked with PyTorch.

LABORATORY

60 min · Pairs

Laboratory 1: a checked CNN on CIFAR-10

- Print every layer's output shape for one batch; count the parameters.
- Hold out 5,000 training images for validation; keep the test set for the end.
- Train SimpleCNN and plot training and validation loss and accuracy.
- Change one thing (BatchNorm and dropout, or a GAP head) and compare on validation.
- Inspect per-class accuracy and eight misclassified images.

NOTEBOOK

`exercises/01_cnn_cifar10/CNN_CIFAR10_Student.ipynb`

EVIDENCE TO SUBMIT

- Shape printout and parameter count
- Training and validation curves
- One controlled comparison, three failure cases

Prepare: torchvision installed and CIFAR-10 (about 170 MB) in ./data. Alternative: exercises/02_cnn_svhn (separate download).

DISCUSSION

Lab 1 debrief: read the curves and the errors

Shapes

- 64 × 8 × 8 = 4,096 inputs must match fc1
- A GAP head removes that coupling

Curves

- Is the train–validation gap growing?
- Compare models at the same epoch budget

Errors

- Which classes are confused, and do they look alike?
- What does 32 × 32 resolution hide?

Evidence

- One change per comparison
- Test set used once, at the end

PART 4 OF 7

04

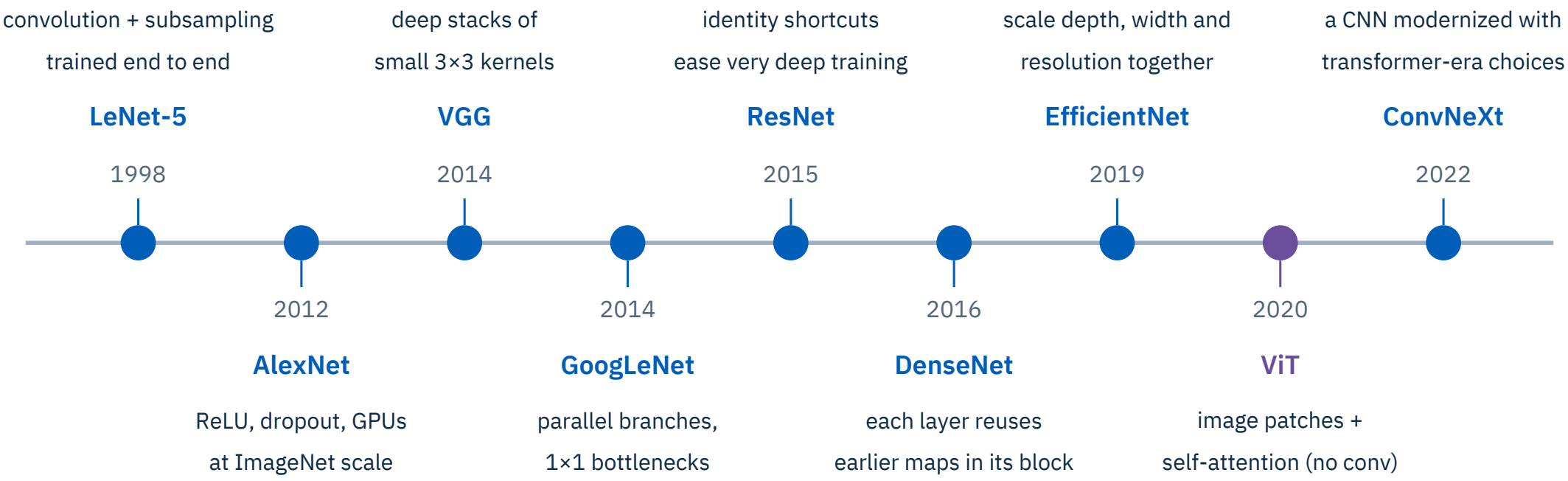
Architecture ideas

From LeNet to ConvNeXt, one reusable design idea at a time.

- A timeline of design ideas
- Residual connections
- Batch normalization in CNNs
- Choosing a backbone

Architectures evolved by adding reusable ideas

Read the history as a toolbox, not a leaderboard: each name contributed an idea still in use.



Schematic timeline, not to scale; years of first publication (conference or preprint); no benchmark results.

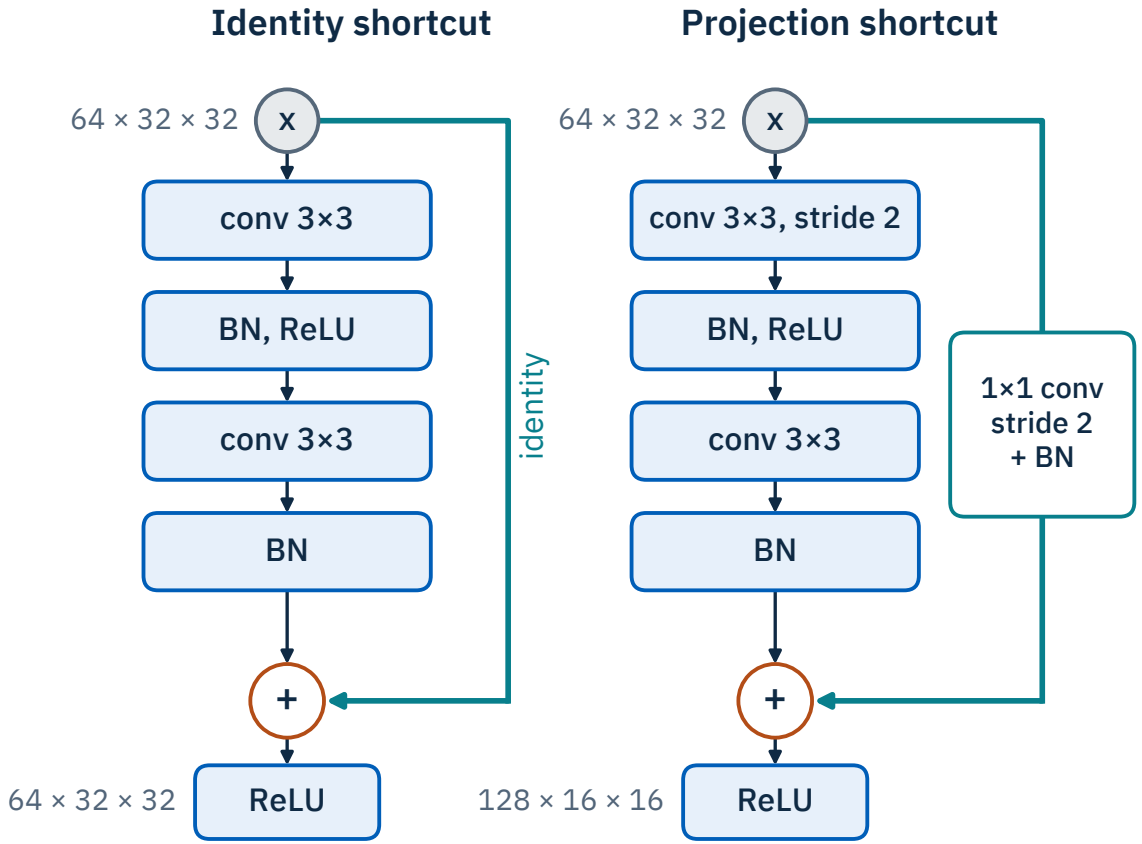
Each architecture family leaves a practical lesson

Family	Idea to retain	Practical lesson
LeNet / AlexNet	Learned convolutional feature hierarchies	Local features can support recognition
VGG	Repeated small kernels	Depth trades compute for capacity
Inception	Parallel branches, 1 × 1 bottlenecks	Mix scales at modest cost
ResNet / DenseNet	Shortcut paths between layers	Identity paths help optimization
MobileNet / EfficientNet	Efficiency-oriented design	Match capacity to deployment constraints

Residual connections preserve an identity path

$$y = x + F(x)$$

- The branch *F* learns a correction to its input.
- Addition needs matching shapes, or a 1 × 1 projection shortcut.
- The identity route helps gradients and signals pass through deep stacks.



Schematic: identity shortcut (left) and projection shortcut (right).

PRACTICE

4 min · Pairs

Make the residual addition shape-compatible

Input x is $64 \times 32 \times 32$. The branch starts with a 3×3 convolution, stride 2, padding 1, 128 filters.

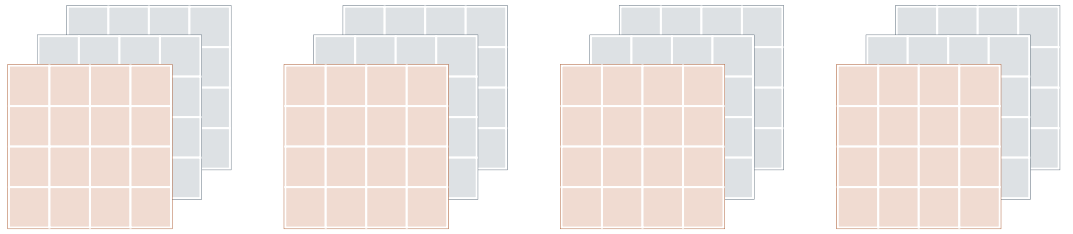
1. What shape does the branch output $F(x)$ have?
2. What must the shortcut do before the addition?
3. How many weights does a 1×1, stride-2 projection from 64 to 128 channels have, without bias?

Batch normalization standardizes each channel

- BatchNorm2d computes one mean and variance per channel over B × H × W.
- Learned scale γ and shift β restore flexibility.
- Evaluation uses running statistics, so train and eval modes differ.
- Tiny batches give noisy statistics; GroupNorm is one alternative.

One mean and variance per channel

each image: C = 3 channel sheets, channel c in front



channel c across the batch and positions: B × H × W values

$$\hat{x} = (x - \mu_c) / \sqrt{\sigma_c^2 + \varepsilon}, \quad y = \gamma_c \hat{x} + \beta_c$$

BatchNorm2d(C): 2C learned (γ, β) + 2C running statistics

Schematic of BatchNorm2d statistics; normalization in general is covered on Day 2.

Frozen weights do not freeze BatchNorm statistics

```python
import torch
from torch import nn

torch.manual_seed(0)
bn = nn.BatchNorm2d(3)
for p in bn.parameters():
    p.requires_grad = False        # gamma and beta frozen

x = torch.randn(16, 3, 8, 8) * 2 + 5
bn.train(); bn(x)                  # training mode: statistics update
print(bn.running_mean)
bn.eval()                          # evaluation mode: running statistics
print(f"{bn(x).mean():.2f}")
```

OUTPUT

```
tensor([0.5045, 0.4930, 0.5014])
3.94
```

ARCHITECTURE IDEAS

Choose a backbone from data, compute and evidence

Little labelled data

- Start from pretrained features
- Freeze most of the backbone

Strong domain shift

- Fine-tune the later stages
- Validate on held-out wells or surveys

Edge or real-time use

- Efficient families such as MobileNet or EfficientNet
- Measure latency on the target hardware

Ample data and compute

- Larger models become affordable
- Still compare with a small baseline

◆ No architecture choice replaces a grouped validation design.

PRACTICE

3 min · Individual

Match each design idea to the problem it solves

Choose from: residual connection, global average pooling, 1 × 1 bottleneck, batch normalization.

1. A deep plain stack trains worse than a shallower one.
2. A flatten-and-dense head holds 99% of the parameters.
3. 256 channels make every 3 × 3 layer expensive.
4. Activation scales drift between layers during training.

PART 5 OF 7

05

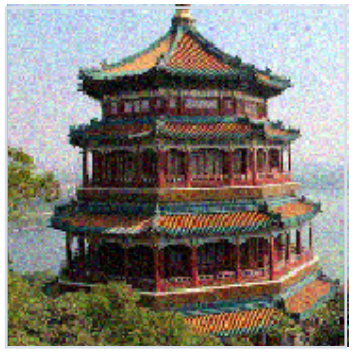
Training and transfer learning

Data, augmentation and honest splits first, then pretrained features.

- Augmentation that preserves the label
- Class imbalance in defect data
- Source-grouped evaluation
- Feature extraction and fine-tuning

Augmentation creates plausible new training views

Each transform encodes a claim: the label does not depend on this change.

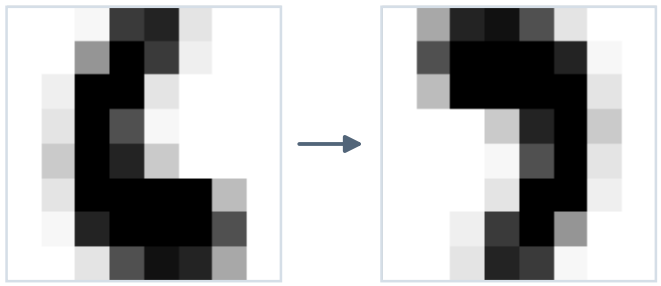
Original	Horizontal flip	Random crop	Rotation	Brightness	Gaussian noise
	p = 0.5	70% side, resized	15°, black corners	× 1.3, clipped	σ = 0.08
					
reference	*mirror symmetry*	*framing and scale*	*orientation*	*illumination*	*sensor noise*

scikit-learn sample image china.jpg (CC BY 2.0), 360 × 360 crop resized to 160 × 160; transforms computed with NumPy and SciPy, seed 3.

Some transforms silently change the label

Whether a transform is safe depends on the physics and on how the target is defined.

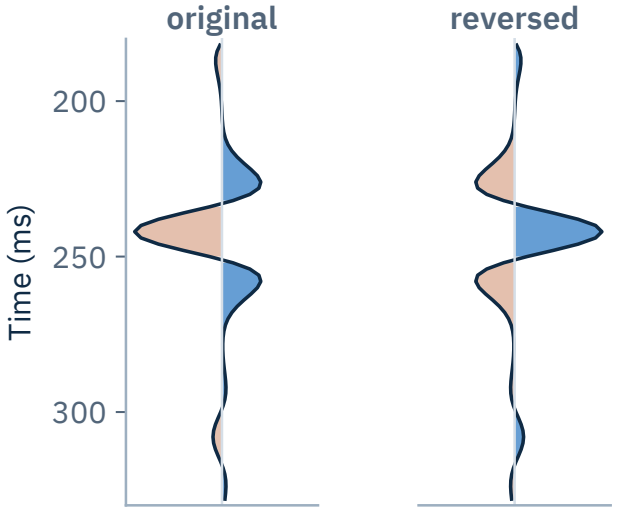
Rotate a “6” by 180°



label 6 → reads as 9

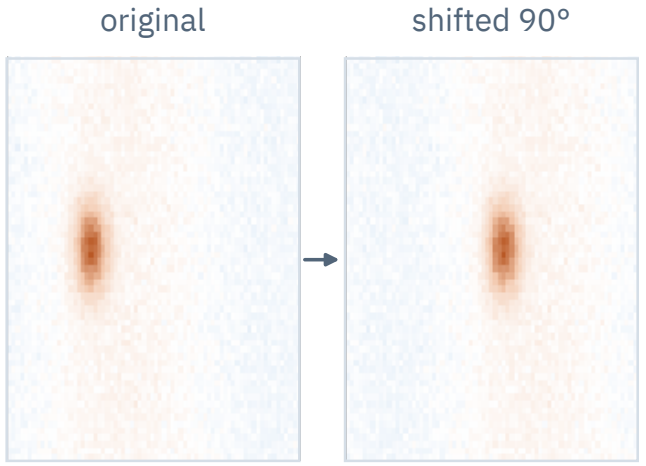
✗ label changes

Reverse seismic polarity



▲ label may change

Rotate a casing map (azimuth)



pits are still pits

✓ label preserved

scikit-learn digit; synthetic seismic trace (seed 7); synthetic thickness map (seed 5) shifted by 18 of 72 sectors.

PRACTICE

4 min · Pairs

Augmentation must preserve the task label

You classify casing defects from inspection images.

1. Is a horizontal flip physically meaningful for this acquisition geometry?
2. Could cropping remove the defect and leave its original label?
3. Which transforms belong only in training?

Training and evaluation use different transforms

```python
import torch
from torchvision.transforms import v2

MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
train_tf = v2.Compose([
    v2.ToImage(),
    v2.RandomResizedCrop(224, scale=(0.8, 1.0), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ColorJitter(brightness=0.2),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(MEAN, STD),
])
eval_tf = v2.Compose([
    v2.ToImage(), v2.Resize(256, antialias=True), v2.CenterCrop(224),
    v2.ToDtype(torch.float32, scale=True), v2.Normalize(MEAN, STD),
])
```

ImageNet statistics shown; use those of your pretrained weights or of your training data.

Accuracy hides rare defects

5,000 inspection patches, 100 of them defective (2%). A model that always answers "normal":

98%	0%	0
accuracy	defect recall	defects found

- Report recall and precision per class and per well; Machine Learning Module 5 covers rare-event metrics.

Class weights rebalance the loss, not the data

```python
import torch
from torch import nn

y_train = torch.tensor([0] * 4900 + [1] * 100)
counts = torch.bincount(y_train, minlength=2)
weights = counts.sum() / (2 * counts.float())
print(weights)
criterion = nn.CrossEntropyLoss(weight=weights)
```

OUTPUT

```
tensor([ 0.5102, 25.0000])
```

- Each defective patch now weighs 49 times a normal one in the loss.
- Alternatives: WeightedRandomSampler, focal loss, threshold tuning.
- Choose the decision threshold on validation data.

A valid image evaluation separates sources

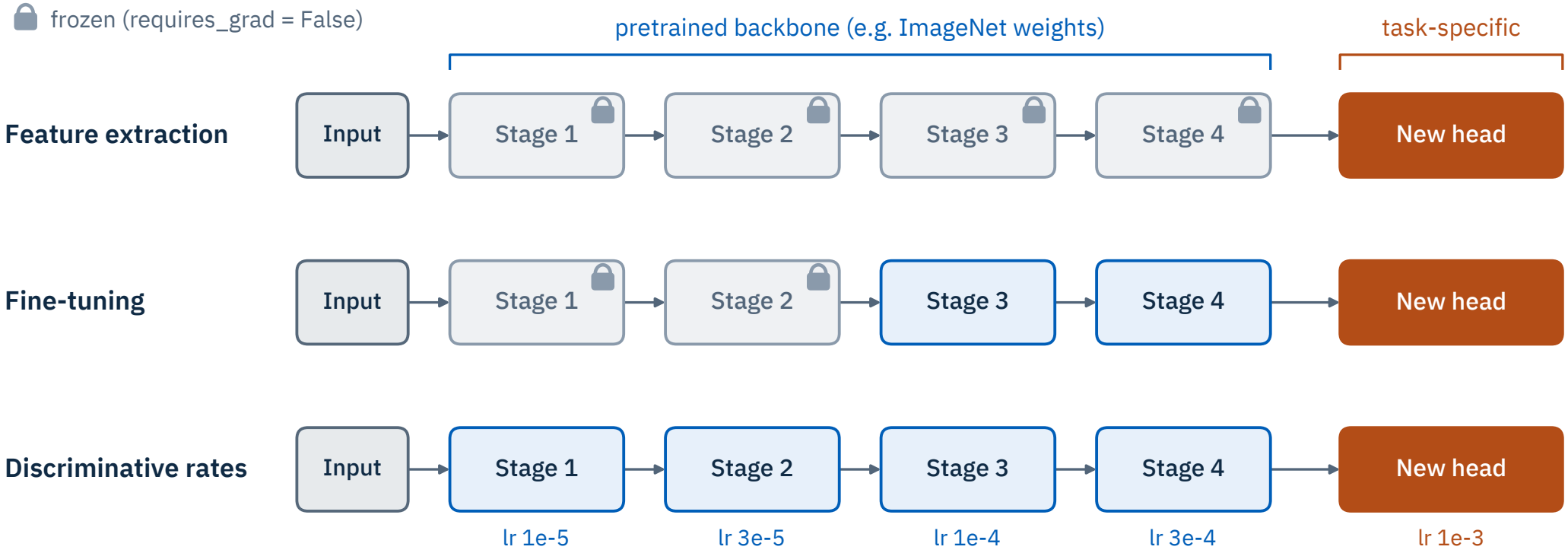
Overlapping windows are near-duplicates: split by well, survey or asset first.



Schematic: 32-row windows with stride 16 (50% overlap); random assignment drawn with seed 8.

Transfer learning reuses a pretrained backbone

Pretrained features are a hypothesis about the new domain; test it.



Schematic of three adaptation strategies; learning rates are illustrative.

Adaptation proceeds from frozen features to fine-tuning

Stage	Trainable parameters	Main question
Feature extraction	New task head	Are the existing features useful?
Fine-tuning	Head and later backbone stages	Does adaptation improve validation performance?
Discriminative rates	All, smaller rates near the input	Does gentler updating of early layers help?

Use the preprocessing that belongs to the chosen pretrained weights.

Load pretrained weights with their own preprocessing

```
from torchvision.models import resnet18, ResNet18_Weights
from torch import nn

weights = ResNet18_Weights.DEFAULT
model = resnet18(weights=weights)
for parameter in model.parameters():
    parameter.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, n_classes)
preprocess = weights.transforms()
```

The new `fc` layer is created after freezing, so only its parameters train.

Discriminative learning rates use parameter groups

```python
import torch

for name, p in model.named_parameters():
    p.requires_grad = name.startswith(("layer4.", "fc."))

optimizer = torch.optim.AdamW([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3},
], weight_decay=1e-4)
```

Frozen stages stay out of the optimizer; keep their BatchNorm layers in eval mode while fine-tuning.

PRACTICE

5 min · Pairs

Choose an adaptation strategy for three datasets

For each dataset: feature extraction, fine-tuning or training from scratch? And how would you split it?

1. 2,000 labelled core photographs of six lithologies from 12 wells.
2. 300 labelled seismic patches from one survey, to be used on a second survey.
3. 50,000 labelled ultrasonic casing patches from 40 wells.

06

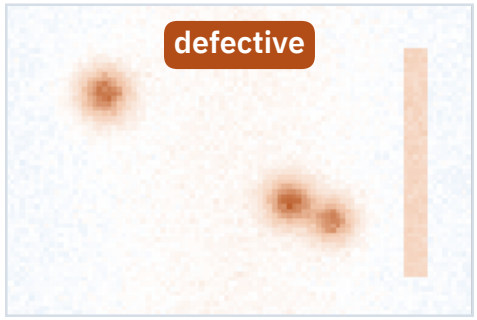
Beyond classification

The same features can return boxes, masks and evidence maps.

- Classification, detection, semantic and instance segmentation
- Intersection over union and U-Net
- Occlusion and gradient maps
- Shortcut learning

Vision tasks differ in what they return

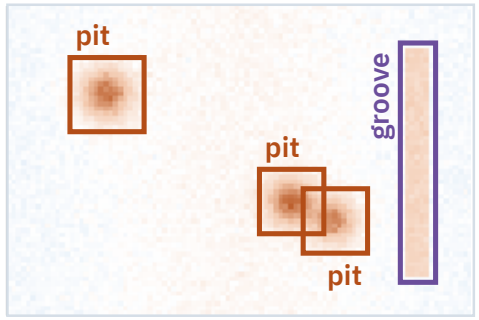
The label format decides the model, the loss and the metric.



Classification

one label per image

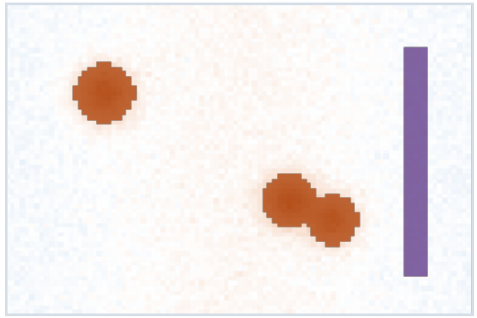
“defects present”



Object detection

box + class + score per object

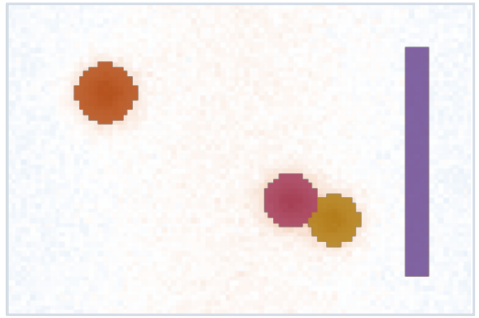
matched to truth by IoU



Semantic segmentation

a class for every pixel

pits merge into one region



Instance segmentation

a separate mask per object

touching pits stay separate

Synthetic thickness patch (seed 12) with three pits and a groove; labels derived from the known geometry, not from a model.

Object detectors predict boxes, classes and scores

A detector turns image features into a list of scored boxes.

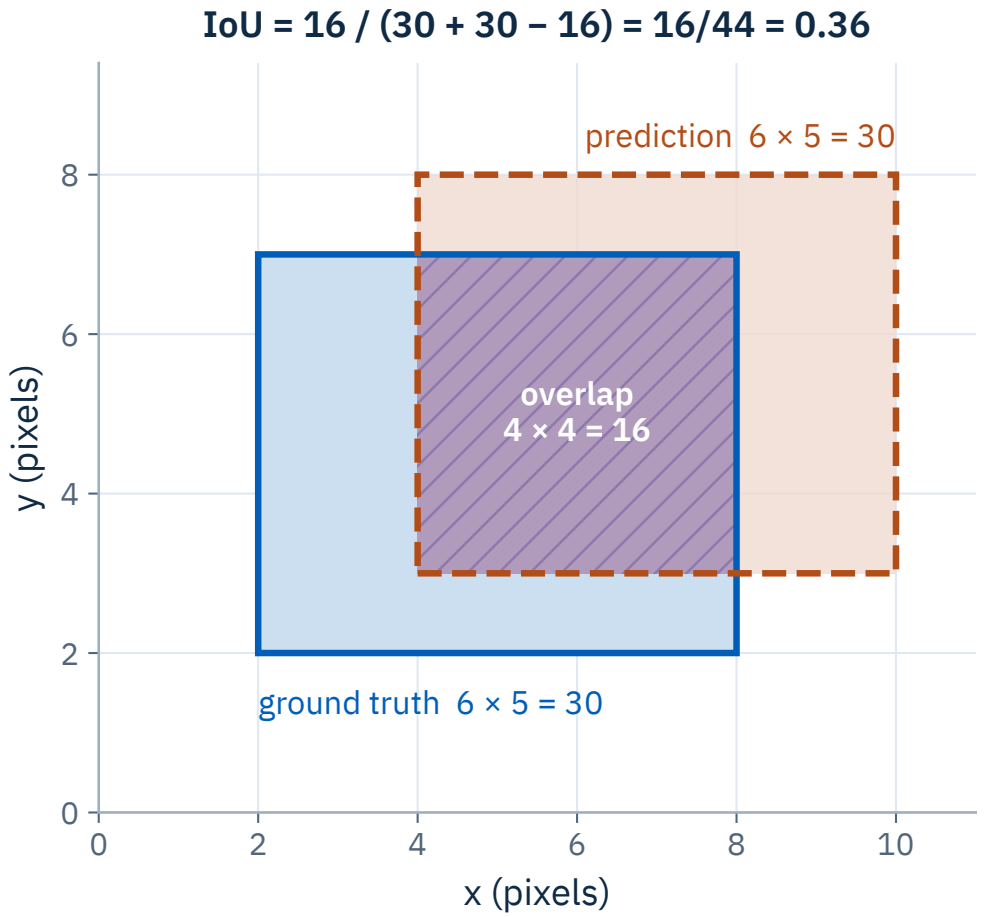
1. **Backbone**
   CNN features for the whole image
2. **Candidates**
   anchors, points or learned queries
3. **Heads**
   class scores and box offsets
4. **Clean-up**
   non-maximum suppression of duplicates
5. **Evaluation**
   match to ground truth by IoU

◆ Two-stage (Faster R-CNN), one-stage (YOLO, RetinaNet) and transformer (DETR) detectors share this output format.

Intersection over union scores box overlap

$$\mathrm{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

- Here the overlap is 16 and the union 44: IoU = 0.36.
- A detection counts only above a threshold, often 0.5.
- COCO-style scores average over thresholds 0.50 to 0.95.



Exact intersection over union for two axis-aligned boxes.

PRACTICE

4 min · Pairs

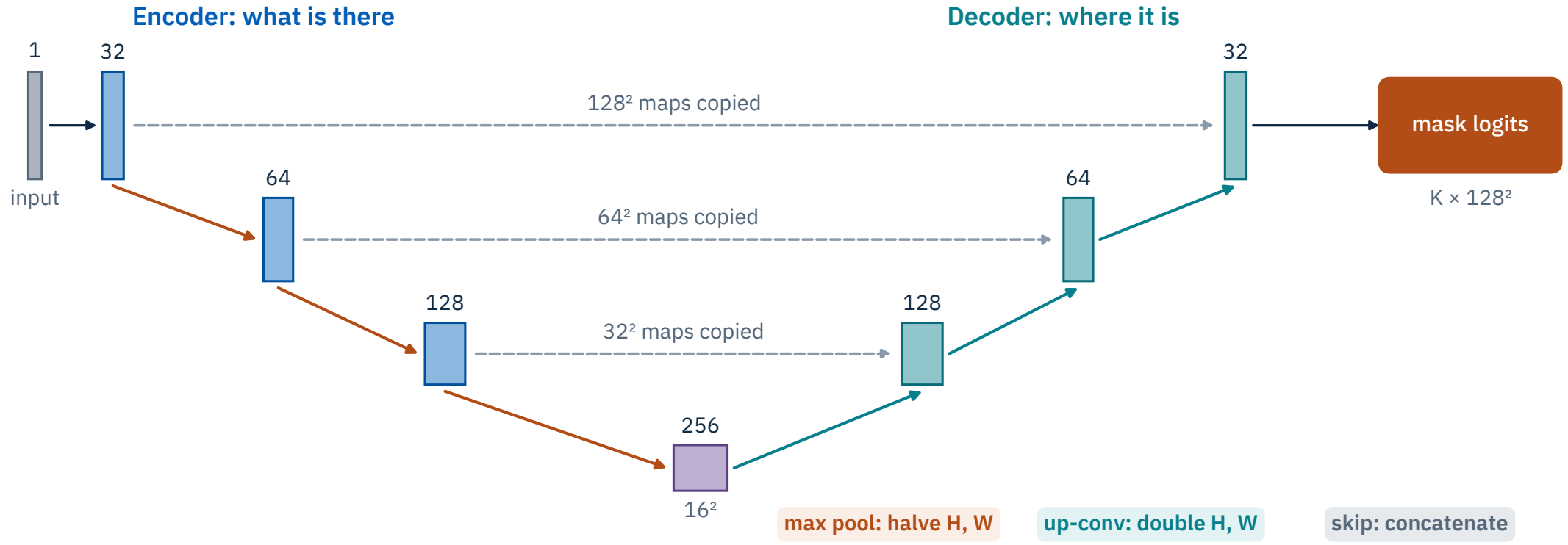
Decide whether each detection is a true positive

Ground-truth box corners: (0, 0) and (4, 4). A detection counts if IoU ≥ 0.5.

1. Detection A has corners (1, 1) and (5, 5). Compute its IoU.
2. Detection B has corners (0, 1) and (4, 5). Compute its IoU.
3. Both are predicted for the same defect. How many true and false positives?

U-Net segments with an encoder, decoder and skips

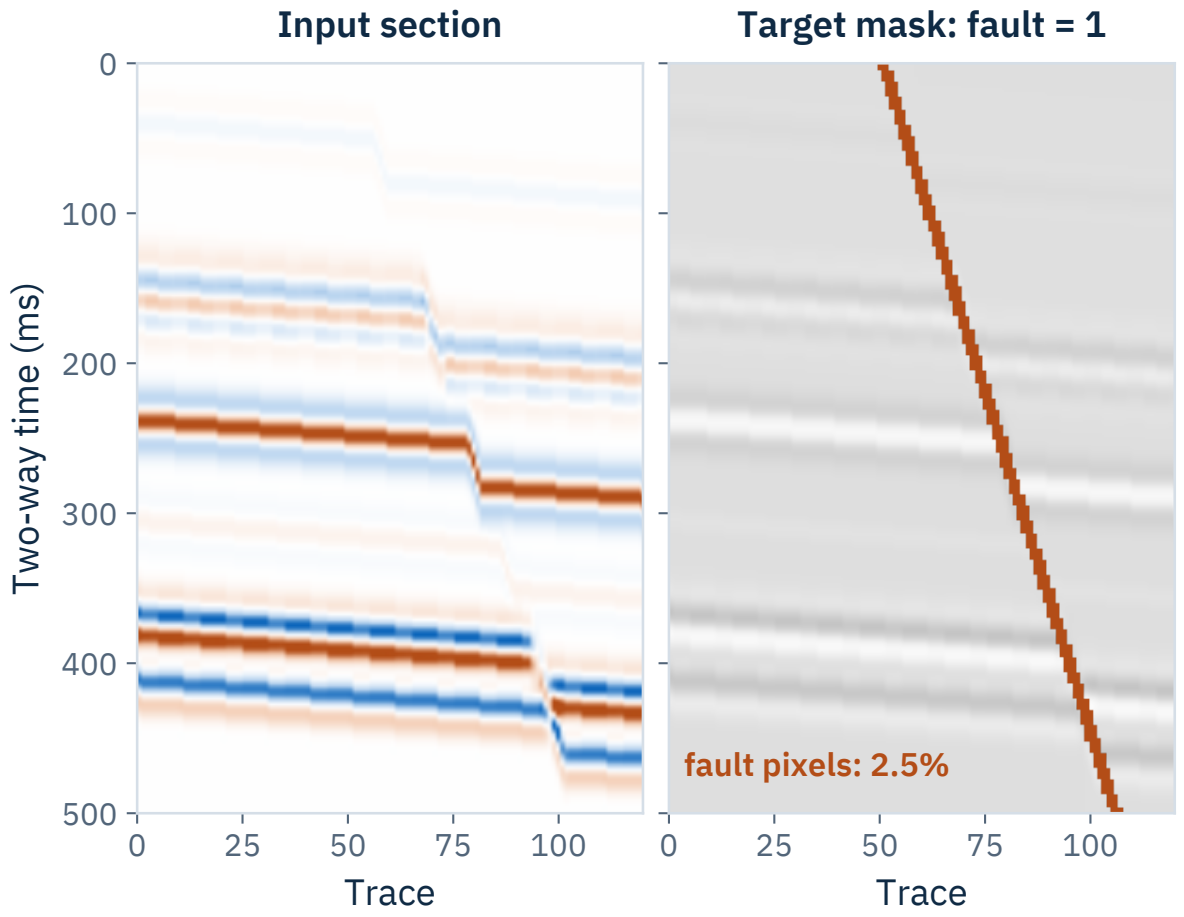
The encoder gathers context, the decoder restores resolution, and skips carry back fine detail.



Schematic of a small U-Net for a 128 × 128 input; numbers are channel counts. After Ronneberger et al. (2015).

Segmentation masks bring their own label problems

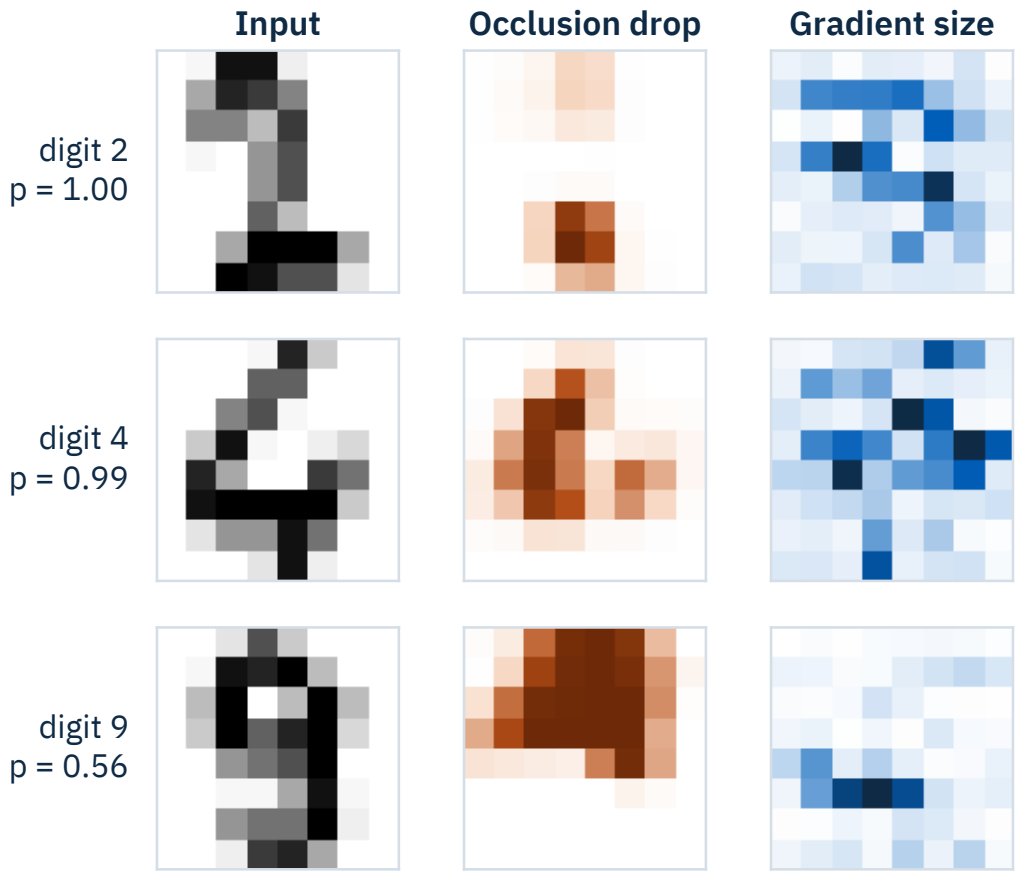
- Pixel labels are slow to draw and differ between interpreters.
- Targets are thin and rare: here 2.5% of pixels are fault.
- Score with Dice or IoU per class, with tolerance near boundaries.
- **Instance segmentation** (e.g. Mask R-CNN) also separates objects.



Synthetic section (seed 7); fault mask: pixels within 1.5 traces of the fault plane.

Occlusion maps show which pixels move the score

- **Occlusion**: blank a patch, record the drop in the class probability.
- Gradient maps: how strongly each pixel changes the class logit.
- Both measure local sensitivity; neither explains the reasoning.



Tiny digits CNN (seed 0): 2 × 2 patches set to 0; absolute input gradients; each map scaled to its maximum.

Shortcuts can pass validation and fail in deployment

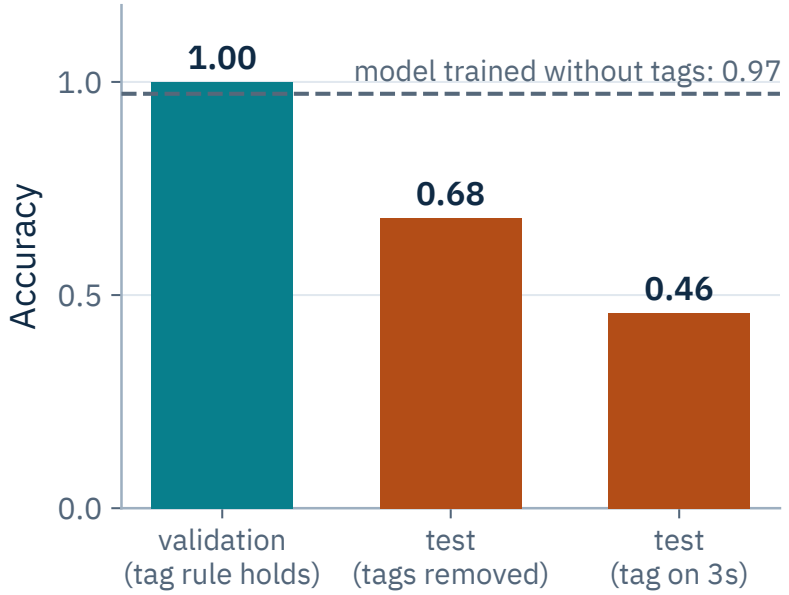
A bright tag on every training 8 is enough: validation is perfect, then accuracy collapses.

Training data

every 8 carries a corner tag

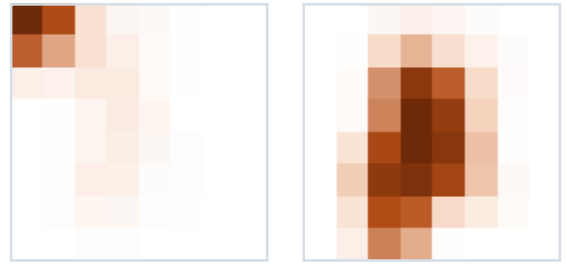


Accuracy of the tagged model



Where the evidence is

occlusion drop for a tagged 8



tagged model

p(8) = 1.00

clean model

p(8) = 0.99

Tiny CNN (seed 0, CPU) on scikit-learn 3s and 8s: 214 training, 71 validation and 72 test images.

Explanations are hypotheses to test

✕ Shortcut sources in energy images

- Tool, run or vendor artefacts that correlate with labels
- Depth tracks, annotations or scale bars in the image
- One well or survey supplying most positives

vs

✓ Tests that expose them

- Grouped splits by well, survey or acquisition run
- Counterfactual edits: remove the tag, move the defect
- Expert review of the evidence maps

PART 7 OF 7

07

Autoencoders and 1-D signals

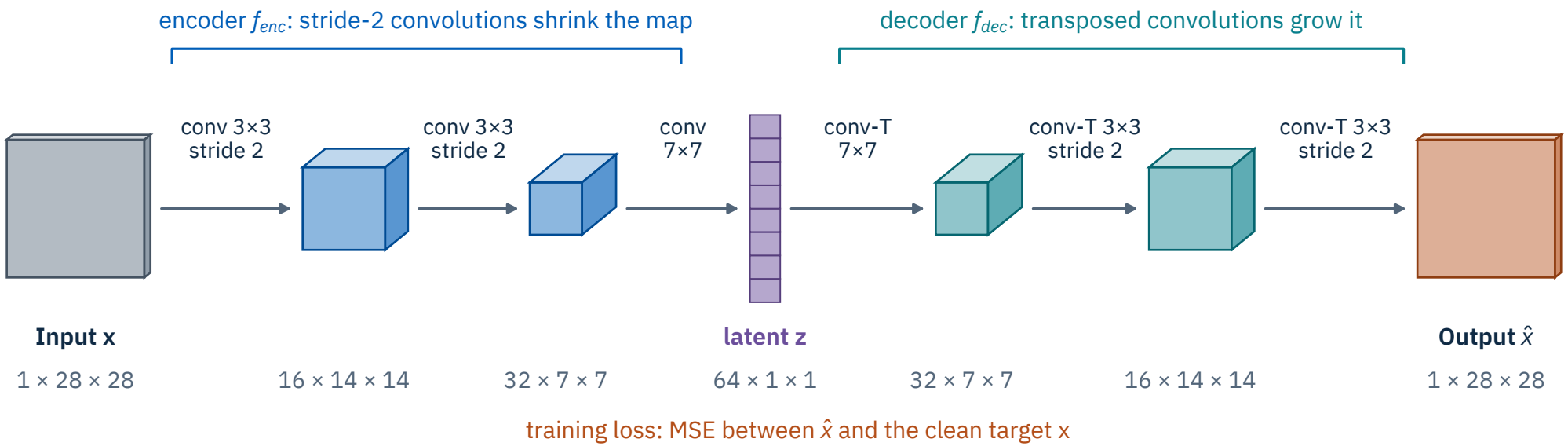
The output can be a reconstruction or an ordered signal.

- Reconstruction objective and denoising
- Reconstruction error for anomaly screening
- 1-D convolutions for sensor series
- Energy application choices

An autoencoder learns a reconstruction objective

$$z = f_{enc}(x), \qquad \hat{x} = f_{dec}(z), \qquad L = \|x - \hat{x}\|^2$$

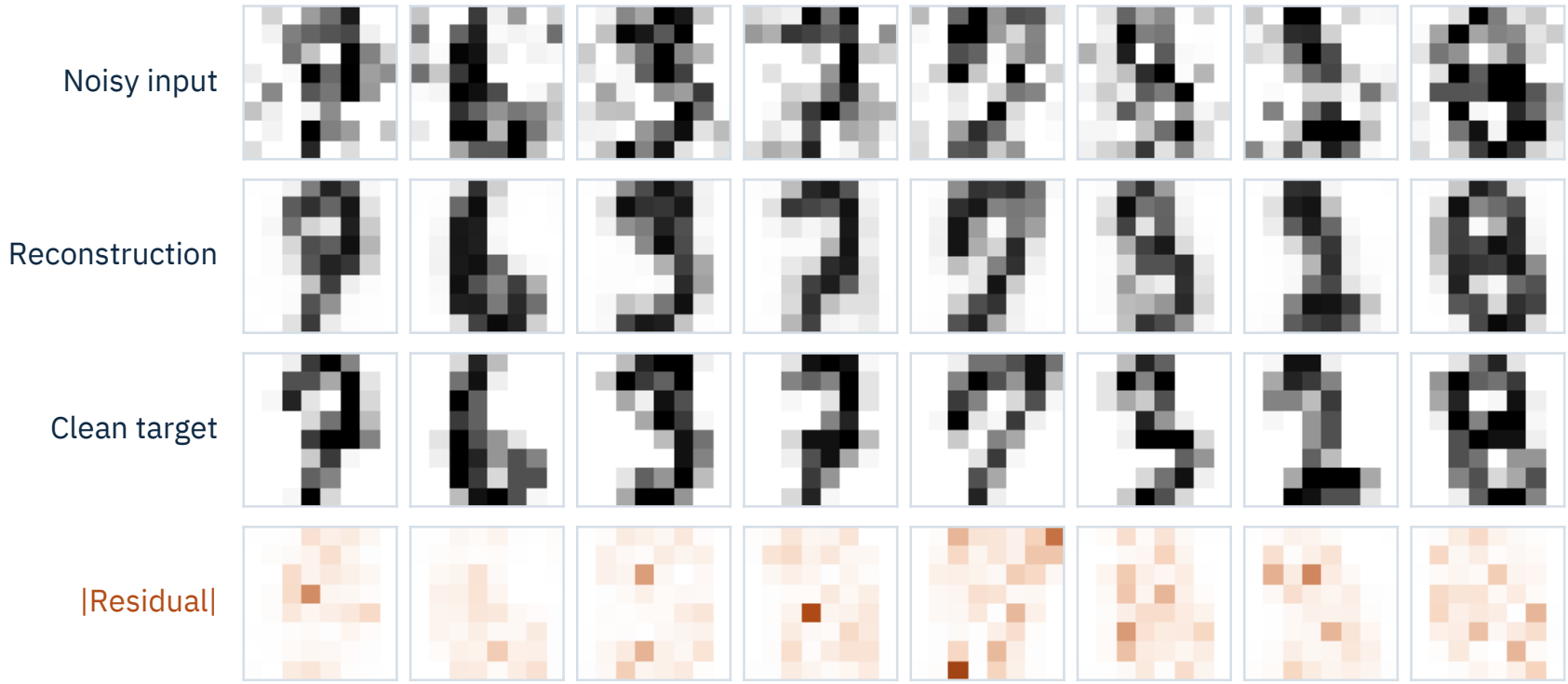
A bottleneck prevents copying; denoising maps corrupted inputs to clean targets.



Schematic of the Lab 2A autoencoder (latent size 64); shapes checked with PyTorch, batch dimension omitted.

Denoising trains from corrupted input to clean target

Held-out error against the clean digits falls from 0.051 for the noisy input to 0.019.



Held-out MSE

vs clean target

noisy input 0.051

reconstruction 0.019

9,569 parameters

noise σ = 0.3, clipped

Small convolutional denoising autoencoder (9,569 parameters, seed 0, CPU) trained on scikit-learn digits; noise σ = 0.3, clipped.

Reconstruction quality needs more than a low loss

Inspect residuals

- Compare input, target, reconstruction, residual
- Structure in residuals means missed signal

Protect small features

- Check whether task-relevant defects vanish
- A smoother image can be less useful

Keep scaling consistent

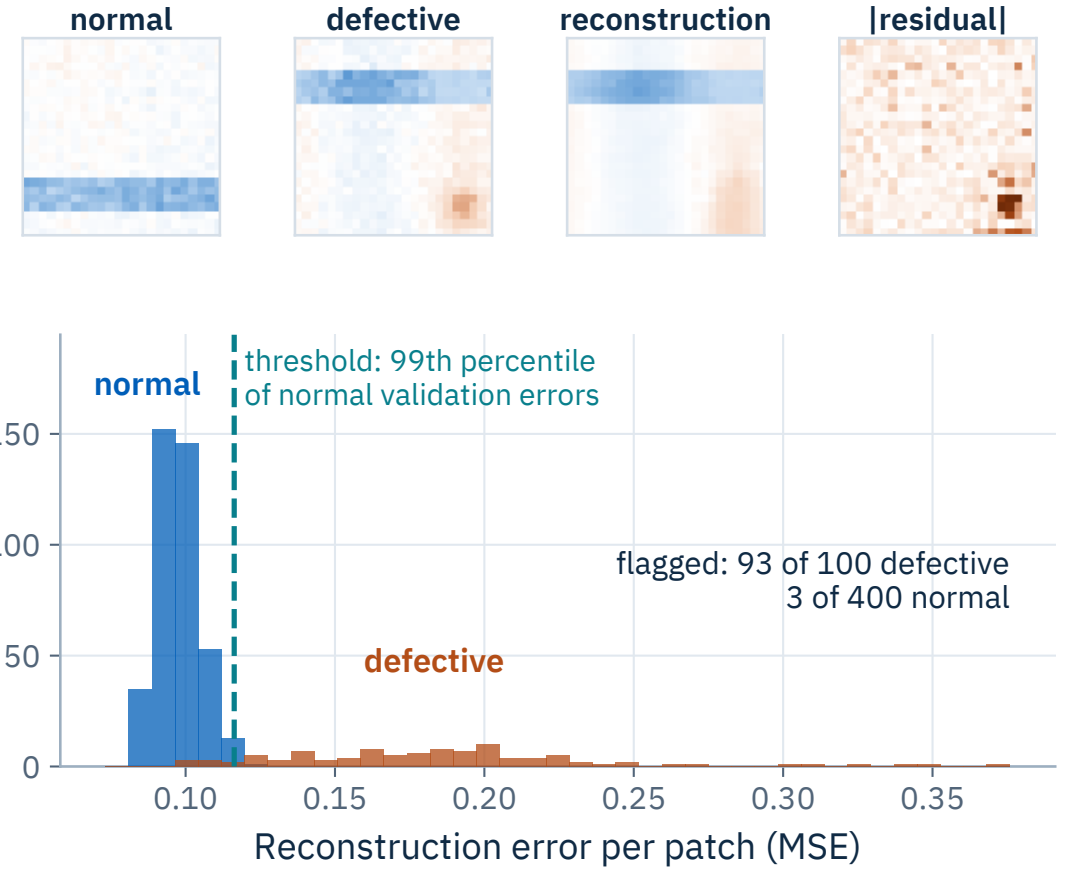
- Same normalization for input and target
- Store the statistics with the model

Match the output range

- Sigmoid only for targets in [0, 1]
- Standardized targets need a linear output

Reconstruction error can screen for anomalies

- Train on normal patches only; score each patch by its error.
- Set the threshold on normal validation data, never on test.
- The score is not a calibrated probability; strong models may rebuild defects too.



Synthetic casing patches (seed 11): 1,500 normal training, 400 normal validation, 400 normal and 100 defective test.

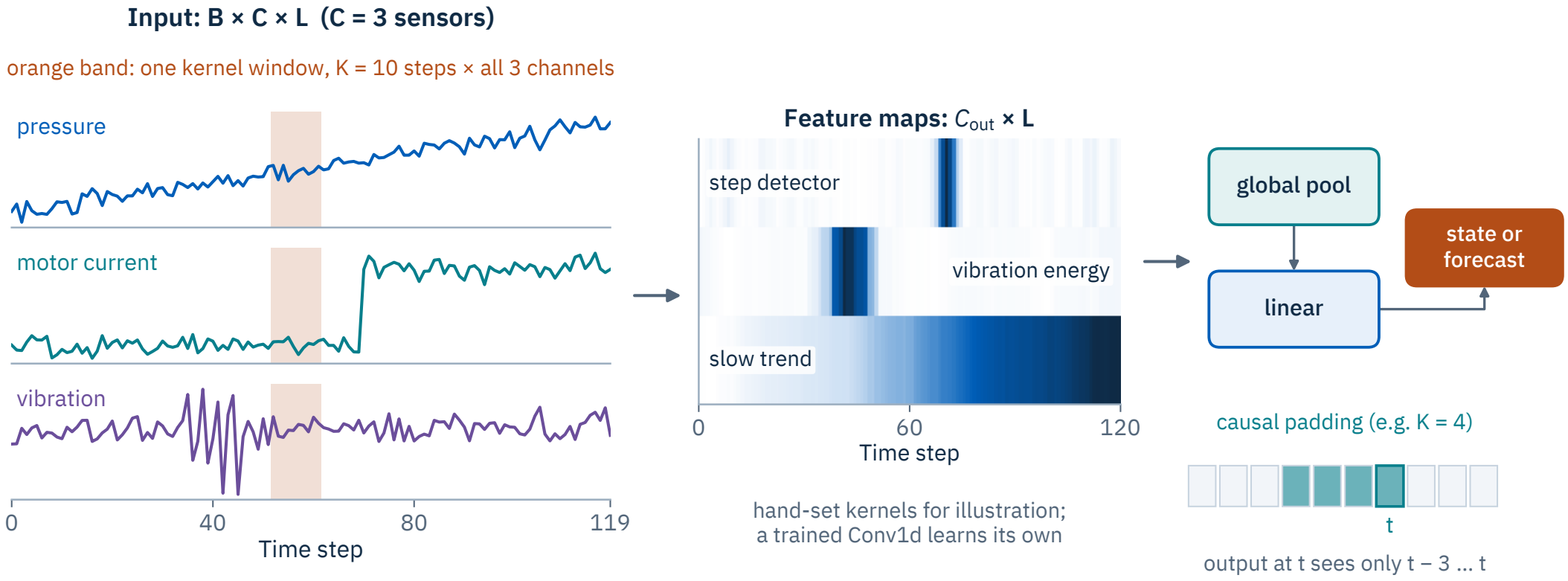
A 1-D convolution uses ordered neighbourhoods

Quantity	PyTorch shape	Example
Input	B × C × L	Sensor channels along time or depth
Kernel	C_{out} × C × K	Local window over the ordered axis
Feature map	B × C_{out} × L_{out}	Learned local signal patterns

Order and sampling interval must have a real meaning.

1-D CNNs read multichannel sensor windows

Kernels slide along time across all channels; causal padding hides the future (Day 5).



Synthetic three-sensor series (seed 21); hand-set kernels for illustration; schematic head and causal-padding inset.

Match the architecture to the data geometry and target

Core imagery

- Classify or segment visible patterns
- Split by well or core run

Well logs

- 1-D convolutions over depth
- Track receptive field in metres

Acoustic and ultrasonic inspection

- Acquisition-aware augmentation
- Respect azimuth wrap-around

Seismic

- Segment faults, horizons or bodies
- Test transfer across surveys and processing

◆ These are application directions, not demonstrated field results.

LABORATORY

55 min · Pairs

Laboratory 2A: a denoising convolutional autoencoder

- ☐ Check the encoder and decoder shapes with one batch.
- ☐ Train with noisy inputs and clean targets; plot training and validation loss.
- ☐ Compare latent sizes 8, 32 and 64 on the same validation images.
- ☐ Show noisy, reconstructed, clean and residual images for eight digits.

NOTEBOOK

`exercises/03_autoencoder/Conv_Autoencoder_Student.ipynb`

EVIDENCE TO SUBMIT

- Loss curves and a latent-size table
- A residual grid with one observation

⚠ Prepare: torchvision installed and MNIST downloaded to ./data. Choose 2A or 2B, not both.

LABORATORY

55 min · Pairs

Laboratory 2B: casing defects with grouped splits

- List runs, wells, missing values and flag fractions in the HDF5 file.
- Check the split by well (GroupShuffleSplit): wells, patches and prevalence per set.
- Train the CNN; choose the epoch and the threshold on validation wells.
- Report precision and recall per test well, once, at natural prevalence.
- Compare with a random patch split and explain the difference.

NOTEBOOK

`exercises/04_defect_detection/Defect_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Split table: wells and patches per set
- Per-test-well metrics; random versus grouped scores

Prepare: install h5py; the HDF5 file is in the exercise folder. Its labels are named "prediction": check their provenance.

DISCUSSION

Lab 2 debrief: evidence before claims

Autoencoder path

- Did small latent codes blur digits or remove noise?
- Were residuals noise-like, or did strokes vanish?

vs

Casing-defect path

- How did metrics change between random and grouped splits?
- Which wells drive the errors, and why?

◆ Neither result is a field-performance claim: data, labels and splits define what the numbers mean.

Key takeaways

- ✓ Images are tensors whose axes have physical meaning; check shapes and spacing first.
- ✓ Convolution shares small local filters; padding, stride and dilation set the output size.
- ✓ Depth grows receptive fields; pooling and striding trade resolution for context.
- ✓ Residual paths, normalization and pretrained backbones make deep CNNs practical.
- ✓ Augmentations and splits must respect the physics and the data sources.
- ✓ Detection, segmentation and reconstruction change only the output.

PRACTICE

5 min · Individual

Exit check: explain the mechanism, not the name

1. Why is a convolution's parameter count independent of image width?
2. How does a stride of 2 change spatial resolution and receptive field growth?
3. Why can frozen weights still coexist with changing BatchNorm behaviour?
4. Name one augmentation you rejected today, and why.
5. What would convince you that a defect model is not using a shortcut?

References and further study

- LeCun, Bottou, Bengio, Haffner (1998). Gradient-based learning applied to document recognition. *Proc. IEEE* 86(11).
- He, Zhang, Ren, Sun (2016). Deep residual learning for image recognition. *CVPR*. Ioffe, Szegedy (2015). Batch normalization. *ICML*.
- Ronneberger, Fischer, Brox (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI*.
- Dumoulin, Visin (2016). A guide to convolution arithmetic for deep learning. arXiv:1603.07285. Zhang (2019). Making convolutional networks shift-invariant again. *ICML*.
- Geirhos et al. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence* 2. Goodfellow, Bengio, Courville (2016). *Deep Learning*, chapter 9.

NEURAL NETWORKS / DAY 5

Sequence models and attention

Forecasting, recurrent networks, attention, uncertainty, and a review of the course

Full day · 09:00–17:00 | 7 parts | 3 lab blocks | Final course review

Today's plan

09:00	Welcome · Part 1: time-series fundamentals	Lecture
09:45	Part 2: splits, baselines and metrics	Lecture
10:30	Break	Break
10:45	Part 3: recurrent networks	Lecture
11:30	Lab 1A: shampoo warm-up, climate baselines	Lab
12:30	Lunch	Lunch
13:30	Lab 1B: climate LSTM against baselines	Lab
14:15	Part 4: convolutions and attention	Lecture
14:50	Part 5: transformers, pretrained models	Lecture
15:20	Break	Break
15:35	Part 6: uncertainty and anomalies	Lecture
16:00	Lab 2: sequence anomalies or retail	Lab
16:35	Part 7: course review, exit check	Discussion

LEARNING OUTCOMES

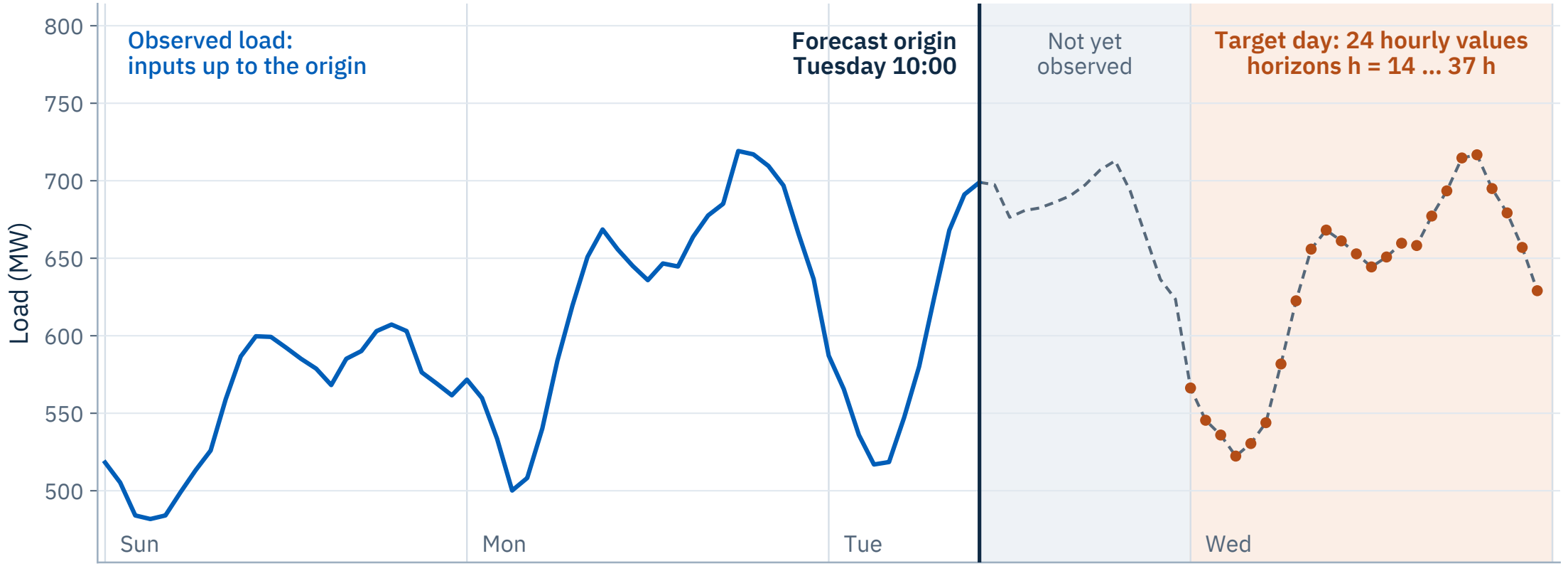
What you will be able to do

By the end of the day you will be able to:

1. Build forecast windows whose inputs are known at the origin, and count the samples.
2. Evaluate a forecast with a time-respecting split, strong baselines and per-horizon metrics.
3. Explain how RNN, LSTM and GRU states update, and check their tensor shapes in PyTorch.
4. Compute a small attention example, apply a causal mask and describe a transformer block.
5. Produce and check quantile forecasts and reconstruction-based anomaly scores.
6. Choose a model family for a new energy problem and defend its evaluation design.

A day-ahead forecast is fixed hours before its target

Next-day hourly load is due at 10:00, so every target lies 14 to 37 hours ahead.



Synthetic hourly load (seed 11) with trend, daily and weekly cycles, and correlated noise. Submission time is hypothetical.

PRACTICE

4 min · Pairs

A forecast is indexed by its origin

At 12:00 you must predict air temperature at 18:00 today.

1. Which measurements are already known, and what is the horizon?
2. Can you use tomorrow's observed maximum temperature?
3. How do known calendar features differ from future sensor readings?

PART 1 OF 7

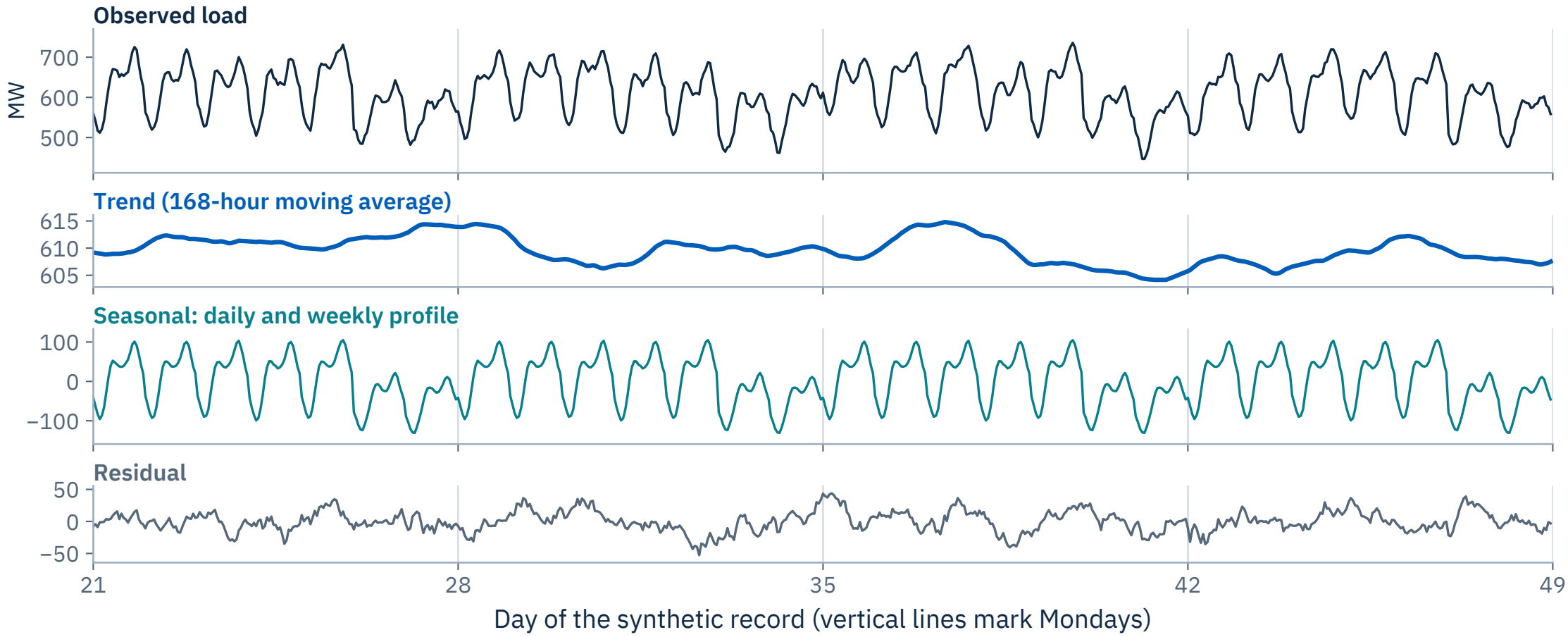
01

Time-series fundamentals

Structure in time, and what a forecast may use.

- Trend, daily and weekly cycles, and noise
- Autocorrelation, stationarity and differencing
- Origin, lookback, horizon and stride
- Sequence task types and covariates

An hourly load series mixes trend, cycles and noise



Synthetic hourly load (seed 11), weeks 4–7. Classical additive decomposition with a 168-hour period.

Classical decomposition takes four steps

The same recipe works for any period: 24 for a daily cycle, 168 for a weekly one.

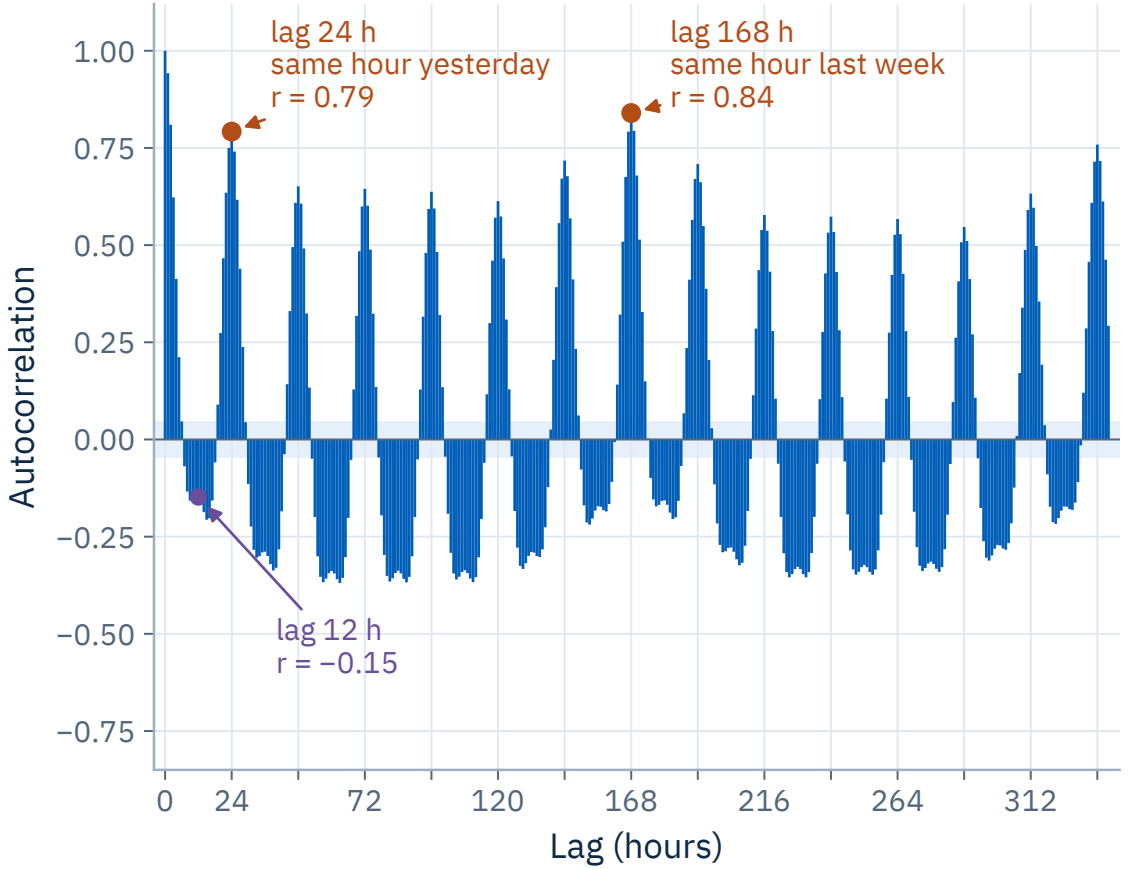
1. **Trend**: a centred 168-hour moving average smooths out the weekly cycle
2. **Detrend**: subtract the trend, leaving the cycles and the noise
3. **Seasonal profile**: average each of the 168 hours of the week, then centre it
4. **Residual**: whatever the trend and the seasonal profile leave unexplained

◆ A centred average reads 84 hours ahead: fine for description, leakage if used as a model input.

Autocorrelation reveals daily and weekly memory

$$r_k = \frac{\sum_t (y_t - \bar{y})(y_{t+k} - \bar{y})}{\sum_t (y_t - \bar{y})^2}$$

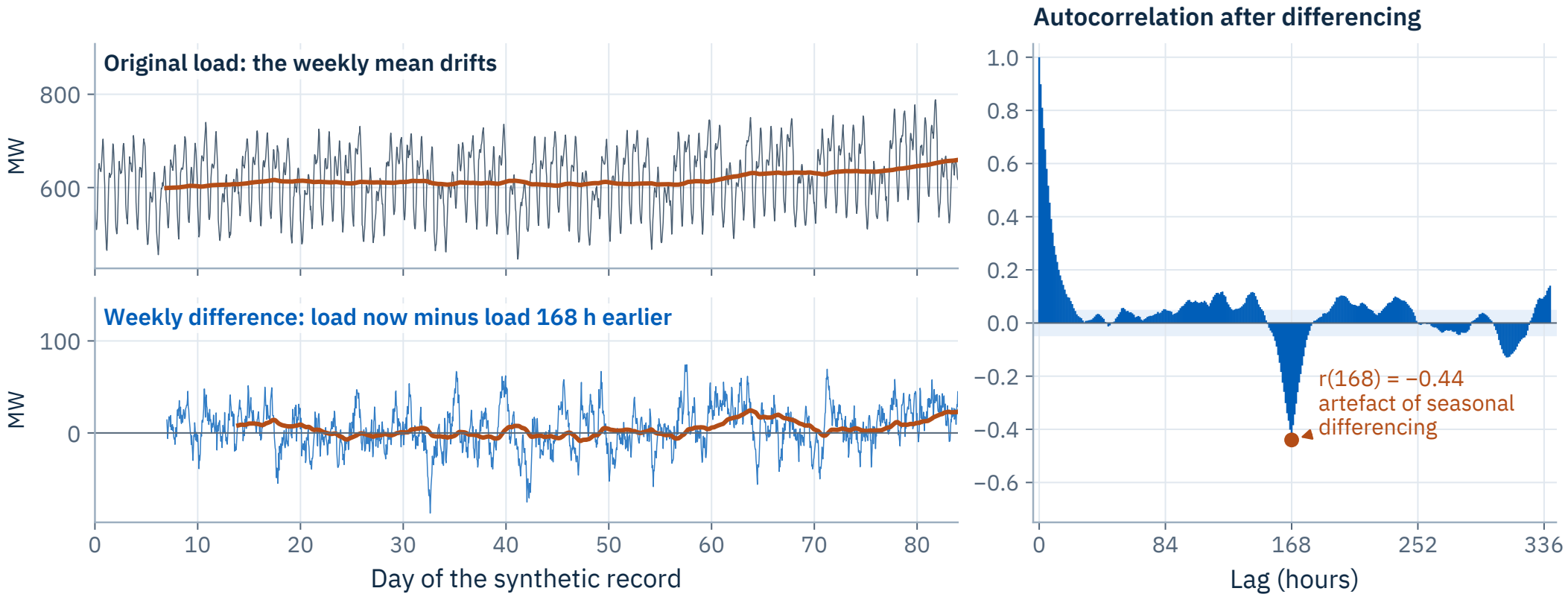
- r_k correlates the series with itself k steps later.
- Peaks at 24 and 168 hours: yesterday and last week.
- Negative at 12 hours: the opposite phase of the day.



Synthetic hourly load (seed 11, 2,016 hours). Band: approximate 95% range for white noise.

Differencing removes trend and seasonality

Weakly stationary: mean, variance and autocorrelation stay the same over time.



Synthetic hourly load (seed 11). Orange: 168-hour rolling means. Right: autocorrelation of the weekly difference.

PRACTICE

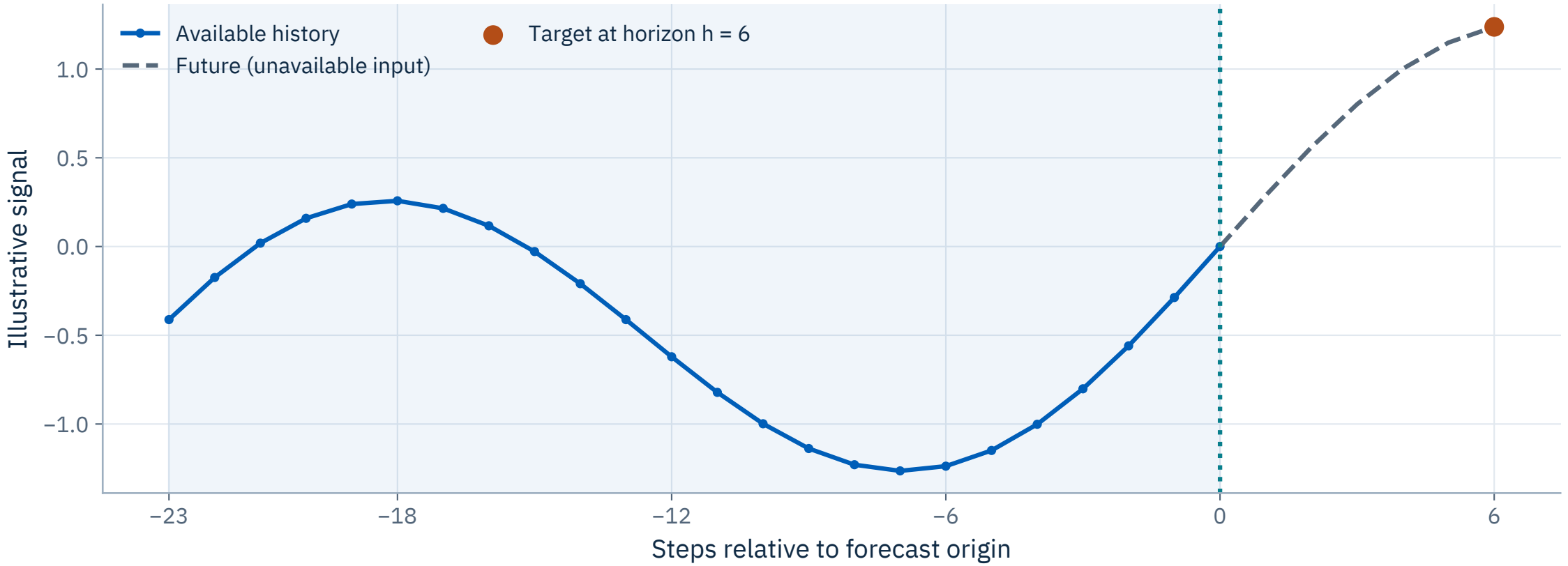
4 min · Pairs

Read the autocorrelation before choosing a lookback

For the hourly load: r(1) = 0.94, r(12) = −0.15, r(24) = 0.79, r(168) = 0.84.

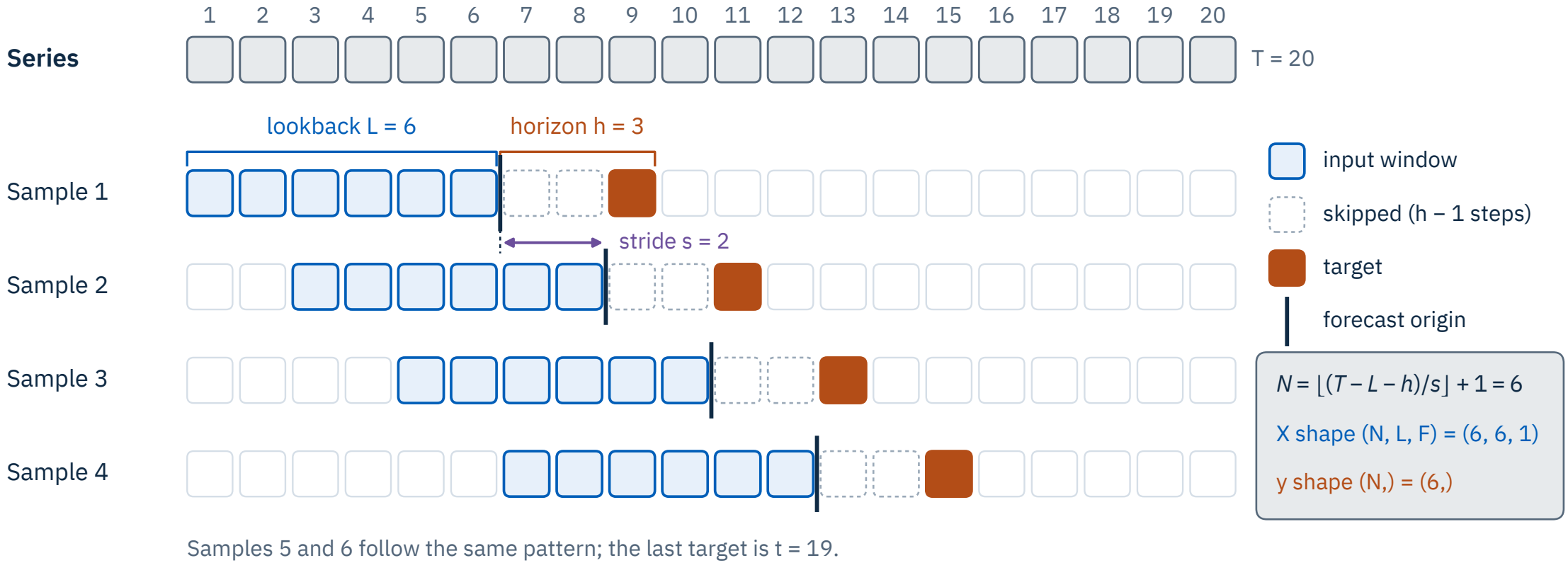
1. Which seasonal-naive forecast does this suggest, and why not lag 24?
2. Why is r(12) negative?
3. For the 10:00 day-ahead case, is lag 24 even available for 18:00 tomorrow?

A window fixes origin, lookback and horizon



Schematic single-origin forecast: lookback 24 observations, horizon 6 steps. Inputs end at the origin; the target lies h steps later.

Sliding the origin turns one series into many samples



Samples 5 and 6 follow the same pattern; the last target is t = 19.

Schematic with exact indexing: T = 20, L = 6, h = 3 and s = 2 give N = 6 samples.

A window builder in a few lines of NumPy

```python
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view

def make_windows(series, L, h, s=1):
    """Inputs end at each origin; the target is h steps later."""
    X = sliding_window_view(series[:len(series) - h], L)[::s]
    y = series[L - 1 + h::s][:len(X)]
    return X[..., None], y            # (N, L, 1) and (N,)

X, y = make_windows(np.arange(1.0, 21.0), L=6, h=3, s=2)
print(X.shape, y.shape)
print(X[0, :, 0], y[0])
```

OUTPUT

```
(6, 6, 1) (6,)
[1. 2. 3. 4. 5. 6.] 9.0
```

Build windows after the chronological split, separately for each asset.

Known and unknown covariates differ at the origin

✓ Known in advance

- Calendar: hour, weekday, holidays
- Planned maintenance and set points
- Weather forecasts issued before the origin

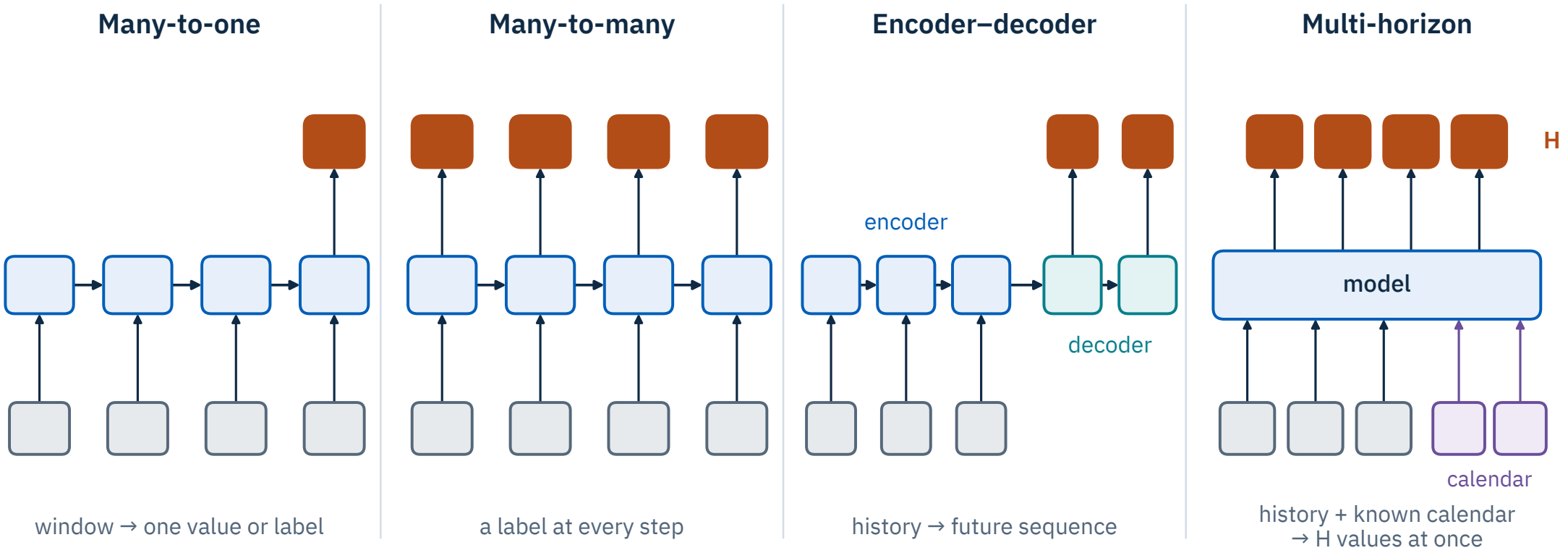
vs

✗ Unknown at the origin

- Future load, production or pressure
- Realized weather after the origin
- Values that arrive after the origin

◆ Check when each value became available, not only when it was measured.

Sequence tasks have different outputs



Schematic. Grey: inputs; blue and teal: learned states; orange: outputs; purple: covariates known in advance.

PRACTICE

5 min · Individual

Count the windows before you train

The climate lab uses hourly data, lookback L = 72 and horizon h = 12.

1. How many windows does a 1,000-hour segment give with stride 1? With stride 6?
2. The training segment has 49,064 hours. How many training windows?
3. Why does the notebook build windows separately inside each split?

PART 2 OF 7

02

Splits, baselines and metrics

A forecast is only as credible as its evaluation.

- Splits that respect target intervals
- Rolling origins and strong baselines
- Metrics and their pitfalls
- Multi-step strategies and horizons

Time needs a different split from rows or wells

Random rows

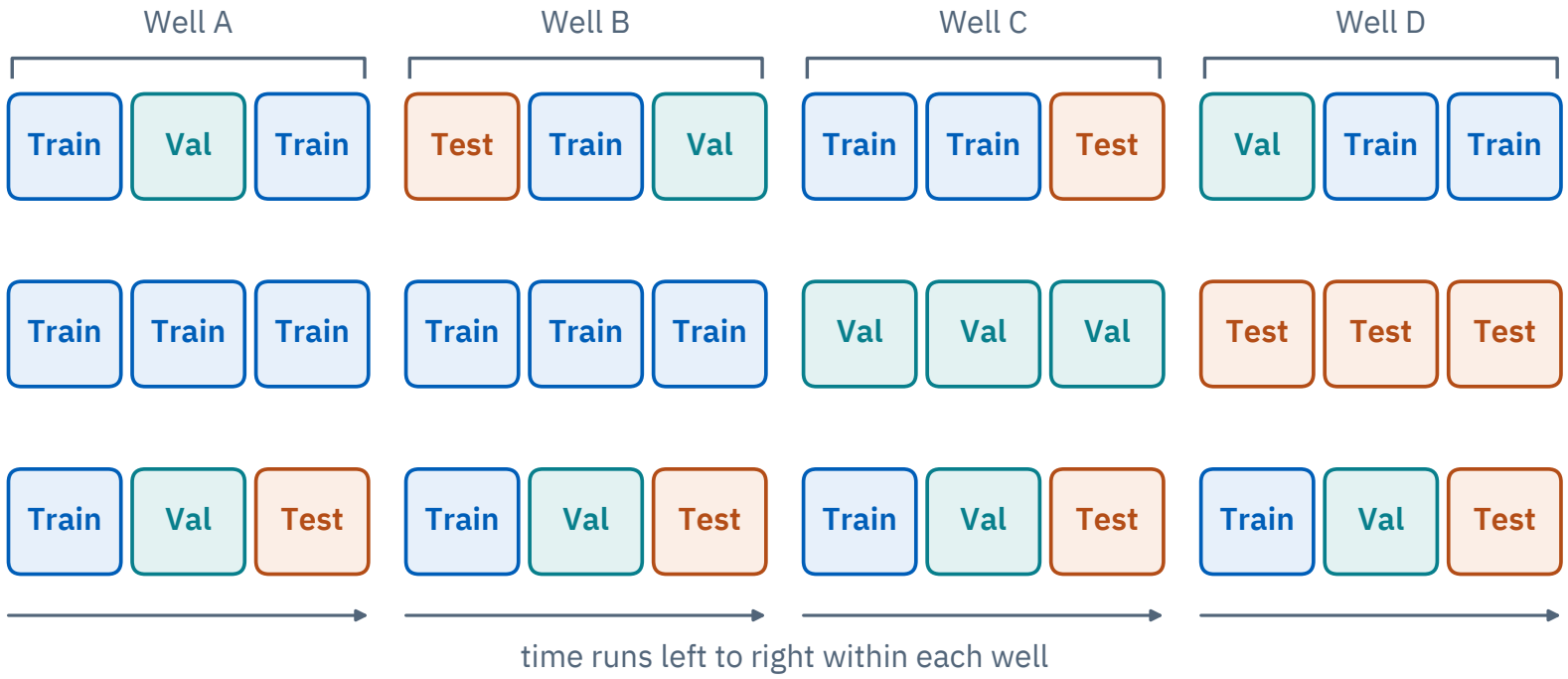
tests new rows from the same wells

Grouped by well

tests wells never seen in training

Ordered in time

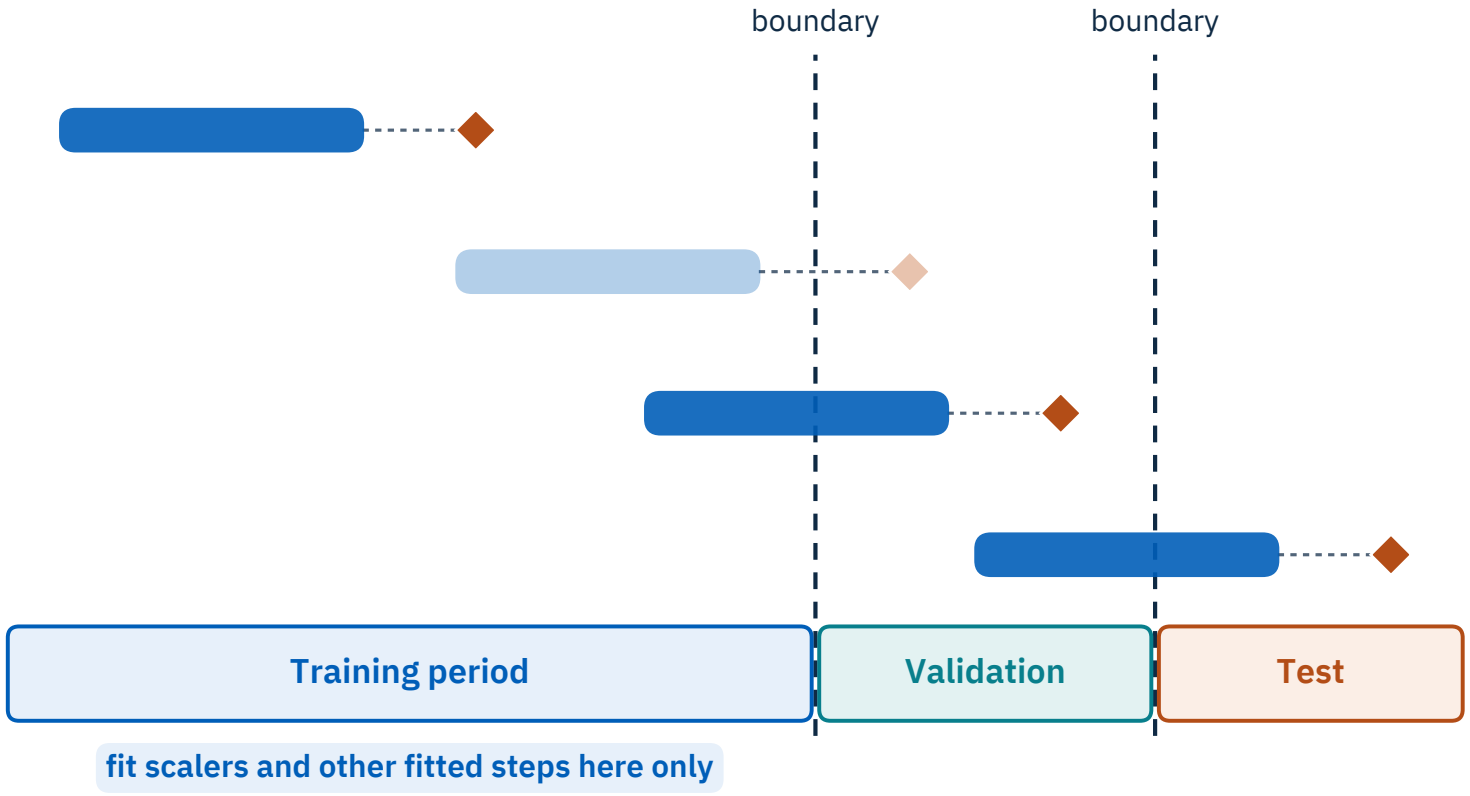
tests the later period of every well



Schematic partition assignments: a time split holds out the later period of every well. The machine learning course covers these splits in more depth.

Targets, not inputs, define the split boundary

- **Training window**
  inputs and target in training
- **Target after the boundary**
  purge it from training
- **Validation window**
  may read earlier, observed history
- **Test window**
  scored once, at the end



Schematic. Bars are input windows, diamonds are targets. The faded window is removed because its target falls after the boundary.

Four rules keep time splits honest

Fit on the past only

- Scalers, imputers, encoders, thresholds
- Estimated on training rows, then frozen

Purge crossing targets

- No training target after the boundary
- Embargo gaps for multi-step labels

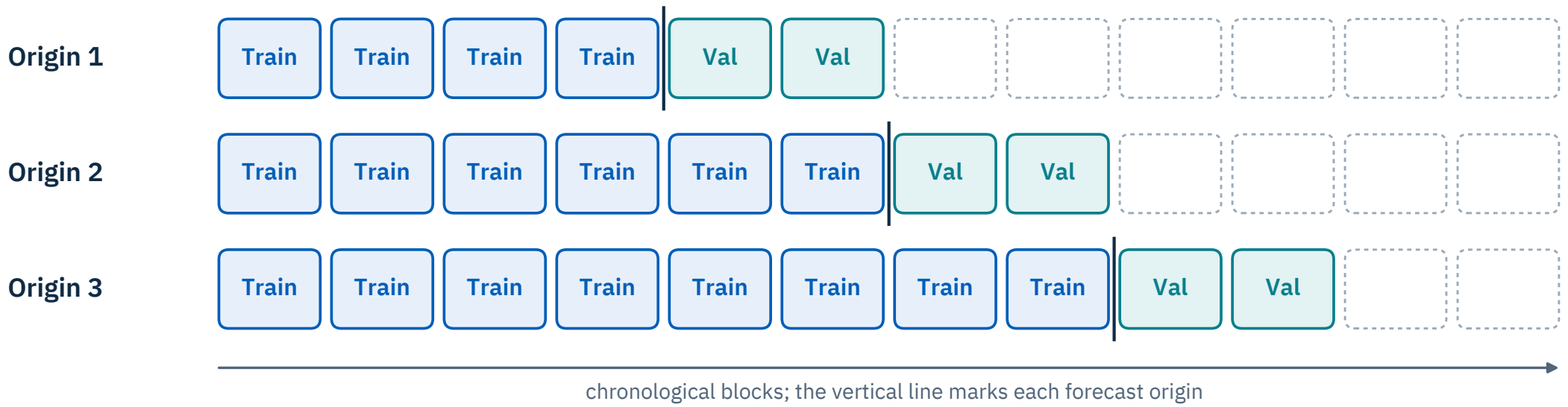
Allow observed history

- Validation inputs may start earlier
- That is what deployment sees

Keep assets separate

- Windows never join two wells or feeders
- Reset recurrent state between assets

Rolling-origin evaluation repeats the real question



Schematic expanding-window evaluation. Each fold forecasts from information available at its origin; keep a final test period outside all folds.

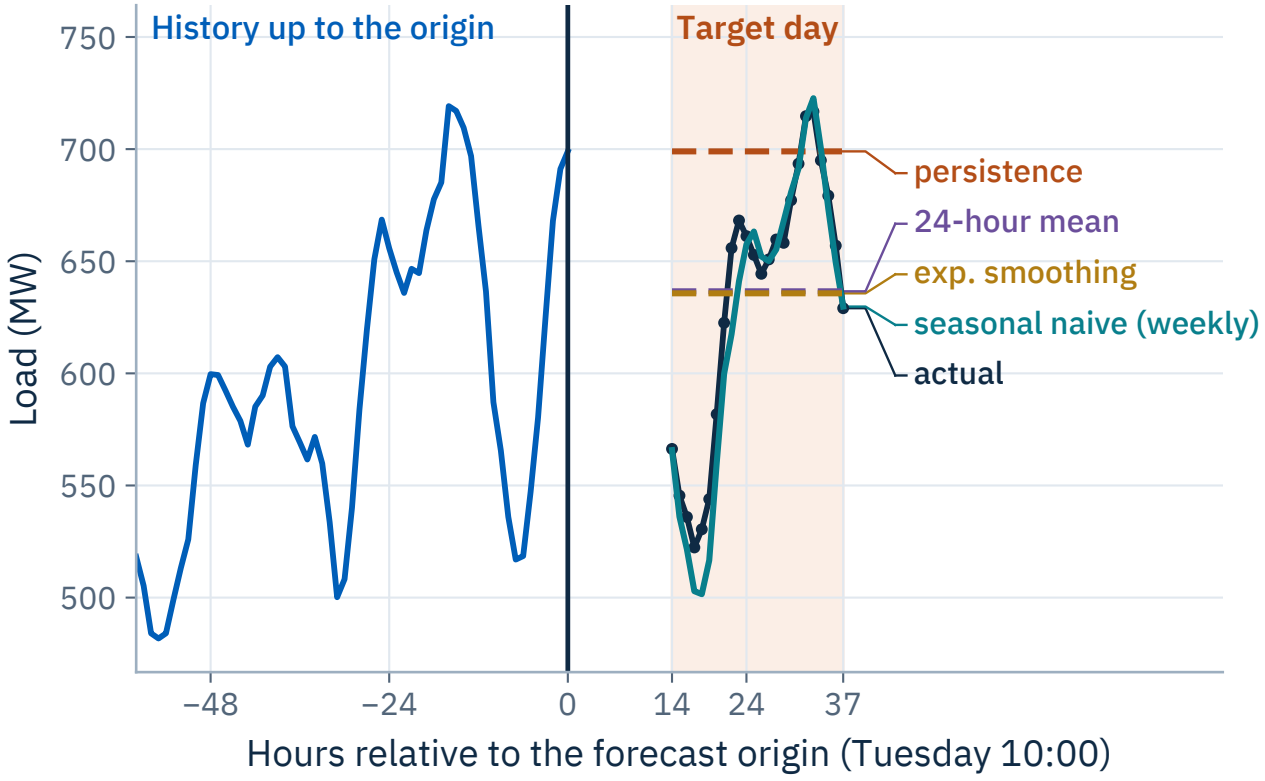
Strong baselines can be very simple

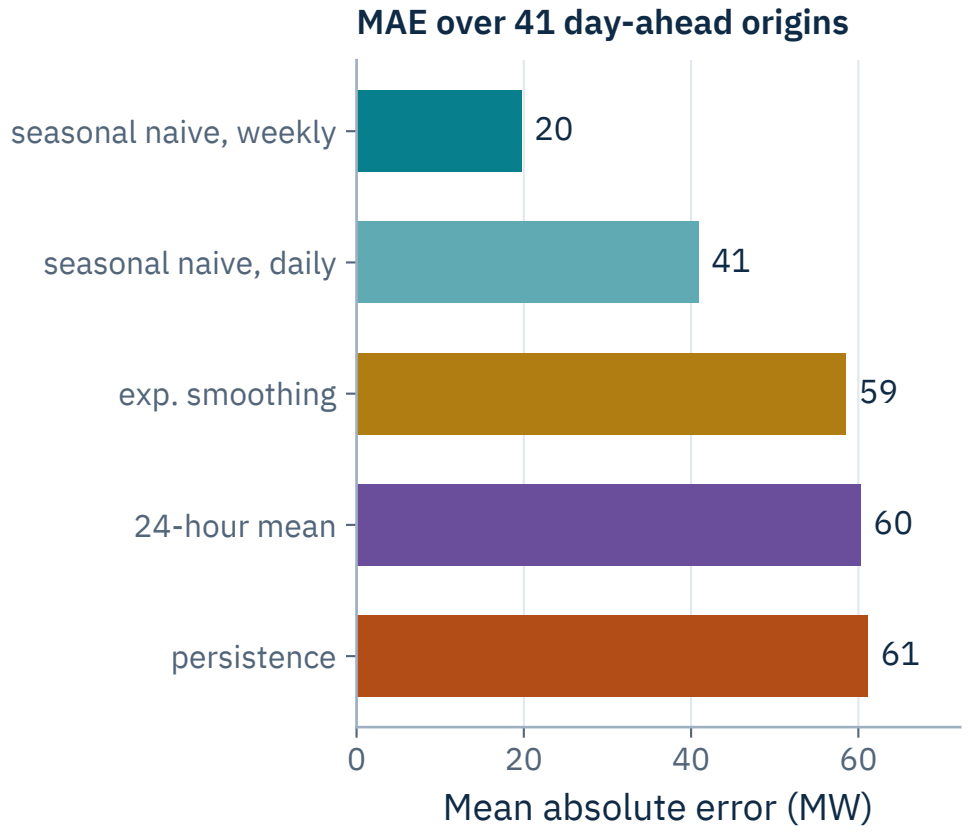
Baseline	Forecast of the value h steps ahead	Informative when
Persistence	The last observed value	Slowly changing signals
Seasonal naive	Same hour, one season earlier	Repeating daily or weekly cycles
Moving average	Mean of the last k values	A noisy level without cycles
Exponential smoothing	A level updated with weight α	A level that drifts slowly
Lag model	Linear or boosted on lags and calendar	A tabular benchmark

Score every model on the same origins, targets and horizons.

◆ A model that cannot beat these has not yet earned its complexity.

Seasonal naive wins at day-ahead horizons





Synthetic hourly load (seed 11): 41 origins at 10:00 on days 42–82, targets 14–37 hours ahead; exponential smoothing α = 0.2.

PRACTICE

3 min · Individual

Exponential smoothing by hand

Simple exponential smoothing with $\alpha = 0.5$ starts from a level $\ell_0 = 100$ MW.

1. The next two observations are 110 and 90 MW. Compute ℓ_1 and ℓ_2.
2. What is the forecast for 1 hour ahead, and for 24 hours ahead?
3. What does the method become when α approaches 1?

Forecast metrics answer different questions

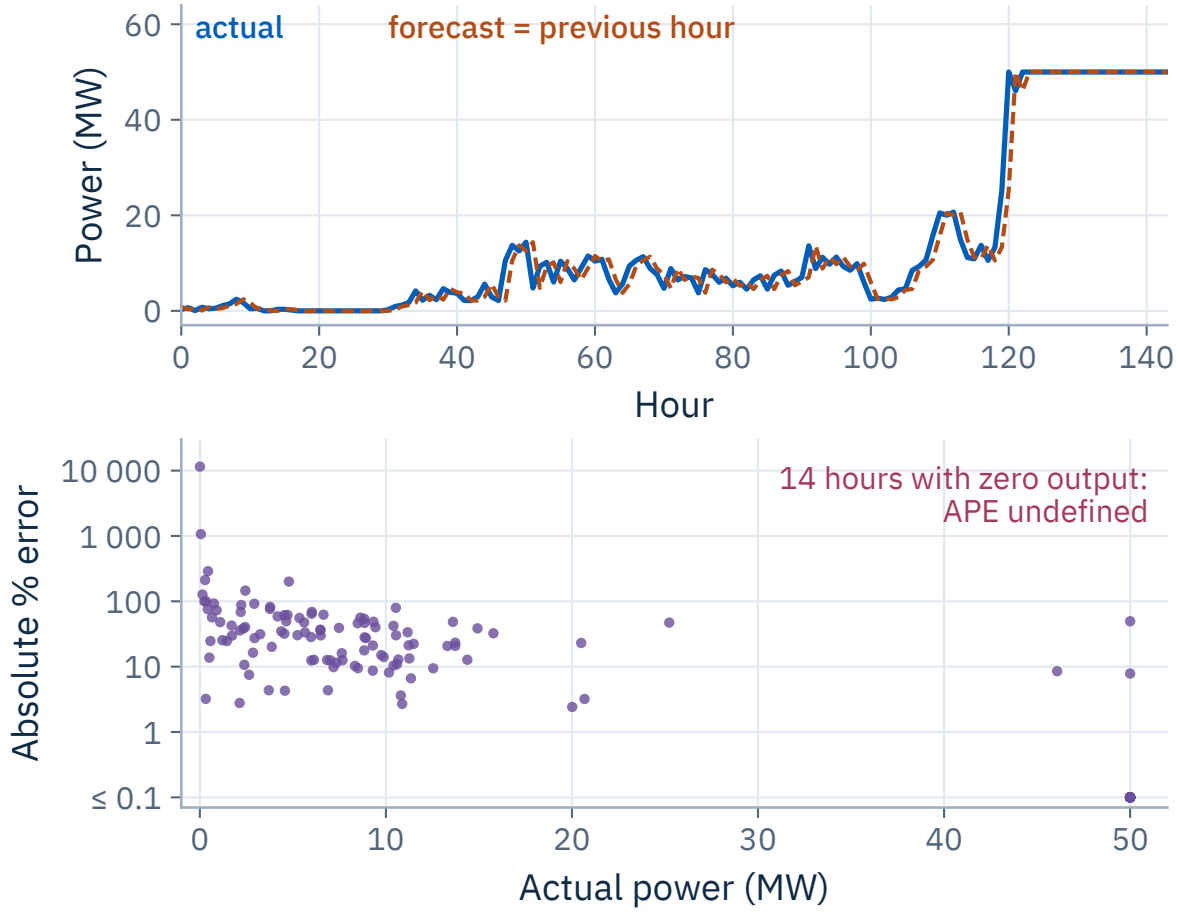
Metric	Definition	Watch out for
MAE	Mean absolute error	In target units; robust to single misses
RMSE	Root of the mean squared error	Dominated by the largest misses
MAPE	Mean of absolute error ÷ actual	Undefined at zero, explodes near zero
MASE	MAE ÷ in-sample naive MAE	Below 1 beats the naive method

Report errors by horizon and by asset, not only one average.

◆ Choose the metric before seeing any results, and use it for every model.

MAPE explodes when actual values approach zero

- One forecast: MAE = 1.7 MW, median APE = 23%.
- MAPE = 130%, driven by a few low-output hours.
- 14 zero-output hours cannot enter MAPE at all.



Synthetic 50 MW wind farm (seed 3), 144 hours; forecast = previous hour's output.

◆ For wind and solar, divide MAE by installed capacity instead.

MASE compares your error with a naive forecast

$$\mathrm{MASE} = \frac{\mathrm{MAE}}{\frac{1}{n-m}\sum_{t=m+1}^{n} |y_t - y_{t-m}|}$$

- The denominator uses the training period only.
- m = 1 scales by persistence; m = 24 or 168 by a seasonal naive.
- MASE below 1 means better than that naive method, on average.
- Scale-free, and defined when the series contains zeros.

◆ Example: MAE 18 MW against an in-sample weekly-naive MAE of 24 MW gives MASE = 0.75.

PRACTICE

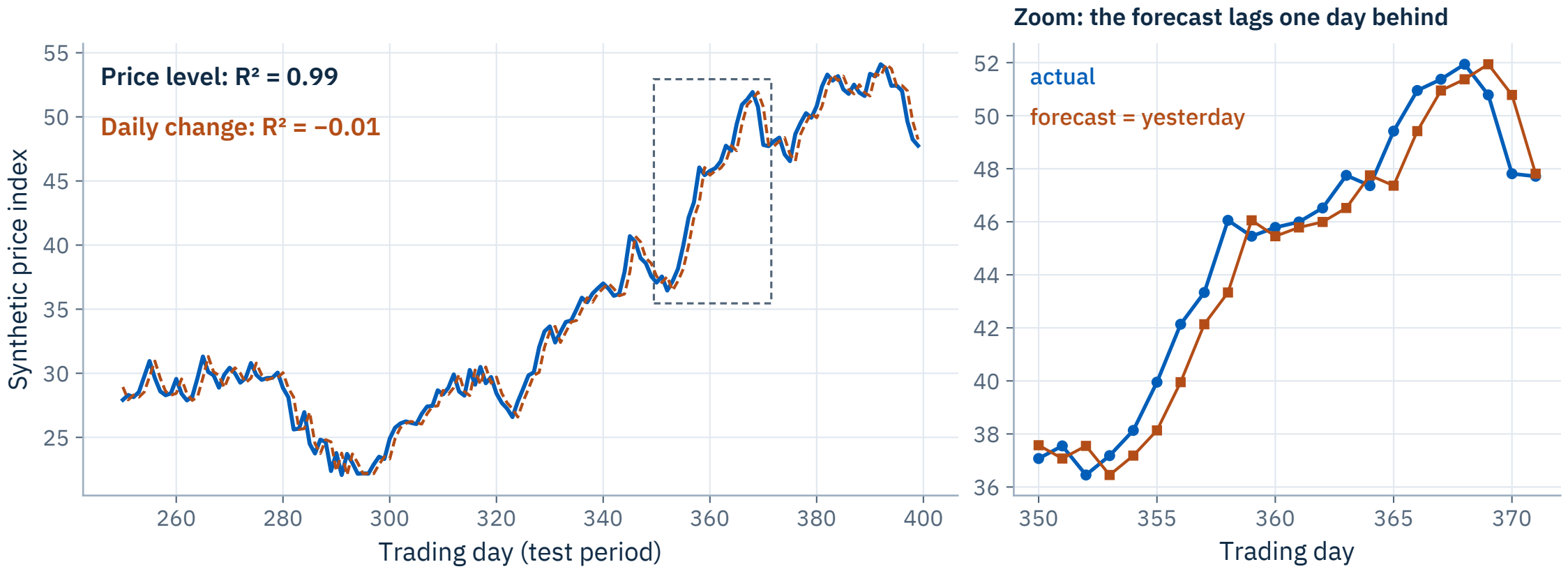
5 min · Pairs

Score one forecast four ways

Wind power (MW). Actual: 4, 2, 0.5, 6. Forecast: 3, 3, 1.5, 5. Training one-step naive MAE: 2.25 MW.

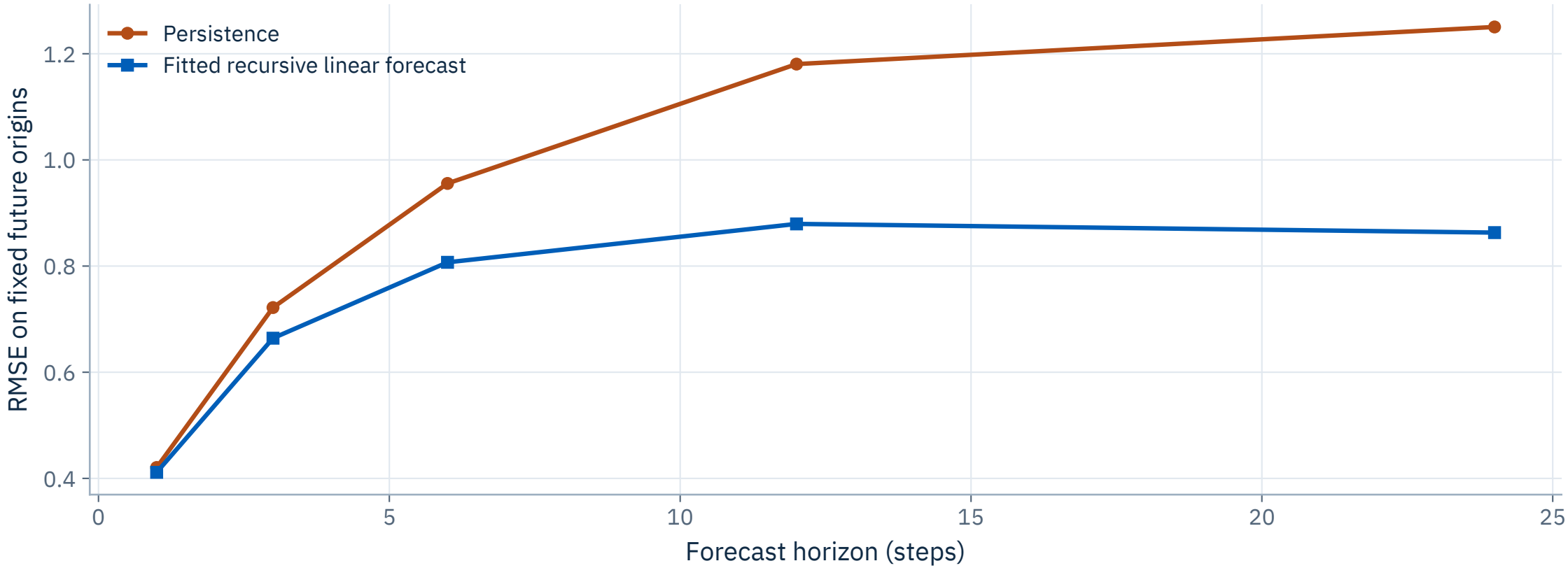
1. Compute MAE and RMSE.
2. Compute MAPE. Which hour dominates it?
3. Compute MASE and interpret it.

A price model can look perfect by copying yesterday



Synthetic Gaussian random walk (seed 21), test days 250–399; the forecast is simply the previous day's value.

Forecast error depends on horizon

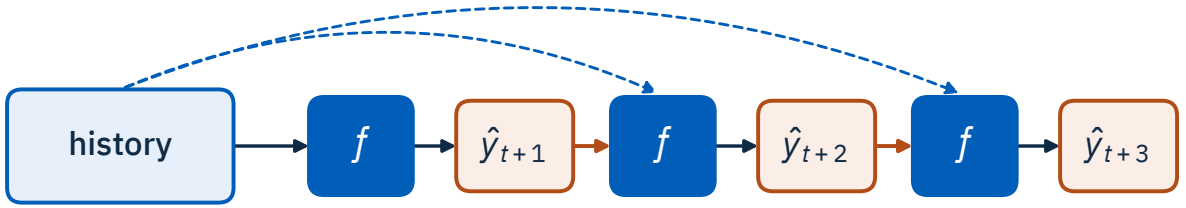


Simulated AR(1) series (coefficient 0.85, seed 42) scored on 320 fixed future origins; persistence and a recursive linear model fitted on earlier data.

Multi-step forecasts need a strategy

Recursive

one model, fed its own output

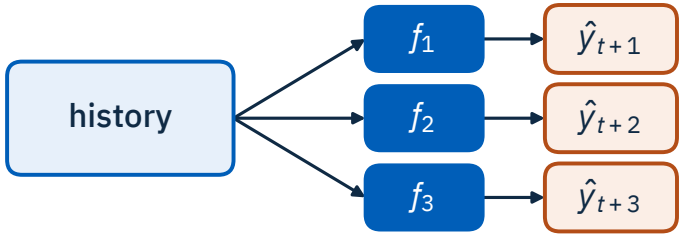


+ one model for any horizon

− errors feed forward and compound

Direct

one model per horizon

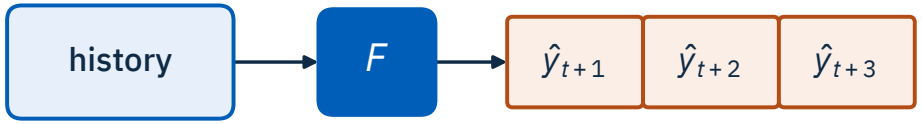


+ no feedback of errors

− H models; horizons trained apart

Multi-output

one model, a vector of outputs



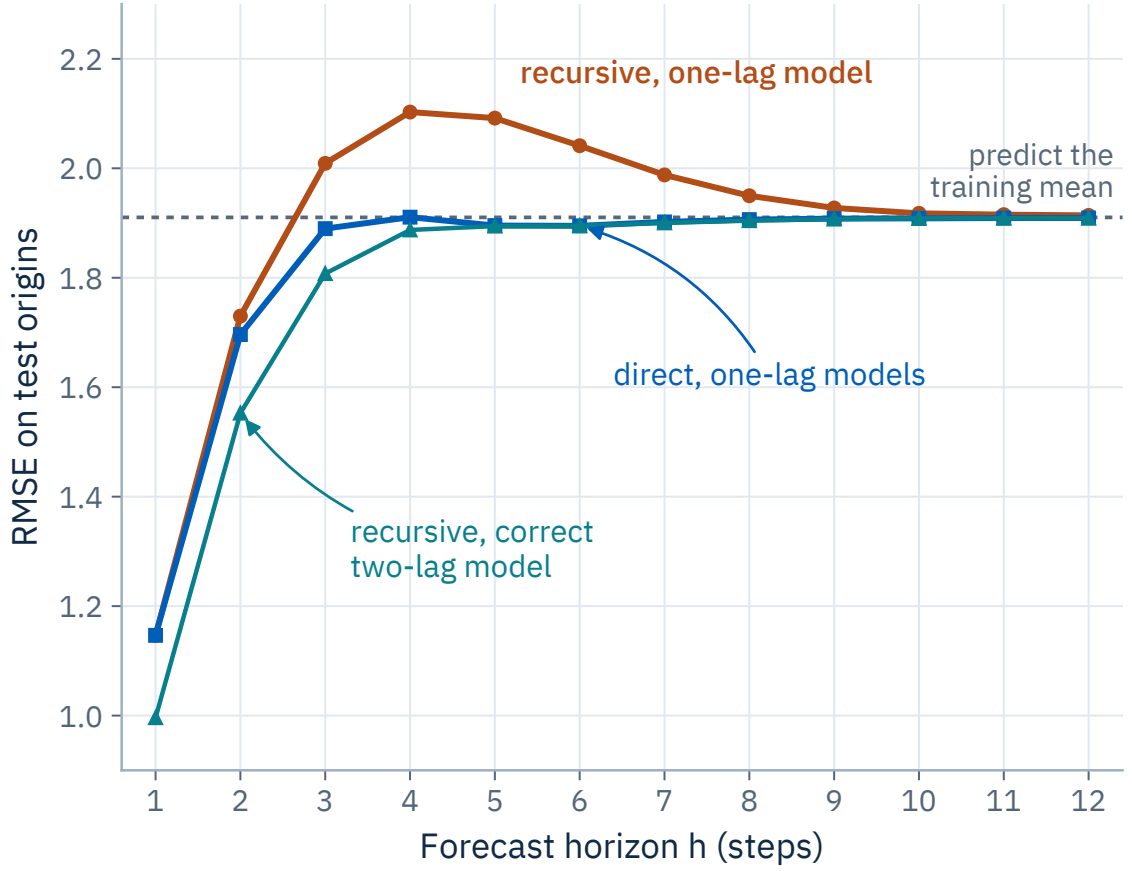
+ one model, horizons trained jointly

− horizon length fixed at design

Schematic of three strategies for horizons 1–3. Dashed arrows: the recursive model also re-reads the observed history.

A misspecified recursive model compounds its error

- True process uses two lags; the one-step model uses one.
- From h = 3, recursive errors exceed the mean forecast.
- Direct models level off at the mean-forecast error.
- With the correct two-lag model, recursion works well.



Simulated AR(2) process (seed 8); models fitted on the first 5,000 steps, scored on 1,986 later origins.

PRACTICE

5 min · Pairs

Design the evaluation for a wind forecast

A 50 MW wind farm needs hourly forecasts 1–48 hours ahead, issued every day at 06:00. You have two years of data.

1. Which baselines must a neural model beat?
2. Which metric would you report, broken down how?
3. Sketch the split and the rolling origins.

PART 3 OF 7

03

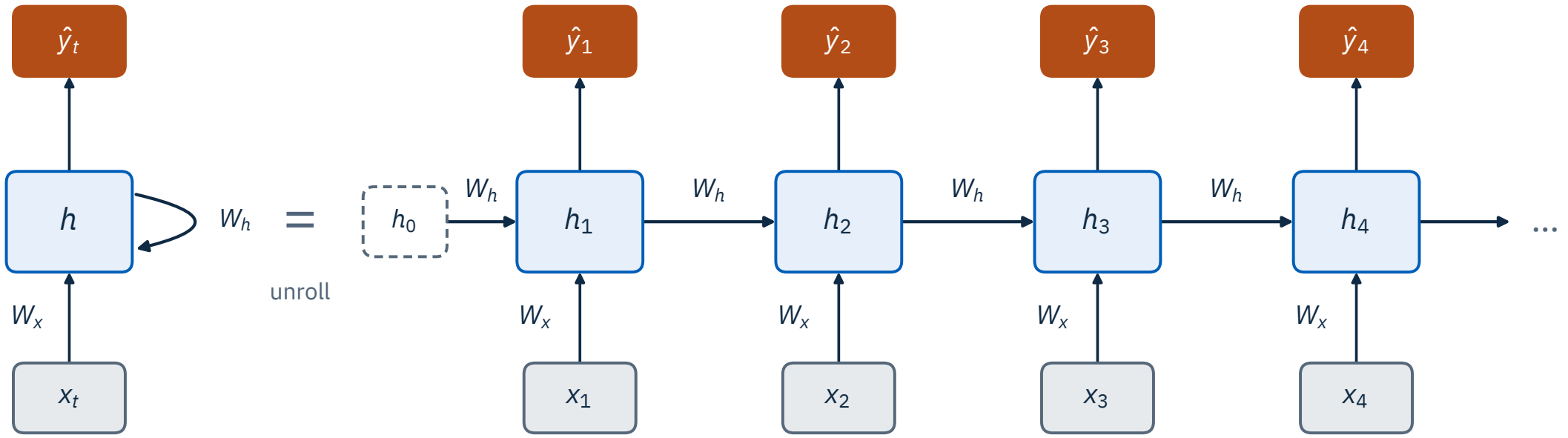
Recurrent networks

One shared update carries a state through time.

- The recurrent update and backpropagation through time
- Vanishing gradients and gated memory
- LSTM, GRU, stacked and bidirectional layers
- Shapes and state handling in PyTorch

One recurrent cell is reused at every step

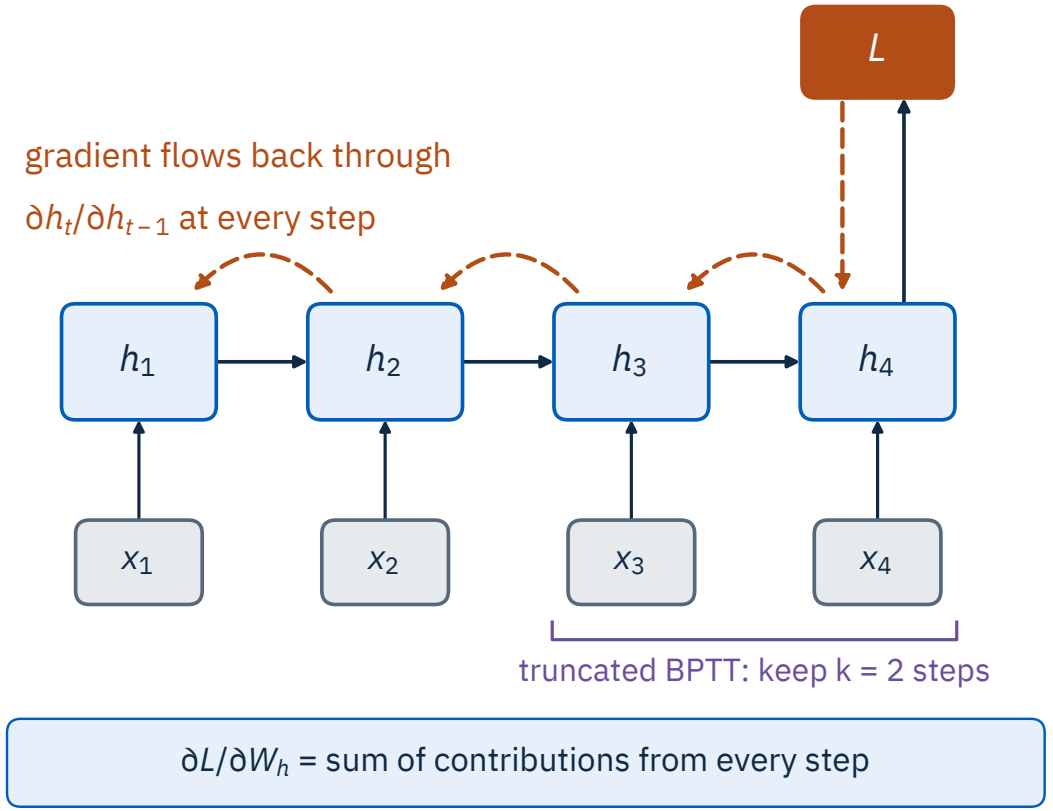
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$



Schematic. The same weights W_x, W_h and bias b are applied at every time step; h_0 is usually zeros.

Backpropagation through time follows the unrolled graph

- Unroll the shared recurrence over the training window.
- Gradients for W_h add up contributions from every step.
- Long products of derivatives can shrink or amplify.
- Truncated BPTT limits history to manage compute.

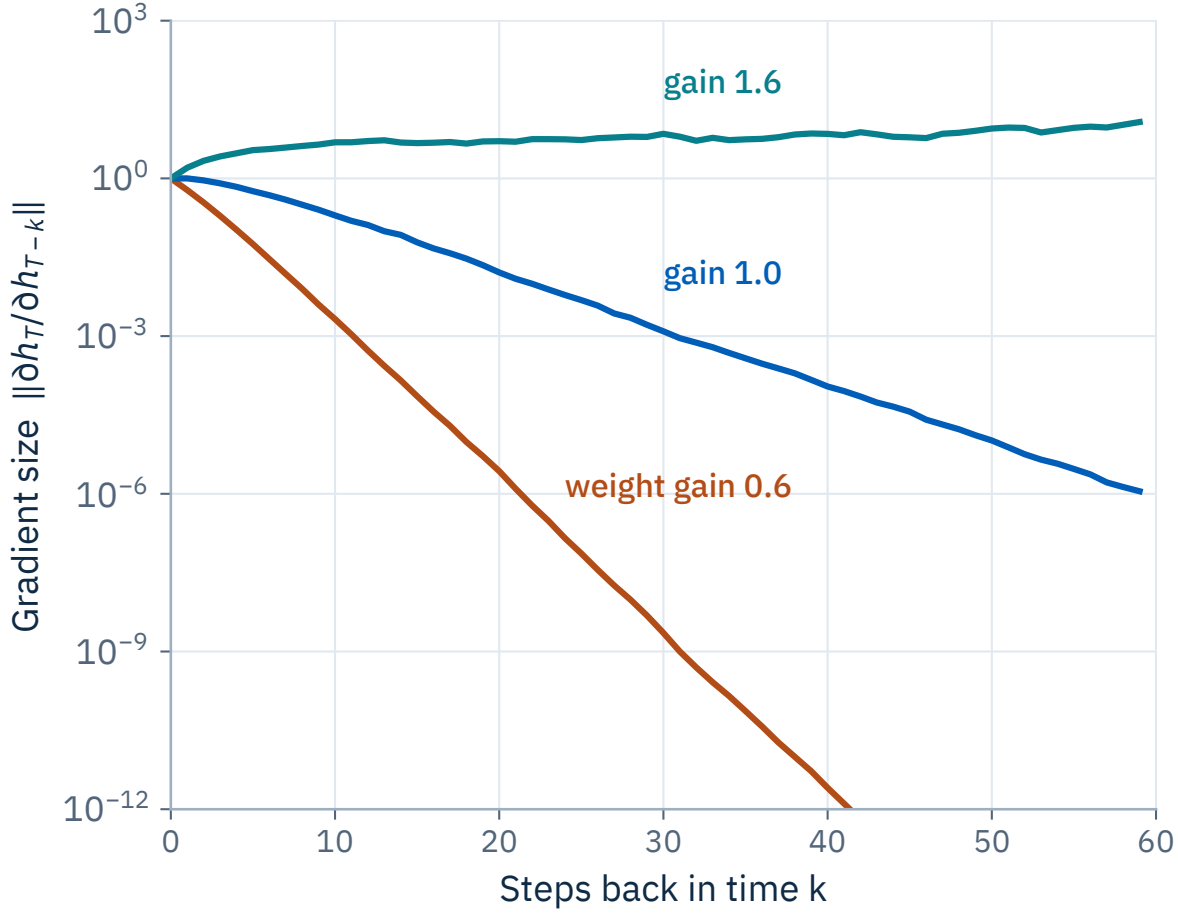


Schematic for a many-to-one RNN; dashed arrows carry gradients backwards through the recurrence.

Gradients shrink geometrically through time

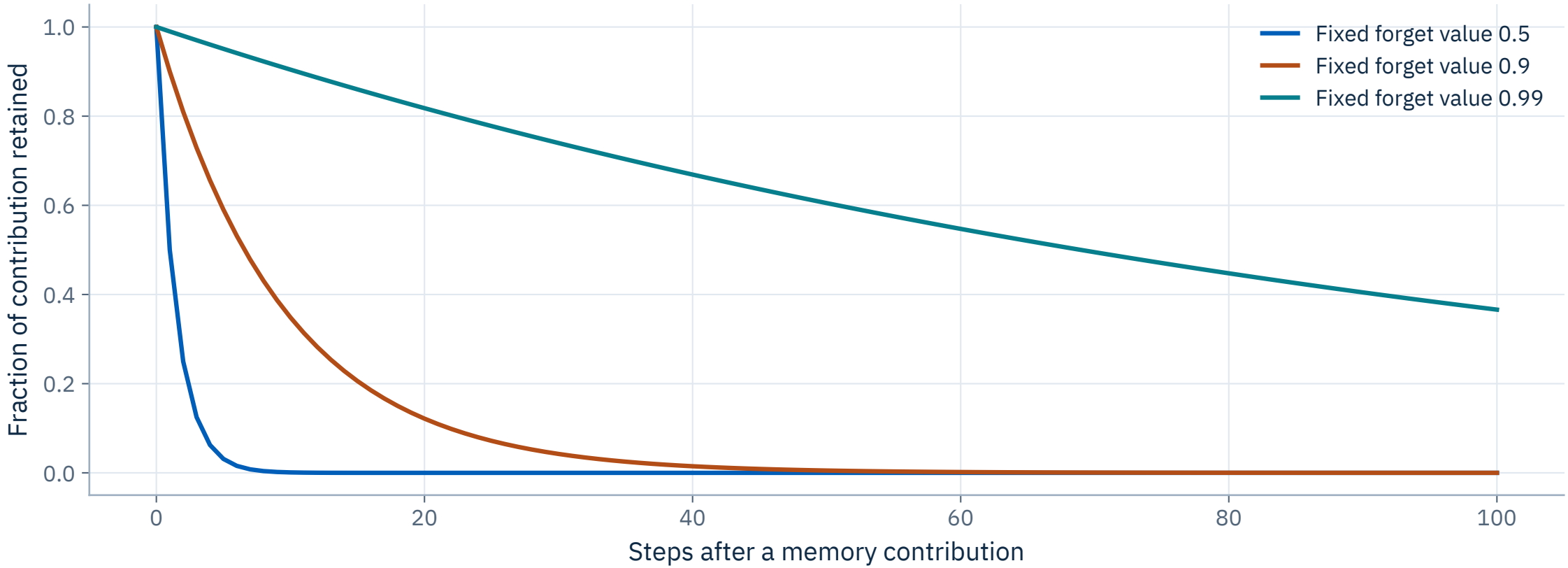
$$\frac{\partial h_T}{\partial h_{T-k}} = \prod_{j=T-k+1}^{T} \operatorname{diag}(1 - h_j^2)\, W_h$$

- Each step multiplies the gradient by a Jacobian.
- Gain 0.6: about 3×10^{-6} of the signal after 20 steps.
- Even gain 1.0 decays, because tanh slopes are below 1.



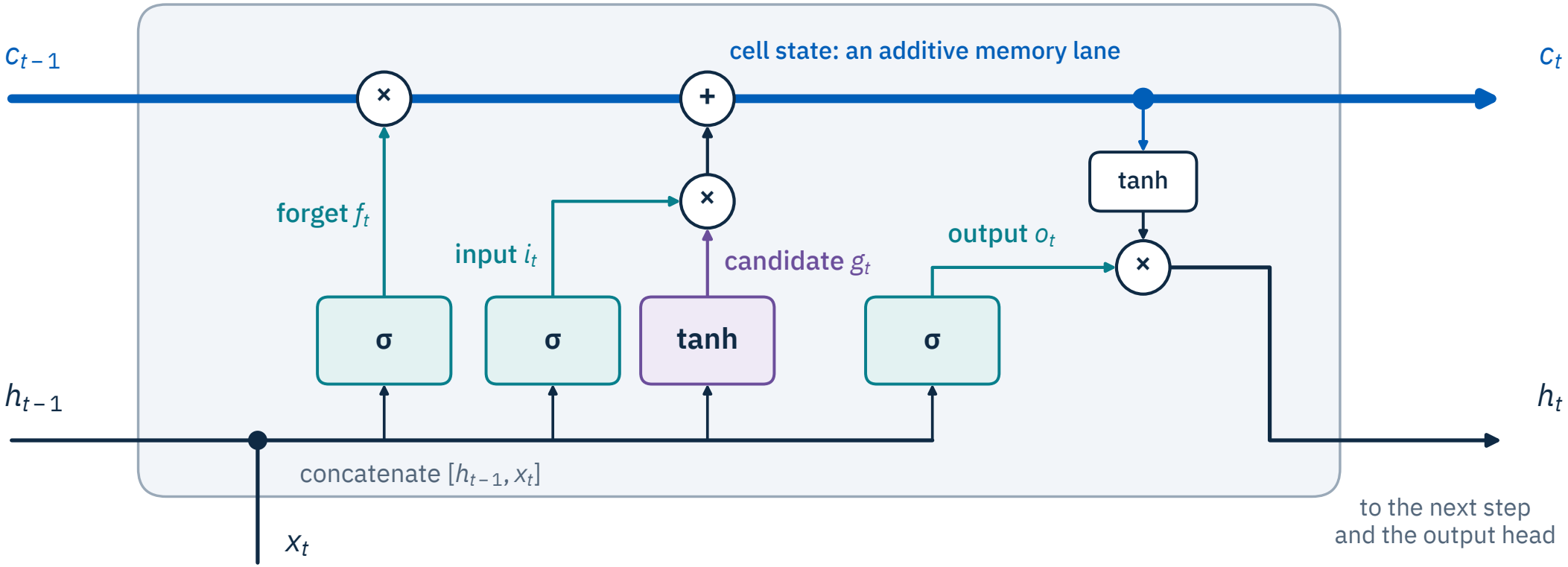
Computed for a 32-unit tanh RNN with random inputs (seed 4); median over 30 runs.

Gated memory offers a more direct path



Simplified retained-signal magnitude under fixed forget-gate values. Real LSTM gates depend on inputs and state.

An LSTM cell adds a protected memory lane



Schematic of one LSTM step. Teal: sigmoid gates; purple: candidate values; blue: cell state. × and + act elementwise.

Four gates define the LSTM update

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \qquad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$g_t = \tanh(W_g[h_{t-1}, x_t] + b_g), \qquad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t)$$

- Gates use sigmoids (0 to 1); the candidate uses tanh (–1 to 1).
- ⊙ multiplies elementwise; c and h both have H values.

◆ PyTorch stacks the four weight blocks in the order i, f, g, o.

PRACTICE

4 min · Individual

A memory update by hand

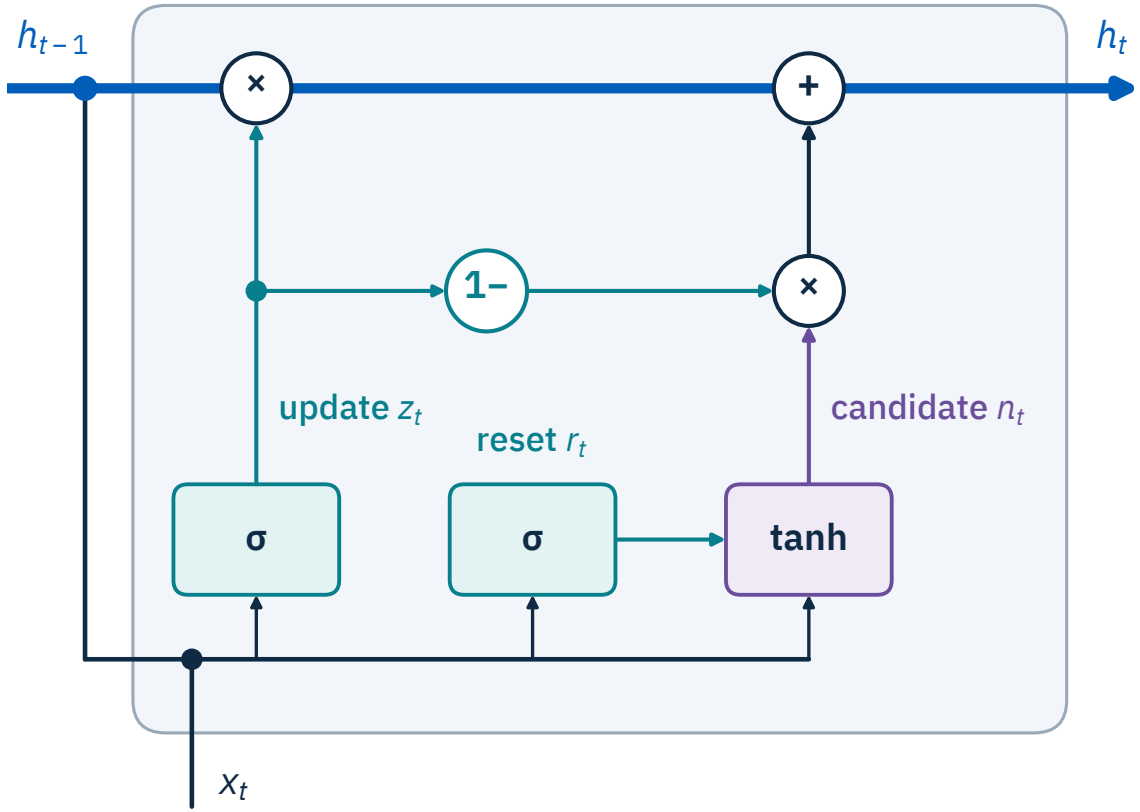
Previous cell value = 2. Forget gate = 0.9. Input gate = 0.2. Candidate = −1.

1. Calculate the new cell value.
2. With output gate 0.5, compute the hidden state.
3. With the forget gate fixed at 0.9 and no new input, what fraction of the memory remains after 10 steps?

A GRU blends old and new state with one gate

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

- Update gate *z* blends the candidate with the old state.
- Reset gate *r* scales the old state inside the candidate.
- No separate cell state; three weight blocks, not four.



Schematic of one GRU step in the PyTorch convention.

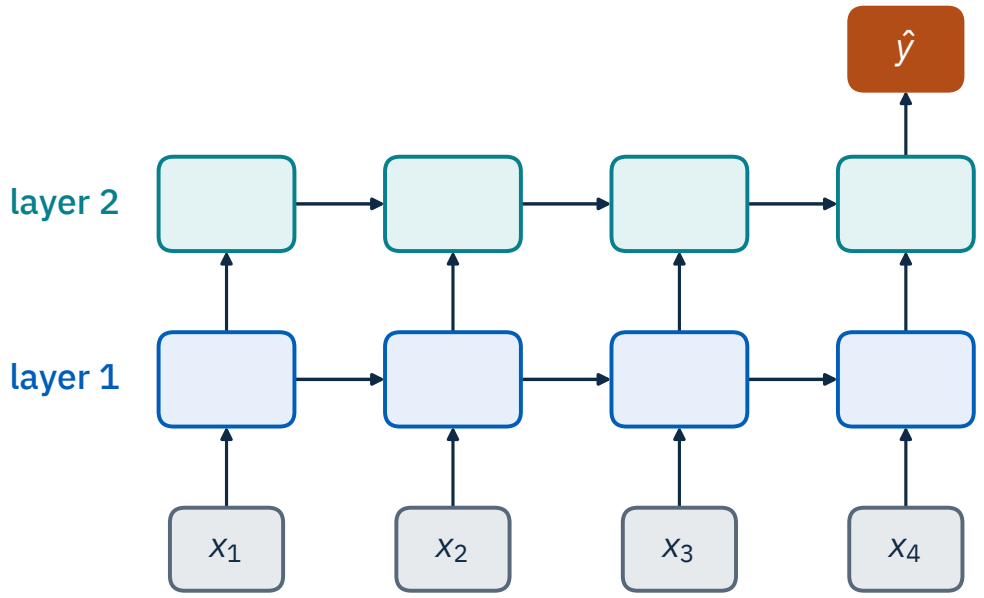
RNN, LSTM and GRU trade simplicity for memory

Model	State	Parameters (14 inputs, 64 units)	Main trade-off
Vanilla RNN	Hidden state	5,120	Simple; long-range training is hard
LSTM	Hidden and cell states	20,480	More gates and parameters
GRU	One gated hidden state	15,360	A simpler gated alternative
1D CNN or TCN	Local feature maps	Depends on kernels and layers	Parallel over time

Compare them with the same split, target and tuning budget.

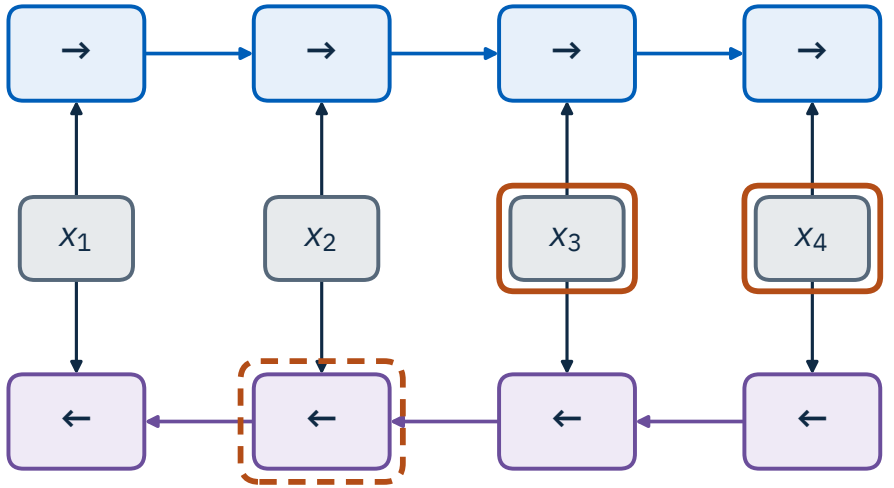
Stacked layers add depth; bidirectional layers read ahead

Stacked: two recurrent layers



Bidirectional: two directions

each output concatenates both directions



the backward state at step 2 has already read x_3 and x_4

Schematic. Left: two stacked recurrent layers, many-to-one. Right: a bidirectional layer; its backward direction runs from the end of the window.

⚠ A bidirectional layer is valid only inside a window that ends at the forecast origin.

A many-to-one LSTM shape check

```python
import torch
from torch import nn

rnn = nn.LSTM(input_size=6, hidden_size=32, batch_first=True)
head = nn.Linear(32, 1)
x = torch.zeros(8, 24, 6)                    # (batch, time, features)
sequence, (hidden, cell) = rnn(x)
prediction = head(sequence[:, -1, :])
print(sequence.shape, hidden.shape, prediction.shape)
```

OUTPUT

```
torch.Size([8, 24, 32]) torch.Size([1, 8, 32]) torch.Size([8, 1])
```

For one layer and equal lengths, `sequence[:, -1]` equals `hidden[0]`.

Hidden state handling is part of the data model

Reset between windows

- The default for independent, shuffled windows
- Every window starts from zeros

Carry state only in order

- Same asset, chronological batches
- Detach at truncation boundaries

Mask padding

- Padded steps are not measurements
- Use packed sequences or masks

Decide the warm-up

- Cold start or a burn-in history
- Identical in training and inference

PRACTICE

5 min · Pairs

Check the lab model before training it

The climate notebook uses `nn.LSTM(14, 128, num_layers=2, batch_first=True)` and a head 128 → 64 → 1.

1. What are the shapes of the LSTM output and of `h_n` for 256 windows of 72 hours?
2. How many parameters does the first LSTM layer have?
3. Which tensor feeds the head, and why not `h_n[0]`?

LABORATORY

60 min

Lab 1A: a warm-up forecast and two baselines

- Shampoo: build 6-month windows, train the LSTM, compare with the previous month.
- Climate: load Jena, subsample hourly, split 70/20/10 in time.
- Fit the scaler on training rows; build 72 h → +12 h windows per split.
- Score persistence and the 24-hour seasonal naive on validation, in °C.
- Start the LSTM training before lunch if time allows.

NOTEBOOK

`exercises/01_lstm_shampoo/LSTM_Shampoo_Student.ipynb`

EVIDENCE TO SUBMIT

- Shampoo RMSE next to the naive RMSE
- Window counts for each split
- Two baseline MAEs on validation

LABORATORY

45 min

Lab 1B: the climate LSTM against both baselines

- [ ] Train the multi-feature LSTM with early stopping; plot both loss curves.
- [ ] Score the test windows in °C: LSTM, persistence and seasonal naive.
- [ ] Use identical target timestamps for all three methods.
- [ ] Plot one test week and name one failure period with a hypothesis.
- [ ] Optional: compare a temperature-only model on validation.

NOTEBOOK

`exercises/02_lstm_climate/LSTM_Climate_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table with both baselines first
- Loss curves with the chosen epoch marked
- One failure period and a hypothesis

DISCUSSION

At 12 hours ahead, persistence is the weak baseline

Jena test windows of the notebook split (6,927 origins); the target is 12 hours after the origin.

4.78 °C

persistence MAE

2.38 °C

24-hour seasonal naive MAE

7.81 °C

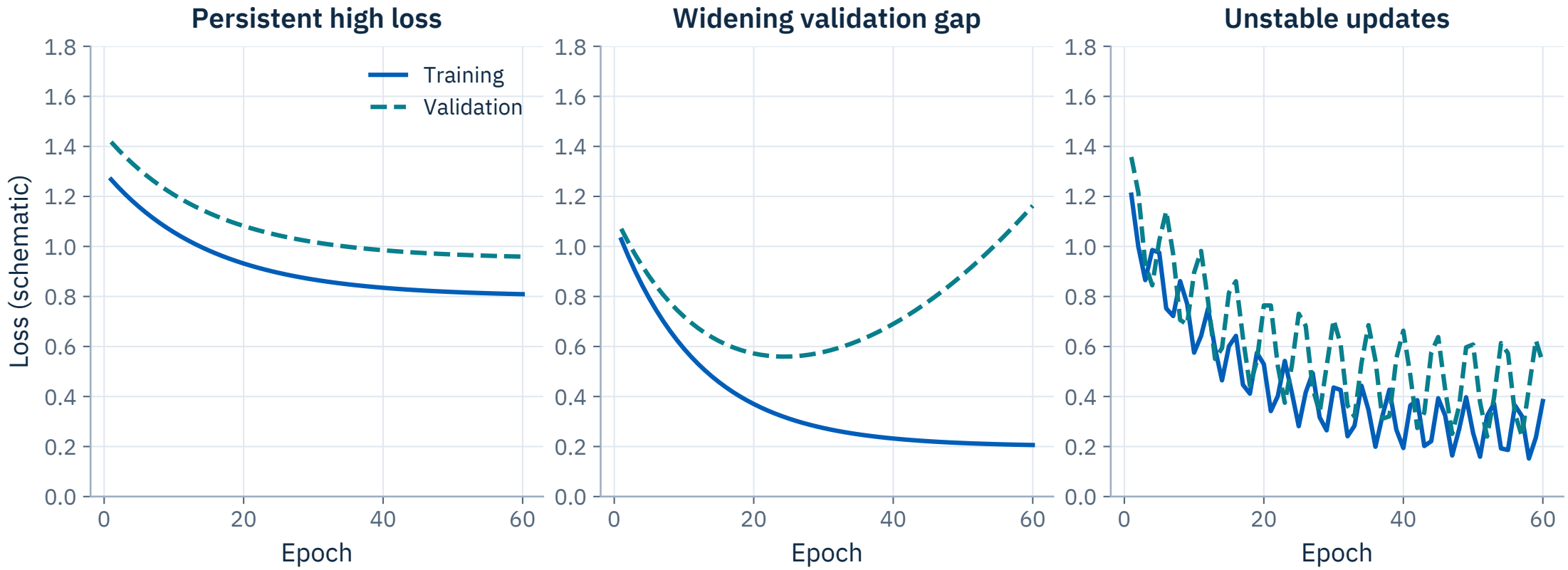
standard deviation of the target

- Report your LSTM next to both; the fair reference is the stronger one.

DISCUSSION

Your loss curves suggest the next experiment

Which shape is closest to your climate LSTM, and what would you change first?



Schematic learning curves (recap of Day 2), not measured training runs; each shape admits several diagnoses.

DISCUSSION

Six errors that flatter a sequence model

✕ Scaler fitted on all rows

- Test statistics leak into training

✕ Windows cross the split

- Training targets inside validation

✕ Different target timestamps

- Baseline and model scored on different rows

Wrong horizon index

- Off by one between origin and target

Test set used for stopping

- The test score is no longer a test

✕ Windows split at random

- Near-copies of test windows in training

PART 4 OF 7

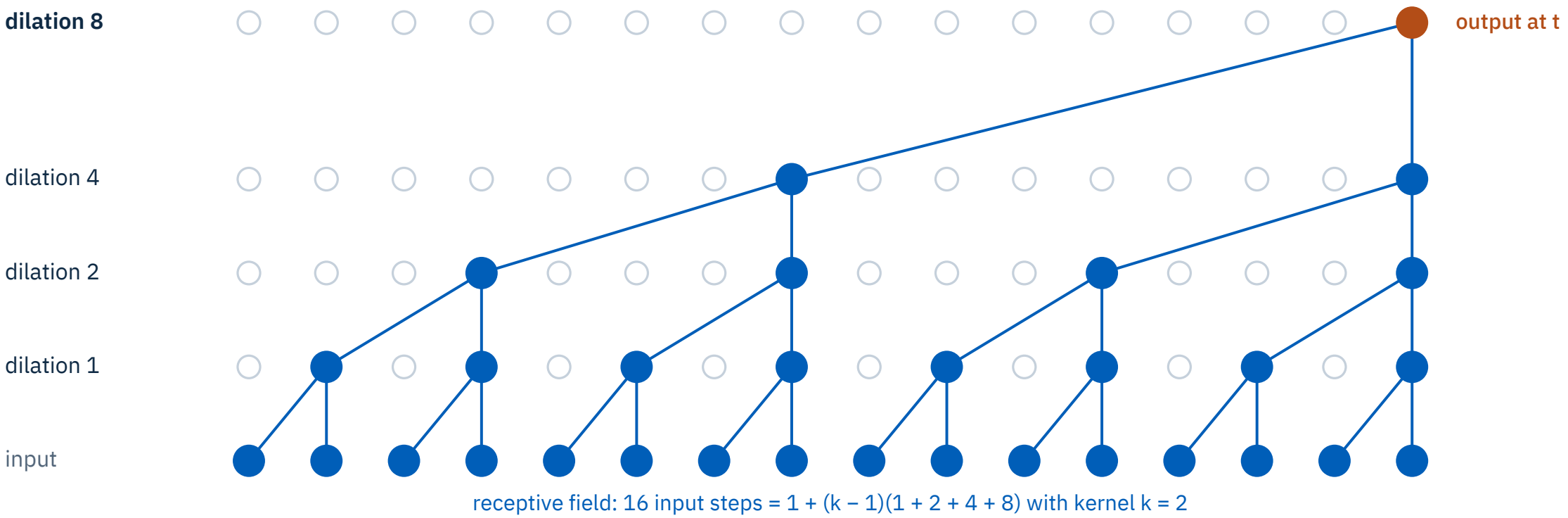
04

Temporal convolutions and attention

Two ways to see far back without a long recurrent chain.

- Dilated causal convolutions
- The encoder–decoder bottleneck
- Queries, keys, values and masks
- Multi-head attention

Dilated causal convolutions reach far back cheaply



Schematic of a causal temporal convolutional stack (kernel size 2, dilations 1, 2, 4, 8). Highlighted: every input that can influence the last output.

Receptive field grows with kernel size and dilation

$$R = 1 + (k - 1)\sum_{l} d_l$$

16

k = 2, dilations 1, 2, 4, 8

31

k = 3, dilations 1, 2, 4, 8

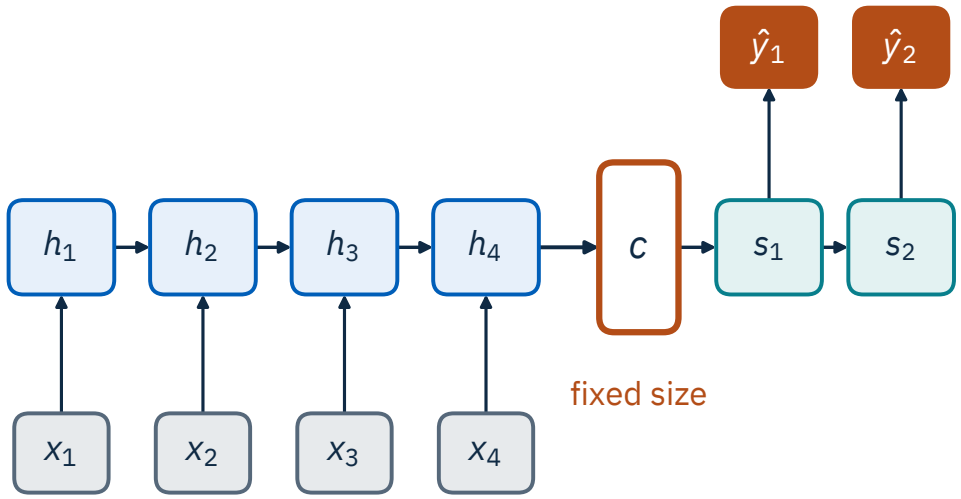
9

k = 3, four layers, no dilation

- One week of hourly data (168 steps) needs 8 doubling layers with k = 2: R = 256.

One context vector is a bottleneck

Without attention: one context vector

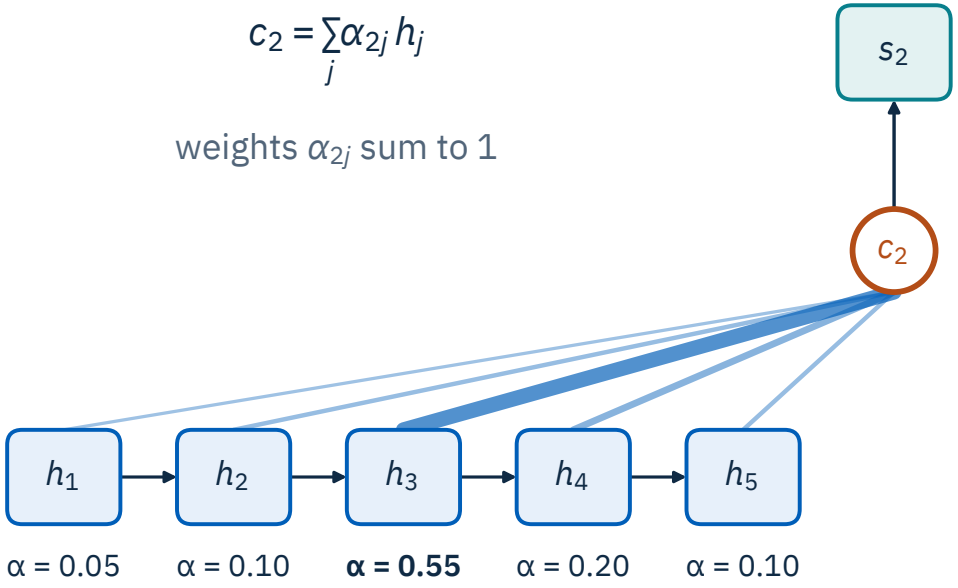


every output step sees only c

With attention: a new context for each output

$$c_2 = \sum_j \alpha_{2j} h_j$$

weights α_{2j} sum to 1

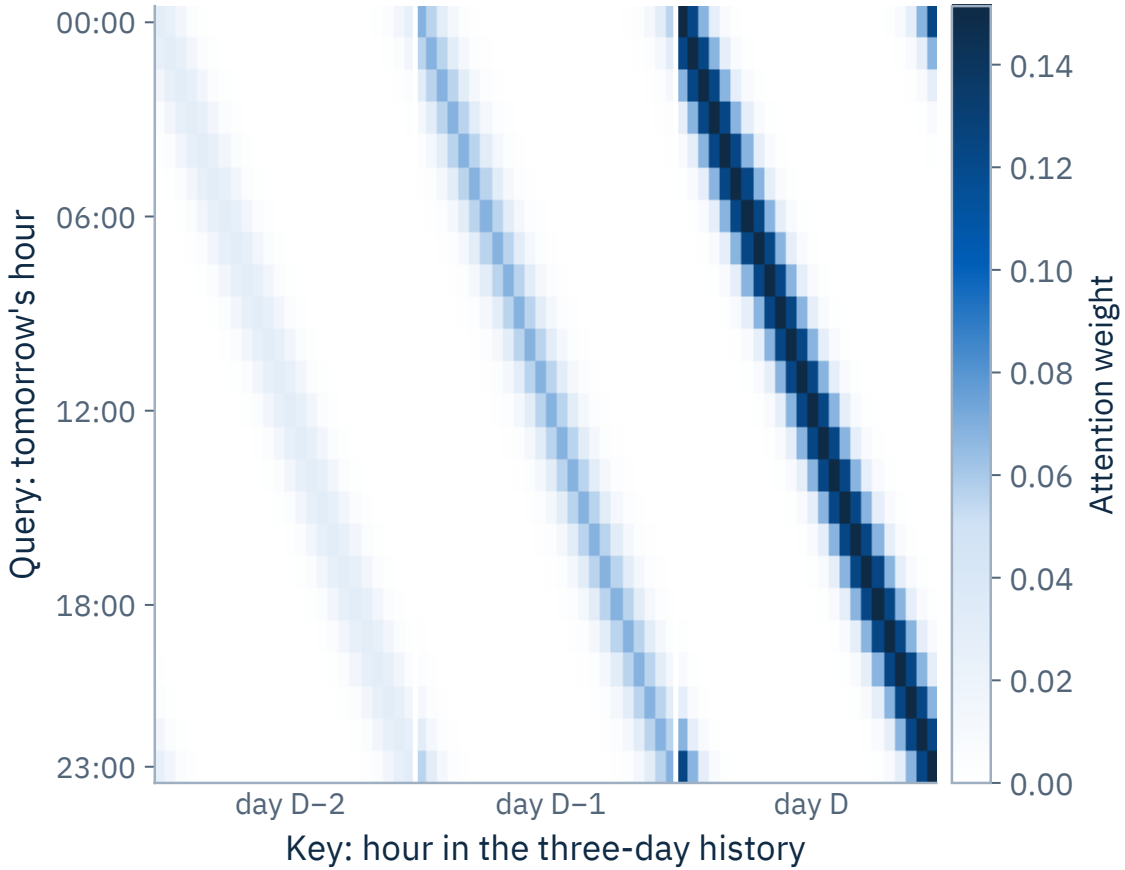


line width shows the attention weight for output step 2

Schematic. Left: an encoder–decoder that passes one fixed-size vector. Right: attention forms a weighted context for each output step; weights are illustrative.

Attention is a learned soft alignment

- Each row is a probability distribution over the keys.
- Each hour of tomorrow looks at the same clock hour.
- Recent days weigh more: day D gets 61% at 00:00.



Hand-built queries and keys (clock-hour encodings, recency penalty); illustrates the mechanism, not a trained model.

◆ Attention weights show where information flowed, not why the output is right.

Queries, keys and values have distinct roles

Quantity	Role	In self-attention
Query Q	What each position seeks	A projection of the input states
Key K	What each position offers for matching	Another projection of the same states
Value V	Information to aggregate	A third projection of the same states

Cross-attention takes queries from one sequence and keys and values from another.

◆ Each output is a sum of values, weighted by how well its query matches each key.

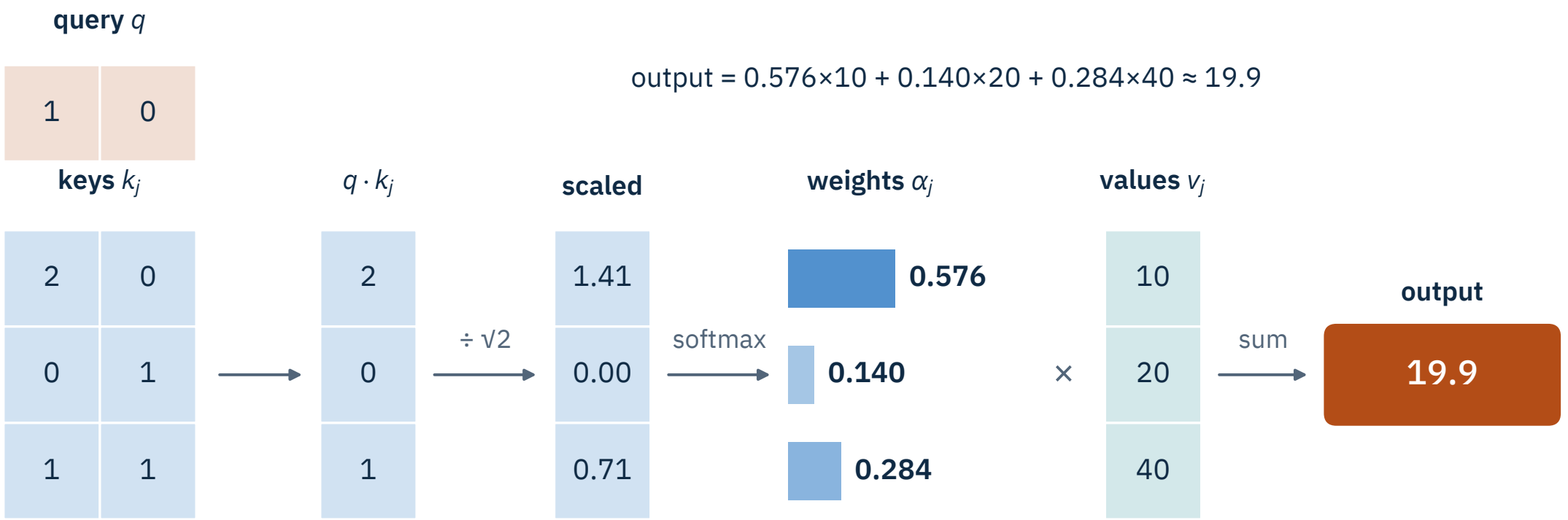
Scaled dot-product attention in one line

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

- Divide by $\sqrt{d_k}$ to keep scores in a useful range.
- Softmax runs over the keys, separately for each query.
- The mask M adds $-\infty$ where access is not allowed.
- Each output is a weighted average of value vectors.

◆ PyTorch: `F.scaled_dot_product_attention(q, k, v, is_causal=True)`.

Attention for one query, step by step



Exact arithmetic for one query, three keys and scalar values; $d_k = 2$.

PRACTICE

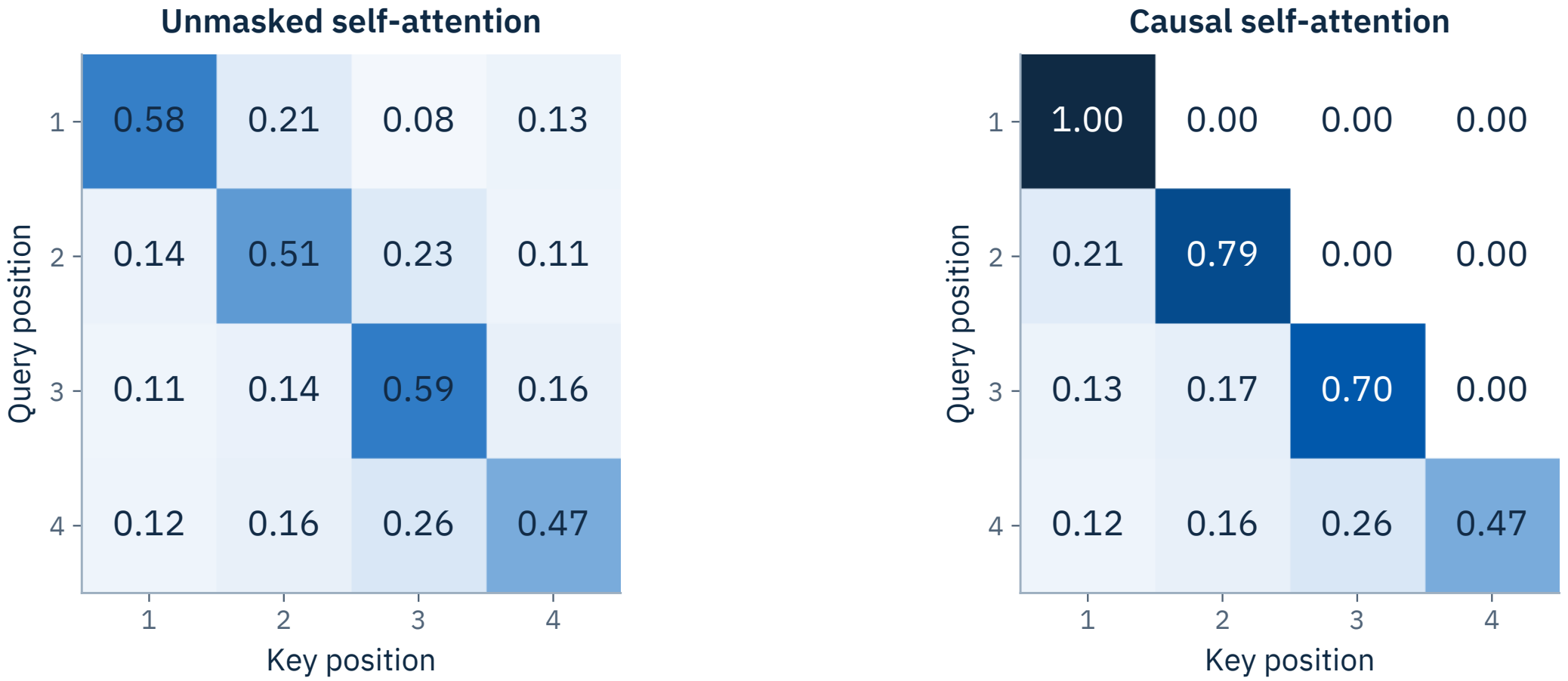
4 min Individual

One attention calculation

A query has scaled scores [0, log 2] for two keys. The values are [10, 20].

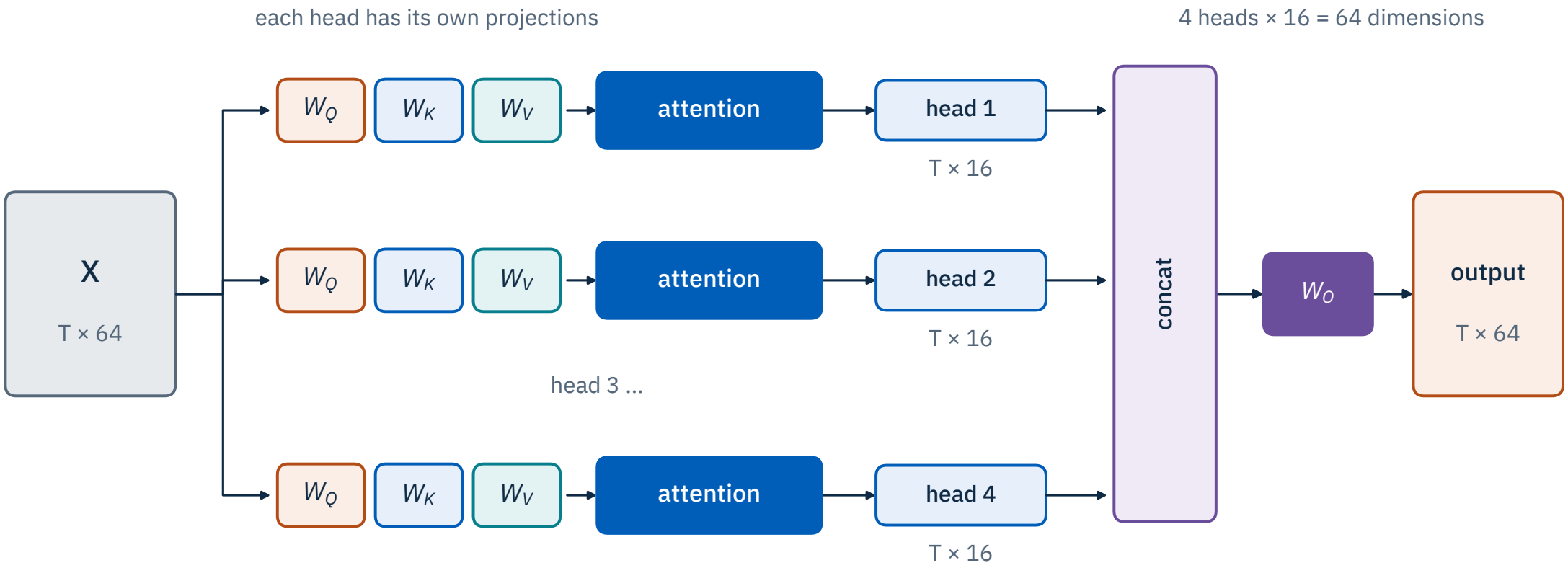
1. Calculate the softmax weights.
2. Calculate the attention output.
3. What changes if the second key is masked out?

A causal mask removes future access



Computed example attention weights with and without an additive causal mask. Every row normalizes over its allowed keys.

Multiple heads attend in different subspaces



Schematic of multi-head self-attention with model width 64 and 4 heads of width 16.

PRACTICE

4 min · Pairs

Why divide by the square root of d_k?

Query and key components are independent, with mean 0 and variance 1; $d_k = 64$.

1. What is the standard deviation of q · k before and after the scaling?
2. Compare softmax([8, 0, −8]) with softmax([1, 0, −1]).
3. Why would near one-hot weights slow down learning?

PART 5 OF 7

05

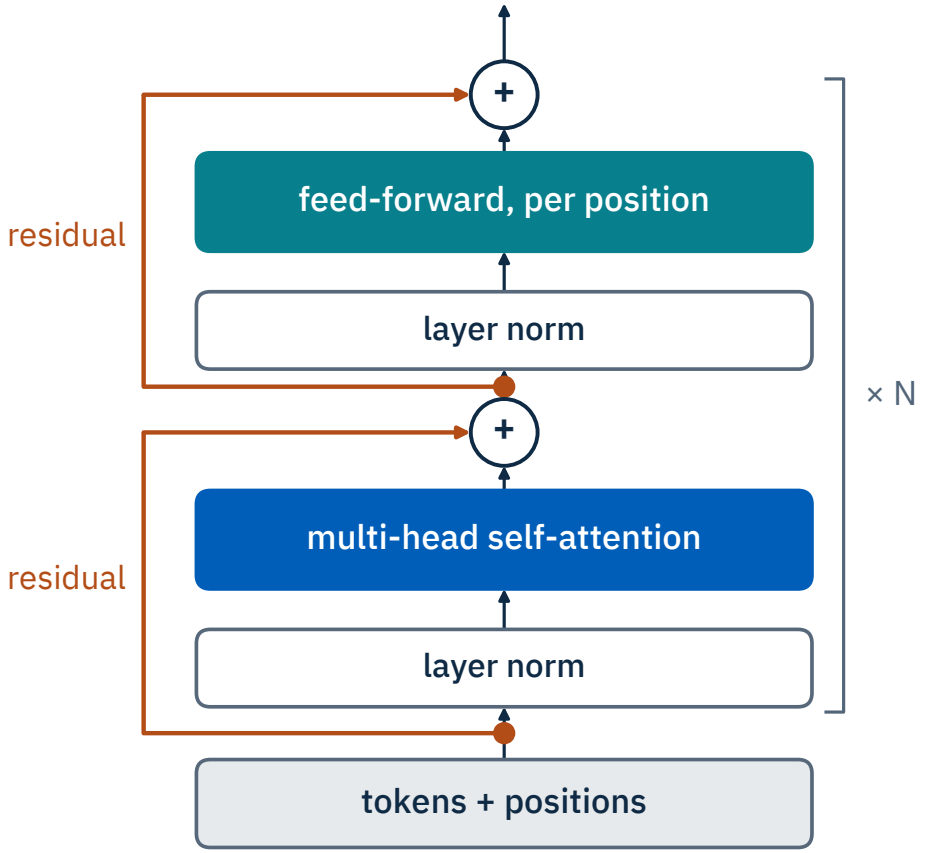
Transformers and pretrained models

Attention blocks, positions, and models trained at scale.

- The transformer block and positional information
- Teacher forcing and the cost of long sequences
- Pretrained forecasters
- Language models in energy workflows

A transformer block alternates mixing and processing

- Self-attention mixes information between positions.
- The feed-forward network transforms each position alone.
- Residual additions keep a direct update path.
- Layer norm controls scale; its placement varies.

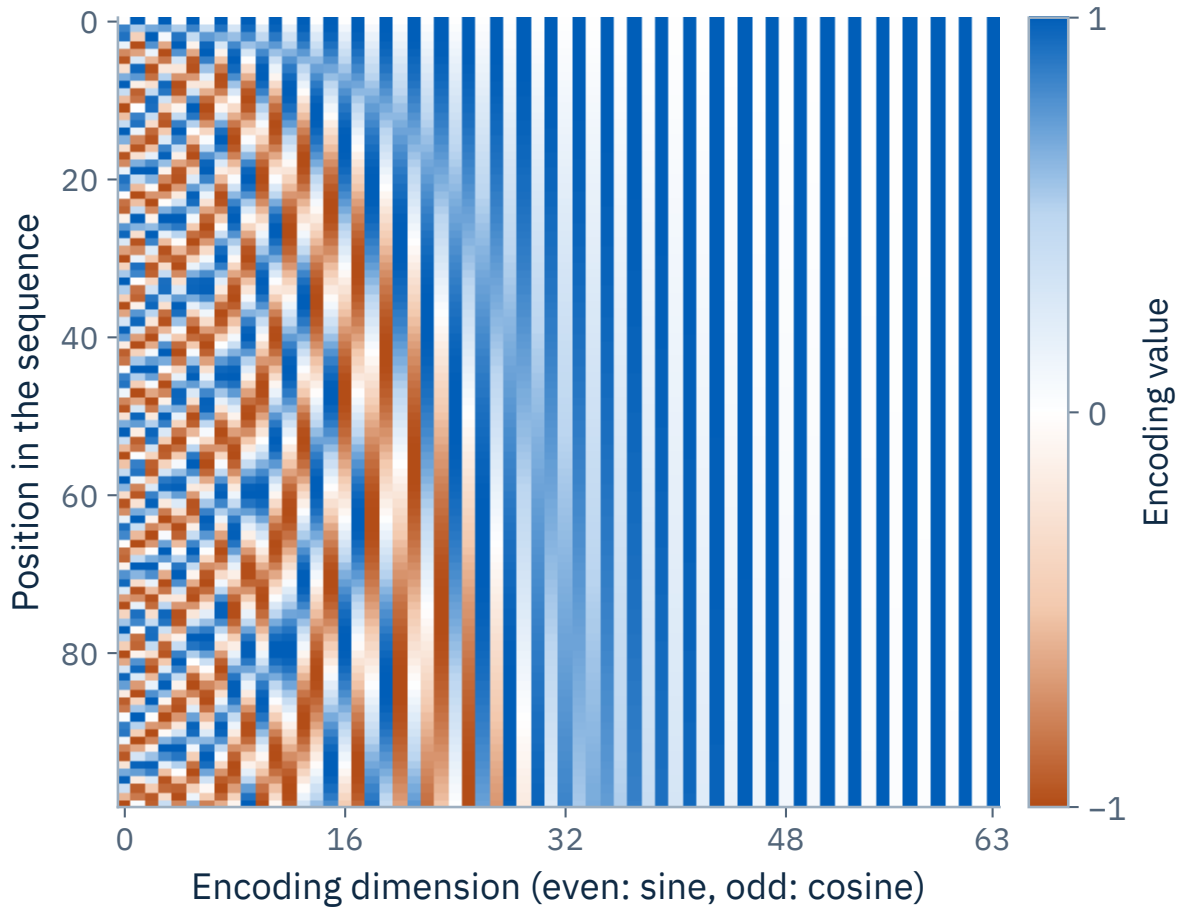


Schematic of a pre-norm encoder block; the 2017 original applied layer norm after each residual addition.

Position must enter the representation

$$PE_{p,\,2i} = \sin\left(p \,/\, 10000^{2i/d}\right)$$
$$PE_{p,\,2i+1} = \cos\left(p \,/\, 10000^{2i/d}\right)$$

- Self-attention alone ignores the order of positions.
- Options: sinusoidal, learned, relative, rotary.
- Time series also need elapsed time and calendar.



Exact sinusoidal encoding for model width 64 and positions 0–99.

Transformer families match different tasks

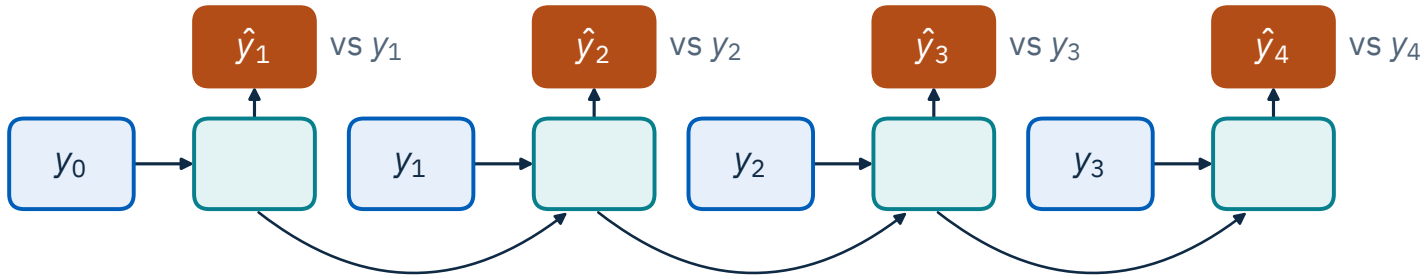
Family	Attention pattern	Example use
Encoder	Often bidirectional over available input	Representation or classification
Decoder-only	Causal over generated positions	Next-token prediction
Encoder–decoder	Input encoding plus cross-attention	Translation or sequence generation

All three can forecast if the masks respect the forecast origin.

- Many large language models are decoder-only transformers trained to predict the next token.

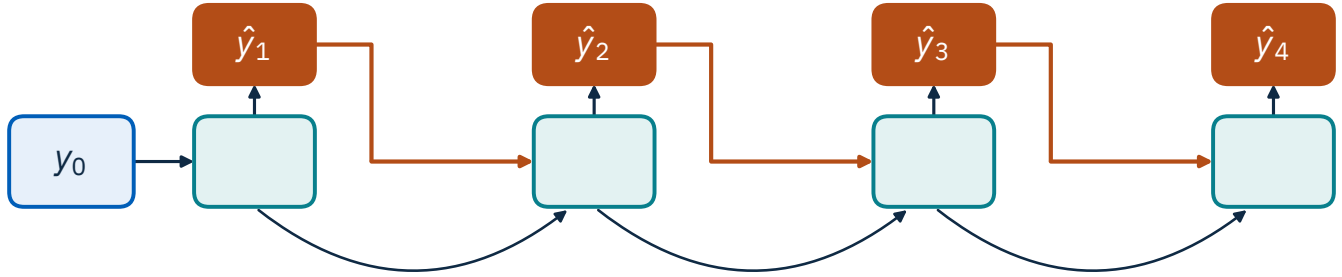
Teacher forcing and inference see different histories

Training with teacher forcing: each step reads the true previous value



loss at every step;
inputs are observed values

Inference, autoregressive: each step reads its own previous forecast

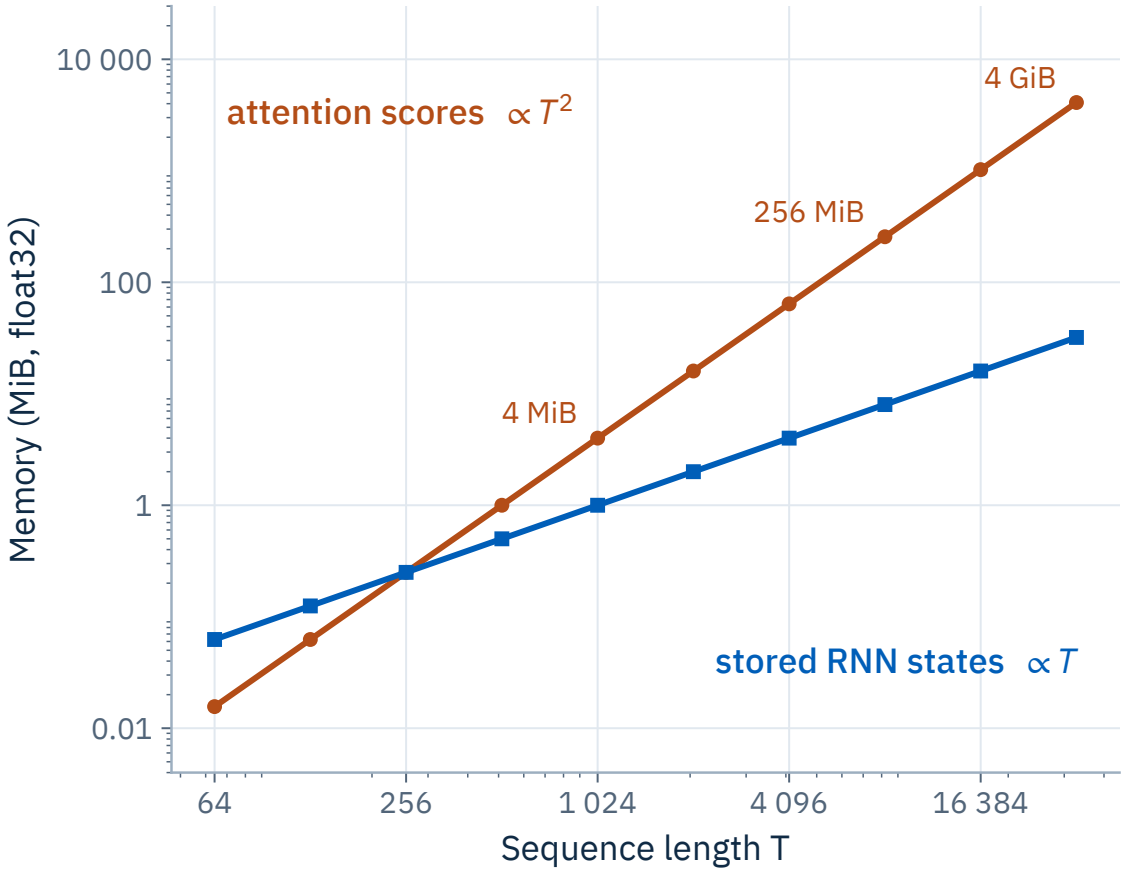


forecast errors are
fed back as inputs

Schematic of a recurrent decoder in training (teacher forcing) and in inference (autoregressive rollout); y_0 is the last observed value.

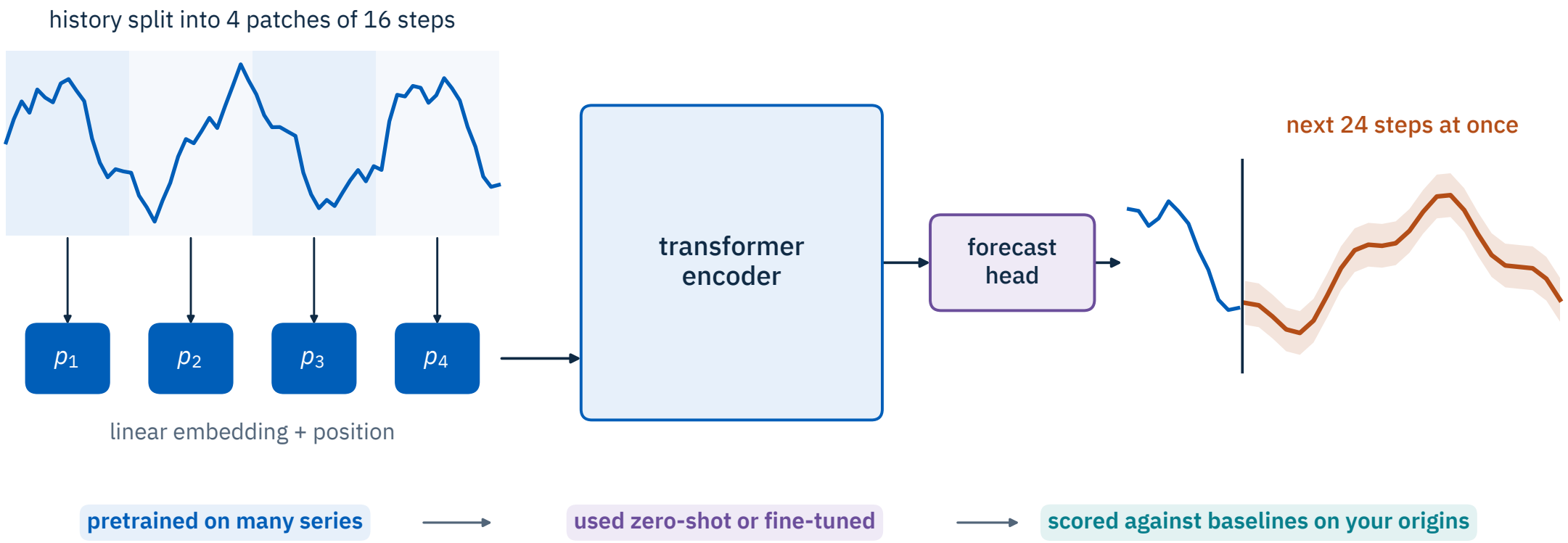
Dense attention cost grows with the square of length

- The score matrix has T^2 entries per head.
- One year of hourly data: T = 8,760, about 293 MiB per head.
- Efficient kernels cut memory traffic; compute stays quadratic.
- Measure memory and latency at your real input lengths.



Exact arithmetic for float32 scores, one head and one sequence, against stored RNN states of width 256.

Time-series transformers turn patches into tokens



Schematic of a patch-based transformer forecaster; the curves are a synthetic illustration, not model output.

Pretrained forecasters still face your baselines

What they are

- Trained on many public series
- Used zero-shot or fine-tuned
- Some accept covariates, some do not

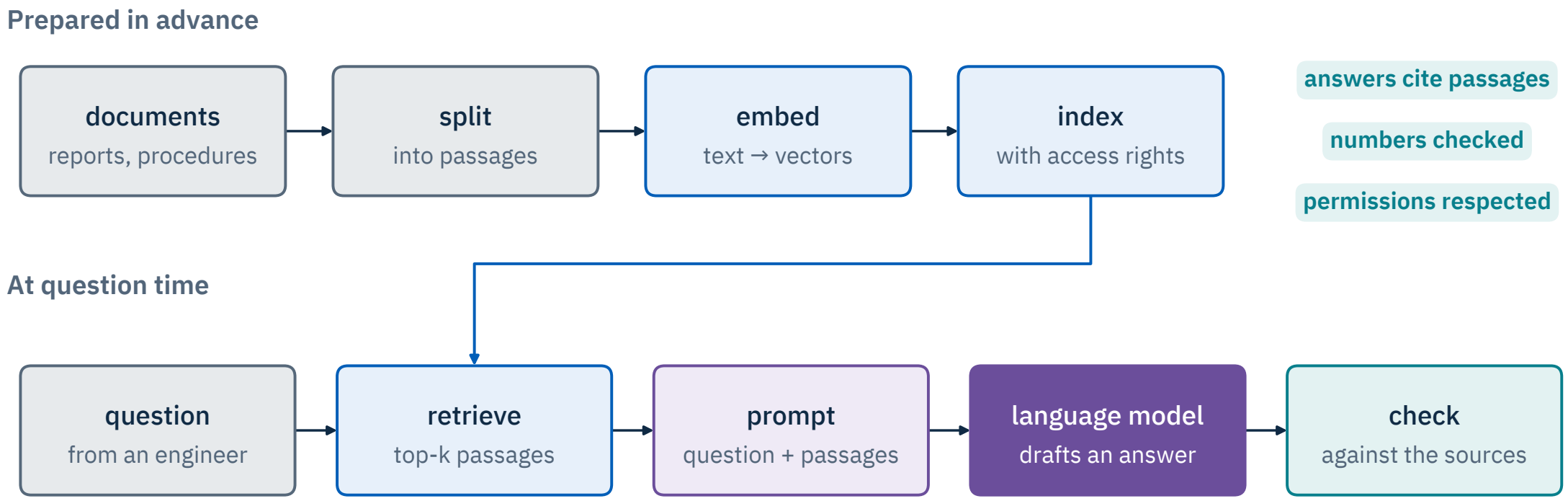
What to check

- Frequency, horizon and context length
- Overlap with the pretraining data
- Probabilistic output: is it calibrated?

How to judge

- Same rolling origins as baselines
- Error by horizon, cost and latency
- Licence and data governance

Language models can search documents and draft code



Schematic of retrieval-augmented generation for technical documents.

Generated answers need checking against sources

✓ Useful with checks

- Search and summarize reports, with citations
- Draft analysis code, then test it
- Extract fields from forms for review

vs

✗ Risky without checks

- Fluent but wrong numbers or units
- Confidential data sent to outside services
- Decisions without domain review

◆ Evaluate a retrieval system on source relevance and answer faithfulness.

PRACTICE

5 min · Pairs

Match the architecture to the constraint

Choose a starting point after the baselines, and name one risk.

1. ESP sensors every 10 minutes; a real-time alarm from the last 2 hours.
2. Hourly load for 300 feeders; one-week context; day-ahead horizon.
3. 5,000 well reports; "which wells had stuck pipe after a mud change?"

PART 6 OF 7

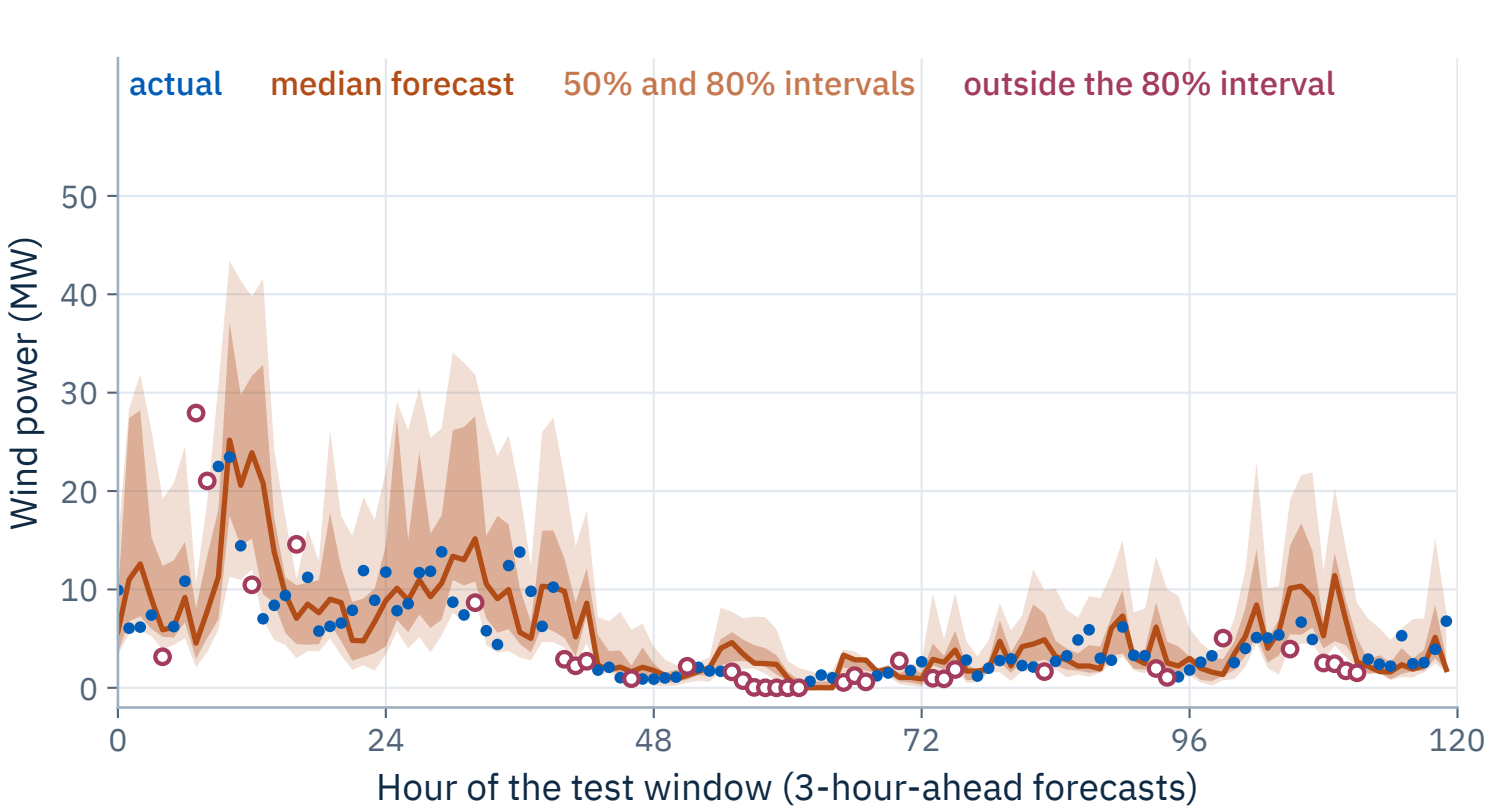
06

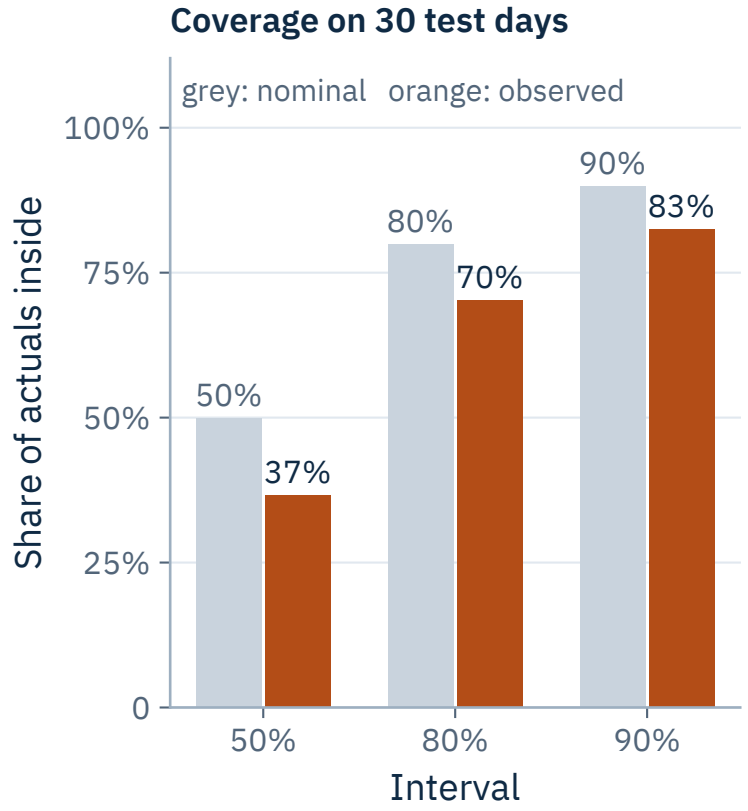
Uncertainty and sequence anomalies

How wrong might the forecast be, and is the process normal?

- Quantile forecasts and the pinball loss
- Prediction intervals and their coverage
- LSTM autoencoders for anomaly detection
- Thresholds, events and applications

A forecast should state its uncertainty



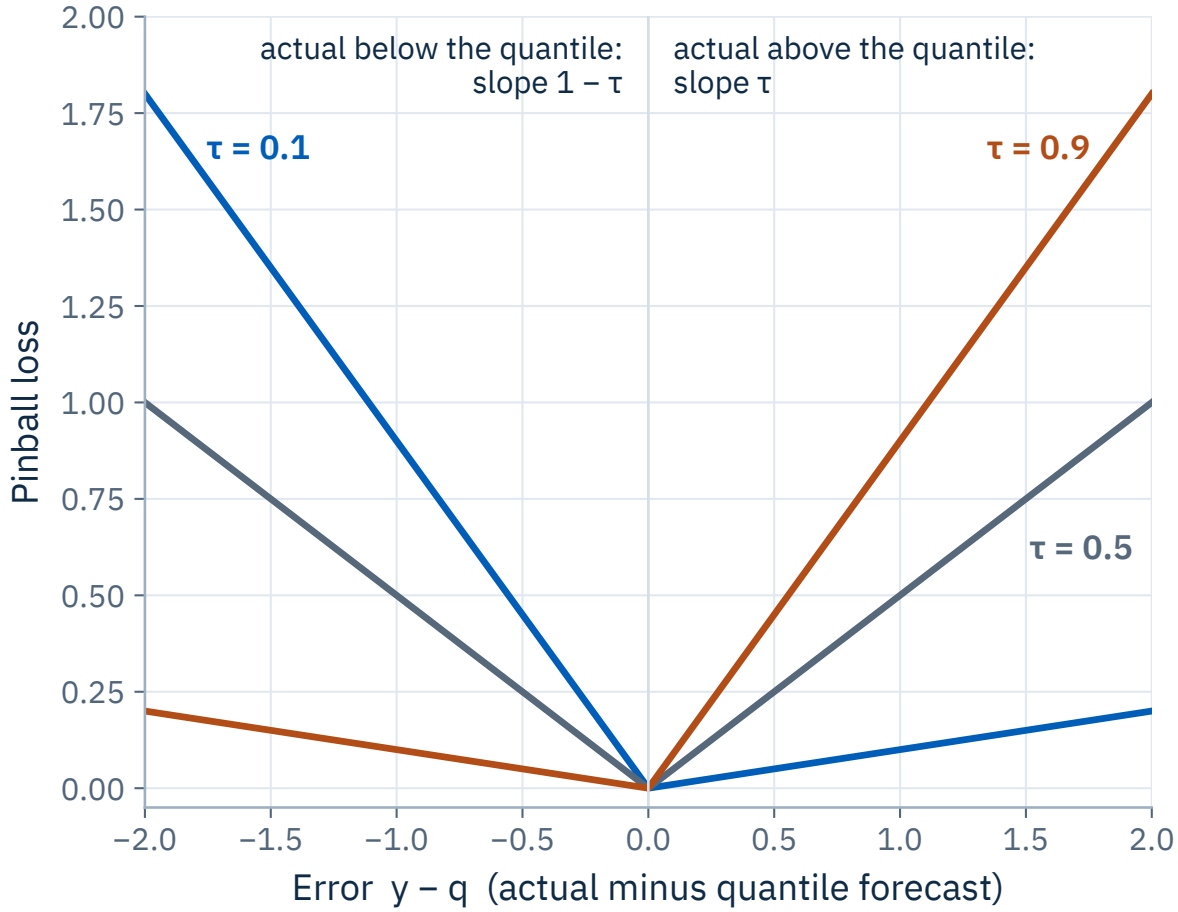


Synthetic 50 MW wind farm (seed 3). Quantile gradient boosting, 3 hours ahead, on the last six hourly values and hour of day; trained on days 1–90, scored on days 91–120.

Quantile forecasts minimize the pinball loss

$$L_\tau(y, q) = \max(\tau(y - q),\ (\tau - 1)(y - q))$$

- τ = 0.9 penalizes under-forecasts nine times more than over-forecasts.
- The minimizer is the τ-quantile of the outcome.
- One output per quantile, trained with one loss each.



Exact quantile (pinball) loss for τ = 0.1, 0.5 and 0.9.

PRACTICE

4 min · Individual

Pinball loss and coverage by hand

1. For $\tau = 0.9$ and an actual of 100 MW, compute the loss for $q = 90$ and for $q = 110$.
2. A nominal 80% interval contains 150 of 200 test hours. What is its coverage?
3. Is the interval too narrow or too wide, and what would you do?

Judge intervals by calibration, then sharpness

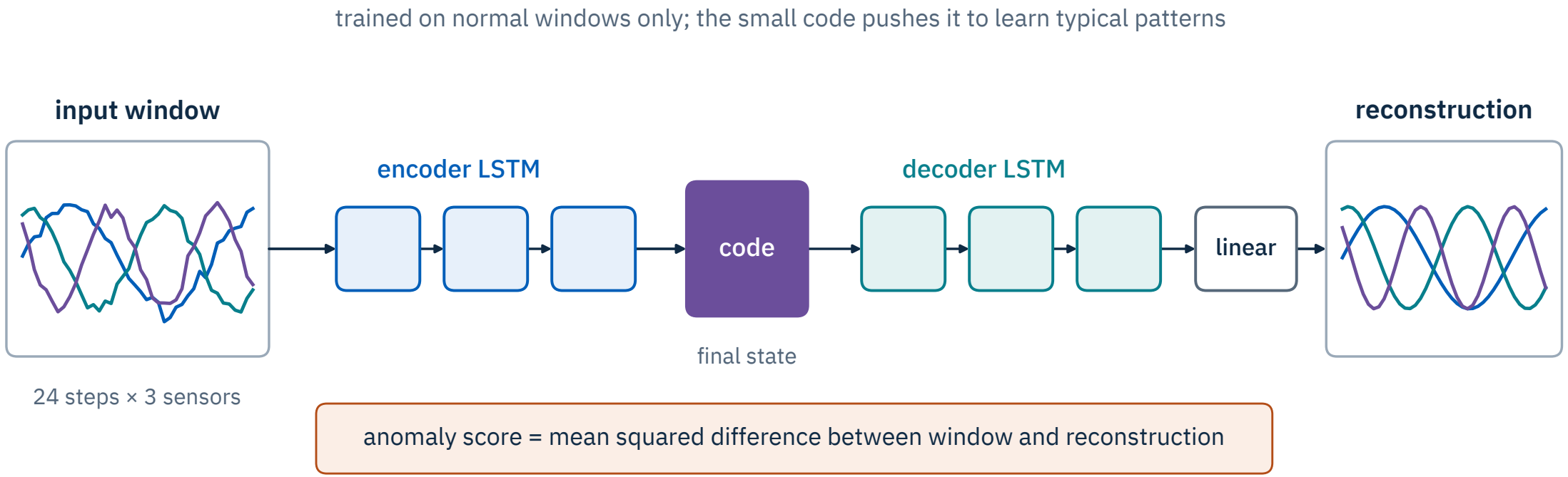
A probabilistic forecast is evaluated in a fixed order, on held-out periods only.

1. **Coverage** — share of actuals inside each interval
2. **Width** — narrower is better, at correct coverage
3. **Pinball loss** — averaged over the quantiles
4. **Breakdown** — by horizon, season and regime
5. **Recalibrate** — on a recent validation period

◆ An interval that is sharp but under-covers promises more certainty than it has.

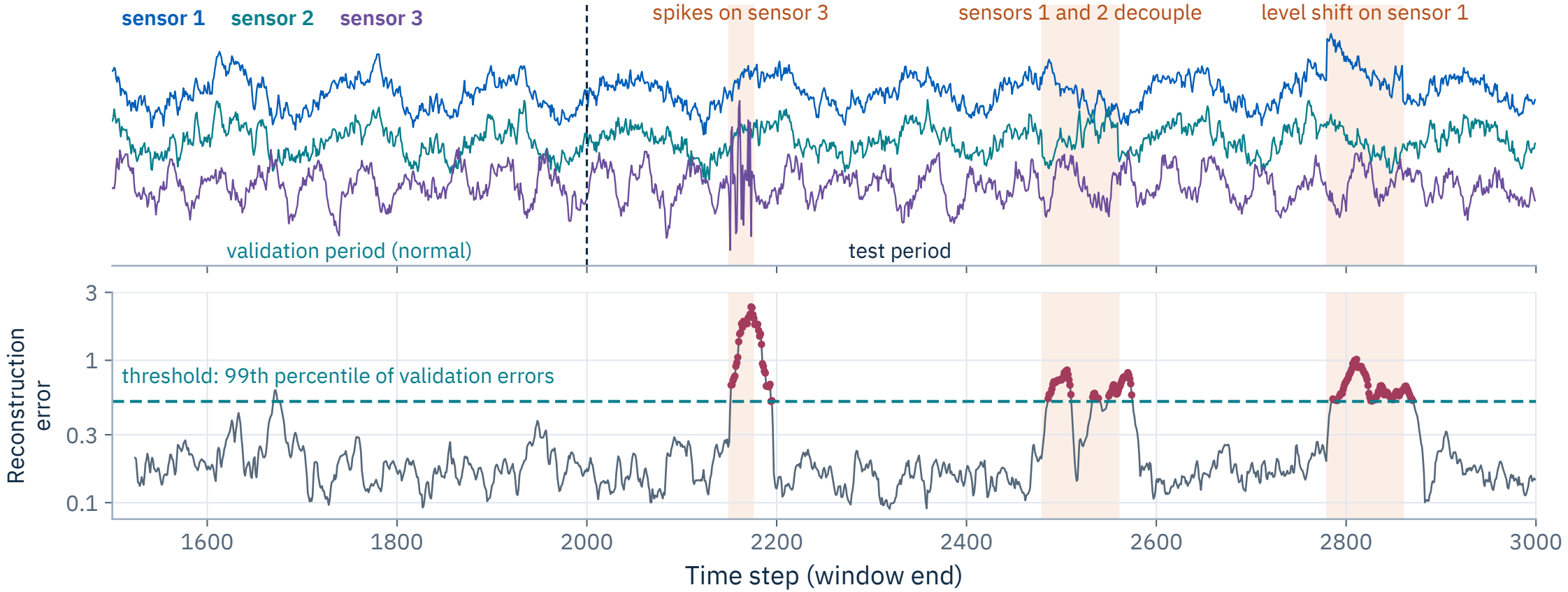
An LSTM autoencoder learns to reconstruct normal windows

Train on normal operation only; windows that differ from normal reconstruct badly.



Schematic (curves illustrative). Notebook 04 uses 30-step windows of the sensor columns.

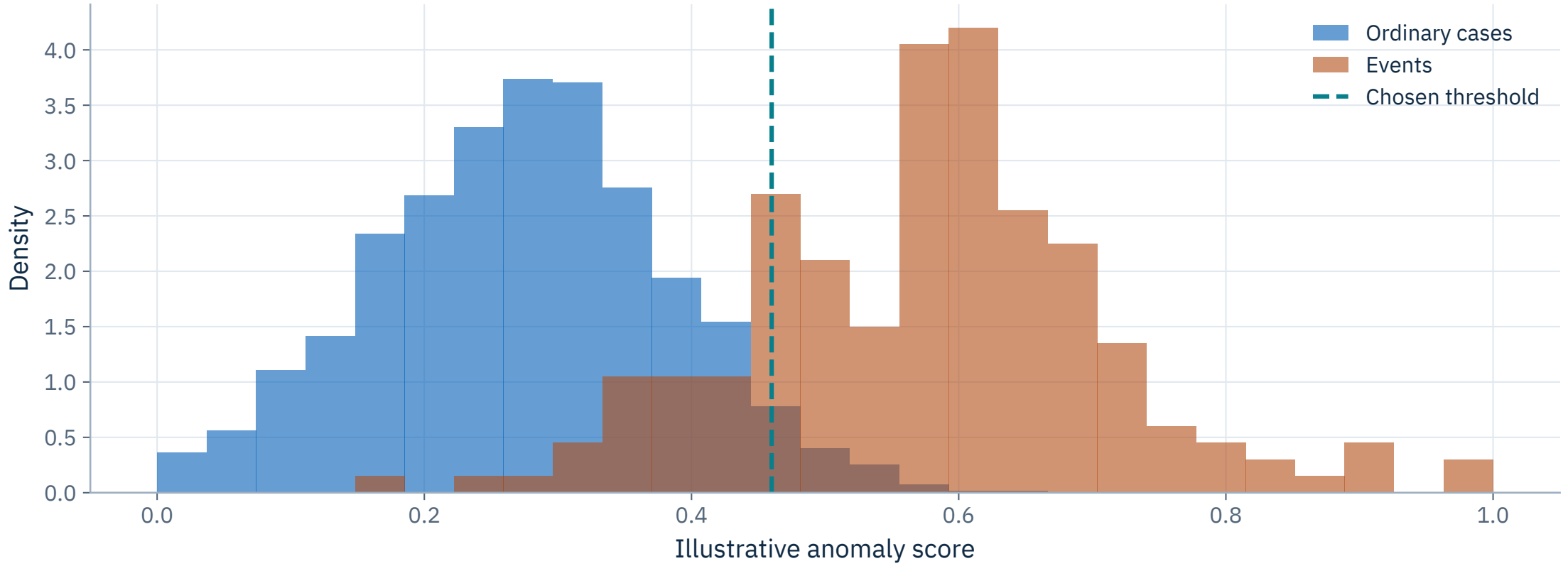
Reconstruction error flags unusual windows



Synthetic three-sensor series (seed 5) with three injected anomalies; LSTM autoencoder (16 units, 24-step windows) trained on steps 0–1,499 (torch seed 0).

Any threshold trades missed events for false alarms

Moving the line right removes false alarms and misses more events; no position removes both.



Simulated score distributions (seed 42), not a fitted detector or calibrated probabilities. The machine learning course uses the same example.

Thresholds and events decide what an alert means

Fit on past normal data

- Scaler and threshold from earlier periods
- Never from errors on training windows

Score events, not windows

- Overlapping windows repeat one event
- Report lead time and false alarms per week

Expect drift

- New regimes look anomalous
- Retrain and re-threshold on a schedule

◆ Lab 2 applies these rules to real paper-machine data.

Each application needs its own target and evaluation

Application	Possible target	Evaluation priority
ESP monitoring	Future event within a defined horizon	Event recall, lead time, false alerts
Production forecasting	Rate at one or more future horizons	Error by well and horizon
Drilling ROP	Future rate of penetration	Covariates available at the origin
Load and renewables	Hourly values, day ahead	Error by horizon, interval coverage
Sequence autoencoder	Reconstruction or prediction residual	Threshold validation and regime effects

These are task definitions to evaluate, not established capabilities.

LABORATORY

35 min

Lab 2: sequence anomalies or a retail forecast

- Option A: train the LSTM autoencoder on an early normal period only.
- Set the threshold on a later normal period; score the remaining rows.
- Report events found, events missed and false alarms, not only F1.
- Option B: retail forecast for one item; beat the weekly seasonal naive.
- Either option: one plot, one table and one caveat.

NOTEBOOK

`exercises/04_timeseries_autoencoder/TS_Autoencoder_Student.ipynb`

EVIDENCE TO SUBMIT

- Error-over-time plot with the threshold
- Event or baseline comparison table
- One caveat about the evaluation

DISCUSSION

What did the alerts, or the forecasts, really show?

- How many distinct events were detected, and how early?
- How many alerts per week would operators receive?
- Did the threshold come from data the model never trained on?
- Retail: did the LSTM beat the weekly seasonal naive on MAE?

◆ Report events and alarms per week; an F1 score on rows hides both.

PART 7 OF 7

07

Course review

Five days, one habit: evidence before architecture.

- A map of the five days
- Choosing a model family
- The final review exercise
- Key takeaways and exit check

COURSE REVIEW

Five days build one chain of evidence

Day 1
Neural network foundations

- neurons and MLPs
- losses, gradients
- backpropagation

Day 2
Training neural networks reliably

- initialization
- regularization
- optimizers, tuning

Day 3
Energy modelling capstone

- data contracts
- tabular-NN playbook
- error analysis

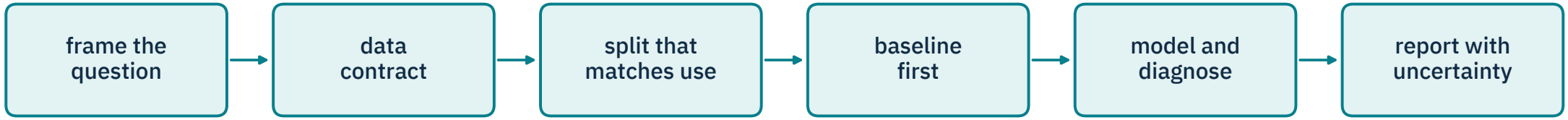
Day 4
Convolutional neural networks

- convolution, pooling
- transfer learning
- 1D CNNs, imaging

Day 5
Sequence models and attention

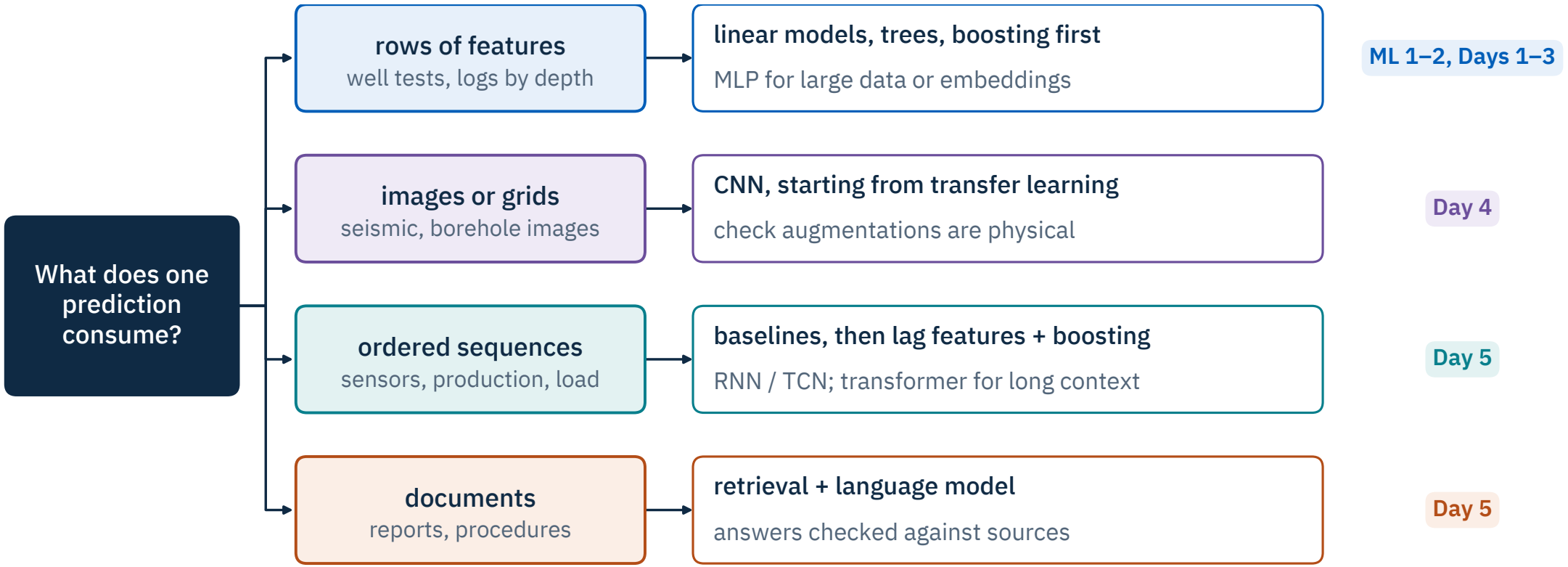
- windows, baselines
- RNN, LSTM, TCN
- attention, intervals

the evidence chain used every day



Schematic summary of the five days of the neural networks course.

Choose the model family from the data geometry



Schematic model-choice guide: a starting point to test against baselines, not a rule.

Every model family shares the same evaluation habits

Data	Split that matches use	First baseline	Report
Rows per well or asset	Grouped by well, often also by time	Mean or linear model	Error by well
Images or grids	By survey, well or field	Pretrained features + linear	Recall per class
Sequences	Rolling origins in time	Persistence, seasonal naive	Error by horizon
Documents	By document set	Keyword search	Faithfulness to sources

Uncertainty and monitoring apply to all four.

PRACTICE

10 min · Pairs

Defend a model choice for one energy task

Pick one: ESP gas-lock warning 1 hour ahead; facies from well-log curves; day-ahead feeder load.

1. Choose a dense model, CNN, RNN or attention model and defend the choice.
2. Define the tensor shapes, target timing, baseline and evaluation split.
3. Explain one plausible failure and the evidence that would reveal it.

DISCUSSION

A strong answer names the timing before the network

1. **Unit and timing**: what one prediction consumes, and when it is made
2. **Shapes**: (batch, time, features) in; the target shape out
3. **Baseline**: the simplest method with exactly the same timing
4. **Split**: grouped or rolling origin, matching deployment
5. **Failure evidence**: the breakdown that would expose the weakness

Key takeaways

- ✓ Every forecast is defined by its origin, horizon and the inputs available then.
- ✓ Beat persistence and seasonal naive on identical targets before claiming skill.
- ✓ Gates, dilations and attention give information and gradients shorter paths.
- ✓ Report errors by horizon, with intervals whose coverage you have checked.
- ✓ Choose the architecture from the data geometry; evaluate every one the same way.

PRACTICE

4 min · Individual

Exit check: five quick questions

1. A forecast issued at 10:00 targets 18:00 tomorrow. What is the horizon?
2. Which baseline would you compute first for hourly load, and why?
3. Which quantity does the LSTM forget gate multiply?
4. Which positions does a causal mask hide from a query?
5. An 80% interval covers 64% of test hours. What do you conclude?

References and further study

- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, 3rd edition (online); Hyndman and Koehler (2006) on MASE.
- Hochreiter and Schmidhuber (1997), long short-term memory; Cho and colleagues (2014), GRU; Bai, Kolter and Koltun (2018), TCN.
- Bahdanau, Cho and Bengio (2014), attention; Vaswani and colleagues (2017), *Attention Is All You Need*; Nie and colleagues (2023), PatchTST.
- Koenker and Bassett (1978), quantile regression; Gneiting and Raftery (2007), proper scoring rules.
- PyTorch documentation (LSTM, GRU, MultiheadAttention) and the Keras weather-forecasting example.